

2004 年上半年 软件设计师 上午试题

● 在面向对象技术中，类属是一种 (1) 机制。一个类属类是关于一组类的一个特性抽象，它强调的是这些类的成员特征中与 (2) 的那些部分，而用变元来表示与 (3) 的那些部分。

- (1) A. 包含多态 B. 参数多态 C. 过载多态 D. 强制多态
(2) A. 具体对象无关 B. 具体类型无关 C. 具体对象相关 D. 具体类型相关
(3) A. 具体对象无关 B. 具体类型无关 C. 具体对象相关 D. 具体类型相关

● (4) 的特点是数据结构中元素的存储地址与其关键字之间存在某种映射关系。

- (4) A. 树形存储结构 B. 链式存储结构 C. 索引存储结构 D. 散列存储结构

● 若循环队列以数组 $Q[0..m-1]$ 作为其存储结构，变量 $rear$ 表示循环队列中队尾元素的实际位置，其移动按 $rear=(rear+1) \bmod m$ 进行，变量 $length$ 表示当前循环队列中的元素个数，则循环队列的队首元素的实际位置是 (5)。

- (5) A. $rear-length$ B. $(rear-length+m) \bmod m$
C. $(1+rear+m-length) \bmod m$ D. $m-length$

● 一个含有 n 个顶点和 e 条边的简单无向图，在其邻接矩阵存储结构中共有 (6) 个零元素。

- (6) A. e B. $2e$ C. n^2-e D. n^2-2e

● 若一棵哈夫曼 (Huffman) 树共有 9 个顶点，则其叶子结点的个数为 (7)。

- (7) A. 4 B. 5 C. 6 D. 7

● 若采用邻接矩阵来存储简单有向图，则其某一个顶点 i 的入度等于该矩阵 (8)。

- (8) A. 第 i 行中值为 1 的元素个数 B. 所有值为 1 的元素总数
C. 第 i 行及第 i 列中值为 1 的元素总个数 D. 第 i 列中值为 1 的元素个数

● 在一棵度为 3 的树中，若有 2 个度为 3 的结点，有 1 个度为 2 的结点，则有 (9) 个度为 0 的结点。

- (9) A. 4 B. 5 C. 6 D. 7

● 设结点 x 和 y 是二叉树中任意的两个结点，在该二叉树的先根遍历序列中 x 在 y 之前，而在其后根遍历序列中 x 在 y 之后，则 x 和 y 的关系是 (10)。

- (10) A. x 是 y 的左兄弟 B. x 是 y 的右兄弟

C. x 是 y 的祖先

D. x 是 y 的后裔

● 设顺序存储的某线性表共有 123 个元素，按分块查找的要求等分为 3 块。若对索引表采用顺序查找方法来确定子块，且在确定的子块中也采用顺序查找方法，则在等概率的情况下，分块查找成功的平均查找长度为 (11)。

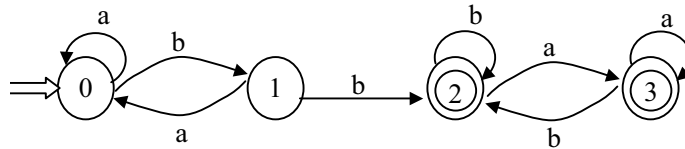
(11) A. 21

B. 23

C. 41

D. 62

● 下图为一确定有限自动机 (DFA) 的状态转换图，与该自动机等价的正规表达式是 (12)，图中的 (13) 是可以合并的状态。



(12) A. $(a|b)^*bb(a^*b^*)^*$

B. $(a|b)^*bba^*|b^*$

C. $(a^*b^*)bb(a|b)^*$

D. $(a^*|b^*)^*bb(a^*|b^*)$

(13) A. 0 和 1

B. 2 和 3

C. 1 和 2

D. 0 和 3

● 已知有一维数组 $A[0..m*n-1]$ ，若要对应为 m 行、 n 列的矩阵，则下面的对应关系 (14) 可将元素 $A[k]$ ($0 \leq k < m*n$) 表示成矩阵的第 i 行、第 j 列的元素 ($0 \leq i < m, 0 \leq j < n$)。

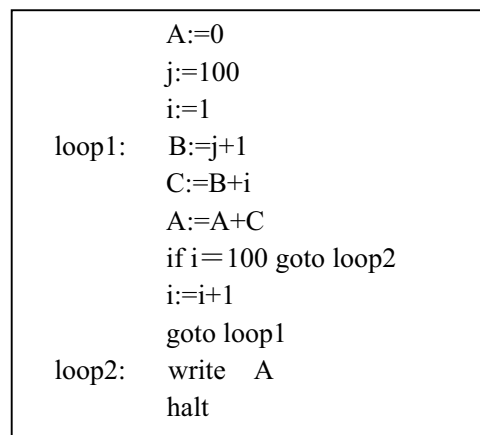
(14) A. $i=k/n, j=k \% m$

B. $i=k/m, j=k \% m$

C. $i=k/n, j=k \% n$

D. $i=k/m, j=k \% n$

● 编译的优化工作对于下面程序段构造的控制流程图有 (15) 个基本块。



(15) A. 1

B. 2

C. 3

D. 4

● 文法 $G[S]: S \rightarrow xSx|y$ 所描述的语言是 (16) ($n \geq 0$)。

- (16) A. $(xyx)^n$ B. xyx^n C. $xy^n x$ D. $x^n yx^n$

● 已知关系 R 如下表所示，关系 R 的主属性为 (17)，候选关键字分别为 (18)。

R			
A	B	C	D
a	b	c	d
a	c	d	e
b	d	e	f
a	d	c	g
b	c	d	g
c	b	e	g

- (17) A. ABC B. ABD C. ACD D. ABCD

- (18) A. ABC B. AB、AD C. AC、AD 和 CD D. AB、AD、BD 和 CD

● 设供应商供应零件的关系模式为 $SP(Sno, Pno, Qty)$ ，其中 Sno 表示供应商号， Pno 表示零件号， Qty 表示零件数量。查询至少包含了供应商“168”所供应的全部零件的供应商号的 SQL 语句如下：

```
SELECT Sno
FROM SP SPX
WHERE (19)
      (SELECT *
       FROM SP SPY
       WHERE (20) AND NOT EXISTS
        (SELECT *
         FROM SP SPZ
         WHERE (21)));
```

- (19) A. EXISTS B. NOT EXISTS C. IN D. NOT IN

- (20) A. $SPY.Sno = '168'$ B. $SPY.Sno <> '168'$
C. $SPY.Sno = SPX.Sno$ D. $SPY.Sno <> SPX.Sno$

- (21) A. $SPZ.Sno = SPY.Sno$ AND $SPZ.Pno = SPY.Pno$
B. $SPZ.Sno = SPX.Sno$ AND $SPZ.Pno = SPX.Pno$
C. $SPZ.Sno = SPX.Sno$ AND $SPZ.Pno = SPY.Pno$
D. $SPY.Sno = '168'$ AND $SPZ.Pno = SPY.Pno$

● 假设某计算机系统的内存大小为 256K，在某一时刻内存的使用情况如图 A 所示。此时，若进程顺序请求 20K、10K 和 5K 的存储空间，系统采用 (22) 算法为进程依次分配内存，则分配后的内存情况如图 B 所示。

起始地址	0K	20K	50K	90K	100K	105K	135K	160K	175K	195K	220K
------	----	-----	-----	-----	------	------	------	------	------	------	------

状态	已用	未用	已用	已用	未用	已用	未用	已用	未用	未用	已用
容量	20K	30K	40K	10K	5K	30K	25K	15K	20K	25K	36K

图 A

起始地址	0K	20K	40K	50K	90K	100K	105K	135K	145K	160K	175K	195K	200K	220K
状态	已用	已用	未用	已用	已用	未用	已用	已用	未用	已用	未用	已用	未用	已用
容量	20K	20K	10K	40K	10K	5K	30K	10K	15K	15K	20K	5K	20K	36K

图 B

(22) A. 最佳适应 B. 最差适应 C. 首次适应 D. 循环首次适应

● 若有一个仓库，可以存放 P1、P2 两种产品，但是每次只能存放一种产品。要求：

① $w = P1 \text{ 的数量} - P2 \text{ 的数量}$

② $-i < w < k$ (i, k 为正整数)

若用 PV 操作实现 P1 和 P2 产品的入库过程，至少需要 (23) 个同步信号量及 (24) 个互斥信号量，其中，同步信号量的初值分别为 (25)，互斥信号量的初值分别为 (26)。

- (23) A. 0 B. 1 C. 2 D. 3
 (24) A. 0 B. 1 C. 2 D. 3
 (25) A. 0 B. $i, k, 0$ C. i, k D. $i-1, k-1$
 (26) A. 1 B. 1, 1 C. 1, 1, 1 D. i, k

● 当在软件工程的环境中考虑风险时，主要基于 Charette 提出的三个概念。以下选项中不属于这三个概念的是 (27)。项目风险关系着项目计划的成败，(28) 关系着软件的生存能力。在进行软件工程风险分析时，项目管理人员要进行四种风险评估活动，这四种活动是 (29) 以及确定风险估计的正确性。

- (27) A. 关心未来 B. 关心变化 C. 关心技术 D. 关心选择
 (28) A. 资金风险 B. 技术风险 C. 商业风险 D. 预算风险
 (29) A. 建立表示风险概率的尺度，描述风险引起的后果，估计风险影响的大小
 B. 建立表示风险概率的尺度，描述风险引起的后果，确定产生风险的原因
 C. 确定产生风险的原因，描述风险引起的后果，估计风险影响的大小
 D. 建立表示风险概率的尺度，确定产生风险的原因，估计风险影响的大小

● 软件的互操作性是指 (30)。

- (30) A. 软件的可移植性
 B. 人机界面的可交互性
 C. 连接一个系统和另一个系统所需的工作量
 D. 多用户之间的可交互性

● 面向对象的测试可分为四个层次，按照由低到高的顺序，这四个层次是 (31)。

- (31) A. 类层——模板层——系统层——算法层

- B. 算法层——类层——模板层——系统层
- C. 算法层——模板层——类层——系统层
- D. 类层——系统层——模板层——算法层

● 在选择开发方法时，有些情况不适合使用原型法，以下选项中不能使用快速原型法的情况是 (32)。

- (32) A. 系统的使用范围变化很大 B. 系统的设计方案难以确定
C. 用户的需求模糊不清 D. 用户的数据资源缺乏组织和管理

● 从信息资源管理的观点出发，信息系统的逻辑结构一般由四部分组成，其中 (33) 利用信息系统提供的信息进行决策和选择，是信息系统服务的对象。

- (33) A. 信息源 B. 信息处理器 C. 信息使用者 D. 信息管理者

● 以下选项中，最适合于用交互式计算机软件解决的问题是 (34)。

- (34) A. 非结构化决策问题 B. 半结构化决策问题
C. 结构化决策问题 D. 确定性问题

● CMU/SEI 推出的 (35) 将软件组织的过程能力分为五个成熟度级别，每一个级别定义了一组过程能力目标，并描述了要达到这些目标应该具备的实践活动。

- (35) A. CMM B. PSP C. TSP D. SSE-CMM

● 我国著作权法中对公民作品的发表权的保护期限是 (36)。

- (36) A. 作者有生之年加死后 50 年 B. 作品完成后 50 年
C. 没有限制 D. 作者有生之年

● 商业秘密是我国 (37) 保护的一项重要内容，它包括技术秘密和经营秘密两项基本内容。

- (37) A. 《专利法》 B. 《著作权法》 C. 《商标法》 D. 《反不正当竞争法》

● 某程序员利用他人已有的财务管理信息系统软件中所用的处理过程和运算方法，为某企业开发出财务管理软件，则该程序员 (38)。

- (38) A. 不侵权，因为计算机软件开发所用的处理过程和运算方法不属于著作权法的保护对象
B. 侵权，因为处理过程和运算是他人已有的
C. 侵权，因为计算机软件开发所用的处理过程和运算是著作权法的保护对象
D. 是否侵权，取决于该程序员是不是合法的受让者

● OSI (Open System Interconnection) 安全体系方案 X.800 将安全性攻击分为两类，即被动攻击和主动攻击。主动攻击包括篡改数据流或伪造数据流，这种攻击试图改变系统资源或影响系统运行。下列攻击方式中不属于主动攻击的为 (39)。

- (39) A. 伪装 B. 消息泄漏 C. 重放 D. 拒绝服务

● 安全机制是实现安全服务的技术手段，一种安全机制可以提供多种安全服务，而一种安全服务也可采用多种安全机制。加密机制不能提供的安全服务是 (40)。

(40) A. 数据保密性 B. 访问控制 C. 数字签名 D. 认证

● 消息摘要算法 MD5 (Message Digest) 是一种常用的 Hash 函数。MD5 算法以一个任意长数据块作为输入，其输出为一个 (41) 比特的消息摘要。

(41) A. 128 B. 160 C. 256 D. 512

● 5 分钟、双声道、22.05kHz 采样、16 位量化的声音，经 5:1 压缩后，其数字音频的数据量约为 (42)。

(42) A. 5.168MB B. 5.047MB C. 26.460MB D. 26.082MB

● 在 YUV 彩色空间中对 YUV 分量进行数字化，对应的数字化位数通常采用 Y:U:V = (43)。

(43) A. 8:4:2 B. 8:4:4 C. 8:8:4 D. 4:8:8

● 若视频图像序列中两帧相邻图像之间存在着极大的相关性，则这种相关性称为 (44) 冗余。

(44) A. 空间 B. 时间 C. 视觉 D. 信息熵

● 下列关于计算机图形图像的描述中，不正确的是 (45)。

(45) A. 图像都是由一些排成行列的点（像素）组成的，通常称为位图或点阵图
B. 图像的最大优点是容易进行移动、缩放、旋转和扭曲等变换
C. 图形是用计算机绘制的画面，也称矢量图
D. 图形文件中只记录生成图的算法和图上的某些特征点，数据量较小

● 若某个计算机系统中，内存地址与 I/O 地址统一编址，访问内存单元和 I/O 设备是靠 (46) 来区分的。

(46) A. 数据总线上输出的数据 B. 不同的地址代码
C. 内存与 I/O 设备使用不同的地址总线 D. 不同的指令

● 在中断响应过程中，CPU 保护程序计数器的主要目的是 (47)。

(47) A. 使 CPU 能找到中断服务程序的入口地址
B. 为了实现中断嵌套
C. 为了使 CPU 在执行完中断服务程序时能回到被中断程序的断点处
D. 为了使 CPU 与 I/O 设备并行工作

● 在 32 位的总线系统中，若时钟频率为 1000MHz，总线上 5 个时钟周期传送一个

32 位字，则该总线系统的数据传送速率约为 (48) 兆字节/秒。

- (48) A. 200 B. 600 C. 800 D. 1000

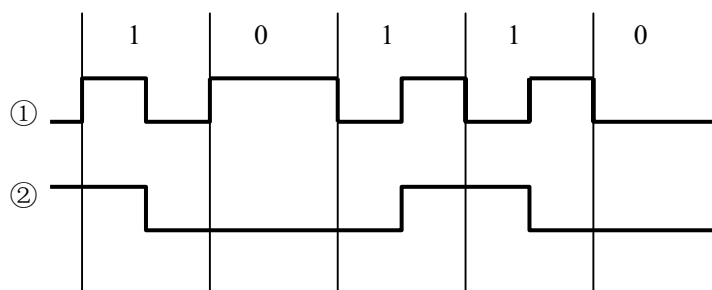
● 现有四级指令流水线，分别完成取指、取数、运算、传送结果四步操作。若完成上述操作的时间依次为 9ns、10ns、6ns、8ns。则流水线的操作周期应设计为 (49) ns。

- (49) A. 6 B. 8 C. 9 D. 10

● 从基本的 CPU 工作原理来看，若 CPU 执行 MOV R1, R0 指令（即将寄存器 R0 的内容传送到寄存器 R1 中），则 CPU 首先要完成的操作是 (50)（其中 PC 为程序计数器；M 为主存储器；DR 为数据寄存器；IR 为指令寄存器；AR 为地址寄存器）。

- (50) A. (R0) → R1 B. PC → AR C. M → DR D. DR → IR

● 若磁盘的写电流波形如下图所示：



图中①波形的记录方式是 (51)；②波形的记录方式是 (52)。

- (51) A. 调频制 (FM) B. 改进调频制 (MFM)
C. 调相制 (PE) D. 不归零制 (NRZ)
(52) A. 调频制 (FM) B. 改进调频制 (MFM)
C. 调相制 (PE) D. 不归零制 (NRZ)

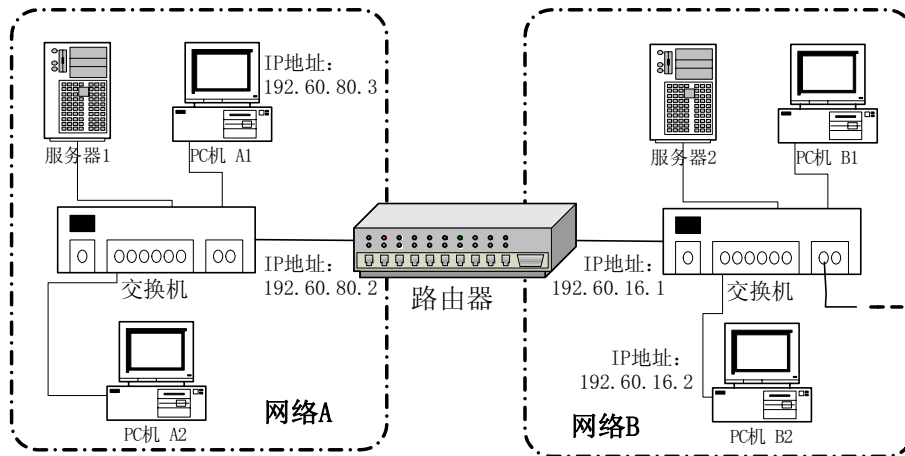
● 关于 RS-232C，以下叙述中正确的是 (53)。

- (53) A. 能提供最高传输率 9600bps
B. 能作为计算机与调制解调器之间的一类接口标准
C. 可以用菊花链式连接
D. 属于一类并行接口

● 某网络的拓扑结构如下图所示，网络 A 中 A2 主机的 IP 地址可以为 (54)；如果网络 B 中有 1000 台主机，那么需要为网络 B 分配 (55) 个 C 类网络地址，其中 B1 主机的 IP 地址可以为 (56)，网络 B 的子网掩码应为 (57)。

- (54) A. 192.60.80.0 B. 192.60.80.2 C. 192.60.80.3 D. 192.60.80.4

- (55) A. 1 B. 2 C. 3 D. 4
 (56) A. 192.60.16.1 B. 192.60.16.2 C. 192.60.16.5 D. 192.60.16.255
 (57) A. 255.255.255.0 B. 255.255.254.0 C. 255.255.253.0 D. 255.255.252.0



● FTP 默认的数据端口号是 (58)。HTTP 默认的端口号是 (59)。

- (58) A. 20 B. 21 C. 22 D. 23
 (59) A. 25 B. 80 C. 1024 D. 8080

● 某个计算机中心有 28 台微机，每台微机有 24 个应用，每个应用占用 1 个端口地址，则这个计算机中心所有应用的地址总数为 (60)。

- (60) A. 24 B. 28 C. 52 D. 672

● 设 f 表示某个二元逻辑运算符， PfQ 的真值表如下表所示，则 PfQ 等价于 (61)。

P	Q	PfQ
0	0	0
0	1	1
1	0	0
1	1	0

- (61) A. $P \vee \neg Q$ B. $\neg P \wedge Q$ C. $\neg P \wedge \neg Q$ D. $\neg P \vee \neg Q$

● 设 Y 表示集合的并运算， I 表示集合的交运算， $\bar{A}$ 表示集合 A 的绝对补， $A - B$ 表示集合 A 与 B 的差，则 $A - B =$ (62)。

- (62) A. $A \vee (A \cap B)$ B. $A \vee \bar{B}$ C. $A \cap (A \vee B)$ D. $A \cap \bar{B}$

● 设集合 $Z_{26} = \{0, 1, \Lambda, 25\}$ ，乘法密码的加密函数为 $E_k: Z_{26} \rightarrow Z_{26}$ ，

$E_k(i) = (ki) \bmod 26$, 密钥 $k \in Z_{26} - \{0\}$, 则加密函数 $E_7(i) = (7i) \bmod 26$ 是一个 (63) 函数。

- (63) A. 单射但非满射 B. 满射但非单射 C. 非单射且非满射 D. 双射

● 类比二分搜索算法, 设计 k 分搜索算法 (k 为大于 2 的整数) 如下: 首先检查 n/k 处 (n 为被搜索集合的元素个数) 的元素是否等于要搜索的值, 然后检查 $2n/k$ 处的元素, ..., 这样, 或者找到要搜索的元素, 或者把集合缩小到原来的 $1/k$; 如果未找到要搜索的元素, 则继续在得到的集合上进行 k 分搜索; 如此进行, 直到找到要搜索的元素或搜索失败。此 k 分搜索算法在最坏情况下搜索成功的时间复杂度为 (64), 在最好情况下搜索失败的时间复杂度为 (65)。

- (64) A. $O(\log n)$ B. $O(n \log n)$ C. $O(\log_k n)$ D. $O(n \log_k n)$

- (65) A. $O(\log n)$ B. $O(n \log n)$ C. $O(\log_k n)$ D. $O(n \log_k n)$

● (66) data effectively is crucial for success in today's competitive environment. Managers must know how to use a variety of tools. Integrated data takes information from different sources and puts it together in a meaningful and useful way. One of the difficulties of this is the (67) in hardware and software. (68) integration uses a base document that contains copies of other objects. (69) integration uses a base document that contains the current or most recent version of the source document it contains.

(70) provides an overview of the program written in "plain" English, without the computer syntax.

- | | | | |
|-----------------------------|---------------------|------------------|----------------|
| (66) A. Generalizing | B. Sharing | C. General-using | D. Globalizing |
| (67) A. similarity | B. interoperability | C. diversity | D. interaction |
| (68) A. Simulated | B. Duplicated | C. Dynamic | D. Static |
| (69) A. Linked | B. Pointed | C. Dynamic | D. Static |
| (70) A. High-level language | | B. Decision tree | |
| C. Pseudocode | | D. Flowchart | |

● Traditional structured analysis techniques focus upon the flow of (71) within a system. Object-oriented analysis emphasizes the building of real-world models. It examines requirements from the perspective of the classes and objects found in the vocabulary of the (72) domain.

Traditional system design method emphasizes the proper and effective structure of a

complex system. Object-oriented design method encompasses the process of object-oriented decomposition and a (73) for depicting both logical and physical as well as static and dynamic models of the system under design.

Object-oriented programming is a method of implementation in which programs are organized as cooperative collections of objects, each of which represents an (74) of some class, and whose classes are all members of a hierarchy of classes united via (75) relationships.

- | | | | |
|------------------|----------------|--------------|-------------------|
| (71) A. control | B. program | C. data | D. reference |
| (72) A. problem | B. solution | C. data | D. program |
| (73) A. mark | B. picture | C. symbol | D. notation |
| (74) A. instance | B. example | C. existence | D. implementation |
| (75) A. control | B. inheritance | C. inference | D. connection |

全国计算机技术与软件专业技术资格（水平）考试

2004 年上半年 软件设计师（原高级程序员）级下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2004 年上半年中国计算机软件专业技术资格和水平考试日期是(1)月(2)日。

因为正确的解答是“5 月 23 日”，故在答题纸的对应栏内写上‘5’和‘23’（参看下表）。

例题	解答栏
(1)	5
(2)	23

试题一至试题四是必答题

试题一

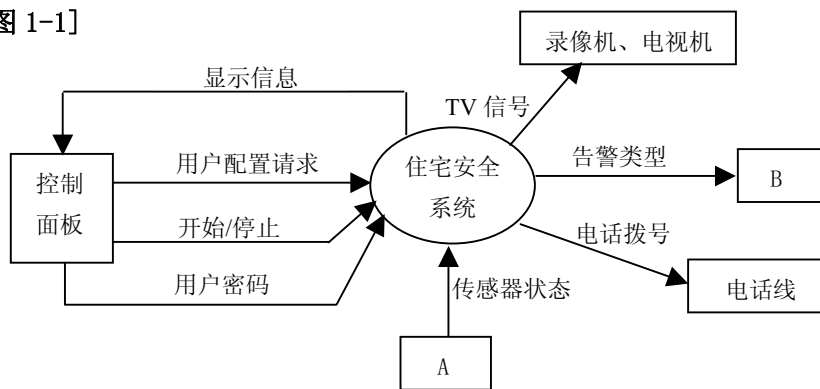
阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

[说明]

某基于微处理器的住宅安全系统，使用传感器（如红外探头、摄像头等）来检测各种意外情况，如非法进入、火警、水灾等。

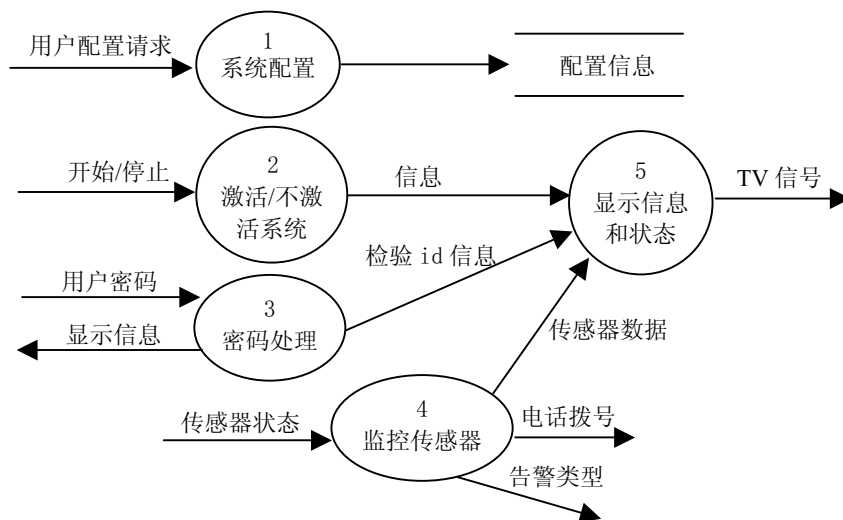
房主可以在安装该系统时配置安全监控设备（如传感器、显示器、报警器等），也可以在系统运行时修改配置，通过录像机和电视机监控与系统连接的所有传感器，并通过控制面板上的键盘与系统进行信息交互。在安装过程中，系统给每个传感器赋予一个编号（即 id）和类型，并设置房主密码以启动和关闭系统，设置传感器事件发生时自动拨出的电话号码。当系统检测到一个传感器事件时，就激活警报，拨出预置的电话号码，并报告关于位置和检测到的事件的性质等信息。

[数据流图 1-1]



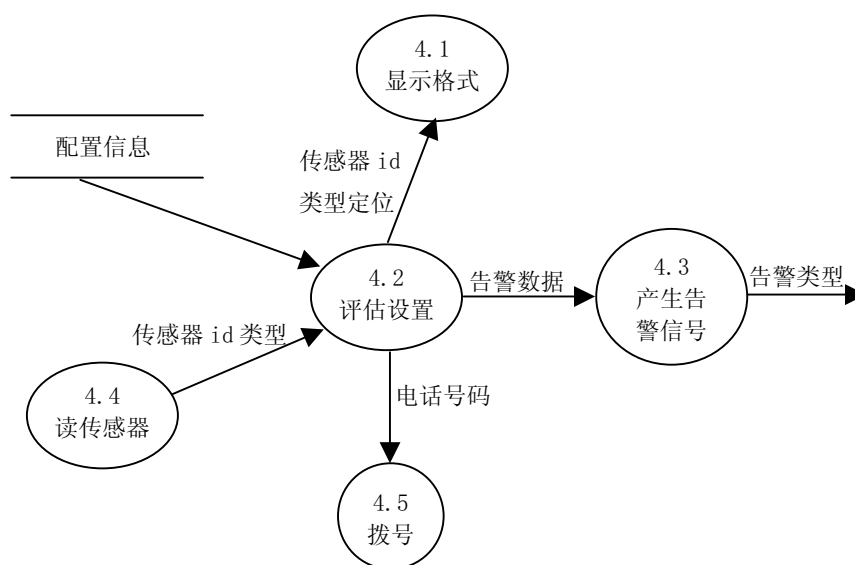
住宅安全系统顶层图

[数据流图 1-2]



住宅安全系统第 0 层 DFD 图

[数据流图 1-3]



加工 4 的细化图

[问题 1]

数据流图 1-1（住宅安全系统顶层图）中的 A 和 B 分别是什么？

[问题 2]

数据流图 1-2（住宅安全系统第 0 层 DFD 图）中的数据存储“配置信息”会影响图中的哪些加工？

[问题 3]

将数据流图 1-3（加工 4 的细化图）中的数据流补充完整，并指明加工名称、数据流的方向（输入/输出）和数据流名称。

[问题 4]

请说明逻辑数据流图(Logical Data Flow Diagram)和物理数据流图(Physical Data Flow Diagram)之间的主要差别。

试题二

阅读下列说明和算法，回答问题 1 和问题 2，将解答填入答题纸的对应栏内。

[说明]

算法 2-1 是用来检查文本文件中的圆括号是否匹配。若文件中存在圆括号没有对应的左括号或者右括号，则给出相应的提示信息，如下所示：

文件	提示信息
(1+2)	
abc)	缺少对应左括号：第 2 行，第 4 列
((def)gx))	缺少对应左括号：第 3 行，第 10 列
((h)	
ij)(k	
(lm)	缺少对应右括号：第 5 行，第 4 列；第 4 行，第 1 列

在算法 2-1 中，stack 为一整数栈。算法中各函数的说明如下表所示：

函数名	函数功能
push(int i)	将整数 i 压入栈 stack 中。
pop()	stack 的栈顶元素出栈。
empty()	判断 stack 栈是否为空。若为空，函数返回 1，否则函数返回 0。
nextch()	读取文本文件中的下一个字符，并返回该字符的 ASCII 值，将字符所在的行号以及字符在行中的位置分别存储到变量 row 和 col 中，若遇到文件结束符，则将变量 EOF 置为 true；
kind(char ch)	判断字符ch是左括号还是右括号，若是左括号，函数返回1，若是右括号，函数返回2，若两者都不是，函数返回0。

[算法 2-1]

```
将栈 stack 置空，置 EOF 为 false
ch ← nextch();
while(not EOF)
    k ← kind(ch);
    if(k == (1))
        push((2)); push((3));
    elseif( k == (4))
        if(not empty())
            pop(); pop();
        else
            显示错误信息（缺少对应左括号或右括号）；
            显示行号 row；显示列号 col；
        endif
    endif
    ch ← nextch();
endwhile
if (not empty())
    显示错误信息（缺少对应左括号或右括号）；
    while(not empty())
```

```

        row ← pop(); col ← pop();
        显示行号 row; 显示列号 col;
    endwhile
endif

```

为了识别更多种类的括号，对算法 2-1 加以改进后得到算法 2-2。算法 2-2 能够识别圆括号，方括号和花括号（不同类型的括号不能互相匹配）。改进后，函数 kind(char ch) 的参数及其对应的返回值如下表所示：

ch	(	)	{	}	[	]	其它
返回值	1	2	3	4	5	6	0

[算法 2-2]

```

将栈 stack 置空，置 EOF 为 false
ch ← nextch();
while(not EOF)
    k ← kind(ch);
    if(k > 0)
        if( 判断条件 1 )
            push( (5) );push( (6) );push( (7) );
        elseif( 判断条件 2 and 判断条件 3 )
            pop(); pop(); pop();
        else
            显示错误信息（缺少对应左括号或右括号）；
            显示行号 row; 显示列号 col;
        endif
    endif
    ch ← nextch();
endwhile
if(not empty())
    显示错误信息（缺少对应左括号或右括号）；
    while(not empty())
        pop(); row ← pop(); col ← pop();
        显示行号 row; 显示列号 col;
    endwhile
endif

```

[问题 1]

请将[算法 2-1]和[算法 2-2]中（1）至（7）处补充完整。

[问题 2]

请从下面的选项中选择相应的判断逻辑填补[算法 2-2]中的“判断条件 1”至“判断条件 3”。注意，若“判断条件 2”的逻辑判断结果为假，就无需对“判断条件 3”进行判断。

- (a) 字符是括号 (b) 字符是左括号 (c) 字符是右括号 (d) 栈空 (e) 栈不空
- (f) 栈顶元素表示的是与当前字符匹配的左括号
- (g) 栈顶元素表示的是与当前字符匹配的右括号

试题三

阅读下列说明以及图 3-1 和图 3-2，回答问题 1、问题 2 和问题 3，将解答填入答题纸的对应栏内。

[说明]

某电话公司决定开发一个管理所有客户信息的交互式网络系统。系统的功能如下：

1. 浏览客户信息：任何使用 Internet 的网络用户都可以浏览电话公司所有的客户信息（包括姓名、住址、电话号码等）。

2. 登录：电话公司授予每个客户一个帐号。拥有授权帐号的客户，可以使用系统提供的页面设置个人密码，并使用该帐号和密码向系统注册。

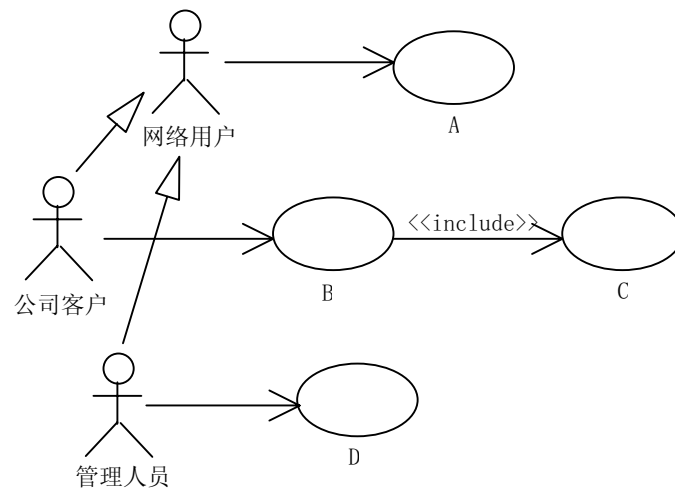
3. 修改个人信息：客户向系统注册后，可以发送电子邮件或者使用系统提供的页面，对个人信息进行修改。

4. 删除客户信息：只有公司的管理人员才能删除不再接受公司服务的客户的信息。

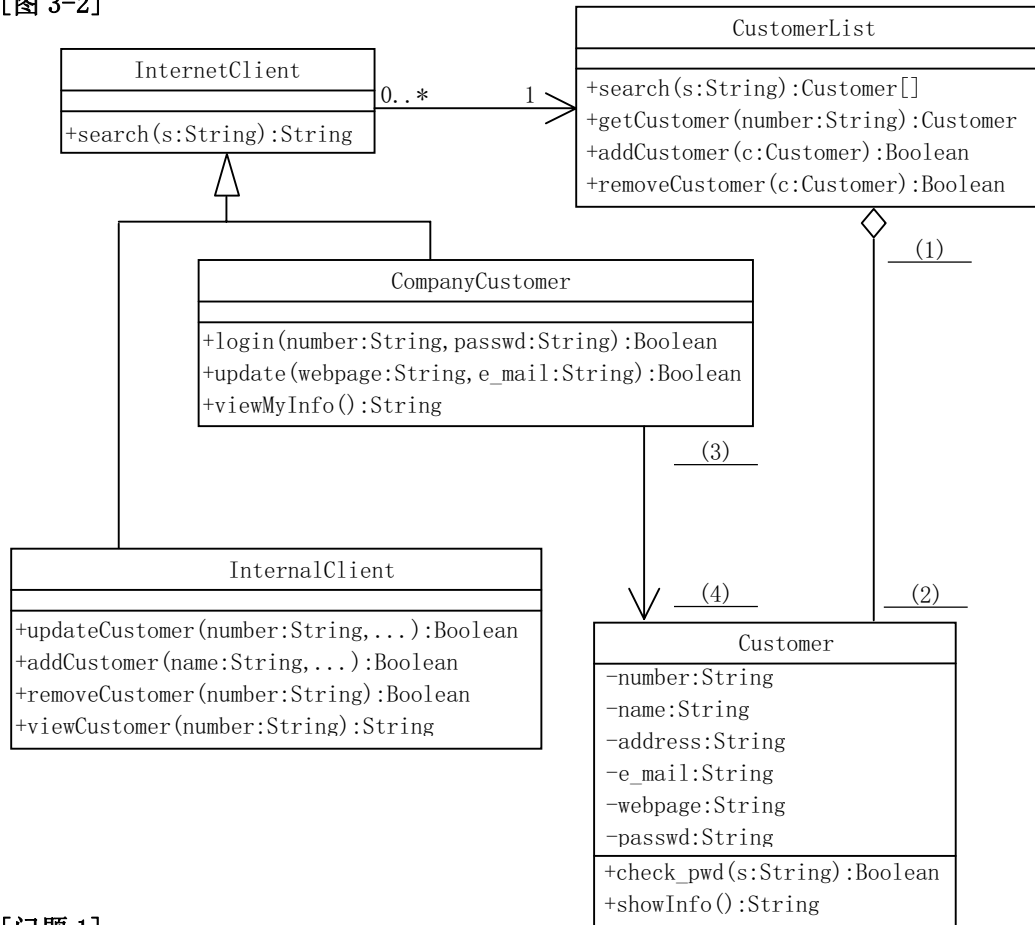
系统采用面向对象方法进行开发，在开发过程中认定出的类如下表所示：

编号	类名	描述
1	InternetClient	网络用户
2	CustomerList	客户信息表，记录公司所有客户的信息
3	Customer	客户信息，记录单个客户的信息
4	CompanyCustomer	公司客户
5	InternalClient	公司的管理人员

[图 3-1]



[图 3-2]



[问题 1]

在需求分析阶段，采用 UML 的用例图（use case diagram）描述系统功能需求，如图 3-1 所示。请指出图中的 A、B、C 和 D 分别是哪个用例？

[问题 2]

在 UML 中，重复度（Multiplicity）定义了某个类的一个实例可以与另一个类的多少个实例相关联。通常把它写成一个表示取值范围的表达式或者一个具体的值。例如图 3-2 中的类 **InternetClient** 和 **CustomerList**，**InternetClient** 端的“0..*”表示：一个 **CustomerList** 的实例可以与 0 个或多个 **InternetClient** 的实例相关联；**CustomerList** 端的“1”表示：一个 **InternetClient** 的实例只能与一个 **CustomerList** 的实例相关。

请指出图 3-2 中 (1) 到 (4) 处的重复度分别为多少？

[问题 3]

类通常不会单独存在，因此当对系统建模时，不仅要识别出类，还必须对类之间的相互关系建模。在面向对象建模中，提供了四种关系：依赖（dependency）、概括（generalization）、关联（association）和聚集（aggregation）。请分别说明这四种关系的含义，并说明关联和聚集之间的主要区别。

试题四

在 COMET 型计算机上可以使用试卷上所附的 CASL 汇编语言，阅读程序说明和 CASL 程序，把应填入____(n)____处的字句写在答卷的对应栏内。

[程序 4 说明]

本程序将 16 位无符号二进制数转换为 5 位十进制数，转换结果用 ASCII 码表示，并从高位至低位依次存放在首地址为 BTASC 的连续 5 个内存单元中。待转换的 16 位无符号二进制数存放在 DATA 内存单元中。

[程序 4]

	START	
PROGBC	LD	GR0, DATA
	LEA	GR1, 0
	LEA	GR3, 48
LOOP1	CPL	GR0, WDT, GR1
	JPZ	LOOP2
	ST	GR3, BTASC, GR1
	LEA	GR1, 1, GR1
	LEA	GR2, -4, GR1
	JNZ	LOOP1
	(1)	
LOOP2	LEA	GR2, 48
LOOP3	CPL	GR0, WDT, GR1
	JMI	NEXT
	(2)	
	LEA	GR2, 1, GR2
	JMP	LOOP3
NEXT	(3)	
	LEA	GR1, 1, GR1
	LEA	GR2, -4, GR1
	JNZ	LOOP2
LAST	(4)	
	(5)	
	EXIT	
C48	DC	48
WDT	DC	10000
	DC	1000
	DC	100
	DC	10
BTASC	DS	5
DATA	DC	#FA59H
	END	

从下列的 2 道试题（试题五至试题六）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五

阅读下列函数说明和 C 函数，将应填入 (n) 处的字句写在答题纸的对应栏内。

[函数 5 说明]

函数 DeleteNode(Bitree *r, int e)的功能是：在树根结点指针为 r 的二叉查找（排序）树上删除键值为 e 的结点，若删除成功，则函数返回 0，否则函数返回-1。二叉查找树结点的类型定义为：

```
typedef struct Tnode{
    int data;                                /*结点的键值*/
    struct Tnode *Lchild,*Rchild;          /*指向左、右子树的指针*/
}*Bitree;
```

在二叉查找树上删除一个结点时，要考虑三种情况：

①若待删除的结点 p 是叶子结点，则直接删除该结点；

②若待删除的结点 p 只有一个子结点，则将这个子结点与待删除结点的父结点直接连接，然后删除结点 p；

③若待删除的结点 p 有两个子结点，则在其左子树上，用中序遍历寻找关键值最大的结点 s，用结点 s 的值代替结点 p 的值，然后删除结点 s，结点 s 必属于上述①、②情况之一。

[函数 5]

```
int DeleteNode(Bitree *r, int e)  {
    Bitree p = *r, pp, s, c;
    while ( _____ (1) _____ ) { /*从树根结点出发查找键值为 e 的结点*/
        pp = p;
        if (e < p->data) p = p->Lchild;
        else p = p->Rchild;
    }
    if (!p) return -1;                /*查找失败*/
    if (p->Lchild && p->Rchild) { /*处理情况③*/
        s = _____ (2) _____; pp = p;
        while ( _____ (3) _____ ) { pp = s; s = s->Rchild; }
        p->data = s->data;      p = s;
    }
    /*处理情况①、②*/
    if ( _____ (4) _____ ) c = p->Lchild;
    else c = p->Rchild;
    if (p == *r) *r = c;
    else if ( _____ (5) _____ ) pp->Lchild = c;
        else pp->Rchild = c;
    free(p);
    return 0;
}
```

试题六

阅读下列说明和 C++ 程序，将应填入 (n) 处的字句写在答题纸的对应栏内。

[程序 6 说明]

C++ 语言本身不提供对数组下标越界的判断。为了解决这一问题，在程序 6 中定义了相应的类模板，使得对于任意类型的二维数组，可以在访问数组元素的同时，对行下标和列下标进行越界判断，并给出相应的提示信息。

[程序 6]

```
#include <iostream.h>
template <class T> class Array;
template <class T> class ArrayBody {
    friend _____ (1) _____;
    T* tpBody;
    int iRows, iColumns, iCurrentRow;
    ArrayBody(int iRsz, int iCsz) {
        tpBody = _____ (2) _____;
        iRows = iRsz; iColumns = iCsz; iCurrentRow = -1;
    }
public:
    T& operator[] (int j) {
        bool row_error, column_error;
        row_error = column_error = false;
        try {
            if (iCurrentRow < 0 || iCurrentRow >= iRows)
                row_error = true;
            if (j < 0 || j >= iColumns)
                column_error = true;
            if (row_error == true || column_error == true)
                _____ (3) _____;
        }
        catch(char) {
            if (row_error == true)
                cerr << "行下标越界[" << iCurrentRow << "]" << " ";
            if (column_error == true)
                cerr << "列下标越界[" << j << "]" << " ";
            cout << "\n";
        }
        return tpBody[iCurrentRow * iColumns + j];
    }
}
```

```

    ~ArrayBody() { delete[] tpBody; }
};

template <class T> class Array {
    ArrayBody<T> tBody;
public:
    ArrayBody<T> & operator[] (int i)    {
        _____ (4) _____;
        return tBody;
    }
    Array(int iRsz, int iCsz) : _____ (5) _____ { }
};

void main()
{
    Array<int> a1(10, 20);
    Array<double> a2(3, 5);
    int b1;
    double b2;
    b1 = a1[-5][10];    // 有越界提示：行下标越界[-5]
    b1 = a1[10][15];    // 有越界提示：行下标越界[10]
    b1 = a1[1][4];      // 没有越界提示
    b2 = a2[2][6];      // 有越界提示：列下标越界[6]
    b2 = a2[10][20];    // 有越界提示：行下标越界[10] 列下标越界[20]
    b2 = a2[1][4];      // 没有越界提示
}

```

全国计算机技术与软件专业技术资格（水平）考试

2004 年下半年 软件设计师级 上午试卷

（考试时间 9：00～11：30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

●2004 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（88） 月 （89） 日。

- | | | | |
|----------|------|-------|-------|
| （88） A.8 | B. 9 | C. 10 | D. 11 |
| （89） A.4 | B. 5 | C. 6 | D. 7 |

因为考试日期是“11 月 6 日”，故（88）选 D，（89）选 C，应在答题卡序号 88 下对 D 填涂，在序号 89 下对 C 填涂（参看答题卡）。

● 内存按字节编址，地址从A4000H到CBFFFH，共有 (1) 字节。若用存储容量为32K×8bit的存储器芯片构成该内存，至少需要 (2) 片。

- (1) A. 80K B. 96K C. 160K D. 192K
(2) A. 2 B. 5 C. 8 D. 10

● 中断响应时间是指 (3) 。

- (3) A. 从中断处理开始到中断处理结束所用的时间
B. 从发出中断请求到中断处理结束所用的时间
C. 从发出中断请求到进入中断处理所用的时间
D. 从中断处理结束到再次中断请求的时间

● 若指令流水线把一条指令分为取指、分析和执行三部分，且三部分的时间分别是 $t_{取指}=2ns$ ， $t_{分析}=2ns$ ， $t_{执行}=1ns$ ，则100条指令全部执行完毕需 (4) ns。

- (4) A. 163 B. 183 C. 193 D. 203

● 在单指令流多数据流计算机 (SIMD) 中，各处理单元必须 (5) 。

- (5) A. 以同步方式，在同一时间内执行不同的指令
B. 以同步方式，在同一时间内执行同一条指令
C. 以异步方式，在同一时间内执行不同的指令
D. 以异步方式，在同一时间内执行同一条指令

● 单个磁头在向盘片的磁性涂层上写入数据时，是以 (6) 方式写入的。

- (6) A. 并行 B. 并一串行 C. 串行 D. 串—并行

● 容量为64块的Cache采用组相联方式映像，字块大小为128个字，每4块为一组。若主存容量为4096块，且以字编址，那么主存地址应为 (7) 位，主存区号应为 (8) 位。

- (7) A. 16 B. 17 C. 18 D. 19
(8) A. 5 B. 6 C. 7 D. 8

● 软件开发中的瀑布模型典型地刻画了软件生存周期的阶段划分，与其最相适应的软件开发方法是 (9) 。

- (9) A. 构件化方法 B. 结构化方法
C. 面向对象方法 D. 快速原型方法

● 下述任务中，不属于软件工程需求分析阶段的是 (10) 。

- (10) A. 分析软件系统的数据要求 B. 确定软件系统的功能需求
C. 确定软件系统的性能要求 D. 确定软件系统的运行平台

● 软件设计的主要任务是设计软件的结构、过程和模块，其中软件结构设计的主要任务是要确定 (11) 。

- (11) A. 模块间的操作细节 B. 模块间的相似性

- C. 模块间的组成关系 D. 模块的具体功能

● 系统测试是将软件系统与硬件、外设和网络等其他因素结合，对整个软件系统进行测试。（12）不是系统测试的内容。

- （12） A. 路径测试 B. 可靠性测试 C. 安装测试 D. 安全测试

● 项目管理工具中，将网络方法用于工作计划安排的评审和检查的是（13）。

- （13） A. Gantt 图 B. PERT 网图 C. 因果分析图 D. 流程图

● 在结构化分析方法中，数据字典是重要的文档。对加工的描述是数据字典的组成内容之一，常用的加工描述方法（14）。

- （14） A. 只有结构化语言 B. 有结构化语言和判定树
C. 有结构化语言、判定树和判定表 D. 有判定树和判定表

● CMM 模型将软件过程的成熟度分为 5 个等级。在（15）使用定量分析来不断地改进和管理软件过程。

- （15） A. 优化级 B. 管理级 C. 定义级 D. 可重复级

● 在面向数据流的设计方法中，一般把数据流图中的数据流划分为（16）两种。

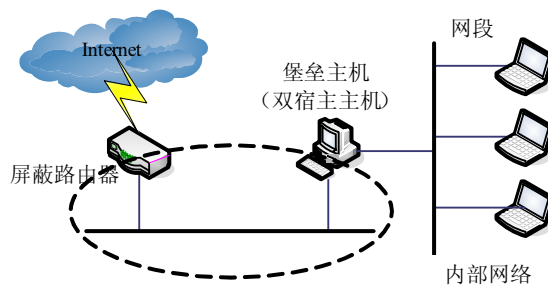
- （16） A. 数据流和事务流 B. 变换流和数据流
C. 变换流和事务流 D. 控制流和事务流

● （17）属于第三层VPN协议。

- （17） A. TCP B. IPsec C. PPOE D. SSL

● 下图所示的防火墙结构属于（18）。

- （18） A. 简单的双宿主主机结构 B. 单 DMZ 防火墙结构
C. 带有屏蔽路由器的单网段防火墙结构 D. 双 DMZ 防火墙结构



● 电子商务交易必须具备抗抵赖性，目的在于防止__ (19) __。

- (19) A. 一个实体假装成另一个实体
B. 参与此交易的一方否认曾经发生过此次交易
C. 他人对数据进行非授权的修改、破坏
D. 信息从被监视的通信过程中泄漏出去

● 知识产权一般都具有法定的保护期限，一旦保护期限届满，权利将自行终止，成为社会公众可以自由使用的知识。__ (20) __ 权受法律保护期限是不确定的，一旦为公众所知悉，即成为公众可以自由使用的知识。

- (20) A. 发明专利 B. 商标 C. 作品发表 D. 商业秘密

● 甲、乙两人在同一时间就同样的发明创造提交了专利申请，专利局将分别向各申请人通报有关情况，并提出多种解决这一问题的办法，不可能采用__ (21) __ 的办法。

- (21) A. 两申请人作为一件申请的共同申请人
B. 其中一方放弃权利并从另一方得到适当的补偿
C. 两件申请都不授予专利权
D. 两件申请都授予专利权

● 《计算机软件产品开发文件编制指南》(GB 8567—88) 是__ (22) __ 标准。

- (22) A. 强制性国家 B. 推荐性国家 C. 强制性行业 D. 推荐性行业

● 虚拟存储管理系统的基础是程序的__ (23) __ 理论，这个理论的基本含义是指程序执行时往往会不均匀地访问主存储器单元。根据这个理论，Denning 提出了工作集理论。工作集是进程运行时被频繁地访问的页面集合。在进程运行时，如果它的工作集页面都在__ (24) __ 内，能够使该进程有效地运行，否则会出现频繁的页面调入/调出现象。

- (23) A. 全局性 B. 局部性 C. 时间全局性 D. 空间全局性
(24) A. 主存储器 B. 虚拟存储器 C. 辅助存储器 D. u 盘

● 在 UNIX 操作系统中，若用户键入的命令参数的个数为 1 时，执行 `cat $1` 命令；若用户键入的命令参数的个数为 2 时，执行 `cat >> $2 < $1` 命令。请将下面所示的 Shell 程序的空缺部分补齐。

```
case __ (25) __ in
    1) cat $1 ;;
    2) cat >> $2 < $1 ;;
    *) echo 'default....' ;
esac
```

- (25) A. \$\$ B. \$@ C. \$# D. \$*

● 进程 P_A 不断地向管道写数据，进程 P_B 从管道中读数据并加工处理，如下图所示。如果采用PV操作来实现进程 P_A 和进程 P_B 间的管道通信，并且保证这两个进程并发执行的正确性，则至少需要 (26)。



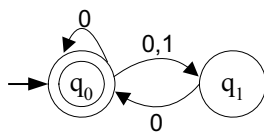
- (26) A. 1 个信号量，信号量的初值为 0
 B. 2 个信号量，信号量的初值分别为 0、1
 C. 3 个信号量，信号量的初值分别为 0、0、1
 D. 4 个信号量，信号量的初值分别为 0、0、1、1

● 假设系统中有三类互斥资源R1、R2和R3，可用资源数分别为9、8和5。在 T_0 时刻系统中有P1、P2、P3、P4和P5五个进程，这些进程对资源的最大需求量和已分配资源数如下表所示。如果进程按 (27) 序列执行，那么系统状态是安全的。

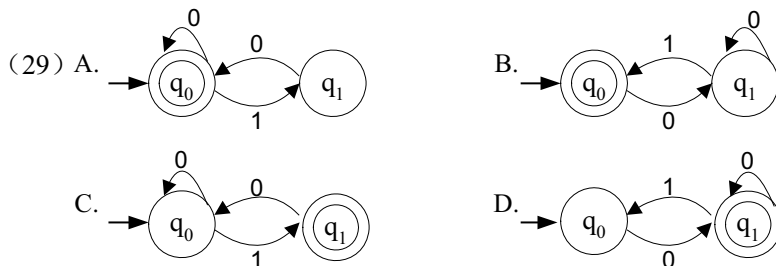
资源 进程	最大需求量			已分配资源数		
	R1	R2	R3	R1	R2	R3
P1	6	5	2	1	2	1
P2	2	2	1	2	1	1
P3	8	0	1	2	1	0
P4	1	2	1	1	2	0
P5	3	4	4	1	1	3

- (27) A. $P1 \rightarrow P2 \rightarrow P4 \rightarrow P5 \rightarrow P3$ B. $P2 \rightarrow P1 \rightarrow P4 \rightarrow P5 \rightarrow P3$
 C. $P2 \rightarrow P4 \rightarrow P5 \rightarrow P1 \rightarrow P3$ D. $P4 \rightarrow P2 \rightarrow P4 \rightarrow P1 \rightarrow P3$

● 某一非确定性有限自动机 (NFA) 的状态转换图如下图所示，与该NFA等价的正规式是 (28)，与该NFA等价的DFA是 (29)。



- (28) A. $0^*(0|1)0$ B. $(0|10)^*$ C. $0^*((0|1)0)^*$ D. $0^*(10)^*$



● 在UML提供的图中, 可以采用__(30)__对逻辑数据库模式建模; __(31)__用于接口、类和协作的行为建模, 并强调对象行为的事件顺序; __(32)__用于系统的功能建模, 并强调对象间的控制流。

- (30) A. 用例图 B. 构件图 C. 活动图 D. 类图
(31) A. 协作图 B. 状态图 C. 序列图 D. 对象图
(32) A. 状态图 B. 用例图 C. 活动图 D. 类图

● 在一棵完全二叉树中, 其根的序号为 1, __(33)__可判定序号为p和q的两个结点是否在同一层。

- (33) A. $\lfloor \log_2 p \rfloor = \lfloor \log_2 q \rfloor$ B. $\log_2 p = \log_2 q$
C. $\lfloor \log_2 p \rfloor + 1 = \lfloor \log_2 q \rfloor$ D. $\lfloor \log_2 p \rfloor = \lfloor \log_2 q \rfloor + 1$

● 堆是一种数据结构, __(34)__是堆。

- (34) A. (10,50,80,30,60,20,15,18) B. (10,18,15,20,50,80,30,60)
C. (10,15,18,50,80,30,60,20) D. (10,30,60,20,15,18,50,80)

● __(35)__从二叉树的任一结点出发到根的路径上, 所经过的结点序列必按其关键字降序排列。

- (35) A. 二叉排序树 B. 大顶堆 C. 小顶堆 D. 平衡二叉树

● 若广义表L=((1,2,3)), 则L的长度和深度分别为__(36)__。

- (36) A. 1 和 1 B. 1 和 2 C. 1 和 3 D. 2 和 2

● 若对 27 个元素只进行三趟多路归并排序, 则选取的归并路数为__(37)__。

- (37) A. 2 B. 3 C. 4 D. 5

● __(38)__是多媒体内容描述接口标准。

- (38) A. MPEG-1 B. MPEG-2 C. MPEG-4 D. MPEG-7

● 未经压缩的数字音频数据传输率的计算公式为__(39)__。

- (39) A. 采样频率(Hz)×量化位数(bit)×声道数×1/8
B. 采样频率(Hz)×量化位数(bit)×声道数
C. 采样频率(Hz)×量化位数(bit)×1/8
D. 采样频率(Hz)×量化位数(bit)×声道数×1/16

● 彩色打印机中所采用的颜色空间是__(40)__。

- (40) A. RGB 彩色空间 B. CMY 彩色空间 C. YUV 彩色空间 D. HSL 彩色空间

● MPEG视频中的时间冗余信息可以采用__(41)___的方法来进行压缩编码。

- (41) A. 帧间预测和变换编码 B. 霍夫曼编码和运动补偿
C. 变换编码和行程编码 D. 帧间预测和运动补偿

● 假定每一车次具有唯一的始发站和终点站。如果实体“列车时刻表”属性为车次、始发站、发车时间、终点站、到达时间，该实体的主键是__(42)___；如果实体“列车运行表”属性为车次、日期、发车时间、到达时间，该实体的主键是__(43)___。通常情况下，上述“列车时刻表”和“列车运行表”两实体间__(44)___联系。

- (42) A. 车次 B. 始发站 C. 发车时间 D. 车次，始发站
(43) A. 车次 B. 始发站 C. 发车时间 D. 车次，日期
(44) A. 不存在 B. 存在一对一 C. 存在一对多 D. 存在多对多

● 关系模式R (U,F)，其中U={W,X,Y,Z}，F={WX→Y,W→X,X→Z,Y→W}。关系模式R的候选键是__(45)___，__(46)___是无损连接并保持函数依赖的分解。

- (45) A. W 和 Y B. WY C. WX D. WZ
(46) A. $\rho = \{ R_1(WY), R_2(XZ) \}$ B. $\rho = \{ R_1(WZ), R_2(XY) \}$
C. $\rho = \{ R_1(WXY), R_2(XZ) \}$ D. $\rho = \{ R_1(WX), R_2(YZ) \}$

● 类的实例化过程是一种实例的合成过程，而不仅仅是根据单个类型进行的空间分配、初始化和绑定。指导编译程序进行这种合成的是__(47)___。

- (47) A. 类层次结构 B. 实例的个数
C. 多态的种类 D. 每个实例的初始状态

● 重置的基本思想是通过__(48)___机制的支持，使得子类在继承父类界面定义的前提下，用适合于自己要求的实现去置换父类中的相应实现。

- (48) A. 静态绑定 B. 对象引用 C. 类型匹配 D. 动态绑定

● OMT定义了三种模型来描述系统。__(49)___可以用状态图来表示；__(50)___可以用数据流图来表示。__(51)___为上述两种模型提供了基本的框架。

- (49) A. 对象模型 B. 功能模型 C. 动态模型 D. 类模型
(50) A. 对象模型 B. 功能模型 C. 动态模型 D. 类模型
(51) A. 对象模型 B. 功能模型 C. 动态模型 D. 类模型

● 采用动态规划策略求解问题的显著特征是满足最优性原理，其含义是__(52)___。

- (52) A. 当前所做出的决策不会影响后面的决策
B. 原问题的最优解包含其子问题的最优解
C. 问题可以找到最优解，但利用贪心法不能找到最优解
D. 每次决策必须是当前看来最优的决策才可以找到最优解

● 下面函数中渐进时间最小的是__(53)___。

- (53) A. $T_1(n)=n+n\log n$ B. $T_2(n)=2n+n\log n$
C. $T_3(n)=n^2-\log n$ D. $T_4(n)=n+100\log n$

● 下面的程序段违反了算法的__(54)___原则。

```
void sam()
{   int n=2;
    while (!odd(n))    n+=2;
    printf(n);
}
```

- (54) A. 有穷性 B. 确定性 C. 可行性 D. 健壮性

● 拉斯维加斯 (Las Vegas) 算法是一种常用的__(55)___算法。

- (55) A. 确定性 B. 近似 C. 概率 D. 加密

● 在分支-限界算法设计策略中, 通常采用__(56)___搜索问题的解空间。

- (56) A. 深度优先 B. 广度优先 C. 自底向上 D. 拓扑序列

● 在下列算法设计方法中, __(57)___在求解问题的过程中并不从整体最优上加以考虑, 而是做出在当前看来是最好的选择。利用该设计方法可以解决__(58)___问题。

- (57) A. 分治法 B. 贪心法 C. 动态规划方法 D. 回溯法
(58) A. 排序 B. 检索 C. 背包 D. 0/1 背包

● 以关键字比较为基础的排序算法在最坏情况下的计算时间下界为 $O(n\log n)$ 。下面的排序算法中, 最坏情况下计算时间可以达到 $O(n\log n)$ 的是__(59)___; 该算法采用的设计方法是__(60)___。

- (59) A. 归并排序 B. 插入排序 C. 选择排序 D. 冒泡排序
(60) A. 分治法 B. 贪心法 C. 动态规划方法 D. 回溯法

● 以太网 100BASE-TX标准规定的传输介质是__(61)___。

- (61) A. 3 类 UTP B. 5 类 UTP C. 单模光纤 D. 多模光纤

● 许多网络通信需要进行组播, 以下选项中不采用组播协议的应用是__(62)___。在IPv4中把__(63)___类地址作为组播地址。

- (62) A. VOD B. Netmeeting C. CSCW D. FTP
(63) A. A B. B C. D D. E

● 将双绞线制作成交叉线(一端按EIA/TIA 568A线序, 另一端按EIA/TIA 568B线序), 该双绞线连接的两个设备可为__(64)___。

- (64) A. 网卡与网卡 B. 网卡与交换机
C. 网卡与集线器 D. 交换机的以太口与下一级交换机的 UPLINK 口

● 以下不属于中间件技术的是__(65)___。

(65) A. Java RMI B. CORBA C. DCOM D. Java Applet

● Networks can be interconnected by different devices. In the physical layer, networks can be connected by __(66)___ or hubs, which just move the bits from one network to an identical network. One layer up we find bridges and switches, which operate at data link layer. They can accept __(67)___, examine the MAC address, and forward the frames to a different network while doing minor protocol translation in the process. In the network layer, we have routers that can connect two networks. If two networks have __(68)___ network layer, the router may be able to translate between the packet formats. In the transport layer we find transport gateway, which can interface between two transport connections. Finally, in the application layer, application gateways translate message __(69)___ . As an example, gateways between Internet e-mail and X.400 e-mail must __(70)___ the e-mail message and change various header fields.

(66) A. reapers B. relays C. connectors D. modems
(67) A. frames B. packets C. packages D. cells
(68) A. special B. dependent C. similar D. dissimilar
(69) A. syntax B. semantics C. language D. format
(70) A. analyze B. parse C. delete D. create

● The purpose of the requirements definition phase is to produce a clear, complete, consistent, and testable __(71)___ of the technical requirements for the software product.

During the requirements definition phase, the requirements definition team uses an iterative process to expand a broad statement of the system requirements into a complete and detailed specification of each function that the software must perform and each __(72)___ that it must meet. The starting point is usually a set of high-level requirements from the __(73)___ that describe the project or problem.

In either case, the requirements definition team formulates an overall concept for the system and then defines __(74)___ showing how the system will be operated, publishes the system and operations concept document, and conducts a system concept review(SCR).

Following the SCR, the team derives __(75)___ requirements for the system from the high level requirements and the system and operations concept. Using structured or object-oriented analysis, the team specifies the software functions and algorithms needed to satisfy each detailed requirement.

(71) A. function B. definition C. specification D. statement
(72) A. criterion B. standard C. model D. system
(73) A. producer B. customer C. programmer D. analyser
(74) A. rules B. principles C. scenarios D. scenes
(75) A. detailed B. outlined C. total D. complete

全国计算机技术与软件专业技术资格（水平）考试

2004 年下半年 软件设计师级 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

- 1.在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
- 2.在答题纸的指定位置填写准考证号、出生年月日和姓名。
- 3.答题纸上除填写上述内容外只能写解答。
- 4.本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。
每题 15 分，满分 75 分。

试题号	一～四	五～七
选择方法	必答题	选答 1 题

- 5.解答时字迹务必清楚，字迹不清时，将不评分。
- 6.仿照下面例题，将解答写在答题纸的对应栏内。

例题

2004 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1） 月 （2） 日。

因为正确的解答是“11 月 6 日”，故在答题纸的对应栏内写上“11”和
“6”（参看下表）。

例题	解答栏
（1）	11
（2）	6

试题一（15 分）

阅读下列说明和数据流图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

某图书管理系统的主要功能是图书管理和信息查询。对于初次借书的读者，系统自动生成读者号，并与读者基本信息（姓名、单位、地址等）一起写入读者文件。

系统的图书管理功能分为四个方面：购入新书、读者借书、读者还书以及图书注销。

1. 购入新书时需要为该书编制入库单。入库单内容包括图书分类目录号、书名、作者、价格、数量和购书日期，将这些信息写入图书目录文件并修改文件中的库存总量（表示到目前为止，购入此种图书的数量）。

2. 读者借书时需填写借书单。借书单内容包括读者号和所借图书分类目录号。系统首先检查该读者号是否有效，若无效，则拒绝借书；若有效，则进一步检查该读者已借图书是否超过最大限制数（假设每位读者能同时借阅的书不超过 5 本），若已达到最大限制数，则拒绝借书；否则允许借书，同时将图书分类目录号、读者号和借阅日期等信息写入借书文件中。

3. 读者还书时需填写还书单。系统根据读者号和图书分类目录号，从借书文件中读出与该图书相关的借阅记录，标明还书日期，再写回到借书文件中，若图书逾期，则处以相应的罚款。

4. 注销图书时，需填写注销单并修改图书目录文件中的库存总量。

系统的信息查询功能主要包括读者信息查询和图书信息查询。其中读者信息查询可得到读者的基本信息以及读者借阅图书的情况；图书信息查询可得到图书基本信息和图书的借出情况。

图书管理系统的顶层图如图 1-1 所示；图书管理系统的第 0 层 DFD 图如图 1-2 所示，其中，加工 2 的细化图如图 1-3 所示。

[数据流图 1-1]

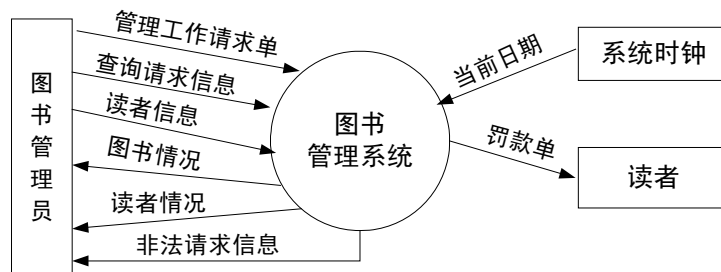


图1-1 图书管理系统顶层图

[数据流图 1-2]

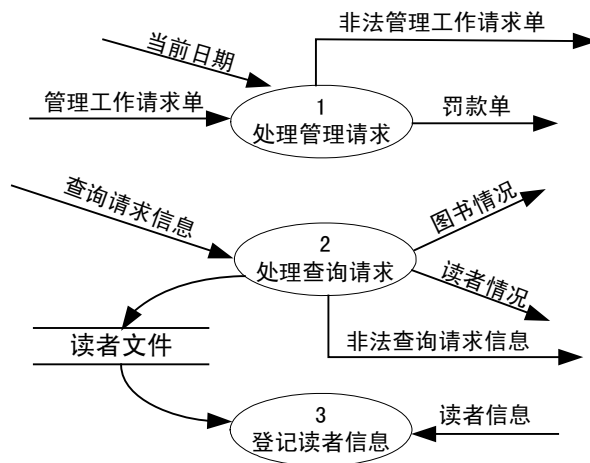


图1-2 图书管理系统第0层DFD图

[数据流图 1-3]

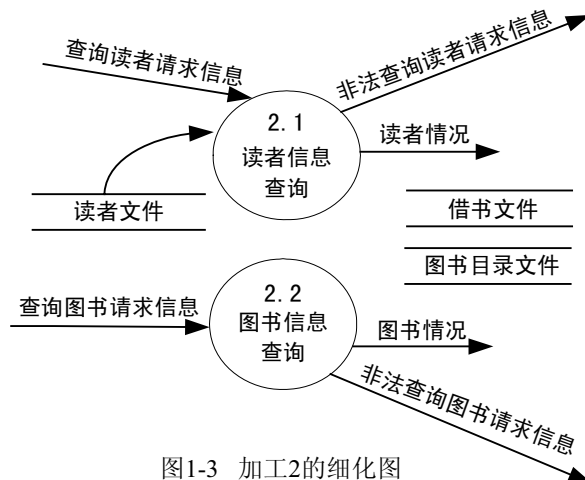


图1-3 加工2的细化图

[问题 1] (2 分)

数据流图 1-2 中有两条数据流是错误的, 请指出这两条数据流的起点和终点。

[问题 2] (6 分)

数据流图 1-3 中缺少三条数据流, 请指出这三条数据流的起点和终点。

[问题 3] (7 分)

根据系统功能和数据流图填充下列数据字典条目中的(1)和(2):

查询请求信息=[查询读者请求信息| 查询图书请求信息]

读者情况=读者号+姓名+所在单位+{借书情况}

管理工作请求单=_____(1)_____

入库单=_____(2)_____

试题二（15 分）

阅读下列说明和 E-R 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

某网上订书系统的 E-R 图(已消除了不必要的冗余)如图 2-1 所示（图中没有标出主码）。图中实体的说明如表 2-1 所示，相关属性说明如表 2-2 所示。

表 2-1

实体	说明
Books	书店内的书
Customers	与书店有业务的顾客
Orders	顾客向书店下的购书单

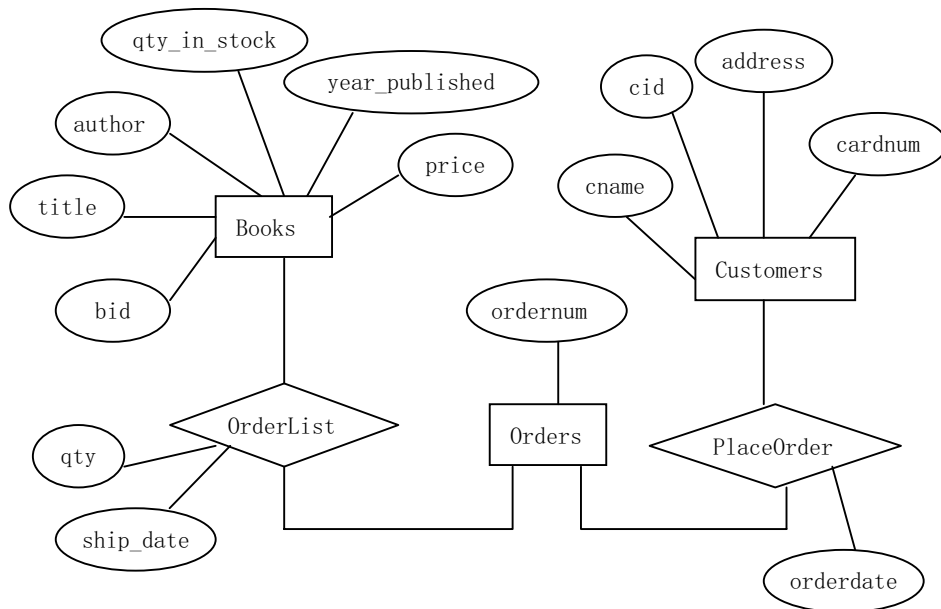
表 2-2

主要属性	说明
qty_in_stock	图书库存量
year_published	出版日期
ordernum	购书单编号
cid	顾客编号
bid	惟一标识每种图书的编码
cardnum	顾客信用卡号码
orderdate	填购书单日期
qty	订购某种图书的数量
ship_date	发货日期

一个顾客可以在同一天填写多张购书单，每张购书单上可填写多种图书，每种图书可以订购多本，bid 相同的图书在同一张购书单上不能出现多次。

注：为简化起见，不考虑信用卡号码泄漏所带来的安全性等问题。

[图 2-1]



[问题 1]（9 分）

根据 E-R 图中给出的词汇，按照“关系模式名（属性, 属性, …）”的格式，将此 E-R

图转换为 4 个关系模式，并指出每个关系模式中的主码和外码，其中模式名根据需要取实体名或联系名。

[问题 2] (2 分)

创建 Customers 表时，cid 使用 INTEGER 数据类型，cname 使用 CHAR(80) 数据类型，address 使用 CHAR(200) 数据类型，cardnum 使用 CHAR(16) 数据类型并且要求此列值惟一。请在下列用于创建表 Customers 的 SQL 语句空缺处填入正确的内容。

```
CREATE TABLE Customers(cid    INTEGER NOT NULL,
                        cname   CHAR(80) NOT NULL,
                        address  CHAR(200),
                        cardnum  CHAR(16) NOT NULL,
                        _____(1)_____,
                        _____(2)_____)
```

[问题 3] (4 分)

如下的 SQL 语句是书店用于查询“所有订购了 bid 为 ‘123-456’ 图书的用户订购其他图书的情况”的不完整语句，请在空缺处填入正确的内容。

```
Select bid
From Orderlist A
Where not exists ( Select * from Orders B
                  where  A.ordernum = B.ordernum and  B.cid _____(3)_____
                  (Select cid from Orderlist C, Orders D
                  where _____(4)_____.bid = '123-456'
                  and _____(5)_____ = D.ordernum))
```

试题三（15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

某指纹门禁系统的体系结构如图 3-1 所示，其主要部件有：主机（MainFrame）、锁控器（LockController）、指纹采集器（FingerReader）和电控锁（Lock）。

（1）系统中的每个电控锁都有一个惟一的编号。锁的状态有两种：“已锁住”和“未锁住”。

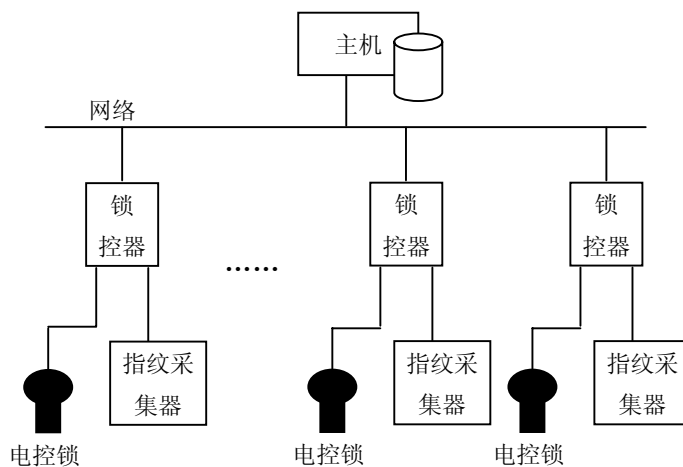
（2）在主机上可以设置每把锁的安全级别以及用户的开锁权限。只有当用户的开锁权限大于或等于锁的安全级别并且锁处于“已锁住”状态时，才能将锁打开。

（3）用户的指纹信息、开锁权限以及锁的安全级别都保存在主机上的数据库中。

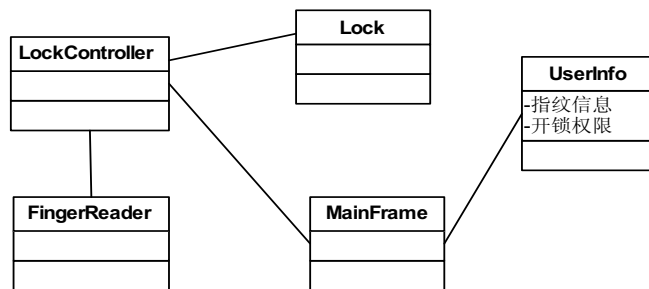
（4）用户开锁时，只需按一下指纹采集器。指纹采集器将发送一个中断事件给锁控器，锁控器从指纹采集器读取用户的指纹并将指纹信息发送到主机，主机根据数据库中存储的信息来判断用户是否具有开锁权限，若有且锁当前处于“已锁住”状态，则将锁打开；否则系统报警。

该系统采用面向对象方法开发，系统中的类以及类之间的关系用 UML 类图表示，图 3-2 是该系统类图的一部分；系统的动态行为采用 UML 序列图表示，图 3-3 是用户成功开锁的序列图。

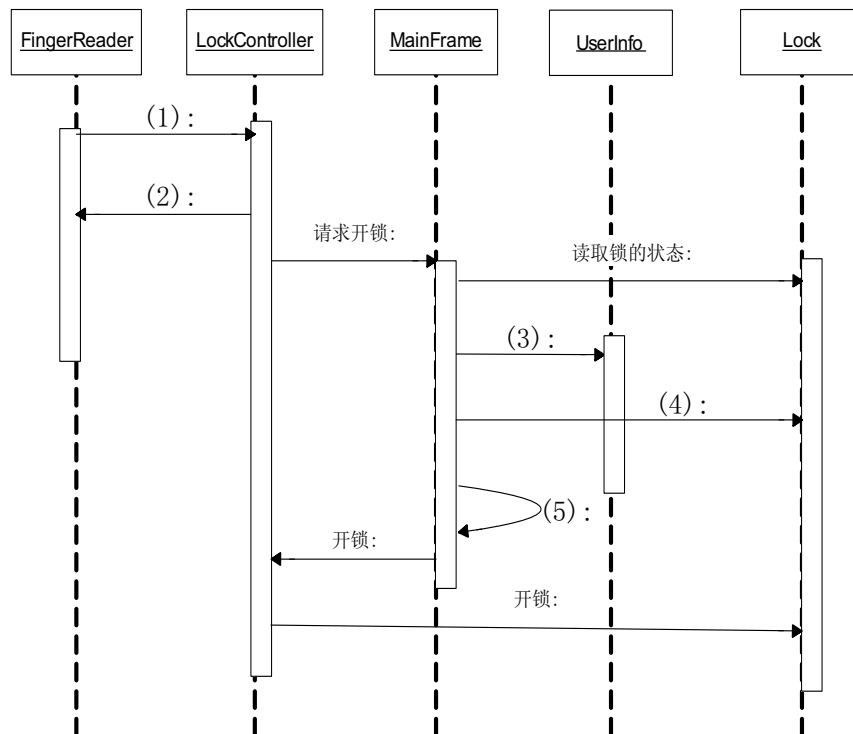
[图 3-1]



[图 3-2]



[图 3-3]



[问题 1] (6 分)

图 3-2 是该系统类图的一部分，依据上述说明中给出的术语，给出类 Lock 的主要属性。

[问题 2] (5 分)

依据上述说明中给出的词语，将图 3-3 中的(1)~(5)处补充完整。

[问题 3] (4 分)

组装 (Composition) 和聚集 (Aggregation) 是 UML 中两种非常重要的关系。请说明组装和聚集分别表示什么含义？两者的区别是什么？

试题四（15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

在并发系统设计中，通过对信号量 S 的 P、V 操作实现进程的同步与互斥控制。

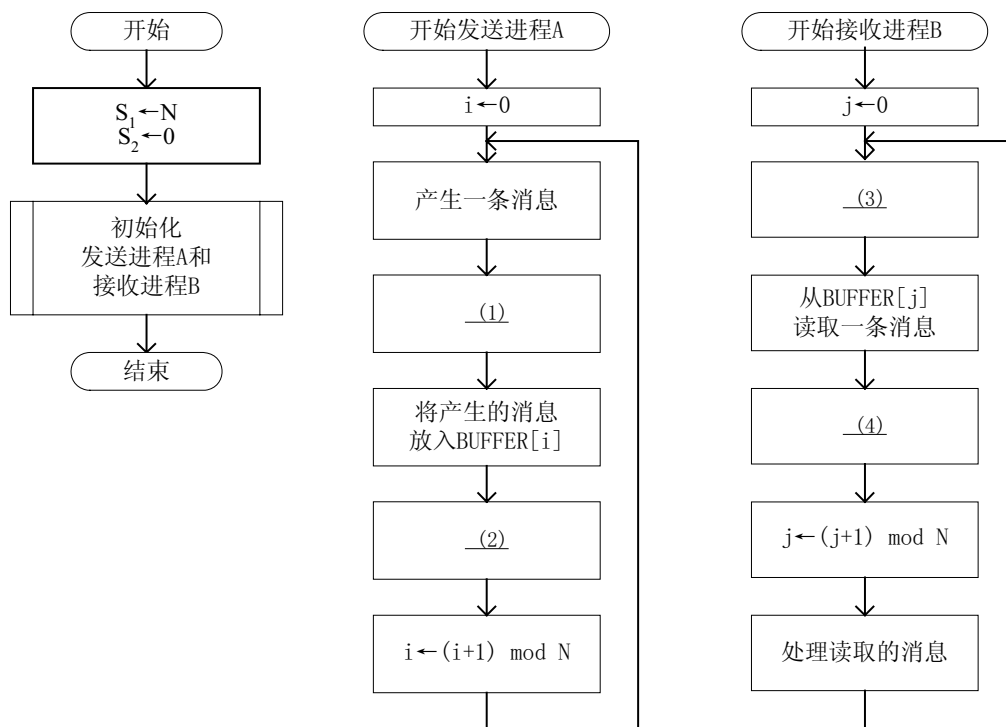
P(S): $S:=S-1$ ，若 $S \geq 0$ ，则执行 P 操作的进程继续执行；若 $S < 0$ ，则置该进程为阻塞状态，并将其插入阻塞队列。

V(S): $S:=S+1$ ，若 $S > 0$ ，则执行 V 操作的进程继续执行；若 $S \leq 0$ ，则从阻塞队列唤醒一个进程，并将其插入就绪队列，然后执行 V 操作的进程继续执行。

[问题 1]（4 分）

在某并发系统中，有一个发送进程 A、一个接收进程 B、一个环形缓冲区 BUFFER、信号量 S_1 和 S_2 。发送进程不断地产生消息并写入缓冲区 BUFFER，接收进程不断地从缓冲区 BUFFER 取消息。假设发送进程和接收进程可以并发地执行，那么，当缓冲区的容量为 N 时，如何使用 P、V 操作才能保证系统的正常工作。发送进程 A 和接收进程 B 的工作流程如图 4-1 所示。请在图 4-1 中的空（1）～（4）处填入正确的内容。

[图 4-1]



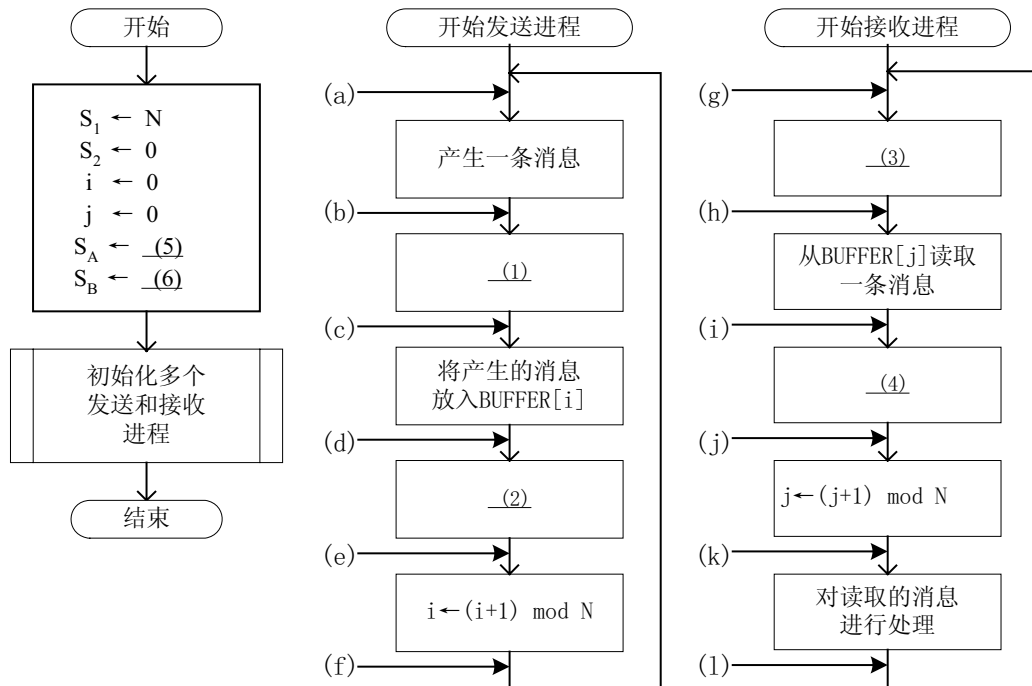
[问题 2]（5 分）

若系统中有多个发送进程和接收进程，进程间的工作流程如图 4-2 所示，其中空（1）～（4）的内容与图 4-1 相同。发送进程产生消息并顺序地写入环形缓冲区 BUFFER，接收者进程顺序地从 BUFFER 中取消息，且每条消息只能读取一次。为了保证进程间的正常通讯，增加了信号量 S_A 和 S_B 。

① 请说明信号量 S_A 和 S_B 的物理意义，并在图 4-2 中的空 (5) 和空 (6) 处填入正确的内容。

② 请从图 4-2 的 (a)~(l) 中选择四个位置正确地插入 $P(S_A)$ 、 $V(S_A)$ 、 $P(S_B)$ 、 $V(S_B)$ 。

[图 4-2]

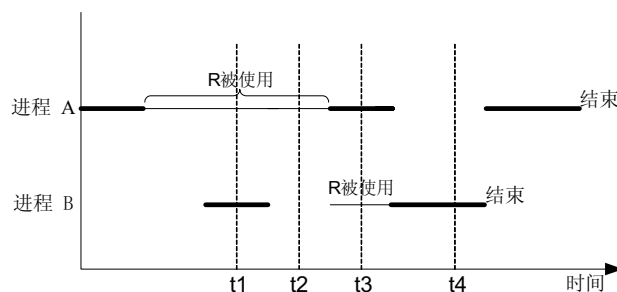


[问题 3] (6 分)

设系统中只有进程 A 和进程 B，除了互斥地使用 CPU 和打印机 R 外，进程 A 和 B 不使用其他资源。另外，进程 B 的优先级比 A 高，而进程 A 先于 B 准备好。进程 A 和 B 的执行情况如图 4-3 所示，其中粗实线表示进程在执行中，细实线表示打印机 R 在使用中。（每个进程具有三种状态：运行、就绪和阻塞）

请分别说明进程 A 和 B 在图 4-3 所示的 t_1 、 t_2 、 t_3 、 t_4 时刻所处的状态；若是阻塞状态，请说明阻塞原因。

[图 4-3]



从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五（15 分，每空 3 分）

阅读下列函数说明和 C 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

函数 `int Topological(LinkedWDigraph G)` 的功能是对图 G 中的顶点进行拓扑排序，并返回关键路径的长度。其中图 G 表示一个具有 n 个顶点的 AOE-网，图中顶点从 $1 \sim n$ 依次编号，图 G 的存储结构采用邻接表表示，其数据类型定义如下：

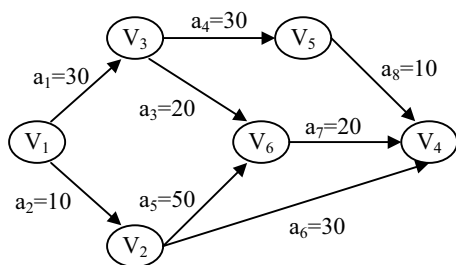
```
typedef struct Gnode{                /* 邻接表的表结点类型 */
    int adjvex;                      /* 邻接顶点编号 */
    int weight;                      /* 弧上的权值 */
    struct Gnode *nextarc;           /* 指示下一个弧的结点 */
}Gnode;

typedef struct Adjlist{              /* 邻接表的头结点类型 */
    char vdata;                     /* 顶点的数据信息 */
    struct Gnode *Firstadj;          /* 指向邻接表的第一个表结点 */
}Adjlist;

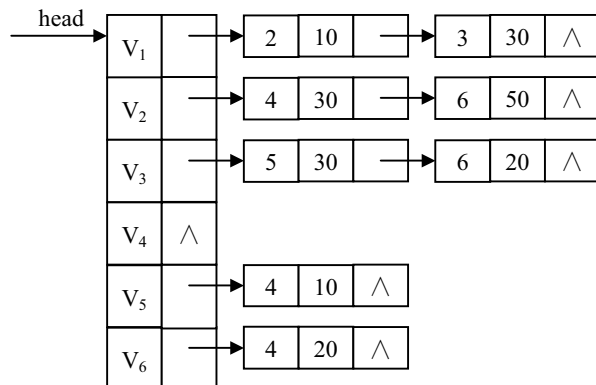
typedef struct LinkedWDigraph{       /* 图的类型*/
    int n, e;                       /* 图中顶点个数和边数 */
    struct Adjlist *head;            /* 指向图中第一个顶点的邻接表的头结点 */
}LinkedWDigraph;
```

例如，某 AOE-网如图 5-1 所示，其邻接表存储结构如图 5-2 所示。

[图 5-1]



[图 5-2]



[函数]

```
int Topological(LinkedWDigraph G)
{ Gnode *p;
  int j, w, top = 0;
  int *Stack, *ve, *indegree;
  ve = (int *)malloc((G.n+1) * sizeof(int));
  indegree = (int *)malloc((G.n+1)*sizeof(int)); /* 存储网中各顶点的入度 */
  Stack = (int *)malloc((G.n+1)*sizeof(int)); /* 存储入度为 0 的顶点的编号 */
  if (!ve || !indegree || !Stack)      exit(0);
  for (j = 1; j <= G.n; j++) {
    ve[j] = 0;    indegree[j] = 0;
  }/*for*/
  for (j = 1; j <= G.n; j++) {          /* 求网中各顶点的入度 */
    p = G.head[j].Firstadj;
    while (p) {
      (1);      p = p->nextarc;
    }/*while*/
  }/*for*/
  for (j = 1; j <= G.n; j++)          /*求网中入度为 0 的顶点并保存其编号*/
    if (!indegree[j])    Stack[++top] = j;
  while (top > 0) {
    w = (2);
    printf("%c ", G.head[w].vdata);
    p = G.head[w].Firstadj;
    while (p) {
      (3);
      if (!indegree[p->adjvex])
        Stack[++top] = p->adjvex;
      if((4))
        ve[p->adjvex] = ve[w] + p->weight;
      p = p->nextarc;
    }/* while */
  }/* while */
  return (5);
} /*Topological*/
```

试题六（15 分，每空 3 分）

阅读以下说明和C++代码，将应填入__(n)__处的字句写在答题纸的对应栏内。

[说明]

通常情况下，用户可以对应用系统进行配置，并将配置信息保存在配置文件中。应用系统在启动时首先将配置文件加载到内存中，这些内存配置信息应该有且仅有一份。下面的代码应用了单身模式(Singleton)以保证 Configure 类只能有一个实例。这样，Configure 类的使用者无法定义该类的多个实例，否则会产生编译错误。

[C++代码]

```
#include <iostream.h>
class Configure {
    (1) :
        Configure() {};           // 构造函数
public:
    static Configure* Instance();
public:
    int GetConfigureData() { return data; } // 获取配置信息
    int SetConfigureDate(int m_data)
    { data = m_data;    return data; }     // 设置配置信息
private:
    static Configure* _instance;
    int data;                             // 配置信息
};

(2) = NULL;

Configure * Configure::Instance () {
    if (_instance == NULL) {
        _instance = (3);
        // 加载配置文件并设置内存配置信息，此处省略
    }
    return (4);
}

void main() {
    Configure* t = NULL;
    t = (5);
    int d = t->GetConfigureData();
    // 获取配置信息后进行其它工作，此处省略
}
```

试题七（15 分，每空 3 分）

阅读以下说明和JAVA代码，将应填入__(n)__处的字句写在答题纸的对应栏内。

[说明]

类 Queue 表示队列，类中的方法如下表所示。

isEmpty()	判断队列是否为空。如果队列不为空，返回 true；否则，返回 false。
enqueue(Object newNode)	入队操作。
dequeue()	出队操作。如果队列为空，则抛出异常。

类 Node 表示队列中的元素；类 EmptyQueueException 给出了队列操作中的异常处理操作。

[JAVA 代码]

```
public class TestMain{                                // 主类
    public static void main(String args[]) {
        Queue q = new Queue();
        q.enqueue("first!");
        q.enqueue("second!");
        q.enqueue("third!");
        __(1)_{
            while(true)
                System.out.println(q.dequeue());
        }
        catch(__(2)_) { }
    }
}

public class Queue {                                // 队列
    Node m_FirstNode;
    public Queue() { m_FirstNode = null; }
    public boolean isEmpty(){
        if (m_FirstNode == null) return true;
        else return false;
    }
    public void enqueue(Object newNode){ // 入队操作
        Node next = m_FirstNode;
        if(next == null) m_FirstNode = new Node(newNode);
        else {
            while(next.getNext() != null) next = next.getNext();
            next.setNext(new Node(newNode));
        }
    }
}
```

```

    }
}

public Object dequeue() ____ (3) ____ { // 出队操作
    Object node;
    if (isEmpty())
        ____ (4) ____; // 队列为空, 抛出异常
    else {
        node = m_FirstNode.getObject();
        m_FirstNode = m_FirstNode.getNext();
        return node;
    }
}

}

public class Node { // 队列中的元素
    Object m_Data;
    Node m_Next;
    public Node(Object data) { m_Data = data; m_Next = null; }
    public Node(Object data, Node next) { m_Data = data; m_Next = next; }
    public void setObject(Object data) { m_Data = data; }
    public Object getObject() { return m_Data; }
    public void setNext(Node next) { m_Next = next; }
    public Node getNext() { return m_Next; }
}

public class EmptyQueueException extends ____ (5) ____ { // 异常处理类
    public EmptyQueueException() {
        System.out.println("队列已空!");
    }
}
}

```

全国计算机技术与软件专业技术资格（水平）考试

2005 年上半年 软件设计师 上午试卷

（考试时间 9:00~11:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2005 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
____（88）____月____（89）____日。

- | | | | |
|------------|-------|-------|-------|
| （88） A. 4 | B. 5 | C. 6 | D. 7 |
| （89） A. 27 | B. 28 | C. 29 | D. 30 |

因为考试日期是“5 月 29 日”，故（88）选 B，（89）选 C，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 C 填涂（参看答题卡）。

● 在计算机中，最适合进行数字加减运算的数字编码是 (1)，最适合表示浮点数阶码的数字编码是 (2)。

- (1) A. 原码 B. 反码 C. 补码 D. 移码
(2) A. 原码 B. 反码 C. 补码 D. 移码

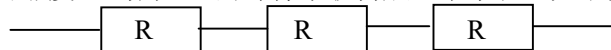
● 如果主存容量为 16M 字节，且按字节编址，表示该主存地址至少应需要 (3) 位。

- (3) A. 16 B. 20 C. 24 D. 32

● 操作数所处的位置，可以决定指令的寻址方式。操作数包含在指令中，寻址方式为 (4)；操作数在寄存器中，寻址方式为 (5)；操作数的地址在寄存器中，寻址方式为 (6)。

- (4) A. 立即寻址 B. 直接寻址
 C. 寄存器寻址 D. 寄存器间接寻址
(5) A. 立即寻址 B. 相对寻址
 C. 寄存器寻址 D. 寄存器间接寻址
(6) A. 相对寻址 B. 直接寻址
 C. 寄存器寻址 D. 寄存器间接寻址

● 三个可靠度 R 均为 0.8 的部件串联构成一个系统，如下图所示：



则该系统的可靠度为 (7)。

- (7) A. 0.240 B. 0.512 C. 0.800 D. 0.992

● 在计算机系统中，构成虚拟存储器 (8)。

- (8) A. 只需要一定的硬件资源便可实现 B. 只需要一定的软件即可实现
 C. 既需要软件也需要硬件方可实现 D. 既不需要软件也不需要硬件

● 某公司使用包过滤防火墙控制进出公司局域网的数据，在不考虑使用代理服务器的情况下，下面描述错误的是“该防火墙能够 (9)”。

- (9) A. 使公司员工只能访问 Internet 上与其有业务联系的公司的 IP 地址
 B. 仅允许 HTTP 协议通过
 C. 使员工不能直接访问 FTP 服务端口号为 21 的 FTP 服务
 D. 仅允许公司中具有某些特定 IP 地址的计算机可以访问外部网络

● 两个公司希望通过 Internet 进行安全通信，保证从信息源到目的地之间的数据传输以密文形式出现，而且公司不希望由于在传输节点使用特殊的安全单元而增加开支，最合适的加密方式是 (10)，使用的会话密钥算法应该是 (11)。

- (10) A. 链路加密 B. 节点加密 C. 端一端加密 D. 混合加密
(11) A. RSA B. RC-5 C. MD5 D. ECC

● 我国著作权法中，____(12)____系指同一概念。

- (12) A. 出版权与版权 B. 著作权与版权
C. 作者权与专有权 D. 发行权与版权

● 由我国信息产业部批准发布，在信息产业部门范围内统一使用的标准，称为____(13)____。

- (13) A. 地方标准 B. 部门标准 C. 行业标准 D. 企业标准

● 某软件设计师自行将他人使用 C 程序语言开发的控制程序转换为机器语言形式的控制程序，并固化在芯片中，该软件设计师的行为____(14)____。

- (14) A. 不构成侵权，因为新的控制程序与原控制程序使用的程序设计语言不同
B. 不构成侵权，因为对原控制程序进行了转换与固化，其使用和表现形式不同
C. 不构成侵权，将一种程序语言编写的源程序转换为另一种程序语言形式，属于一种“翻译”行为
D. 构成侵权，因为他不享有原软件作品的著作权

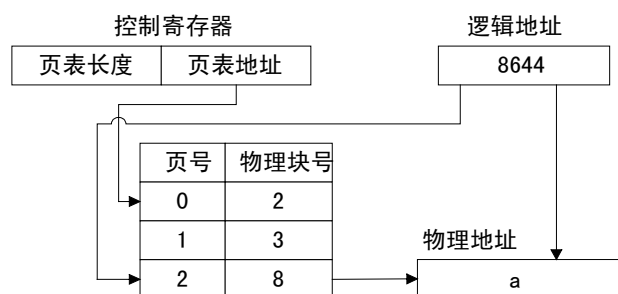
● 数据存储磁盘上的排列方式会影响 I/O 服务的总时间。假设每磁道划分成 10 个物理块，每块存放 1 个逻辑记录。逻辑记录 R1, R2, ..., R10 存放在同一个磁道上，记录的安排顺序如下表所示：

物理块	1	2	3	4	5	6	7	8	9	10
逻辑记录	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10

假定磁盘的旋转速度为 20ms/周，磁头当前处在 R1 的开始处。若系统顺序处理这些记录，使用单缓冲区，每个记录处理时间为 4ms，则处理这 10 个记录的最长时间为____(15)____；若对信息存储进行优化分布后，处理 10 个记录的最少时间为____(16)____。

- (15) A. 180ms B. 200ms C. 204ms D. 220ms
(16) A. 40ms B. 60ms C. 100ms D. 160ms

● 页式存储系统的逻辑地址是由页号和页内地址两部分组成。假定页面的大小为 4K，地址变换过程如下图所示，图中逻辑地址用十进制表示。



图中有效地址经过变换后，十进制物理地址 a 应为____(17)____。

- (17) A. 33220 B. 8644 C. 4548 D. 2500

● 下列叙述中，与提高软件可移植性相关的是 (18)。

- (18) A. 选择时间效率高的算法
B. 尽可能减少注释
C. 选择空间效率高的算法
D. 尽量用高级语言编写系统中对效率要求不高的部分

● 在系统转换的过程中，旧系统和新系统并行工作一段时间，再由新系统代替旧系统的策略称为 (19)；在新系统全部正式运行前，一部分一部分地代替旧系统的策略称为 (20)。

- (19) A. 直接转换 B. 位置转换 C. 分段转换 D. 并行转换
(20) A. 直接转换 B. 位置转换 C. 分段转换 D. 并行转换

● 下列要素中，不属于 DFD 的是 (21)。当使用 DFD 对一个工资系统进行建模时，(22) 可以被认定为外部实体。

- (21) A. 加工 B. 数据流 C. 数据存储 D. 联系
(22) A. 接收工资单的银行 B. 工资系统源代码程序
C. 工资单 D. 工资数据库的维护

● 在系统验收测试中，(23) 是在一个模拟的环境下使用模拟数据运行系统；(24) 是在一个实际环境中使用真实数据运行系统。

- (23) A. 验证测试 B. 审计测试 C. 确认测试 D. 模块测试
(24) A. 验证测试 B. 审计测试 C. 确认测试 D. 模块测试

● 采用瀑布模型进行系统开发的过程中，每个阶段都会产生不同的文档。以下关于产生这些文档的描述中，正确的是 (25)。

- (25) A. 外部设计评审报告在概要设计阶段产生
B. 集成测试计划在程序设计阶段产生
C. 系统计划和需求说明在详细设计阶段产生
D. 在进行编码的同时，独立的设计单元测试计划

● 在一个单 CPU 的计算机系统中，有两台外部设备 R1、R2 和三个进程 P1、P2、P3。系统采用可剥夺式优先级的进程调度方案，且所有进程可以并行使用 I/O 设备，三个进程的优先级、使用设备的先后顺序和占用设备时间如下表所示：

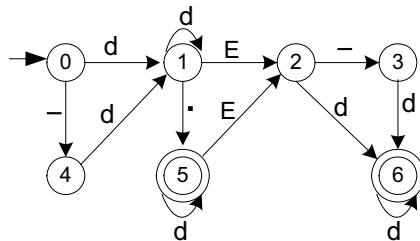
进程	优先级	使用设备的先后顺序和占用设备时间
P1	高	R2(30 ms)→CPU(10 ms)→R1(30 ms)→CPU(10 ms)
P2	中	R1(20 ms)→CPU(30 ms)→R2(40 ms)
P3	低	CPU(40 ms)→R1(10 ms)

假设操作系统的开销忽略不计，三个进程从投入运行到全部完成，CPU 的利用率约为__ (26) __%； R2 的利用率约为__ (27) __%（设备的利用率指该设备的使用时间与进程组全部完成所占用 时间的比率）。

- (26) A. 60 B. 67 C. 78 D. 90
 (27) A. 70 B. 78 C. 80 D. 89

● 某一确定性有限自动机（DFA）的状态转换图如下图所示，令 $d=0|1|2|\dots|9$ ，则以下字符串中，不能被该 DFA 接受的是__ (28) __，与该 DFA 等价的正规式是__ (29) __。（其中， ε 表示空字符）

- ① 3857 ② 1.2E+5 ③ -123. ④ .576E10



- (28) A. ①、②、③ B. ①、②、④ C. ②、③、④ D. ①、②、③、④
 (29) A. $(-d|d)d^* E(-d|d)d^* | (-d|d)d^*.d^*(\varepsilon | E(-d|d)d^*)$
 B. $(-d|d)dd^*(.| \varepsilon)d^*(\varepsilon | E(-d|d)d^*)$
 C. $(-d|d)dd^* E(-d|d)d^* | (-d|d)dd^*.d^*(\varepsilon | E(-d|d)d^*)$
 D. $(-d|d)dd^* E(-d|d)d^* | (-d|d)dd^*.d^*(\varepsilon | E(-dd^*|dd^*))$

● 对于以下编号为①、②、③的正规式，正确的说法是__ (30) __。

- ① $(aa^*|ab)^*b$ ② $(a|b)^*b$ ③ $((a|b)^*|aa)^*b$
 (30) A. 正规式①、②等价 B. 正规式①、③等价
 C. 正规式②、③等价 D. 正规式①、②、③互不等价

● 在 UML 提供的图中，__ (31) __用于描述系统与外部系统及用户之间的交互；__ (32) __用于按时间顺序描述对象间的交互。

- (31) A. 用例图 B. 类图 C. 对象图 D. 部署图
 (32) A. 网络图 B. 状态图 C. 协作图 D. 序列图

● 某数据库中有供应商关系 S 和零件关系 P，其中：供应商关系模式 S (Sno, Sname, SZip, City) 中的属性分别表示：供应商代码、供应商名、邮编、供应商所在城市；零件关系模式 P (Pno, Pname, Color, Weight, City) 中的属性分别表示：零件号、零件名、颜色、重量、产地。要求一个供应商可以供应多种零件，而一种零件可以由多个供应商供应。请将下面的 SQL 语句空缺部分补充完整。

```
CREATE TABLE SP (Sno CHAR(5),
                  Pno CHAR(6),
                  Status CHAR(8),
                  Qty NUMERIC(9),
                  (33) (Sno, Pno),
                  (34) (Sno),
                  (35) (Pno));
```

查询供应了“红”色零件的供应商号、零件号和数量 (Qty) 的元组演算表达式为：

$\{t \mid (\exists u)(\exists v)(\exists w)(\text{(36)} \wedge u[1] = v[1] \wedge v[2] = w[1] \wedge w[3] = \text{'红'} \wedge \text{(37)})\}$

- (33) A. FOREIGN KEY
 B. PRIMARY KEY
 C. FOREIGN KEY (Sno) REFERENCES S
 D. FOREIGN KEY (Pno) REFERENCES P
- (34) A. FOREIGN KEY
 B. PRIMARY KEY
 C. FOREIGN KEY (Sno) REFERENCES S
 D. FOREIGN KEY (Pno) REFERENCES P
- (35) A. FOREIGN KEY
 B. PRIMARY KEY
 C. FOREIGN KEY (Sno) REFERENCES S
 D. FOREIGN KEY (Pno) REFERENCES P

- (36) A. $S(u) \wedge SP(v) \wedge P(w)$ B. $SP(u) \wedge S(v) \wedge P(w)$
 C. $P(u) \wedge SP(v) \wedge S(w)$ D. $S(u) \wedge P(v) \wedge SP(w)$
- (37) A. $t[1] = u[1] \wedge t[2] = w[2] \wedge t[3] = v[4]$ B. $t[1] = v[1] \wedge t[2] = u[2] \wedge t[3] = u[4]$
 C. $t[1] = w[1] \wedge t[2] = u[2] \wedge t[3] = v[4]$ D. $t[1] = u[1] \wedge t[2] = v[2] \wedge t[3] = v[4]$

● 循环链表的主要优点是 (38)。

- (38) A. 不再需要头指针了
 B. 已知某个结点的位置后，能很容易找到它的直接前驱结点
 C. 在进行删除操作后，能保证链表不断开
 D. 从表中任一结点出发都能遍历整个链表

● 表达式 $a*(b+c)-d$ 的后缀表达形式为 (39)。

- (39) A. $abcd*+-$ B. $abc+*d-$ C. $abc*+d-$ D. $-+*abcd$

● 若二叉树的先序遍历序列为 ABDECF，中序遍历序列 DBEAF C，则其后序遍历序列为 (40)。

- (40) A. DEBAFC B. DEFBCA C. DEBCFA D. DEBFCA

● 无向图中一个顶点的度是指图中 (41)。

- (41) A. 通过该顶点的简单路径数 B. 通过该顶点的回路数
C. 与该顶点相邻接的顶点数 D. 与该顶点连通的顶点数

● 利用逐点插入法建立序列 (50, 72, 43, 85, 75, 20, 35, 45, 65, 30) 对应的二叉排序树以后，查找元素 30 要进行 (42) 次元素间的比较。

- (42) A. 4 B. 5 C. 6 D. 7

● 已知 3 个类 O、P 和 Q，类 O 中定义了一个私有方法 F1 和一个公有方法 F2；类 P 中定义了一个公有方法 F3，类 P 为类 O 的派生类；类 Q 为类 P 的派生类，它们的继承方式如下所示：

```
class P : public O {…};
```

```
class Q : private P {…};
```

在关于类 P 的描述中正确的是 (43)；在关于类 Q 的描述中正确的是 (44)。

- (43) A. 类 P 的对象可以访问 F1，但不能访问 F2
B. 类 P 的对象可以访问 F2，但不能访问 F1
C. 类 P 的对象既可以访问 F1，也可以访问 F2
D. 类 P 的对象既不能访问 F1，也不能访问 F2
- (44) A. 类 Q 的对象可以访问 F1、F2 和 F3
B. 类 Q 的对象可以访问 F2 和 F3，但不能访问 F1
C. 类 Q 的成员可以访问 F2 和 F3，但不能访问 F1
D. 类 Q 的成员不能访问 F1、F2 和 F3

● 在关于类的实例化的描述中，正确的是 (45)。

- (45) A. 同一个类的对象具有不同的静态数据成员值
B. 不同的类的对象具有相同的静态数据成员值
C. 同一个类的对象具有不同的对象自身引用 (this) 值
D. 不同的类的对象具有相同的对象自身引用 (this) 值

● 在某信息系统中，存在如下的业务陈述：①一个客户提交 0 个或多个订单；②一个订单由一个且仅由一个客户提交。系统中存在两个类：“客户”类和“订单”类。对应每个“订单”类的实例，存在(46)“客户”类的实例；对应每个“客户”类的实例，存在(47)个“订单”类的实例。

- (46) A. 0 个 B. 1 个 C. 1 个或多个 D. 0 个或多个
(47) A. 0 个 B. 1 个 C. 1 个或多个 D. 0 个或多个

● 在常用的描述二叉排序树的存储结构中，关键字值最大的结点(48)。

- (48) A. 左指针一定为空 B. 右指针一定为空
C. 左右指针均为空 D. 左右指针均不为空

● 一个具有 n ($n > 0$) 个顶点的连通无向图至少有(49)条边。

- (49) A. $n+1$ B. n C. $\frac{n}{2}$ D. $n-1$

● 由权值为 9, 2, 5, 7 的四个叶子结点构造一棵哈夫曼树，该树的带权路径长度为(50)。

- (50) A. 23 B. 37 C. 44 D. 46

● 在最好和最坏情况下的时间复杂度均为 $O(n \log n)$ 且稳定的排序方法是(51)。

- (51) A. 基数排序 B. 快速排序 C. 堆排序 D. 归并排序

● 已知一个线性表 (38, 25, 74, 63, 52, 48)，假定采用散列函数 $h(\text{key}) = \text{key} \% 7$ 计算散列地址，并散列存储在散列表 $A[0..6]$ 中，若采用线性探测方法解决冲突，则在该散列表上进行等概率成功查找的平均查找长度为(52)。

- (52) A. 1.5 B. 1.7 C. 2.0 D. 2.3

● 为在状态空间树中(53)，可以利用 LC-检索 (Least Cost Search) 快速找到一个答案结点。在进行 LC-检索时，为避免算法过分偏向于作纵深检查，应该(54)。

- (53) A. 找出任一个答案结点 B. 找出所有的答案结点
C. 找出最优的答案结点 D. 进行遍历
(54) A. 使用精确的成本函数 $c(.)$ 来作 LC-检索
B. 使用广度优先检索
C. 使用深度优先检索
D. 在成本估计函数 $\hat{c}(.)$ 中考虑根结点到当前结点的成本 (距离)

● 以比较为基础的排序算法在最坏情况下的计算时间下界为 (55)。

- (55) A. $O(n)$ B. $O(n^2)$ C. $O(\log n)$ D. $O(n \log n)$

● 利用动态规划方法求解每对结点之间的最短路径问题 (all pairs shortest path problem) 时, 设有向图 $G = \langle V, E \rangle$ 共有 n 个结点, 结点编号 $1 \sim n$, 设 C 是 G 的成本邻接矩阵, 用 $D^k(i, j)$ 表示从 i 到 j 并且不经过编号比 k 还大的结点的最短路径的长度 ($D^n(i, j)$ 即为图 G 中结点 i 到 j 的最短路径长度), 则求解该问题的递推关系式为 (56)。

- (56) A. $D^k(i, j) = D^{k-1}(i, j) + C(i, j)$
B. $D^k(i, j) = \min\{D^{k-1}(i, j), D^{k-1}(i, j) + C(i, j)\}$
C. $D^k(i, j) = D^{k-1}(i, k) + D^{k-1}(k, j)$
D. $D^k(i, j) = \min\{D^{k-1}(i, j), D^{k-1}(i, k) + D^{k-1}(k, j)\}$

● PC 机处理人耳能听得到的音频信号, 其频率范围是 (57)。

- (57) A. 80~3400Hz B. 300~3400Hz C. 20~20kHz D. 20~44.1kHz

● 电视系统采用的颜色空间中, 其亮度信号和色度信号是相分离的。下列颜色空间中, (58) 颜色空间不属于电视系统的颜色空间。

- (58) A. YUV B. YIQ C. YCbCr D. HSL

● 双层双面只读 DVD 盘片的存储容量可以达到 (59)。

- (59) A. 4.7GB B. 8.5GB C. 17GB D. 6.6GB

● 静态图像压缩标准 JPEG2000 中使用的是 (60) 算法。

- (60) A. K-L 变换 B. 离散正弦变换 C. 离散余弦变换 D. 离散小波变换

● 一个局域网中某台主机的 IP 地址为 176.68.160.12, 使用 22 位作为网络地址, 那么该局域网的子网掩码为 (61), 最多可以连接的主机数为 (62)。

- (61) A. 255.255.255.0 B. 255.255.248.0
C. 255.255.252.0 D. 255.255.0.0
(62) A. 254 B. 512 C. 1022 D. 1024

● 以下选项中, 可以用于 Internet 信息服务器远程管理的是 (63)。

- (63) A. Telnet B. RAS C. FTP D. SMTP

● 在 TCP/IP 网络中, 为各种公共服务保留的端口号范围是 (64)。

- (64) A. 1~255 B. 1~1023 C. 1~1024 D. 1~65535

● 在以下网络应用中, 要求带宽最高的应用是 (65)。

- (65) A. 可视电话 B. 数字电视 C. 拨号上网 D. 收发邮件

● DOM is a platform- and language- (66) API that allows programs and scripts to dynamically access and update the content, structure and style of WWW documents (currently, definitions for HTML and XML documents are part of the specification). The document can be further processed and the results of that processing can be incorporated back into the presented (67). DOM is a (68)-based API to documents, which requires the whole document to be represented in (69) while processing it. A simpler alternative to DOM is the event-based SAX, which can be used to process very large (70) documents that do not fit into the memory available for processing.

- | | | | |
|------------------|--------------|--------------|------------|
| (66) A. specific | B. neutral | C. contained | D. related |
| (67) A. text | B. image | C. page | D. graphic |
| (68) A. table | B. tree | C. control | D. event |
| (69) A. document | B. processor | C. disc | D. memory |
| (70) A. XML | B. HTML | C. script | D. Web |

● Melissa and LoveLetter made use of the trust that exists between friends or colleagues. Imagine receiving an (71) from a friend who asks you to open it. This is what happens with Melissa and several other similar email (72). Upon running, such worms usually proceed to send themselves out to email addresses from the victim's address book, previous emails, web pages (73).

As administrators seek to block dangerous email attachments through the recognition of well-known (74), virus writers use other extensions to circumvent such protection. Executable (.exe) files are renamed to .bat and .cmd plus a whole list of other extensions and will still run and successfully infect target users.

Frequently, hackers try to penetrate networks by sending an attachment that looks like a flash movie, which, while displaying some cute animation, simultaneously runs commands in the background to steal your passwords and give the (75) access to your network.

- | | | | |
|--------------------|------------|-------------|---------------|
| (71) A. attachment | B. packet | C. datagram | D. message |
| (72) A. virtual | B. virus | C. worms | D. bacteria |
| (73) A. memory | B. caches | C. ports | D. registers |
| (74) A. names | B. cookies | C. software | D. extensions |
| (75) A. cracker | B. user | C. customer | D. client |

全国计算机技术与软件专业技术资格（水平）考试

2005 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2005 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1） 月 （2） 日。

因为正确的解答是“5 月 29 日”，故在答题纸的对应栏内写上“5”和“29”
（参看下表）。

例题	解答栏
（1）	5
（2）	29

试题一(15 分)

阅读以下说明和数据流图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

学生住宿服务系统帮助学生在就学的城市内找到所需的住房，系统对出租的房屋信息、房主信息、需要租房的学生信息以及学生和房主的会面信息进行管理和维护。

房主信息包括姓名、地址、电话号码以及系统分配的唯一身份标识（ID）和密码；房屋信息包括房屋地址、类型（单间/套间）、适合住宿的人数、房租、房主的 ID 以及现在是否可以出租（例如由于装修原因，需等到装修后才可出租或者房屋已被租出）。每当房屋信息发生变化时，房主须通知系统，系统将更新房屋文件以便学生能够获得准确的可租用房屋信息。房主向系统中加入可租用的房屋信息时，需交纳一定的费用，由系统自动给出费用信息。房主可随时更新房屋的各种属性。

学生可通过系统查询现有的可租用的房屋，但必须先系统中注册。学生信息包括姓名、现住址、电话号码、出生日期、性别以及系统分配的唯一身份标识（ID）和密码。若学生希望租用某房屋，则需要发出租房请求，请求中包含房屋的详细信息，系统将安排学生与房主会面的时间和地点，并将会面信息通知学生和房主，会面信息包括会面时间、地点以及会面双方的基本信息，系统将记录会面信息。

学生住宿服务系统的顶层图如图 1-1 所示；学生住宿服务系统的第 0 层 DFD 图如图 1-2 所示，其中，加工 3 的细化图如图 1-3 所示。

[问题 1]（6 分）

（1）数据流图 1-1 缺少了一条数据流（在图 1-2 中也未给出该数据流），请给出此数据流的起点和终点，并采用说明中的词汇给出此数据流名。

（2）数据流图 1-2 中缺少了与“查询房屋”加工相关的数据流，请指出此数据流的起点和终点。

[问题 2]（4 分）

“安排会面”加工除需要写入会面文件外，还需要访问哪些文件？

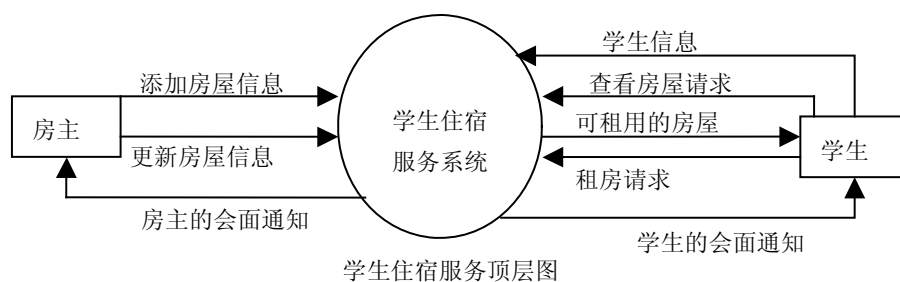
[问题 3]（5 分）

请补齐下列数据字典条目：

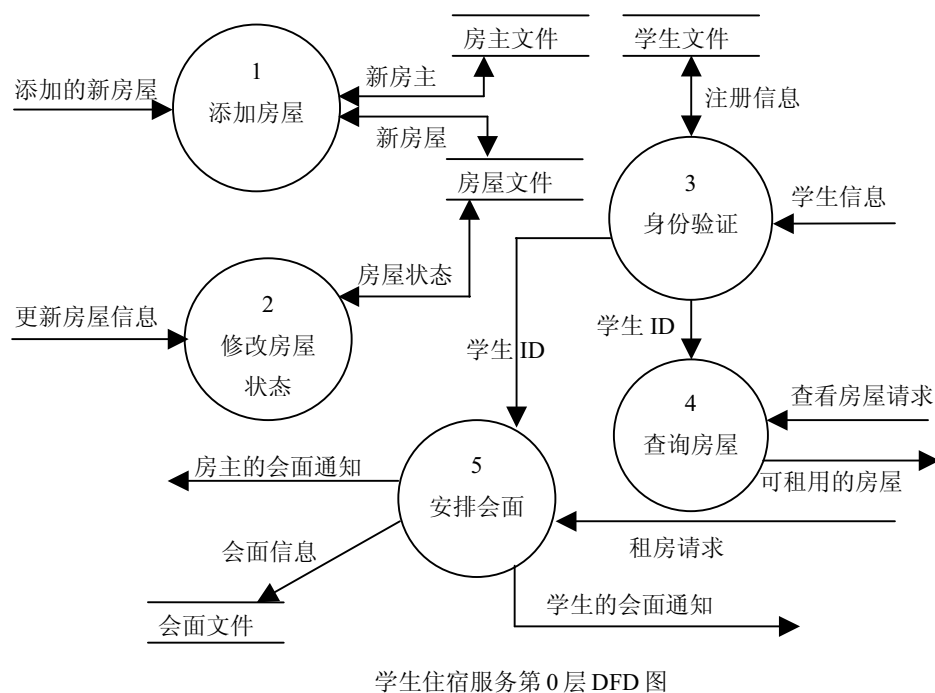
登录信息 = 学生 ID + 密码

注册信息 = _____

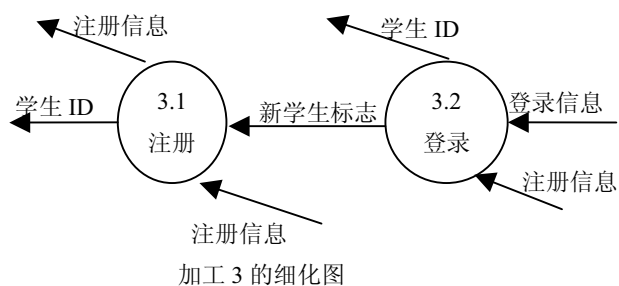
[数据流图 1-1]



[数据流图 1-2]



[数据流图 1-3]



试题二(15 分)

阅读以下说明和表，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

[说明]

某公司信息管理系统的需求分析和部分关系模式设计的结果描述如下：

1. 公司有多个部门，每个部门有一名负责人、一间办公室、一部电话、多名职员，每个职员最多属于一个部门，负责人也是一名公司职员。
2. 公司职员的月工资大于等于 1000 元且小于等于 8000 元。
3. 数据库的部分关系模式设计如下：
职员（职员号，职员姓名，月工资，部门号，办公室，电话）
部门（部门号，部门名，负责人代码，任职时间）
4. “职员”和“部门”的关系示例分别如表 2-1 和表 2-2 所示。

[表 2-1]

“职员”关系

职员号	职员姓名	月工资	部门号	办公室	电话
60801	汪俊华	1000	1	A 座 201	6883122
60802	杨晓军	3200	1	A 座 201	6883122
60803	王晓华	4300	2	B 座 202	6883123
60804	邢彦军	2800	2	B 座 202	6883123
60805	吕靖原	5300	3	A 座 301	6883124
60806	芦文峰	3200	3	A 座 301	6883124
60807	牟雪松	2800	3	A 座 301	6883124
60808	高亚南	1200	4	B 座 302	6883125
60810	周 黎	3200	4	B 座 302	6883125
60820	姚应磊	1200	4	B 座 302	6883125
60821	程文驰	3200	5	B 座 303	6883126
60836	许俊坤	0	Null	...	...

[表 2-2]

“部门”关系

部门号	部门名	负责人代码	任职时间
1	财务部	60802	2001-8-5
2	市场部	60803	2002-6-3
3	研发部	60805	2002-6-3
4	生产部 1	60810	2003-8-1
5	生产部 2	60821	2004-6-3

[问题 1] (4 分)

根据上述说明, 请给出

- (1) “职员”关系模式的主键和外键。
- (2) “部门”关系模式的主键和外键。

[问题 2] (4 分)

- (1) 用 SQL 定义“职员”关系模式, 请在空缺处填入正确的内容。

```
Create Table 职员( 职员号 CHAR(5) _____ (a) _____,
                  职员姓名 CHAR(8),
                  月工资  NUMBER(4),
                  部门号  CHAR(1),
                  办公室  CHAR(20),
                  电话    CHAR(8),
                  _____ (b) _____ (部门号),
                  CHECK (月工资 >=1000 AND 月工资<=8000 ));
```

- (2) 针对人数大于等于 2 的部门创建视图 D_View(Dept, D_num, D_Totals, D_AvgPay), 其中, Dept 为部门号, D_num 为部门人数, D_Totals 为工资总数, D_AvgPay 为平均工资, 请在空缺处填入正确的内容。

```
Create View D_View(Dept, D_num, D_Totals, D_AvgPay) As
( Select 部门号, _____ (c) _____
  from 职员
  _____ (d) _____ count(*)>=2 WHERE 部门号 IS NOT NULL);
```

[问题 3] (3 分)

对于表 2-1、表 2-2 所示的“职员”和“部门”关系, 请指出下列各行是否可以插入“职员”关系, 为什么?

(1)	60811	芦 峰	800	1	A 座 201	6883122
(2)	60802	李晓啸	3500	2	B 座 202	6883123
(3)	60812	高亚南	2600			

[问题 4] (4 分)

原来的“职员”关系模式存在什么问题? 在不增加新关系模式的前提下, 请给出修改后的“职员”和“部门”关系模式。

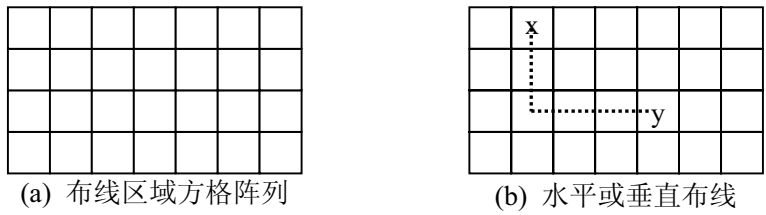
试题三（15 分）

阅读以下说明和流程图，从供选择的答案中选出应填入流程图 (n) 处的字句写在答题纸的对应栏内。

【说明】

一个印刷电路板的布线区域可分成 $n \times m$ 个方格，如图 3-1(a) 所示，现在需要确定电路板中给定的两个方格的中心点之间的最短布线方案。电路只能沿水平或垂直方向布线，如图 3-1(b) 中虚线所示。为了避免线路相交，应将已布过线的方格作封锁标记，其他线路不允许穿过被封锁的方格。

【图 3-1】



设给定印刷电路板的起始方格 x 与目的方格 y 尚未布线，求这两个方格间最短布线方案的基本思路是：从起始方格 x 开始，先考查距离起始方格距离为 1 的可达方格并用一个路径长度值标记，然后依次考查距离为 2、3、... 的可达方格，直到距离为 k 的某一个可达方格就是目标方格 y 时为止，或者由于不存在从 x 到 y 的布线方案而终止。布线区域中的每一个方格与其相邻的上、下、左、右四个方格之间的距离为 1，依次沿下、右、上、左这四个方向考查，并用一个队列记录可达方格的位置。表 3-1 给出了沿这四个方向前进 1 步时相对于当前方格的相对偏移量。

【表 3-1】

搜索顺序 i	方向	行偏移量	列偏移量
0	下	1	0
1	右	0	1
2	上	-1	0
3	左	0	-1

例如，设印刷电路板的布线区域可划分为一个 6×8 的方格阵列，如图 3-2(a) 所示，其中阴影表示已封锁方格。从起始方格 x (位置 [3, 2]，标记为 0) 出发，按照下、右、上、左的方向依次考查，所标记的可达方格如图 3-2(a) 所示，目标方格为 y (位置 [4, 7]，标记为 10)，相应的最短布线路径如图 3-2(b) 虚线所示。

【图 3-2】

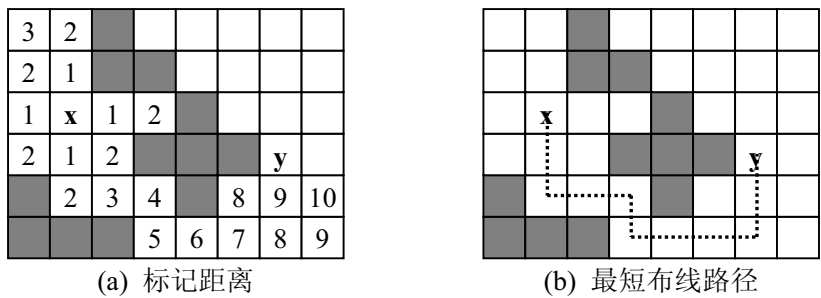
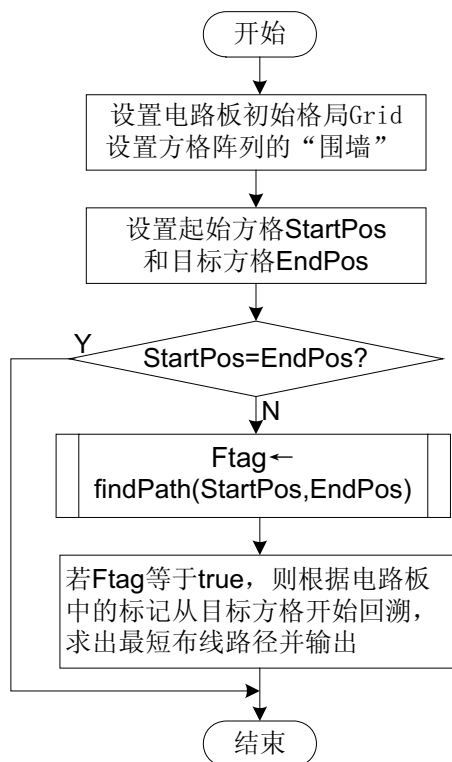


图 3-3 和图 3-4 所示的流程图即利用上述思路，在电路板方格阵列中进行标记，图中使用的主要符号如表 3-2 所示。在图 3-4 中，设置电路板初始格局即将**可布线方格置为数值-1、已布线方格（即封锁方格）置为-9**。设置方格阵列“围墙”的目的是省略方格位置的边界条件判定，方法是在四周附加方格，并将其标记为-9（与封锁标记相同）。

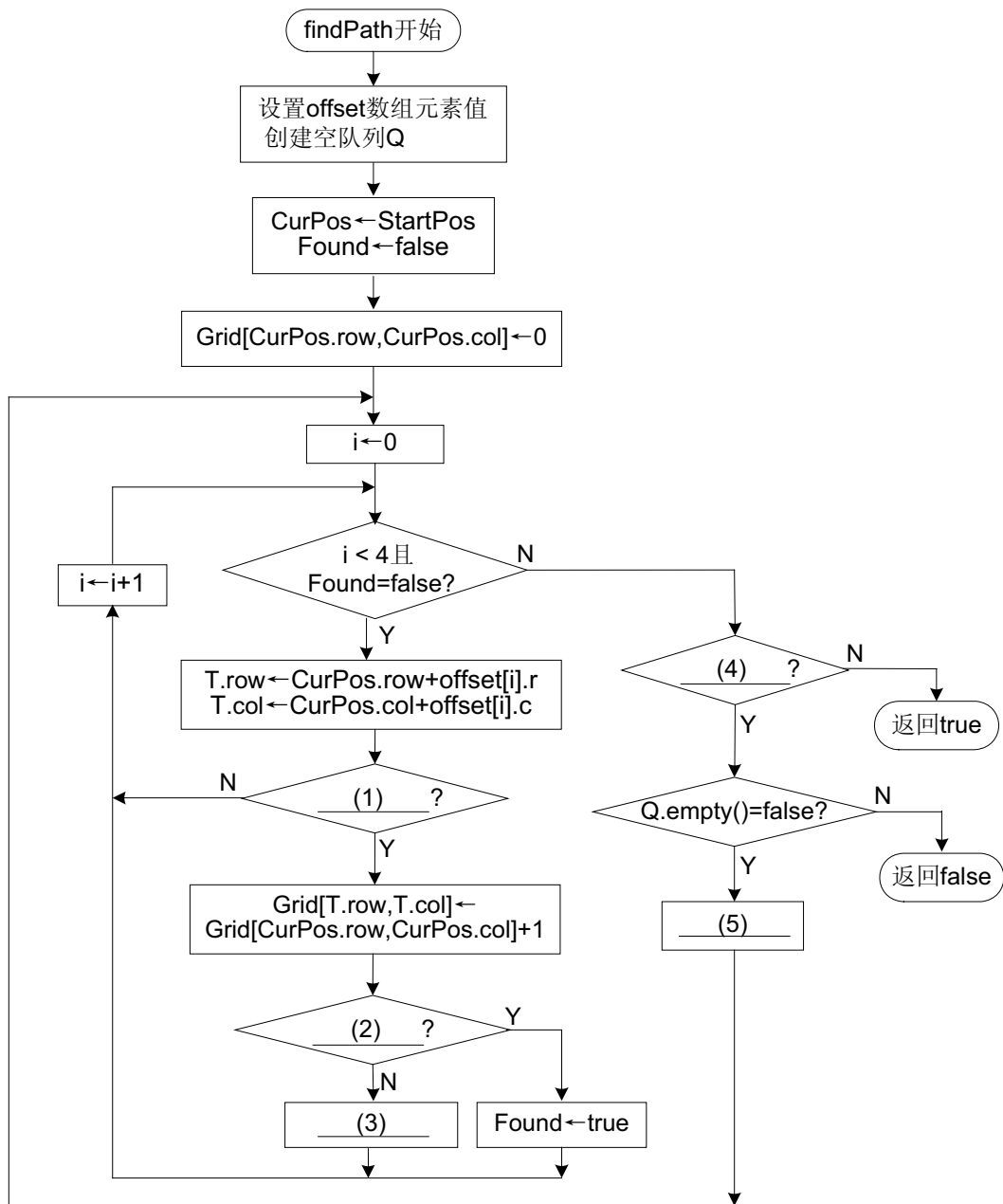
[表 3-2]

符号	含义
Grid	全局二维数组 $\text{Grid}[N+2, M+2]$ ，表示电路板方格阵列，初始时数组元素 $\text{Grid}[i, j]$ 的值为-1 表示当前方格可布线，为-9 表示当前方格不可布线。
offset	一维数组 $\text{offset}[4]$ ： $\text{offset}[i]$ ($0 \leq i \leq 3$) 的分量为 r （行偏移量）和 c （列偏移量），按照表 3-1 的内容设置其值。
StartPos、EndPos、CurPos、T	分别表示起始方格、目标方格、当前方格和临时方格，其位置用分量 row 和 col 确定。
$Q.\text{insert}(s)$	将方格 s 的位置信息加入队列。
$Q.\text{delete}()$	删除非空队列的队头元素，并返回该元素。
$Q.\text{empty}()$	若队列 Q 为空，则返回 true ；否则返回 false 。

[图 3-3]



[图 3-4]



● 供选择的答案

[a] Found ≠ true

[b] Found = true

[c] T = EndPos

[d] Q.insert(T)

[e] T ← Q.delete()

[f] CurPos = EndPos

[g] i ≥ 4

[h] CurPos ← Q.delete()

[i] Grid[T.row, T.col] = -1

[j] Grid[T.row, T.col] ≠ -1

试题四(15 分)

阅读以下说明和 C 程序，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

假设需要将 N 个任务分配给 N 个工人同时去完成，每个人都能承担这 N 个任务，但费用不同。下面的程序用回溯法计算总费用最小的一种工作分配方案，在该方案中，为每个人分配 1 个不同的任务。

程序中，N 个任务从 0 开始依次编号，N 个工人也从 0 开始依次编号，主要的变量说明如下：

c[i][j]:将任务 i 分配给工人 j 的费用；

task[i]:值为 0 表示任务 i 未分配，值为 j 表示任务 i 分配给工人 j；

worker[k]:值为 0 表示工人 k 未分配任务，值为 1 表示工人 k 已分配任务；

mincost:最小总费用。

[C 程序]

```
#include <stdio.h>
#define N 8                                /*N 表示任务数和工人数*/
int c[N][N];
unsigned int mincost = 65535;              /*设置 min 的初始值，大于可能的总费用*/
int task[N],temp[N],worker[N];
void Plan(int k,unsigned int cost)
{ int i;
  if (     (1)     && cost < mincost ) {
    mincost = cost;
    for(i = 0; i < N; i++) temp[i] = task[i];
  }
  else {
    for(i = 0; i < N; i++)                /*分配任务 k*/
      if (worker[i] == 0 &&     (2)    ) {
        worker[i] = 1; task[k] =     (3)    ;
        Plan(    (4)     , cost+c[k][i]);
            (5)    ; task[k] = 0;
      }/*if*/
  }
}/*Plan*/

void main()
{ int i,j;
  for(i = 0;i < N;i++) { /*设置每个任务由不同工人承担时的费用及全局数组的初值*/
    worker[i] = 0; task[i] = 0; temp[i] = 0;
    for(j=0;j < N;j++)
      scanf("%d",&c[i][j]);
  }
  Plan(0,0);                             /*从任务 0 开始分配*/
  printf("\n 最小费用 = %d\n", mincost);
  for(i = 0;i < N;i++)
    printf("Task%d is assigned to Worker%d\n",i,temp[i]);
}/*main*/
```

从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五(15 分)

阅读以下说明和 C++代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

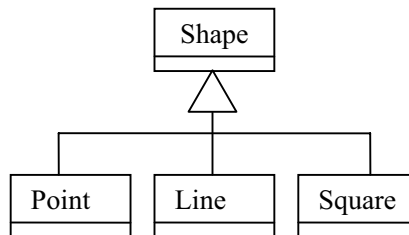
[说明]

某绘图系统存在 Point、Line、Square 三种图元，它们具有 Shape 接口，图元的类图关系如图 5-1 所示。现要将 Circle 图元加入此绘图系统以实现功能扩充。已知某第三方库已经提供了 XCircle 类，且完全满足系统新增的 Circle 图元所需的功能，但 XCircle 不是由 Shape 派生而来，它提供的接口不能被系统直接使用。代码 5-1 既使用了 XCircle 又遵循了 Shape 规定的接口，既避免了从头开发一个新的 Circle 类，又可以不修改绘图系统中已经定义的接口。代码 5-2 根据用户指定的参数生成特定的图元实例，并对之进行显示操作。

绘图系统定义的接口与 XCircle 提供的显示接口及其功能如下表所示：

Shape	XCircle	功能
display()	displayIt()	显示图元

[图 5-1]



[代码 5-1]

```
class Circle:public   (1)  {
private:
      (2)   m_circle;
public:
    void display(){
        m_circle.  (3)  ;
    }
};
```


[代码 5-2]

```
class Factory{
public:
    (4) getShapeInstance(int type){ //生成特定类实例
        switch(type){
            case 0: return new Point;
            case 1: return new Rectangle;
            case 2: return new Line;
            case 3: return new Circle;
            default: return NULL;
        }
    }
};

void main(int argc, char *argv[]){
    if(argc != 2) {
        cout << "error parameters !" << endl;
        return;
    }
    int type = atoi(argv[1]);
    Factory factory;
    Shape *s;
    s = factory.(5);
    if(s == NULL){
        cout << "Error get the instance !" << endl;
        return;
    }
    s->display();
    (6);
    return;
}
```

试题六(15 分)

阅读以下说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

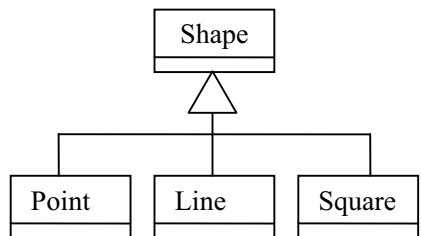
[说明]

某绘图系统存在 Point、Line、Square 三种图元，它们具有 Shape 接口，图元的类图关系如图 6-1 所示。现要将 Circle 图元加入此绘图系统以实现功能扩充。已知某第三方库已经提供了 XCircle 类，且完全满足系统新增的 Circle 图元所需的功能，但 XCircle 不是由 Shape 派生而来，它提供的接口不能被系统直接使用。代码 6-1 既使用了 XCircle 又遵循了 Shape 规定的接口，既避免了从头开发一个新的 Circle 类，又可以不修改绘图系统中已经定义的接口。代码 6-2 根据用户指定的参数生成特定的图元实例，并对之进行显示操作。

绘图系统定义的接口与 XCircle 提供的显示接口及其功能如下表所示：

Shape	XCircle	功能
display()	displayIt()	显示图元

[图 6-1]



[代码 6-1]

```
class Circle   (1)   {
    private   (2)   pxc;
    public Circle() { pxc = new   (3)  ;
    }
    public void display() {
        pxc.  (4)  ;
    }
}
```

[代码 6-2]

```
public class Factory{
    public ____ (5) ____ getShapeInstance(int type){ //生成特定类实例
        switch(type){
            case 0: return new Point();
            case 1: return new Rectangle();
            case 2: return new Line();
            case 3: return new Circle();
            default: return null;
        }
    }
};

public class App{
    public static void main(String argv[]){
        if(argv.length != 1) {
            System.out.println("error parameters !");
            return;
        }
        int type = (new Integer(argv[0])).intValue();
        Factory factory = new Factory();
        Shape s;
        s = factory. ____ (6) ____;
        if(s == null){
            System.out.println("Error get instance !");
            return;
        }
        s.display();
        return;
    }
}
```

试题七(15 分)

阅读以下说明和 Visual Basic 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

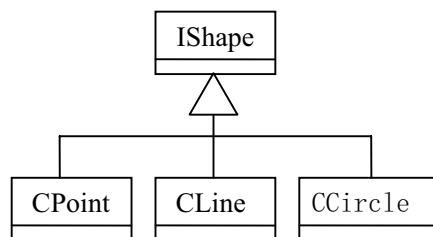
[说明]

某绘图系统定义了一个抽象类 IShape，现有三个类 CPoint、CLine 和 CCircle，它们都具有 IShape 界面。相应的类图关系如图 7-1 所示。

已知某第三方库已经提供了 XCircle 类，且完全满足 CCircle 图元显示时所需的功能。代码 7-1 是抽象类 IShape 的类模块内容，代码 7-2 实现了类 CCircle 的 IShape 界面，并使用了 XCircle 提供的显示功能。

XCircle 提供的显示功能方法接口为 displayIt。

[图 7-1]



[代码 7-1]

```
Public Color As Long
```

```
Sub draw()
```

```
    ' 方法体不包括可执行语句
```

```
End Sub
```

```
Sub move(stepx As Single, stepy As Single)
```

```
    ' 方法体不包括可执行语句
```

```
End Sub
```

[代码 7-2]

```
_____(1)_____  
Private color As Long  
...          ' 其他定义省略  
Private bridged As _____(2)_____  
  
Private Sub Class_Initialize()  
    Set bridged = _____(3)_____  
End Sub  
  
Private Property _____(4)_____( ) As Long  
    IShape_Color = color  
End Property  
  
Private Property _____(5)_____(ByVal newColor As Long)  
    color = newColor  
End Property  
  
Private Sub IShape_draw()          ' 使用 XCircle 提供的显示功能  
    _____(6)_____  
End Sub  
  
Private Sub IShape_move(stepx As Single, stepy As Single)  
    ...          ' 省略描述  
End Sub
```

全国计算机技术与软件专业技术资格（水平）考试

2005 年下半年 软件设计师 上午试卷

（考试时间 9:00~11:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2005 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
____(88)____月____(89)____日。

- | | | | |
|-----------|-------|-------|-------|
| (88) A. 9 | B. 10 | C. 11 | D. 12 |
| (89) A. 4 | B. 5 | C. 6 | D. 7 |

因为考试日期是“11 月 5 日”，故（88）选 C，（89）选 B，应在答题卡序号 88 下对 C 填涂，在序号 89 下对 B 填涂（参看答题卡）。

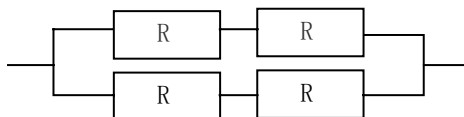
● 阵列处理机属于 (1) 计算机。

- (1) A. SISD B. SIMD C. MISD D. MIMD

● 采用 (2) 不能将多个处理机互连构成多处理机系统。

- (2) A. STD 总线 B. 交叉开关 C. PCI 总线 D. Centronic 总线

● 某计算机系统的可靠性结构是如下图所示的双重串并联结构，若所构成系统的每个部件的可靠度均为 0.9，即 $R=0.9$ ，则该系统的可靠度为 (3) 。



- (3) A. 0.9997 B. 0.9276 C. 0.9639 D. 0.6561

● 若每一条指令都可以分解为取指、分析和执行三步。已知取指时间 $t_{\text{取指}}=5\Delta t$ ，分析时间 $t_{\text{分析}}=2\Delta t$ ，执行时间 $t_{\text{执行}}=5\Delta t$ 。如果按顺序方式从头到尾执行完 500 条指令需 (4) Δt 。如果按照 [执行]_k、[分析]_{k+1}、[取指]_{k+2} 重叠的流水线方式执行指令，从头到尾执行完 500 条指令需 (5) Δt 。

- (4) A. 5590 B. 5595 C. 6000 D. 6007
(5) A. 2492 B. 2500 C. 2510 D. 2515

● 在开发一个系统时，如果用户对系统的目标不是很清楚，难以定义需求，这时最好使用 (6) 。

- (6) A. 原型法 B. 瀑布模型
C. V-模型 D. 螺旋模型

● 应该在 (7) 阶段制定系统测试计划。

- (7) A. 需求分析 B. 概要设计 C. 详细设计 D. 系统测试

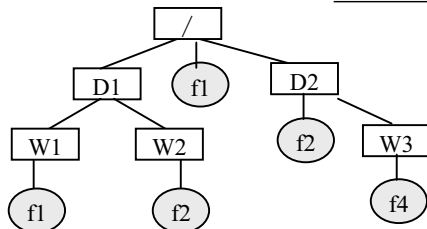
● 已经发布实施的现有标准（包括已确认或修改补充的标准），经过实施一定时期后，对其内容再次审查，以确保其有效性、先进性和适用性，其周期一般不超过 (8) 年。

- (8) A. 1 B. 3 C. 5 D. 7

● (9) 不需要登记或标注版权标记就能得到保护。

- (9) A. 专利权 B. 商标权 C. 著作权 D. 财产权

● 在下图所示的树型文件系统中，方框表示目录，圆圈表示文件，“/” 表示路径中的分隔符，“/” 在路径之首时表示根目录。图中， (10) 。



假设当前目录是 D1，进程 A 以如下两种方式打开文件 f1。

方式① `fd1=open("__ (11) __/f1",o_RDONLY);`

方式② `fd1=open("/D1/W1/f1",o_RDONLY);`

其中，方式①的工作效率比方式②的工作效率高，因为采用方式①的文件系统__ (12) __。

- (10) A. 子目录 W2 中文件 f2 和子目录 D2 中文件 f2 是完全相同的
B. 子目录 W2 中文件 f2 和子目录 D2 中文件 f2 是不相同的
C. 子目录 W2 中文件 f2 和子目录 D2 中文件 f2 可能相同也可能不相同
D. 树型文件系统中不允许出现相同名字的文件
- (11) A. /D1/W1 B. D1/W1 C. W1 D. f1
- (12) A. 可以直接访问根目录下的文件 f1
B. 可以从当前路径开始查找需要访问的文件 f1
C. 只需要访问一次磁盘，就可以读取文件 f1，而方式②需要二次
D. 只需要访问一次磁盘，就可以读取文件 f1，而方式②需要三次

● 正在开发的软件项目可能存在一个未被发现的错误，这个错误出现的概率是 0.5%，给公司造成的损失将是 1 000 000 元，那么这个错误的风险曝光度 (risk exposure) 是__ (13) __元。

- (13) A. 5 000 000 B. 50 000 C. 5 000 D. 500

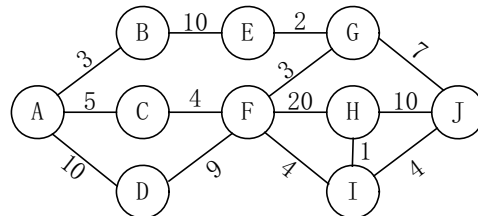
● 代码走查 (code walkthrough) 和代码审查 (code inspection) 是两种不同的代码评审方法，这两种方法的主要区别是__ (14) __。

- (14) A. 在代码审查中由编写代码的程序员来组织讨论，而在代码走查中由高级管理人员来领导评审小组的活动
B. 在代码审查中只检查代码中是否有错误，而在代码走查中还要检查程序与设计文档的一致性
C. 在代码走查中只检查程序的正确性，而在代码审查中还要评审程序员的编程能力和工作业绩
D. 代码审查是一种正式的评审活动，而代码走查的讨论过程是非正式的

● 在软件项目管理中可以使用各种图形工具来辅助决策，下面对 Gantt 图的描述不正确的是__ (15) __。

- (15) A. Gantt 图表现各个活动的顺序和它们之间的因果关系
B. Gantt 图表现哪些活动可以并行进行
C. Gantt 图表现了各个活动的起始时间
D. Gantt 图表现了各个活动完成的进度

● 在活动图中，结点表示项目中各个工作阶段的里程碑，连接各个结点的边表示活动，边上的数字表示活动持续的时间。在下面的活动图中，从A到J的关键路径是 (16)，关键路径的长度是 (17)，从E开始的活动启动的最早时间是 (18)。

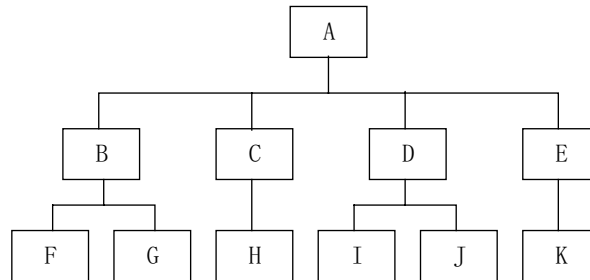


- (16) A. ABEGJ B. ADFHJ C. ACFGJ D. ADFIJ
 (17) A. 22 B. 49 C. 19 D. 35
 (18) A. 10 B. 12 C. 13 D. 15

● 一个故障已经被发现，而且也被排除了，为了检查修改是否引起了其它故障，这时应该进行 (19)。

- (19) A. 程序走查 B. 退化测试 C. 软件评审 D. 接收测试

● 一个系统的模块结构图如下所示，用 $\{ \times, \times, \times \}$ 表示这个系统的测试模块组合。下面的选项中 (20) 表示自顶向下的测试，(21) 表示三明治式测试。



- (20) A. $\{A\} \{A,B,C,D,E\} \{A,B,C,D,E,F,G,H,I,J,K\}$
 B. $\{F\} \{G\} \{H\} \{I\} \{J\} \{K\} \{B,F,G\} \{C,H\} \{D,I,J\} \{E,K\} \{A,B,C,D,E,F,G,H,I,J,K\}$
 C. $\{K\} \{J\} \{I\} \{H\} \{G\} \{F\} \{B\} \{C\} \{D\} \{E\} \{A,B,C,D,E,F,G,H,I,J,K\}$
 D. $\{A\} \{F\} \{G\} \{H\} \{I\} \{J\} \{K\} \{B,F,G\} \{C,H\} \{D,I,J\} \{E,K\} \{A,B,C,D,E,F,G,H,I,J,K\}$
 (21) A. $\{A\} \{A,B,C,D,E\} \{A,B,C,D,E,F,G,H,I,J,K\}$
 B. $\{F\} \{G\} \{H\} \{I\} \{J\} \{K\} \{B,F,G\} \{C,H\} \{D,I,J\} \{E,K\} \{A,B,C,D,E,F,G,H,I,J,K\}$
 C. $\{K\} \{J\} \{I\} \{H\} \{G\} \{F\} \{B\} \{C\} \{D\} \{E\} \{A,B,C,D,E,F,G,H,I,J,K\}$
 D. $\{A\} \{F\} \{G\} \{H\} \{I\} \{J\} \{K\} \{B,F,G\} \{C,H\} \{D,I,J\} \{E,K\} \{A,B,C,D,E,F,G,H,I,J,K\}$

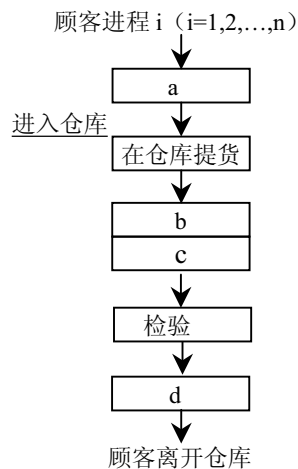
● 某公司服务器上存储了大量的数据，员工使用服务器前首先必须登录。为了保证安全，使用认证技术 (22)。为保证传输效率，使用 (23) 加密算法对传输的数据进行加密。

- (22) A. 对登录系统的用户身份进行认可 B. 保护数据在传输中的机密性
 C. 保证数据在传输中不被非法修改 D. 防止登录信息被泄漏出去
 (23) A. RSA B. ECC C. MD4 D. 3DES

● (24) 不能保障公司内部网络边界的安全。

- (24) A. 在公司网络与 Internet 或外界其它接口处设置防火墙
 B. 公司以外网络上用户要访问公司网时, 使用认证授权系统
 C. 禁止公司员工使用公司外部的电子邮件服务器
 D. 禁止公司内部网络的用户私自设置拨号上网

● 某仓库有两名发货员, 一名审核员。当顾客提货时, 只要发货员空闲, 允许顾客进入仓库提货, 顾客离开时, 审核员检验顾客提货是否正确。其工作流程如下图所示。为了利用PV操作正确地协调他们之间的工作, 设置了两个信号量S1和S2, 且S1的初值为2, S2的初值为1。图中的a应填写(25); 图中的b、c和d应分别填写(26)。

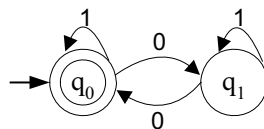


- (25) A. P(S1) B. P(S2) C. V(S1) D. V(S2)
 (26) A. P(S2)、V(S2)和 V(S1) B. P(S1)、V(S1)和 V(S2)
 C. V(S1)、P(S2)和 V(S2) D. V(S2)、P(S1)和 V(S1)

● 编译程序进行词法分析时不能(27)。

- (27) A. 过滤源程序中的注释 B. 扫描源程序并识别记号
 C. 指出出错行号 D. 查出拼错的保留字(关键字)

● 某一确定有限自动机(DFA)的状态转换图如下图所示, 该DFA接受的字符串集是(28), 与之等价的正规式是(29)。



- (28) A. 以 1 开头的二进制代码串组成的集合
 B. 以 1 结尾的二进制代码串组成的集合
 C. 包含偶数个 0 的二进制代码串组成的集合
 D. 包含奇数个 0 的二进制代码串组成的集合
 (29) A. $1^*0(0|1)^*$ B. $((0|1^*0)^*1^*)^*$ C. $1^*((0|1)0)^*$ D. $(1^*(01^*0)^*)^*$

● 下列关于静态多态实现重载的描述中正确的是 (30)。

- (30) A. 重载的方法（函数）的方法名（函数名）可以不同
B. 重载的方法（函数）必须在返回值类型以及参数类型上有所不同
C. 重载的方法（函数）必须在参数顺序或参数类型上有所不同
D. 重载的方法（函数）只需在返回值类型上有所不同

● 程序设计语言提供了基本类型及其相关的操作，而 (31) 则允许开发者自定义一种新的类型及其相关的操作。

- (31) A. 对象 B. 实例 C. 类 D. 引用

● (32) 表示了对象间“is-a”的关系。

- (32) A. 组合 B. 引用 C. 聚合 D. 继承

● 在关系代数运算中，关系S、SP和R如下表所示。若先 (33)，则可以从S和SP获得R。其对应的关系表达式为 (34)。如下的SQL语句可以查询销售总量大于 1000 的部门名。

Select 部门名 From S

Where 部门号 in (Select 部门号 From SP Group by (35))

S		SP			R		
部门号	部门名	部门号	商品号	销售量	部门名	商品号	销售量
010	家电部	010	01020210	500	家电部	01020210	500
021	百货部	010	01020211	780	家电部	01020211	780
030	食品部	010	01020212	990	家电部	01020212	990
035	五金部	021	02110200	580			
		025	02520100	1290			
		035	03530311	4680			

- (33) A. 对 S 进行选择运算，再与 SP 进行自然连接运算
B. 对 S 进行选择运算，再与 SP 进行自然连接运算，最后进行投影运算
C. 对 S 和 SP 进行笛卡儿积运算，再对运算的结果进行投影运算
D. 分别对 S 和 SP 进行投影运算，再对运算的结果进行笛卡儿积运算

- (34) A. $\sigma_{1='010'}(S) \times SP$ B. $\pi_{1,3,4}(\sigma_{1='010'}(S) \times SP)$

- C. $\pi_{2,3,4}(\sigma_{1='010'}(S) \times SP)$ D. $\pi_{2,3,4}(S \times SP)$

- (35) A. 部门号 where sum(销售量)>1000 B. 部门号 having sum(销售量)>1000
C. 商品号 where sum(销售量)>1000 D. 商品号 having sum(销售量)>1000

● 给定关系 $R(A_1, A_2, A_3, A_4)$ 上的函数依赖集 $F = \{A_1 A_3 \rightarrow A_2, A_2 \rightarrow A_3\}$ ，R 的候选关键字为 (36)。分解 $\rho = \{(A_1, A_2), (A_1, A_3)\}$ (37)。

- (36) A. A_1A_3 B. $A_1A_2A_3$ C. $A_1A_3A_4$ D. A_2 和 A_1A_3
 (37) A. 是无损联接的 B. 是保持函数依赖的
 C. 既是无损联接又保持函数依赖 D. 既是有损联接又不保持函数依赖

● 已知某二叉树的中序、层序序列分别为DBAFCE、FDEBCA，则该二叉树的后序序列为(38)。

- (38) A. BCDEAF B. ABDCEF C. DBACEF D. DABECF

● 在二叉树的顺序存储中，每个结点的存储位置与其父结点、左右子树结点的位置都存在一个简单的映射关系，因此可与三叉链表对应。若某二叉树共有 n 个结点，采用三叉链表存储时，每个结点的数据域需要 d 个字节，每个指针域占用4个字节，若采用顺序存储，则最后一个结点的下标为 k （起始下标为1），那么(39)时采用顺序存储更节省空间。

- (39) A. $d < \frac{12n}{k-n}$ B. $d > \frac{12n}{k-n}$ C. $d < \frac{12n}{k+n}$ D. $d > \frac{12n}{k+n}$

● 简单无向图的邻接矩阵是对称的，可以对其进行压缩存储。若无向图 G 有 n 个节点，其邻接矩阵为 $A[1..n, 1..n]$ ，且压缩存储在 $B[1..k]$ 中，则 k 的值至少为(40)。若按行压缩存储对称矩阵的上三角元素，则当 n 等于10时，边 (V_6, V_3) 的信息存储在 $B[(41)]$ 中。

- (40) A. $\frac{n(n+1)}{2}$ B. $\frac{n^2}{2}$ C. $\frac{(n-1)(n+1)}{2}$ D. $\frac{n(n-1)}{2}$

- (41) A. 18 B. 19 C. 20 D. 21

● 以下内容中，(42)应写入操作手册。

- (42) A. 描述系统对各种输入数据的处理方法
 B. 说明系统升级时厂商提供的服务
 C. 描述系统处理过程的各个界面
 D. 说明系统各部分之间的接口关系

● 某软件工程项目各开发阶段工作量的比例如下表所示

需求分析	概要设计	详细设计	编 码	测 试
0.29	0.13	0.17	0.10	0.31

假设当前已处于编码阶段，3000行程序已完成了1200行，则该工程项目开发进度已完成的比例是(43)。

- (43) A. 29% B. 45% C. 59% D. 63%

● 在 11 个元素的有序表A[1..11]中进行折半查找($\lfloor (low + high) / 2 \rfloor$), 查找元素A[11]时, 被比较的元素的下标依次是__ (44) __。

(44) A. 6, 8, 10, 11 B. 6, 9, 10, 11 C. 6, 7, 9, 11 D. 6, 8, 9, 11

● 关于维护软件所需的成本, 以下叙述正确的是__ (45) __。

(45) A. 纠正外部和内部设计错误比纠正源代码错误需要更大的成本
B. 与需求定义相比, 源代码的文字量大得多, 所以源代码的维护成本更高
C. 用户文档需要经常更新, 其维护成本超过了纠正设计错误的成本
D. 需求定义的错误会在设计时被发现并纠正, 因此需求定义纠错的成本小于源代码纠错的成本

● 由元素序列(27,16,75,38,51)构造平衡二叉树, 则首次出现的最小不平衡子树的根(即离插入结点最近且平衡因子的绝对值为 2 的结点)为__ (46) __。

(46) A. 27 B. 38 C. 51 D. 75

● 若排序前后关键字相同的两个元素相对位置不变, 则称该排序方法是稳定的。__ (47) __排序是稳定的。

(47) A. 归并 B. 快速 C. 希尔 D. 堆

● 若对象A可以给对象B发送消息, 那么__ (48) __。

(48) A. 对象 B 可看见对象 A B. 对象 A 可看见对象 B
C. 对象 A、B 相互不可见 D. 对象 A、B 相互可见

● 类描述了一组对象共同的特性, 下列叙述中正确的是__ (49) __。

(49) A. 类本身不能具有变量
B. 对象具有类定义的所有变量的一份拷贝
C. 对象间不能共享类定义的变量
D. 可通过类名访问静态变量(类变量)

● __ (50) __反映了类间的一种层次关系, 而 __ (51) __反映了一种整体与部分的关系。

(50) A. 继承 B. 组合 C. 封装 D. 多态
(51) A. 继承 B. 组合 C. 封装 D. 多态

● 下列叙述中正确的是__ (52) __。

(52) A. 面向对象程序设计语言都不支持过程化的程序设计
B. 面向对象系统只可采用面向对象程序设计语言实现
C. 某些过程化程序设计语言也可实现面向对象系统
D. 面向对象程序设计语言不支持对成员变量的直接访问

- 设求解某问题的递归算法如下：

```

F(int n){
    if ( n == 1 ) {
        Move ( 1 );
    } else {
        F ( n - 1 );
        Move ( n );
        F ( n - 1 );
    }
}

```

求解该算法的计算时间时，仅考虑算法Move所做的计算为主要计算，且Move为常数级算法。则算法F的计算时间 $T(n)$ 的递推关系式为__ (53) __；设算法Move的计算时间为 k ，当 $n=4$ 时，算法F的计算时间为__ (54) __。

- (53) A. $T(n) = T(n-1) + 1$ B. $T(n) = 2T(n-1)$
 C. $T(n) = 2T(n-1) + 1$ D. $T(n) = 2T(n+1) + 1$
- (54) A. $14k$ B. $15k$ C. $16k$ D. $17k$

● 利用贪心法求解 0/1 背包问题时，__ (55) __能够确保获得最优解。用动态规划方法求解 0/1 背包问题时，将“用前 i 个物品来装容量是 X 的背包”的 0/1 背包问题记为KNAP(1, i , X)，设 $f_i(X)$ 是KNAP(1, i , X)最优解的效益值，第 j 个物品的重量和放入背包后取得效益值分别为 w_j 和 p_j ($j=1\sim n$)。则依次求解 $f_0(X)$ 、 $f_1(X)$ 、...、 $f_n(X)$ 的过程中使用的递推关系式为__ (56) __。

- (55) A. 优先选取重量最小的物品 B. 优先选取效益最大的物品
 C. 优先选取单位重量效益最大的物品 D. 没有任何准则
- (56) A. $f_i(X) = \min \{f_{i-1}(X), f_{i-1}(X) + p_i\}$ B. $f_i(X) = \max \{f_{i-1}(X), f_{i-1}(X - w_i) + p_i\}$
 C. $f_i(X) = \min \{f_{i-1}(X - w_i), f_{i-1}(X - w_i) + p_i\}$ D. $f_i(X) = \max \{f_{i-1}(X - w_i), f_{i-1}(X) + p_i\}$

- __ (57) __ 标准中定义了MP3 音乐标准。

- (57) A. JPEG B. MPEG-1 C. MPEG-2 D. MPEG-3

● 使用 200DPI的扫描分辨率扫描一幅 2×2.5 英寸的黑白图像，可以得到一幅__ (58) __像素的图像。

- (58) A. 200×2 B. 2×2.5 C. 400×500 D. 800×1000

- 要在网络上发布彩色动画文件，可以采用的存储格式是__ (59) __。

- (59) A. BMP B. JPEG C. MP3 D. GIF

- 为保证用户在网络上边下载边观看视频信息，需要采用__ (60) __技术。

- (60) A. 流媒体 B. 数据库 C. 数据采集 D. 超链接

● 通过代理服务器使内部局域网中各客户机访问Internet时，__ (61) __不属于代理服务器的功能。

- (61) A. 共享 IP 地址 B. 信息缓存 C. 信息转发 D. 信息加密

● 下列 (62) 设备可以隔离ARP广播帧。

(62) A. 路由器 B. 网桥 C. 以太网交换机 D. 集线器

● 在Windows系统中, (63) 不是网络服务组件。

(63) A. RAS B. HTTP C. IIS D. DNS

● 在OSI参考模型中, 数据链路层处理的数据单位是 (64) 。

(64) A. 比特 B. 帧 C. 分组 D. 报文

● 在OGSA标准中定义了 (65) 的概念, 它提供一组遵守特定的约定并定义明确的接口, 是实体之间产生、管理和交换信息的机制。

(65) A. Object B. Grid Service C. Web Service D. XML

● MIDI enables people to use (66) computers and electronic musical instruments. There are actually three components to MIDI, the communications " (67) ", the Hardware interface and a distribution (68) called "Standard MIDI Files". In the context of the WWW, the most interesting component is the (69) Format. In principle, MIDI files contain sequences of MIDI Protocol messages. However, when MIDI Protocol (70) are stored in MIDI files, the events are also time-stamped for playback in the proper sequence. Music delivered by MIDI files is the most common use of MIDI today.

(66) A. personal	B. electronic	C. multimedia	D. network
(67) A. device	B. protocol	C. network	D. controller
(68) A. format	B. text	C. wave	D. center
(69) A. Video	B. Faxmail	C. Graphic	D. Audio
(70) A. messages	B. packets	C. frame	D. information

● Certificates are (71) documents attesting to the (72) of a public key to an individual or other entity. They allow verification of the claim that a given public key does in fact belong to a given individual. Certificates help prevent someone from using a phony key to (73) someone else. In their simplest form, Certificates contain a public key and a name. As commonly used, a Certificate also contains an (74) date, the name of the CA that issued the Certificate, a serial number, and perhaps other information. Most importantly, it contains the digital (75) of the certificate issuer. The most widely accepted format for certificates is X.509, thus, Certificates can be read or written by any application complying with X.509.

(71) A. text	B. data	C. digital	D. structured
(72) A. connecting	B. binding	C. composing	D. conducting
(73) A. impersonate	B. personate	C. damage	D. control
(74) A. communication	B. computation	C. expectation	D. expiration
(75) A. signature	B. mark	C. stamp	D. hypertext

全国计算机技术与软件专业技术资格（水平）考试

2005 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2005 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月
（2）日。

因为正确的解答是“11 月 5 日”，故在答题纸的对应栏内写上“11”和“5”
（参看下表）。

例题	解答栏
（1）	11
（2）	5

试题一（共 15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

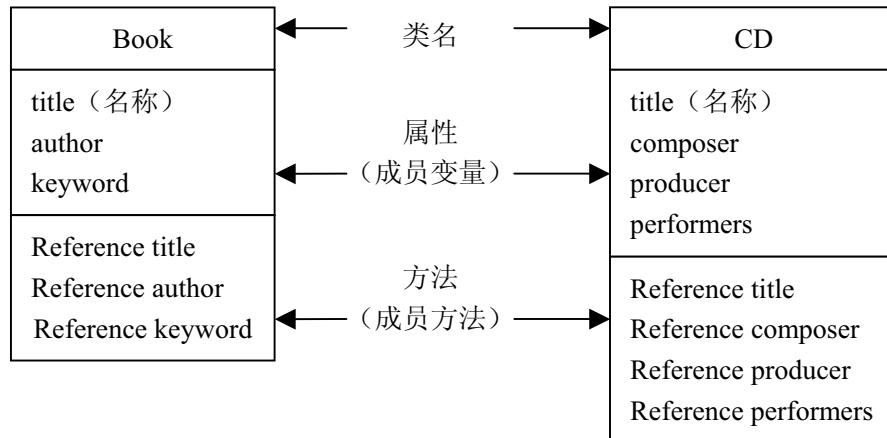
[说明]

某公司的主要业务是出租图书和唱碟。由于业务需求，该公司委托软件开发公司 A 开发一套信息管理系统。该系统将记录所有的图书信息、唱碟信息、用户信息、用户租借信息等。A 公司决定采用面向对象的分析和设计方法开发此系统。图 1-1 所示为某类图书或唱碟被借阅时应记录的信息，图 1-2 描述了系统定义的两个类 Book 和 CD，分别表示图书和唱碟的信息。

[图 1-1]

图书/唱碟 名称: _____		
借出时间	归还时间	用户

[图 1-2]



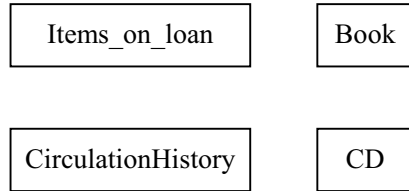
[问题 1]（3 分）

经过进一步分析，设计人员决定定义一个类 Items_on_loan，以表示类 Book 和 CD 的共有属性和方法。请采用图 1-2 中属性和方法的名称给出类 Items_on_loan 应该具有的属性和方法。（注意：不同名称的属性和方法表示不同的含义，如 CD 中的 composer 与

Book 中的 author 无任何关系)

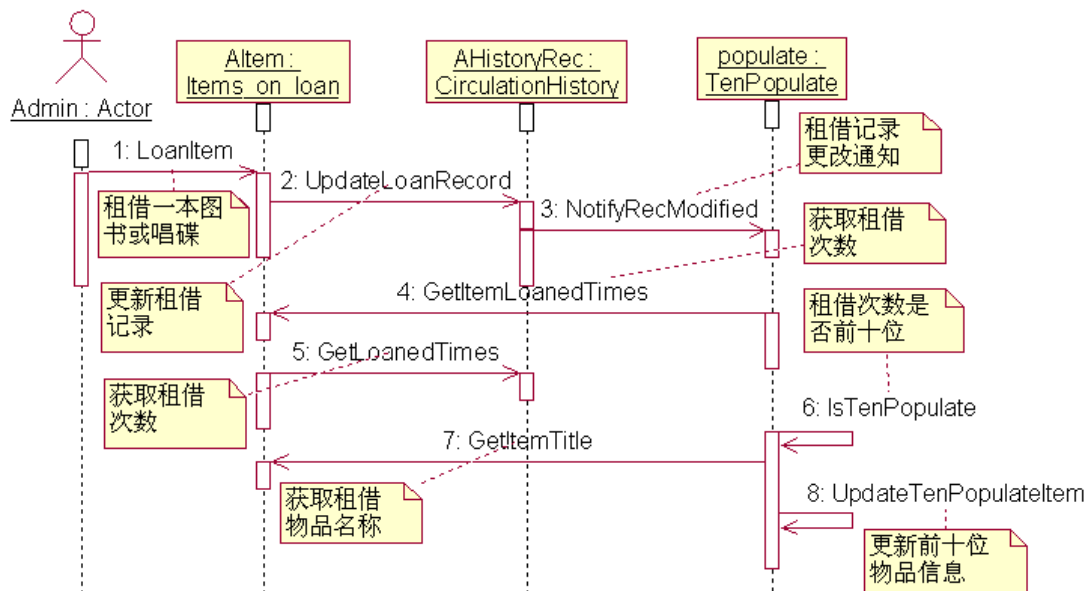
[问题 2] (6 分)

为了记录每种图书或唱碟租借的历史记录, 引入类 **CirculationHistory**, 类中存储的信息是图 1-1 中所表示的内容。请采用 UML 表示法将下列四个类间的关系表示出来。



[问题 3] (6 分)

现需了解十大最畅销(借出次数最多)图书或唱碟。为此, 引入 **TenPopulate** 类以存储所有十大畅销图书或 CD 的名称及其被借出的次数。下列顺序图描述了某类图书或唱碟被借出后成为十大畅销图书或唱碟时对象间的消息交互。系统在一次运行过程中, 应有 (1) 个 **TenPopulate** 实例对象最合适, 一个 **TenPopulate** 类实例对象最多需要和 (2) 个 **Items_on_loan** 实例对象交互。



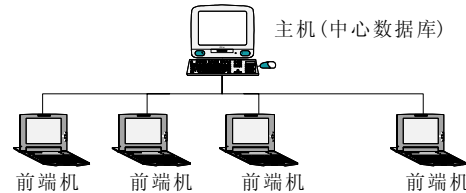
试题二（共 15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某企业决定开发一个企业仓储管理系统，由李工承担系统的设计工作。该系统的网络连接如图 2-1 所示。

【图 2-1】



该企业有多个仓库，图 2-1 所示的中心数据库存储了各个仓库中每种货物的库存信息。每个仓库配备一台前端机，进出货物均由前端机辅助实现。管理员每天上班时，通过前端机从中心数据库的库存表中读取本仓库各种货物的库存数，每个仓库的当日业务数据也都暂存在前端机，当天业务结束后，再将前端机中存储的数据传输到主机进行存储与汇总。

每个仓库可以存放多种货物，但同一种货物不能存放在不同的仓库中。每个仓库有多个管理员，但每个管理员只管理一个仓库。货物出库/入库时，由仓库管理员将货物的条码通过阅读器输入前端机中，货物数量的默认值为 1，可以由管理员修改。前端机根据输入的货物信息，打印“出库/入库”清单。出库/入库单中同一种货物最多只出现一次，每份出库/入库单由流水号唯一标识。图 2-2 是一个出库单的实例。

【图 2-2】

流水号：200408080001300101 时间：2005-10-01 13:22

货物编号	货物名称	单价	数量
6900100180988	全自动洗衣机	1680.00	26
6900100170655	32 寸彩色电视机	7580.00	20
6901100160126	1P 空调	1360.00	60

管理员：01105 出库/入库： 出库

该系统处理业务的过程如下：

1. **初始化：**前端机根据仓库号从货物表中读取本仓库中每种货物的货物编码、库存量、货物名称和单价；
2. **登记出库/入库信息：**由前端机存储每一笔“出库/入库”记录；
3. **汇总：**在每个工作日结束前汇总当日各种货物的“出库/入库”量至日汇总表；
4. **更新库存表：**根据当日的汇总信息更新货物的库存。

李工经过分析，设计出如图 2-3 所示的关系模式。

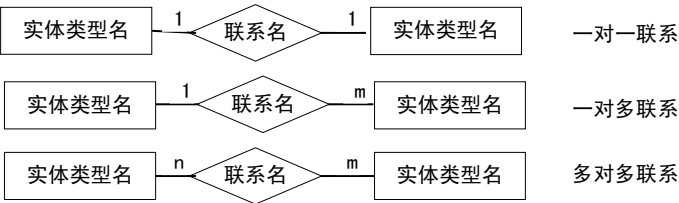
[图 2-3]

出入库单(流水号，出入库标志，管理员号，时间)
出入库记录(货物编码，数量，流水号)
日汇总表(日期，货物编码，数量，出入库标志)
仓库(仓库号，仓库名，仓库电话)
管理员(管理员号，姓名，仓库号)
货物(_____ (a) _____)

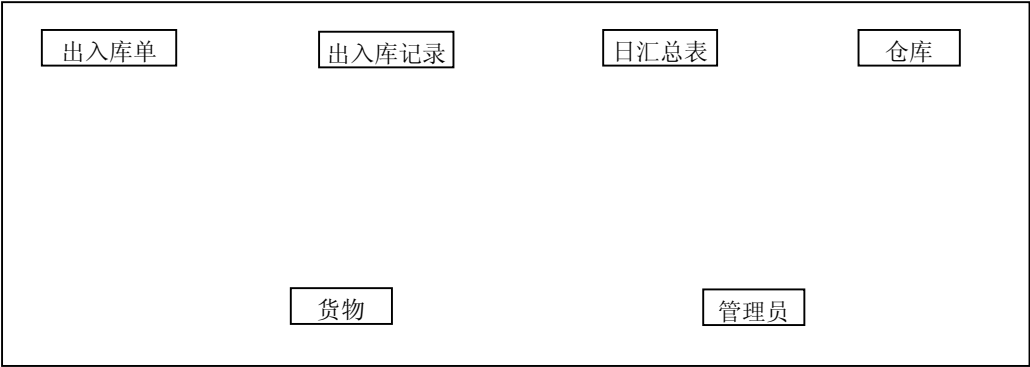
注：时间格式为年-月-日 时:分；日期格式为年-月-日。

实体联系图的表示方法如图 2-4 所示，其中方框表示实体，菱形表示联系，联系的类型在实体与联系的边上标出。图 2-5 为与该系统对应的实体联系图。

[图 2-4]



[图 2-5]



[问题 1](3 分)

根据题意，补充图 2-3 中 (a) 处的空缺，即货物关系模式的属性。

[问题 2](6 分)

根据题意，补充图 2-5 中缺失的联系和联系的类型，使其成为完善的实体联系图。其中，联系名分别取名为联系 1，联系 2，联系 3，...

[问题 3](6 分)

写出每种关系模式的主键，将其填写在答题纸的对应栏内。

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

[说明]

某公司计划与客户通过 Internet 交换电子邮件和数据（以下统一称为“消息”）。为保障安全，在对传输的数据进行加密的同时，还要对参与通信的实体进行身份认证。因此，需同时使用对称与非对称密钥体系。图 3-1 描述了接收者 B 使用非对称密钥体系对发送者 A 进行认证的过程。

[图 3-1]

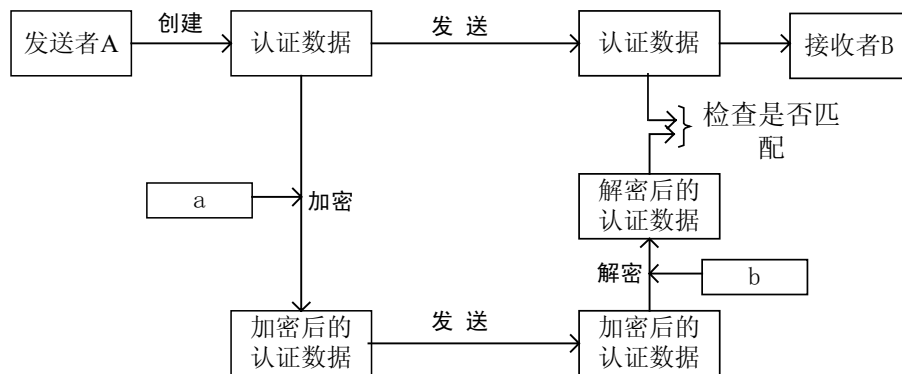


图 3-2 描述了发送和接收消息的过程，其中的认证过程使用了图 3-1 中的方法。图 3-1 中的方框 a 和方框 b 与图 3-2 中的方框 a 和方框 b 相同。

[图 3-2]

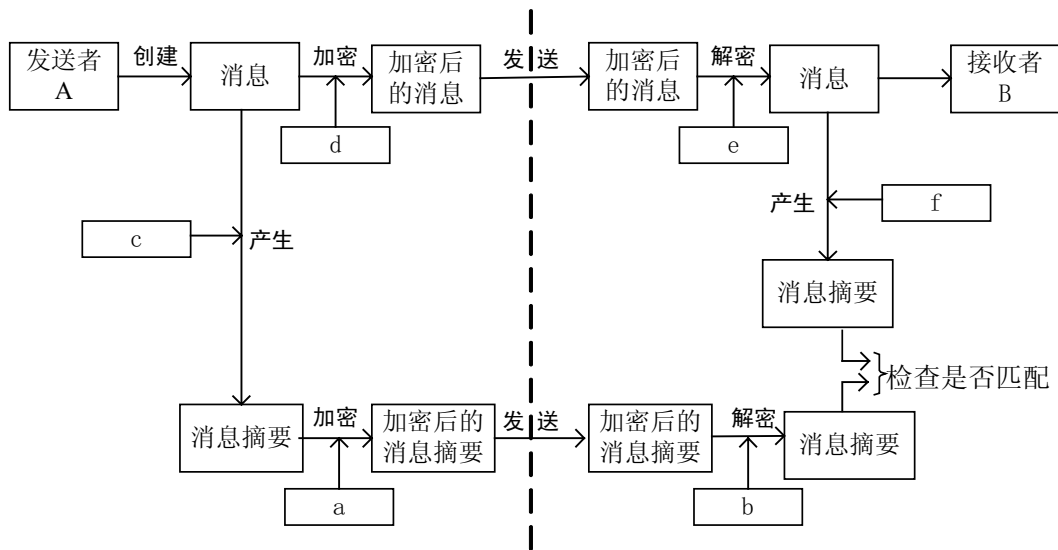


图3-2中发送和接收消息的过程是：

- 1) 发送者A使用与接收者B共享的对称密钥体系的密钥加密将要发送的消息。
- 2) 为了实现身份认证，A使用与B共享的摘要算法生成消息摘要，并使用公钥密码体系把生成的消息摘要加密后发送给B（这里假设A和B都能通过安全的方法获得对方的公钥）。
- 3) B使用非对称密钥体系解密收到的消息摘要，使用与A共享的对称密钥体系的密钥解密加密后的消息，再使用与A共享的摘要算法针对解密后的消息生成消息摘要。
- 4) B对比自己生成的消息摘要与接收到的A发送的消息摘要是否相同，从而验证发送者A的身份。

[问题 1](2 分)

请在下列选项中选择合适的答案，填入图 3-1、3-2 的方框 a 和方框 b。

B 的公钥，B 的私钥，摘要算法，A 的私钥，A 的公钥，会话密钥

[问题 2](4 分)

请在下列选项中选择合适的答案，填入图 3-2 的方框 c 至方框 f。

B 的公钥，B 的私钥，摘要算法，A 的私钥，A 的公钥，会话密钥

[问题 3](5 分)

按照图 3-2 中的方法发送邮件时，使用不同的密码体制加密消息和消息摘要，请用 150 字以内文字简要说明这样做的理由。

[问题 4](4 分)

请从下面关于摘要函数的说法中选出所有正确的描述。

- [a] 很容易使不同的输入数据生成相同的输出数据。
- [b] 根据输入数据获取输出数据的时间非常短。
- [c] 根据输入数据获取输出数据的时间非常长。
- [d] 输出数据的长度比输入数据的长度要长。
- [e] 根据输出数据无法还原出输入数据。

试题四（15 分）

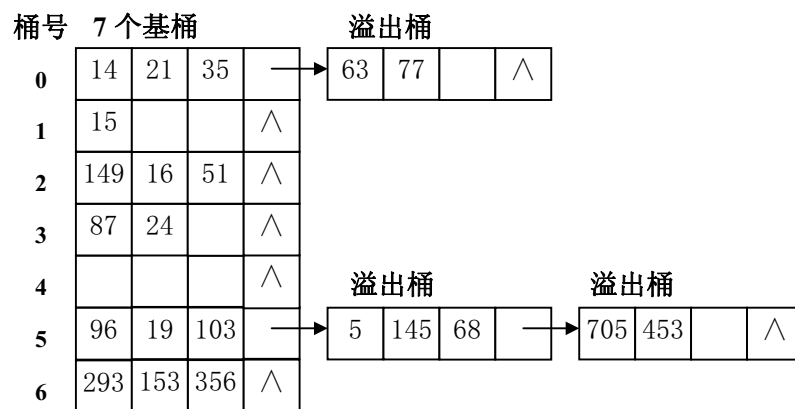
阅读下列函数说明、图和 C 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

散列文件的存储单位称为桶(BUCKET)。假如一个桶能存放 m 个记录，当桶中已有 m 个同义词(散列函数值相同)的记录时，存放第 $m+1$ 个同义词会发生“溢出”。此时需要将第 $m+1$ 个同义词存放到另一个称为“溢出桶”的桶中。相对地，称存放前 m 个同义词的桶为“基桶”。溢出桶和基桶大小相同，用指针链接。查找指定元素记录时，首先在基桶中查找。若找到，则成功返回，否则沿指针到溢出桶中进行查找。

例如：设散列函数为 $\text{Hash}(\text{Key}) = \text{Key} \bmod 7$ ，记录的关键字序列为 15, 14, 21, 87, 96, 293, 35, 24, 149, 19, 63, 16, 103, 77, 5, 153, 145, 356, 51, 68, 705, 453，建立的散列文件内容如图 4-1 所示。

[图 4-1]



为简化起见，散列文件的存储单位以内存单元表示。

函数 $\text{InsertToHashTable}(\text{int NewElemKey})$ 的功能是：若新元素 NewElemKey 正确插入散列文件中，则返回值 1；否则返回值 0。

采用的散列函数为 $\text{Hash}(\text{NewElemKey}) = \text{NewElemKey} \% P$ ，其中 P 为设定的基桶数目。

函数中使用的预定义符号如下：

```
#define NULLKEY -1          /*散列桶的空闲单元标识*/
#define P      7           /*散列文件中基桶的数目*/
#define ITEMS  3           /*基桶和溢出桶的容量*/
typedef struct BucketNode{ /*基桶和溢出桶的类型定义*/
    int KeyData[ITEMS];
    struct BucketNode *Link;
}BUCKET;
BUCKET Bucket[P];          /*基桶空间定义*/
```

[函数]

```
int InsertToHashTable(int NewElemKey) {
    /*将元素 NewElemKey 插入散列桶中, 若插入成功则返回 0, 否则返回-1*/
    /*设插入第一个元素前基桶的所有 KeyData[], Link 域已分别初始化为 NULLKEY、NULL*/

    int Index;      /*基桶编号*/
    int i, k;
    BUCKET *s, *front, *t;
    _____(1)_____;
    for(i = 0; i < ITEMS; i++) /*在基桶查找空闲单元, 若找到则将元素存入*/
        if (Bucket[Index].KeyData[i] == NULLKEY) {
            Bucket[Index].KeyData[i] = NewElemKey;    break;
        }
    if (_____(2)_____) return 0;
    /*若基桶已满, 则在溢出桶中查找空闲单元, 若找不到则申请新的溢出桶*/
    _____(3)_____;    t = Bucket[Index].Link;
    if (t != NULL) { /*有溢出桶*/
        while (t != NULL) {
            for(k = 0; k < ITEMS; k++)
                if (t->KeyData[k] == NULLKEY) { /*在溢出桶链表中找到空闲单元*/
                    t->KeyData[k] = NewElemKey;    break;
                } /*if*/
            front = t;
            if (_____(4)_____) t = t->Link;
            else break;
        } /*while*/
    } /*if*/
    if (_____(5)_____) { /*申请新溢出桶并将元素存入*/
        s = (BUCKET *)malloc(sizeof(BUCKET));
        if (!s) return -1;
        s->Link = NULL;
        for(k = 0; k < ITEMS; k++)
            s->KeyData[k] = NULLKEY;
        s->KeyData[0] = NewElemKey;
        _____(6)_____;
    } /*if*/
    return 0;
} /*InsertToHashTable*/
```


从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五(15 分)

阅读以下说明和 C++代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

在一公文处理系统中，开发者定义了一个公文类 OfficeDoc，其中定义了公文具有的属性和处理公文的相应方法。当公文的内容或状态发生变化时，关注此 OfficeDoc 类对象的相应的 DocExplorer 对象都要更新其自身的状态。一个 OfficeDoc 对象能够关联一组 DocExplorer 对象。当 OfficeDoc 对象的内容或状态发生变化时，所有与之相关联的 DocExplorer 对象都将得到通知，这种应用被称为观察者模式。以下代码写在一个 C++ 源文件中，能够正确编译通过。

[C++代码]

```
#include <iostream>

const OBS_MAXNUM=20; //最多与 OfficeDoc 对象相关联的 DocExplorer 对象的个数

_____(1)_____;
```

```
class DocExplorer{ //关注 OfficeDoc 公文对象的类
public:
    DocExplorer (_____(2)_____ *doc); //构造函数
    _____(3)_____ void update(OfficeDoc *doc)=0; //更新自身状态的函数
    //其它相关属性和方法省略
};
```

```
class OfficeDoc{ //公文类
private:
    DocExplorer *myObs[OBS_MAXNUM];
    //关注此公文类的 DocExplorer 类对象指针数组
    int index; //与 OfficeDoc 对象关联的 DocExplorer 对象的个数
public:
    OfficeDoc() {
        index=0;
    }
    void attach(DocExplorer *o) {
        //将一 DocExplorer 对象与 OfficeDoc 对象相关联
        if (index >= OBS_MAXNUM || o == NULL) return;
        for(int loop = 0; loop < index; loop++)
```

```

        if(myObs[loop] == o)    return;
myObs[index] = o;
index++;
}
void detach(DocExplorerer *o) {
    //解除某 DocExplorerer 对象与 OfficeDoc 对象的关联
    if(o==NULL) return;
    for(int loop = 0; loop < index; loop ++){
        if(myObs[loop] == o){
            if(loop <= index-2) myObs[loop] = myObs[index-1];
            myObs[index-1]=NULL;
            index--;
            break;
        }
    }
}
private:
    void notifyObs() { //通知所有的 DocExplorerer 对象更改自身状态
        for(int loop = 0; loop < index; loop++){
            myObs[loop]->____(4)____; //DocExplorerer 对象更新自身状态
        }
    }
//其它公文类的相关属性和方法
};

DocExplorerer::DocExplorerer(OfficeDoc *doc) { //DocExplorerer 类对象的构造函数
    doc->____(5)____; //将此 DocExplorerer 对象与 doc 对象相关联
}

```

试题六(共 15 分)

阅读以下说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

在一公文处理系统中，开发者定义了一个公文类 OfficeDoc，其中定义了公文具有的属性和处理公文的相应方法。当公文的内容或状态发生变化时，关注此 OfficeDoc 类对象的相应的 DocExplorer 对象都要更新其自身的状态。一个 OfficeDoc 对象能够关联一组 DocExplorer 对象。当 OfficeDoc 对象的内容或状态发生变化时，所有与之相关联的 DocExplorer 对象都将得到通知，这种应用被称为观察者模式。以下代码采用 Java 语言实现，能够正确编译通过。

[Java 代码]

// Subject. java 文件

```
public interface Subject {  
    public void attach(Observer DocExplorer);  
    public void detach(Observer DocExplorer);  
    void notifyObservers();  
}
```

// Observer. java 文件

```
public interface Observer{  
    void update(          (1)          );  
}
```

// OfficeDoc. java 文件

```
import java.util.*;  
public class OfficeDoc implements Subject{ //OfficeDoc 类实现 Subject 接口  
    private Vector ObserverVector = new java.util.Vector();  
    // 存储与 OfficeDoc 相关联的 DocExplorer 对象  
  
    public void attach(Observer observer){  
        //将某 DocExplorer 对象与 OfficeDoc 相关联  
        ObserverVector.addElement(observer);  
    }  
  
    public void detach(Observer observer){  
        //解除某 DocExplorer 对象与 OfficeDoc 的关联关系  
        ObserverVector.removeElement(observer);  
    }  
    public void notifyObservers(){
```

```

//当 OfficeDoc 对象状态已发生变化时，通知所有的 DocExplorer 对象
Enumeration enumeration = _____(2)_____；
while (enumeration.hasMoreElements()) {
    ((Observer) enumeration.nextElement()). _____(3)_____；
}
}

public Enumeration Observers() {
    return  ObserverVector.elements();
}
//其它公文类的属性和方法省略
}

// DocExplorer.java 文件
public class DocExplorer implements _____(4)_____ {
    public void update(_____ (5) _____) {
        //更新 DocExplorer 自身的状态，代码省略
    }
}
}

```

试题七(共 15 分)

阅读以下说明和 C 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

在一公文处理系统中，开发者定义了一个公文结构 OfficeDoc，其中定义了公文应该具有的属性（字段）。当公文的内容或状态发生变化时，与之相关联的 DocExplorer 结构的值都需要发生改变。一个 OfficeDoc 结构能够关联一组 DocExplorer 结构。当 OfficeDoc 结构的内容或状态发生变化时，所有与之相关联的 DocExplorer 结构都将被更新，这种应用被称为观察者模式。以下代码采用 C 语言实现，能够正确编译通过。

[C 代码]

```
#include <stdio.h>
#define OBS_MAXNUM 20 /*一个 OfficeDoc 变量最多能够关联的*/
                        /*DocExplorer 变量的个数*/
typedef void (____(1)____)(struct OfficeDoc *, struct DocExplorer * );

struct DocExplorer{
    func update; /* DocExplorer 结构采用的更新函数*/
    /*其它的结构字段省略*/
};

struct OfficeDoc{
    _____(2)_____myObs[OBS_MAXNUM];
    /*存储所有与 OfficeDoc 相关联的 DocExplorer 结构指针*/
    int index; /*与 OfficeDoc 结构变量相关联的 DocExplorer 结构变量的个数*/
};

void attach(struct OfficeDoc *doc, struct DocExplorer *ob) {
    /*关联 Obersver 结构 ob 与 OfficeDoc 结构 doc*/
    int loop = 0;
    if (doc->index >= OBS_MAXNUM || ob == NULL) return;
    for(loop = 0; loop < doc->index; loop++){
        if(doc->myObs[loop] == ob) return;
        doc->myObs[doc->index] = ob;
        doc->index++;
    }

    void detach(struct OfficeDoc *doc, struct DocExplorer *ob){
        /*解除 doc 结构与 ob 结构间的关系*/
        int loop;
        if(ob==NULL) return;
        for(loop = 0; loop < doc->index; loop++){
            if(doc->myObs[loop] == ob){
```

```

        if(loop <= doc->index-2)
            doc->myObs[loop] = doc->myObs[_____(3)_____];
        doc->myObs[doc->index-1] = NULL;
        doc->index--;
        break;
    }
}

void update1(struct OfficeDoc *doc, struct DocExplorer *ob){
    /*更新 ob 结构的值，更新代码省略*/
}

void update2(struct OfficeDoc *doc, struct DocExplorer *ob){
    /*更新 ob 结构的值，更新代码省略*/
}

void notifyObs(struct OfficeDoc *doc){
    /*当 doc 结构的值发生变化时，通知与之关联的所有 DocExplorer 结构变量*/
    int loop;
    for(loop = 0; loop < doc->index; loop ++){
        (doc->myObs[loop])->update(_____(4)_____);
    }
}

void main(){
    struct OfficeDoc doc; /* 定义一 OfficeDoc 变量*/
    struct DocExplorer explorer1, explorer2; /* 定义两个 DocExplorer 变量*/
    /*初始化与 OfficeDoc 变量相关的 DocExplorer 变量个数为 0*/
    doc.index = 0;
    explorer1.update = update1; /*设置 explorer1 变量的更新函数*/
    explorer2.update = update2; /*设置 explorer2 变量的更新函数*/
    attach(&doc,&explorer1); /*关联 explorer1 与 doc 对象*/
    attach(&doc,&explorer2); /*关联 explorer2 与 doc 对象*/
    /* 其它代码省略 */
    _____(5)_____; /*通知与 OfficeDoc 相关的所有 DocExplorer 变量*/
    return;
}

```

全国计算机技术与软件专业技术资格（水平）考试

2006 年上半年 软件设计师 上午试卷

（考试时间 9:00~11:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2006 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
____(88)____月____(89)____日。

(88) A. 4 B. 5 C. 6 D. 7

(89) A. 18 B. 19 C. 20 D. 21

因为考试日期是“5 月 20 日”，故 (88) 选 B，(89) 选 C，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 C 填涂（参看答题卡）。

● 两个同符号的数相加或异符号的数相减，所得结果的符号位 SF 和进位标志 CF 进行__ (1) __运算为 1 时，表示运算的结果产生溢出。

- (1) A. 与 B. 或 C. 与非 D. 异或

● 若浮点数的阶码用移码表示，尾数用补码表示。两规格化浮点数相乘，最后对结果规格化时，右规的右移位数最多为__ (2) __位。

- (2) A. 1 B. 2 C. 尾数位数 D. 尾数位数-1

● 高速缓存 Cache 与主存间采用全相联地址映像方式，高速缓存的容量为 4MB，分为 4 块，每块 1MB，主存容量为 256MB。若主存读写时间为 30ns，高速缓存的读写时间为 3ns，平均读写时间为 3.27ns，则该高速缓存的命中率为__ (3) __%。若地址变换表如下所示，则主存地址为 8888888H 时，高速缓存地址为__ (4) __H。

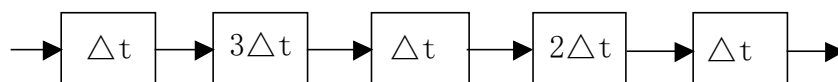
地址变换表	
0	38H
1	88H
2	59H
3	67H

- (3) A. 90 B. 95 C. 97 D. 99
 (4) A. 488888 B. 388888 C. 288888 D. 188888

● 若某计算机系统是由 500 个元器件构成的串联系统，且每个元器件的失效率均为 $10^{-7}/H$ ，在不考虑其他因素对可靠性的影响时，该计算机系统的平均故障间隔时间为__ (5) __小时。

- (5) A. 2×10^4 B. 5×10^4 C. 2×10^5 D. 5×10^5

● 某指令流水线由 5 段组成，各段所需要的时间如下图所示。



连续输入 10 条指令时的吞吐率为__ (6) __。

- (6) A. $10/70\Delta t$ B. $10/49\Delta t$ C. $10/35\Delta t$ D. $10/30\Delta t$

● 相对于 DES 算法而言，RSA 算法的__ (7) __，因此，RSA__ (8) __。

- (7) A. 加密密钥和解密密钥是不相同的 B. 加密密钥和解密密钥是相同的
 C. 加密速度比 DES 要高 D. 解密速度比 DES 要高
 (8) A. 更适用于对文件加密 B. 保密性不如 DES
 C. 可用于对不同长度的消息生成消息摘要 D. 可以用于数字签名

● 驻留在多个网络设备上的程序在短时间内同时产生大量的请求消息冲击某 Web 服务器，导致该服务器不堪重负，无法正常响应其他合法用户的请求，这属于 (9)。

- (9) A. 网上冲浪 B. 中间人攻击 C. DDoS 攻击 D. MAC 攻击

● 上海市标准化行政主管部门制定并发布的工业产品的安全、卫生要求的标准，在其行政区域内是 (10)。

- (10) A. 强制性标准 B. 推荐性标准 C. 自愿性标准 D. 指导性标准

● 小王购买了一个“海之久”牌活动硬盘，而且该活动硬盘还包含有一项实用新型专利，那么，小王享有 (11)。

- (11) A. “海之久” 商标专用权 B. 该盘的所有权
C. 该盘的实用新型专利权 D. 前三项权利之全部

● MPC (Multimedia PC) 与 PC 的主要区别是增加了 (12)。

- (12) A. 存储信息的实体 B. 视频和音频信息的处理能力
C. 光驱和声卡 D. 大容量的磁介质和光介质

● 人眼看到的任一彩色光都是亮度、色调和饱和度三个特性的综合效果，其中 (13) 反应颜色的种类。

- (13) A. 色调 B. 饱和度 C. 灰度 D. 亮度

● CD 上声音的采样频率为 44.1kHz，样本精度为 16b/s，双声道立体声，那么其未经压缩的数据传输率为 (14)。

- (14) A. 88.2 kb/s B. 705.6 kb/s C. 1411.2 kb/s D. 1536.0 kb/s

● 在软件项目管理中可以使用各种图形工具来辅助决策，下面对 Gantt 图的描述中，不正确的是 (15)。

- (15) A. Gantt 图表现各个活动的持续时间
B. Gantt 图表现了各个活动的起始时间
C. Gantt 图反映了各个活动之间的依赖关系
D. Gantt 图表现了完成各个活动的进度

● 耦合度描述了 (16)。

- (16) A. 模块内各种元素结合的程度
B. 模块内多个功能之间的接口
C. 模块之间公共数据的数量
D. 模块之间相互关联的程度

● 内聚是一种指标，表示一个模块(17)。

- (17) A. 代码优化的程度 B. 代码功能的集中程度
C. 完成任务的及时程度 D. 为了与其他模块连接所要完成的工作量

● 在软件项目开发过程中，评估软件项目风险时，(18)与风险无关。

- (18) A. 高级管理人员是否正式承诺支持该项目
B. 开发人员和用户是否充分理解系统的需求
C. 最终用户是否同意部署已开发的系统
D. 开发需要的资金是否能按时到位

● 开发专家系统时，通过描述事实和规则由模式匹配得出结论，这种情况下适用的开发语言是(19)。

- (19) A. 面向对象语言 B. 函数式语言 C. 过程式语言 D. 逻辑式语言

● 高级程序设计语言中用于描述程序中的运算步骤、控制结构及数据传输的是(20)。

- (20) A. 语句 B. 语义 C. 语用 D. 语法

● 为了解决进程间的同步和互斥问题，通常采用一种称为(21)机制的方法。若系统中有 5 个进程共享若干个资源 R，每个进程都需要 4 个资源 R，那么使系统不发生死锁的资源 R 的最少数目是(22)。

- (21) A. 调度 B. 信号量 C. 分派 D. 通讯
(22) A. 20 B. 18 C. 16 D. 15

● 在 UNIX 操作系统中，把输入/输出设备看作是(23)。

- (23) A. 普通文件 B. 目录文件 C. 索引文件 D. 特殊文件

● 某软盘有 40 个磁道，磁头从一个磁道移至另一个磁道需要 5ms。文件在磁盘上非连续存放，逻辑上相邻数据块的平均距离为 10 个磁道，每块的旋转延迟时间及传输时间分别为 100ms 和 25ms，则读取一个 100 块的文件需要(24)时间。

- (24) A. 17500ms B. 15000ms C. 5000ms D. 25000ms

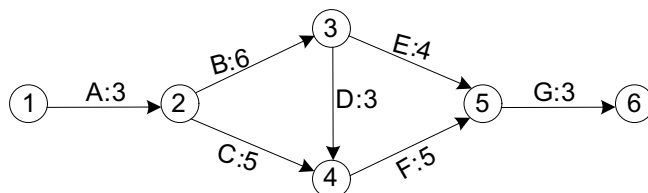
● 文件系统中，设立打开文件（Open）系统功能调用的基本操作是(25)。

- (25) A. 把文件信息从辅存读到内存
B. 把文件的控制管理信息从辅存读到内存
C. 把磁盘的超级块从辅存读到内存
D. 把文件的 FAT 表信息从辅存读到内存

● 最主要的软件质量度量指标有正确性、可维护性、完整性和可用性，下面关于这些质量指标的描述中正确的是（26）。

- (26) A. 软件维护的工作量比开发阶段的工作量小
B. 正确性用每千行代码的故障（fault）数来度量
C. 软件完整性是指软件功能与需求符合的程度
D. 可用性与用户的操作效率和用户对软件的主观评价有关

● 某工程计划图如下图所示，弧上的标记为作业编码及其需要的完成时间（天），作业 E 最迟应在第（27）天开始。



- (27) A. 7 B. 9 C. 12 D. 13

● 正式的技术评审 FTR(Formal Technical Review)是软件工程师组织的软件质量保证活动，下面关于 FTR 指导原则中不正确的是（28）。

- (28) A. 评审产品，而不是评审生产者的能力
B. 要有严格的评审计划，并遵守日程安排
C. 对评审中出现的问题要充分讨论，以求彻底解决
D. 限制参与者人数，并要求评审会之前做好准备

● 两个小组独立地测试同一个程序，第一组发现 25 个错误，第二组发现 30 个错误，在两个小组发现的错误中有 15 个是共同的，那么可以估计程序中的错误总数是（29）个。

- (29) A. 25 B. 30 C. 50 D. 60

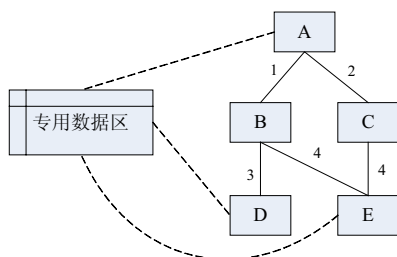
● 下面有关测试的说法正确的是（30）。

- (30) A. 测试人员应该在软件开发结束后开始介入
B. 测试主要是软件开发人员的工作
C. 要根据软件详细设计中设计的各种合理数据设计测试用例
D. 严格按照测试计划进行，避免测试的随意性

● 在绘制数据流图时，应遵循父图与子图平衡的原则，所谓平衡是指（31）。

- (31) A. 父图和子图都不得改变数据流的性质
B. 子图不改变父图数据流的一致性
C. 父图的输入/输出数据流与子图的输入/输出数据流一致
D. 子图的输出数据流完全由父图的输入数据流确定

● 下图中的程序由 A、B、C、D、E 5 个模块组成，下表中描述了这些模块之间的接口，每一个接口有一个编号。此外，模块 A、D 和 E 都要引用一个专用数据区。那么 A 和 E 之间耦合关系是 (32)。



编号	参数	返回值
1	数据项	数据项
2	数据项	数据项
3	功能码	无
4	无	列表

- (32) A. 公共耦合 B. 数据耦合 C. 内容耦合 D. 无耦合

● 在 C++ 语言中，若类 C 中定义了一个方法 `int f(int a, int b)`，那么方法 (33) 不能与该方法同时存在于类 C 中。

- (33) A. `int f(int x, int y)` B. `int f(float a, int b)`
 C. `float f(int x, float y)` D. `int f(int x, float y)`

● 已知 3 个类 O、P 和 Q，类 O 中定义了一个私有方法 F1、一个公有方法 F2 和一个受保护的方法 F3；类 P 和类 Q 是类 O 的派生类，其继承方式如下所示：

```
class P : protected O { ... };
class Q : public O { ... };
```

关于方法 F1 的描述中正确的是 (34)；关于方法 F2 的描述中正确的是 (35)；关于方法 F3 的描述中正确的是 (36)。

- (34) A. 方法 F1 无法被访问
 B. 只有在类 O 内才能访问方法 F1
 C. 只有在类 P 内才能访问方法 F1
 D. 只有在类 Q 内才能访问方法 F1
- (35) A. 类 O、P 和 Q 的对象都可以访问方法 F2
 B. 类 P 和 Q 的对象都可以访问方法 F2
 C. 类 O 和 Q 的对象都可以访问方法 F2
 D. 只有在类 P 内才能访问方法 F2
- (36) A. 类 O、P 和 Q 的对象都可以访问方法 F3
 B. 类 O、P 和 Q 的对象都不可以访问方法 F3
 C. 类 O 和 Q 的对象都可以访问方法 F3
 D. 类 P 和 Q 的对象都可以访问方法 F3

● 在面向对象软件开发过程中，采用设计模式__（37）__。

- (37) A. 允许在非面向对象程序设计语言中使用面向对象的概念
B. 以复用成功的设计和体系结构
C. 以减少设计过程创建的类的个数
D. 以保证程序的运行速度达到最优值

● 以下关于单身模式（Singleton）的描述中，正确的是__（38）__。

- (38) A. 它描述了只有一个方法的类的集合
B. 它能够保证一个类只产生一个唯一的实例
C. 它描述了只有一个属性的类的集合
D. 它能够保证一个类的方法只能被一个唯一的类调用

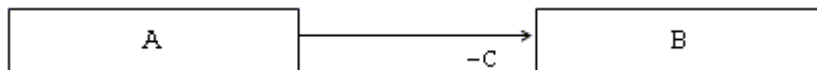
● 下列关于一个类的静态成员的描述中，不正确的是__（39）__。

- (39) A. 该类的对象共享其静态成员变量的值
B. 静态成员变量可被该类的所有方法访问
C. 该类的静态方法只能访问该类的静态成员变量
D. 该类的静态数据成员变量的值不可修改

● 在 MVC（模型/视图/控制器）模式中，视图部分描述的是__（40）__。

- (40) A. 将应用问题域中包含的抽象领域知识呈现给用户的方式
B. 应用问题域中所包含的抽象类
C. 用户界面对用户输入的响应方式
D. 应用问题域中包含的抽象领域知识

● 对于如下所示的 UML 类图，正确的描述是__（41）__。



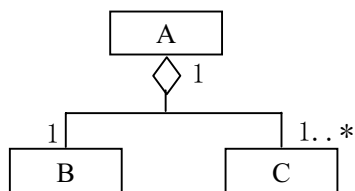
- (41) A. 类 B 的实例中包含了对类 C 的实例的引用
B. 类 A 的实例中包含了对类 B 的实例的引用
C. 类 A 的实例中包含了对类 C 的实例的引用
D. 类 B 的实例中包含了对类 A 的实例的引用

● UML 中关联的多重度是指__（42）__。

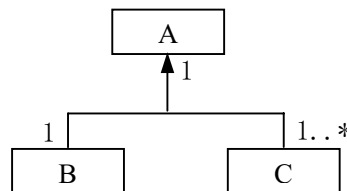
- (42) A. 一个类有多少个方法被另一个类调用
B. 一个类的实例能够与另一个类的多少个实例相关联
C. 一个类的某个方法被另一个类调用的次数
D. 两个类所具有的相同的方法和属性

● 已知 3 个类 A、B 和 C，其中类 A 由类 B 的一个实例和类 C 的 1 个或多个实例构成。能够正确表示类 A、B 和 C 之间关系的 UML 类图是 (43)。

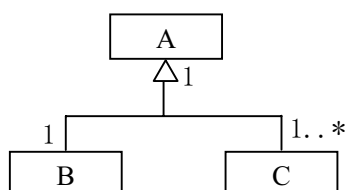
(43) A.



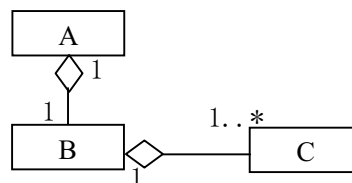
B.



C.



D.



● 对于下面的文法 $G[S]$ ，(44) 是其句子（从 S 出发开始推导）。

$G[S]: S \rightarrow M|(S,M) \quad M \rightarrow P|MP \quad P \rightarrow a|b|c|\dots|x|x|z$

(44) A. ((a,f)) B. ((fac,bb),g) C. (abc) D. (c,(da))

● 与逆波兰式 $ab+ -c*d$ 对应的中缀表达式是 (45)。

(45) A. $a-b-c*d$ B. $-(a+b)*c-d$ C. $-a+b*c-d$ D. $(a+b)*(-c-d)$

● 下面的 C 程序代码段在运行中会出现 (46) 错误。

```
int i = 0;
while (i < 10);
{ i = i + 1; }
```

(46) A. 语法 B. 类型不匹配 C. 变量定义 D. 动态语义

● 设有关系 R、S 如下所示，则关系代数表达式 $R \div S$ 的结果集为 (47)。

关系 R

A	B	C
a_1	b_1	c_1
a_1	b_2	c_1
a_2	b_2	c_2

关系 S

B	D
b_1	d_1
b_2	d_1

(47)

A.

A
a_1
a_1

B.

A
a_1

C.

A	C
a_1	c_1

D.

A	C
a_1	c_1
a_2	c_2

● 某数据库中有员工关系 E、产品关系 P、仓库关系 W 和库存关系 I，其中：员工关系 E (employeeID, name, department) 中的属性为：员工编号，姓名，部门；产品关系 P (productID, name, model, size, color) 中的属性为：产品编号，产品名称，型号，尺寸，颜色；仓库关系 W (warehouseID, name, address, employeeID) 中的属性为：仓库编号，仓库名称，地址，负责人编号；库存关系 I (warehouseID, productID, quantity) 中的属性为仓库编号，产品编号和产品数量。

a. 若要求仓库关系的负责人引用员工关系的员工编号，员工关系 E 的员工编号、仓库关系 W 的仓库编号和产品关系 P 的产品编号不能为空且惟一标识一个记录，并且仓库的地址不能为空，则依次要满足的完整性约束是__ (48) __。

- (48) A. 实体完整性、参照完整性、用户定义完整性
B. 参照完整性、实体完整性、用户定义完整性
C. 用户定义完整性、实体完整性、参照完整性
D. 实体完整性、用户定义完整性、参照完整性

b. 若需得到每种产品的名称和该产品的总库存量，则对应的查询语句为：

SELELCT name, SUM(quantity)

FROM P, I

WHERE __ (49) __

- (49) A. P.productID=I.productID;
B. P.productID=I.productID ORDER BY name;
C. P.productID=I.productID GROUP BY name;
D. P.productID=I.productID GROUP BY name, quantity;

c. 若需得到在所有仓库中都存在的产品的名称，则对应的查询语句为：

SELECT name FROM P

WHERE __ (50) __

(SELECT * FROM W

WHERE NOT EXISTS

(SELECT * FROM I

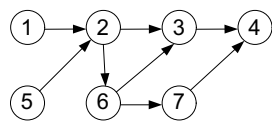
WHERE P.productID=I.productID AND W.warehouseID=I.warehouseID))

- (50) A. EXISTS B. NOT EXISTS C. IN D. NOT IN

● 对事务回滚的正确描述是__ (51) __。

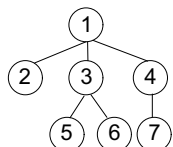
- (51) A. 将该事务对数据库的修改进行恢复
B. 将事务对数据库的更新写入硬盘
C. 跳转到事务程序的开头重新执行
D. 将事务中修改的变量值恢复到事务开始时的初值

● 拓扑序列是无环有向图中所有顶点的一个线性序列，图中任意路径中的各个顶点在该图的拓扑序列中保持先后关系，(52)为下图所示有向图的一个拓扑序列。

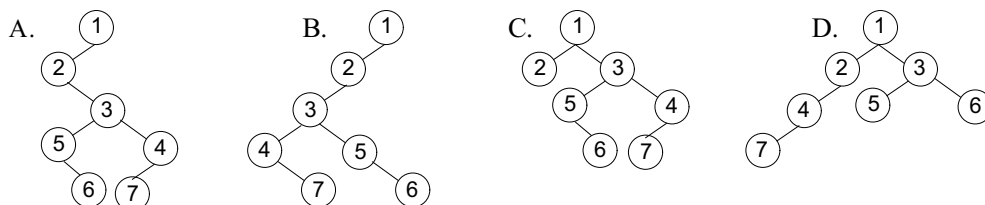


- (52) A. 1 2 3 4 5 6 7 B. 1 5 2 6 3 7 4 C. 5 1 2 6 3 4 7 D. 5 1 2 3 7 6 4

● 为便于存储和处理一般树结构形式的信息，常采用孩子-兄弟表示法将其转换成二叉树（左子关系表示父子、右子关系表示兄弟），与下图所示的树对应的二叉树是(53)。



(53)



● 给定一个有 n 个元素的有序线性表。若采用顺序存储结构，则在等概率前提下，删除其中的一个元素平均需要移动(54)个元素。

- (54) A. $\frac{(n+1)}{2}$ B. $\frac{n}{2}$ C. $\frac{(n-1)}{2}$ D. 1

● 在平衡二叉树中，(55)。

- (55) A. 任意结点的左、右子树结点数目相同
 B. 任意结点的左、右子树高度相同
 C. 任意结点的左右子树高度之差的绝对值不大于 1
 D. 不存在度为 1 的结点

● 在(56)存储结构中，数据结构中元素的存储地址与其关键字之间存在某种映射关系。

- (56) A. 顺序 (Sequence) B. 链表 (Link)
 C. 索引 (Index) D. 散列 (Hash)

● 对于求取两个长度为 n 的字符串的最长公共子序列 (LCS) 问题, 利用 (57) 策略可以有效地避免子串最长公共子序列的重复计算, 得到时间复杂度为 $O(n^2)$ 的正确算法。串 $\langle 1,0,0,1,0,1,0,1 \rangle$ 和 $\langle 0,1,0,1,1,0,1,1 \rangle$ 的最长公共子序列的长度为 (58)。

- (57) A. 分治 B. 贪心 C. 动态规划 D. 分支-限界
(58) A. 3 B. 4 C. 5 D. 6

● 设某算法的计算时间可用递推关系式 $T(n) = 2T(n/2) + n$ 表示, 则该算法的时间复杂度为 (59)。

- (59) A. $O(\lg n)$ B. $O(n \lg n)$ C. $O(n)$ D. $O(n^2)$

● (60) 在其最好情况下的算法时间复杂度为 $O(n)$ 。

- (60) A. 插入排序 B. 归并排序 C. 快速排序 D. 堆排序

● 与多模光纤相比较, 单模光纤具有 (61) 等特点。

- (61) A. 较高的传输率、较长的传输距离、较高的成本
B. 较低的传输率、较短的传输距离、较高的成本
C. 较高的传输率、较短的传输距离、较低的成本
D. 较低的传输率、较长的传输距离、较低的成本

● CDMA 系统中使用的多路复用技术是 (62)。我国自行研制的移动通信 3G 标准是 (63)。

- (62) A. 时分多路 B. 波分多路
C. 码分多址 D. 空分多址
(63) A. TD-SCDMA B. WCDMA
C. CDMA2000 D. GPRS

● “`<title style="italic">science</title>`” 是 XML 中一个元素的定义, 其中元素的内容是 (64)。

- (64) A. title B. style C. italic D. science

● 某校园网用户无法访问外部站点 210.102.58.74, 管理人员在 Windows 操作系统下可以使用 (65) 判断故障发生在校园网内还是校园网外。

- (65) A. ping 210.102.58.74 B. tracert 210.102.58.74
C. netstat 210.102.58.74 D. arp 210.102.58.74

● Originally introduced by Netscape Communications, (66) are a general mechanism which HTTP Server side applications, such as CGI (67), can use to both store and retrieve information on the HTTP (68) side of the connection. Basically, Cookies can be used to compensate for the (69) nature of HTTP. The addition of a simple, persistent, client-side state significantly extends the capabilities of WWW-based (70).

- | | | | |
|------------------|-----------------|----------------|---------------|
| (66) A. Browsers | B. Cookies | C. Connections | D. Scripts |
| (67) A. graphics | B. processes | C. scripts | D. texts |
| (68) A. Client | B. Editor | C. Creator | D. Server |
| (69) A. fixed | B. flexible | C. stable | D. stateless |
| (70) A. programs | B. applications | C. frameworks | D. constrains |

● WebSQL is a SQL-like (71) language for extracting information from the web. Its capabilities for performing navigation of web (72) make it a useful tool for automating several web-related tasks that require the systematic processing of either all the links in a (73), all the pages that can be reached from a given URL through (74) that match a pattern, or a combination of both. WebSQL also provides transparent access to index servers that can be queried via the Common (75) Interface.

- | | | | |
|-------------------|----------------|------------------|----------------|
| (71) A. query | B. transaction | C. communication | D. programming |
| (72) A. browsers | B. servers | C. hypertexts | D. clients |
| (73) A. hypertext | B. page | C. protocol | D. operation |
| (74) A. paths | B. chips | C. tools | D. directories |
| (75) A. Router | B. Device | C. Computer | D. Gateway |

全国计算机技术与软件专业技术资格（水平）考试

2006 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2006 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是(1)月(2)日。

因为正确的解答是“5 月 20 日”，故在答题纸的对应栏内写上“5”和“20”（参看下表）。

例题	解答栏
(1)	5
(2)	20

试题一（15分）

阅读下列说明以及数据流程图，回答问题1、问题2和问题3，将解答填入答题纸的对应栏内。

【说明】

某学校建立了一个网上作业提交与管理系统，基本功能描述如下：

（1）帐号和密码。任课老师用帐号和密码登录系统后，提交所有选课学生的名单。系统自动为每个选课学生创建登录系统的帐号和密码。

（2）作业提交。学生使用帐号和密码登录系统后，可以向系统申请所选课程的作业。系统首先检查学生的当前状态，如果该学生还没有做过作业，则从数据库服务器申请一份作业。若申请成功，则显示需要完成的作业。学生需在线完成作业，单击“提交”按钮上交作业。

（3）在线批阅。系统自动在线批改作业，显示作业成绩，并将该成绩记录在作业成绩统计文件中。

【问题1】（3分）

如果将数据库服务器（记为DB）作为一个外部实体，那么在绘制该系统的数据流程图时，还应有哪些外部实体和数据存储？

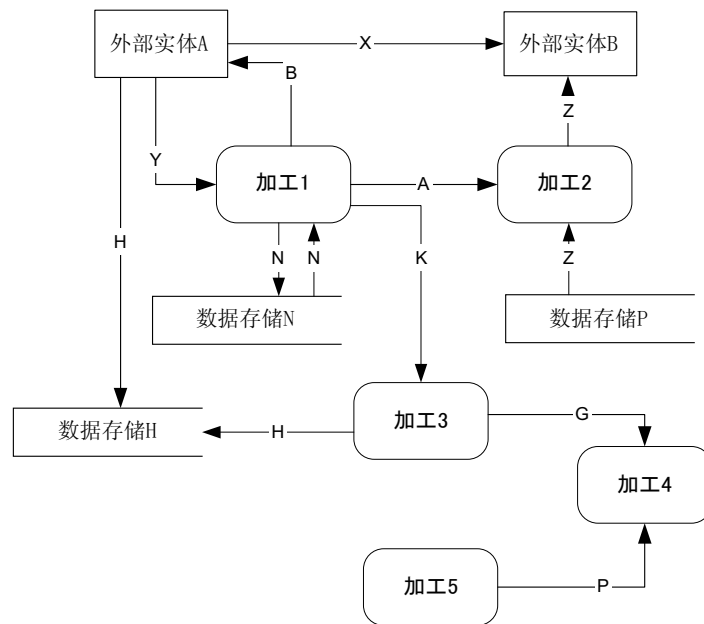
【问题2】（7分）

根据说明结合问题1的解答，指出在该系统的顶层数据流图中应有哪些数据流。请采用说明中的词汇给出这些数据流的起点、终点以及数据流名称，下表给出了数据流的部分信息，请填写空缺处。

序号	起点	终点	数据流名称
1	____(1)____	网上作业提交与管理系统	作业申请
2	____(2)____	网上作业提交与管理系统	提交的作业
3	网上作业提交与管理系统	____(3)____	需完成的作业
4	网上作业提交与管理系统	____(4)____	____(5)____
5	网上作业提交与管理系统	____(6)____	作业申请
6	网上作业提交与管理系统	____(7)____	____(8)____
7	____(9)____	网上作业提交与管理系统	选课学生名单
8	____(10)____	网上作业提交与管理系统	____(11)____
9	____(12)____	网上作业提交与管理系统	帐号和密码
10	____(13)____	网上作业提交与管理系统	帐号和密码

【问题3】(5分)

根据数据流图的设计原则，阅读下图所示的数据流图，找出其中的错误之处。



试题二(15 分)

阅读下列说明以及UML类图，回答问题1、问题2和问题3，将解答填入答题纸的对应栏内。

【说明】

某客户信息管理系统中保存着两类客户的信息：

(1) 个人客户。对于这类客户，系统保存了其客户标识（由系统生成）和基本信息（包括姓名、住宅电话和 email）。

(2) 集团客户。集团客户可以创建和管理自己的若干名联系人。对于这类客户，系统除了保存其客户标识（由系统生成）之外，也保存了其联系人的信息。联系人的信息包括姓名、住宅电话、email、办公电话以及职位。

该系统除了可以保存客户信息之外，还具有以下功能：

- (1) 向系统中添加客户（addCustomer）；
- (2) 根据给定的客户标识，在系统中查找该客户（getCustomer）；
- (3) 根据给定的客户标识，从系统中删除该客户（removeCustomer）；
- (4) 创建新的联系人（addContact）；
- (5) 在系统中查找指定的联系人（getContact）；
- (6) 从系统中删除指定的联系人（removeContact）。

该系统采用面向对象方法进行开发。在面向对象分析阶段，根据上述描述，得到如表 2-1 所示的类。

表 2-1

类名	说明
CustomerInformationSystem	客户信息管理系统
IndividualCustomer	个人客户
InstitutionalCustomer	集团客户
Contact	联系人

描述该客户信息管理系统的 UML 类图如图 2-1 所示。

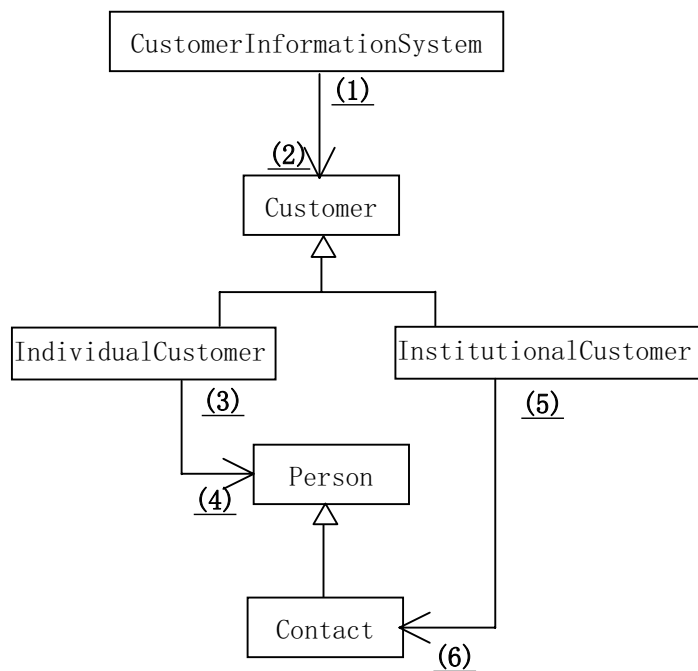


图 2-1 客户信息管理系统 UML 类图

【问题 1】（3 分）

请使用说明中的术语，给出图 2-1 中类 Customer 和类 Person 的属性。

【问题 2】（6 分）

识别关联的多重度是面向对象建模过程中的一个重要步骤。根据说明中给出的描述，完成图中的 (1)~(6)。

【问题 3】（6 分）

根据说明中的叙述，抽象出如表 2-2 所示的方法，请指出图 2-1 中的类 CustomerInformationSystem 和 InstitutionalCustomer 应分别具有其中的哪些方法。

表 2-2

功能描述	方法名
向系统中添加客户	addCustomer
根据给定的客户标识，在系统中查找该客户	getCustomer
根据给定的客户标识，从系统中删除该客户	removeCustomer
创建新的联系人	addContact
在系统中查找指定的联系人	getContact
从系统中删除指定的联系人	removeContact

试题三(15分)

阅读下列说明，回答问题1、问题2和问题3，将解答填入答题纸的对应栏内。

【说明】

某单位资料室需要建立一个图书管理系统，初步的需求分析结果如下：

(1) 资料室有图书管理员若干名，他们负责已购入图书的编目和借还工作，每名图书管理员的信息包括工号和姓名；

(2) 读者可在阅览室读书，也可通过图书流通室借还图书，读者信息包括读者 ID、姓名、电话和 Email，系统为不同读者生成不同的读者 ID；

(3) 每部书在系统中对应惟一的一条图书在版编目数据（CIP，以下简称书目），书目的基本信息包括 ISBN 号、书名、作者、出版商、出版年月，以及本资料室拥有该书的册数（以下简称册数），不同书目的 ISBN 号不相同；

(4) 资料室对于同一书目的图书可拥有多册（本），图书信息包括图书 ID、ISBN 号、存放位置、当前状态，每一本书在系统中被赋予惟一的图书 ID；

(5) 一名读者最多只能借阅十本图书，且每本图书最多只能借两个月，读者借书时需由图书管理员登记读者 ID、所借图书 ID、借阅时间和应还时间，读者还书时图书管理员在对应的借书信息中记录归还时间；

(6) 当某书目的可借出图书的数量为零时，读者可以对其进行预约登记，即记录读者 ID、需要借阅的图书的 ISBN 号、预约时间。

某书目的信息如表 3-1 所示，与该书目对应的图书信息如表 3-2 所示。

表 3-1 书目信息

书 名	作 者	出版商	ISBN 号	出版年月	册数	经办人
《数据结构》	严蔚敏 吴伟民	清华大学出版社	ISBN7-302-02368-9	1997.4	4	01

表 3-2 图书信息

图书 ID	ISBN 号	存放位置	状态	经办人
C832.1	ISBN7-302-02368-9	图书流通室	已借出	01
C832.2	ISBN7-302-02368-9	图书阅览室	不外借	01
C832.3	ISBN7-302-02368-9	图书流通室	未借出	01
C832.4	ISBN7-302-02368-9	图书流通室	已预约	01

系统的主要业务处理如下：

(1) 入库管理：图书购进入库时，管理员查询本资料室的书目信息，若该书的书目尚未建立，则由管理员编写该书的书目信息并录入系统，然后编写并录入图书信息；否则，修改该书目的册数，然后编写并录入图书信息，对于进入流通室的书，其初始状态为“未借出”，而送入阅览室的书的状态始终为“不外借”。

(2) 借书管理：读者借书时，若有，则由管理员为该读者办理借书手续，并记录该读者的借书信息，同时将借出图书的状态修改为“已借出”。

(3) 预约管理：若图书流通室没有读者要借的书，则可为该读者建立预约登记，需要记录读者 ID、书的 ISBN 号、预约时间和预约期限（最长为 10 天）。一旦其他读者归还这种书，就自动通知该预约读者。系统将自动清除超出预约期限的预约记录并修改相关信息。

(4) 还书管理：读者还书时，则记录相应借还信息中的“归还时间”，对于超期归还者，系统自动计算罚金（具体的计算过程此处省略）。系统同时自动查询预约登记表，若存在其他读者预约该书的记录，则将该图书的状态修改为“已预约”，并将该图书 ID 写入相应的预约记录中（系统在清除超出预约期限的记录时解除该图书的“已预约”状态）；否则，将该图书的状态修改为“未借出”。

(5) 通知处理：对于已到期且未归还的图书，系统通过 Email 自动通知读者；若读者预约的书已到，系统则自动通过 Email 通知该读者来办理借书手续。

【问题 1】（4 分）

根据以上说明设计的实体联系图如图 3-1 所示，请指出读者与图书、书目与读者、书目与图书之间的联系类型。

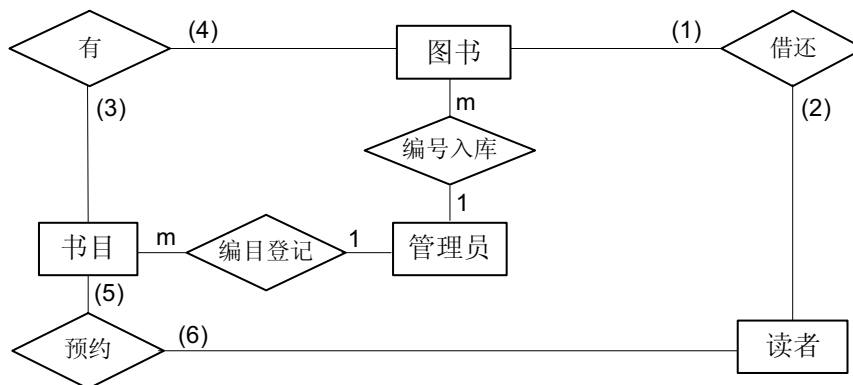


图 3-1 图书管理系统的实体联系图

【问题 2】（4 分）

该图书管理系统的主要关系模式如下，请补充“借还记录”和“预约登记”关系中的空缺。

管理员(工号, 姓名)

读者(读者 ID, 姓名, 电话, Email)

书目(ISBN 号, 书名, 作者, 出版商, 出版年月, 册数, 经办人)

图书(图书 ID, ISBN 号, 存放位置, 状态, 经办人)

借还记录(_____(a)_____, 借出时间, 应还时间, 归还时间)

预约登记(_____(b)_____, 预约时间, 预约期限, 图书 ID)

注：时间格式为“年.月.日 时:分:秒”

【问题 3】（7 分）

请指出问题 2 中给出的读者、书目关系模式的主键，以及图书、借还记录和预约登记关系模式的主键和外键。

试题四(15分)

阅读以下说明，回答问题 1、问题 2 和问题 3，将解答填入答题纸的对应栏内。

【说明】

某单位正在使用一套 C/S 模式的应用软件系统，现在需要升级为 B/S 应用模式，但需要保持业务的连续性。开发人员提出用 Web Service 作为中间层的接口进行开发。

【问题 1】(6 分)

请用 120 字以内文字，从业务的继承性、升级成本（时间、工作量）和扩展性三个方面简要说明开发人员所提方案的优点。

【问题 2】(3 分)

Web Service 的三个基本技术是 WSDL、SOAP、UDDI，它们都是以 XML 为基础定义的。请用 120 字以内文字，简要说明 WSDL、SOAP 和 UDDI 的作用。

【问题 3】(6 分)

服务注册中心、服务提供者和服务请求者之间的交互和操作构成了 Web Service 的体系结构，如图 4-1 所示。请用 180 字以内文字，说明这三者的主要功能及其交互过程。

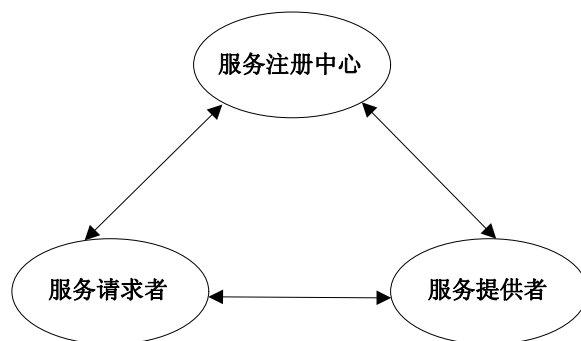


图 4-1 Web Service 的工作模式

试题五(15 分)

阅读下列说明、图和 C 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明 5-1】

B 树是一种多叉平衡查找树。一棵 m 阶的 B 树，或为空树，或为满足下列特性的 m 叉树：

- ① 树中每个结点至多有 m 棵子树；
- ② 若根结点不是叶子结点，则它至少有两棵子树；
- ③ 除根之外的所有非叶子结点至少有 $\lceil m/2 \rceil$ 棵子树；
- ④ 所有的非叶子结点中包含下列数据信息

$(n, A_0, K_1, A_1, K_2, A_2, \dots, K_n, A_n)$

其中： $K_i (i=1, 2, \dots, n)$ 为关键字，且 $K_i < K_{i+1} (i=1, 2, \dots, n-1)$ ； $A_i (i=0, 1, \dots, n)$ 为指向子树根结点的指针，且指针 A_{i-1} 所指子树中所有结点的关键字均小于 K_i ， A_{i+1} 所指子树中所有结点的关键字均大于 K_i ， n 为结点中关键字的数目。

⑤ 所有的叶子结点都出现在同一层次上，并且不带信息（可以看作是外部结点或查找失败的结点，实际上这些结点不存在，指向这些结点的指针为空）。

例如，一棵 4 阶 B 树如图 5-1 所示（结点中关键字的数目省略）。

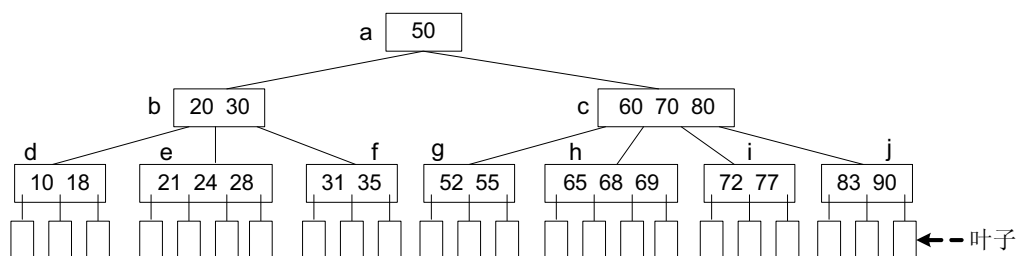


图 5-1 4 阶 B 树示例

B 树的阶 M 、bool 类型、关键字类型及 B 树结点的定义如下：

```
#define M 4 /*B 树的阶*/
typedef enum {FALSE = 0, TRUE = 1} bool;
typedef int ElemKeyType;
typedef struct BTreeNode{
    int numkeys; /*结点中关键字的数目*/
    struct BTreeNode *parent; /*指向父结点的指针，树根的父结点指针为空*/
    struct BTreeNode *A[M]; /*指向子树结点的指针数组*/
    ElemKeyType K[M]; /*存储关键字的数组，K[0]闲置不用*/
}BTreeNode;
```

函数 SearchBtree(BTreeNode* root, ElemKeyType akey, BTreeNode **ptr)的功能是：在给定的一棵 M 阶 B 树中查找关键字 akey 所在结点，若找到则返回 TRUE，否则返回 FALSE。其中, root 是指向该 M 阶 B 树根结点的指针，参数 ptr 返回 akey 所在结点的指针，若 akey 不在该 B 树中，则 ptr 返回查找失败时空指针所在结点的指针。例如，在图 5-1 所示的 4 阶 B 树中查找关键字 25 时，ptr 返回指向结点 e 的指针。

注：在结点中查找关键字 akey 时采用二分法。

【函数 5-1】

```
bool SearchBtree(BTreeNode* root, ElemKeyType akey, BTreeNode **ptr)
{
    int lw, hi, mid;
    BTreeNode *p = root;
    *ptr = NULL;
    while ( p ) {
        lw = 1;    hi = ____ (1) ____;
        while (lw <= hi) {
            mid = (lw + hi) / 2;
            if (p -> K[mid] == akey) {
                *ptr = p;
                return TRUE;
            }
            else
                if (____ (2) ____ )
                    hi = mid - 1;
                else
                    lw = mid + 1;
        }
        *ptr = p;
        p = ____ (3) ____;
    }
    return FALSE;
}
```

【说明 5-2】

在 M 阶 B 树中插入一个关键字时，首先在最接近外部结点的某个非叶子结点中增加一个关键字，若该结点中关键字的个数不超过 M-1，则完成插入；否则，要进行结点的“分裂”处理。所谓“分裂”，就是把结点中处于中间位置上的关键字取出来并插入其父结点中，然后以该关键字为分界线，把原结点分成两个结点。“分裂”过程可能会一直持续到树根，若树根结点也需要分裂，则整棵树的高度增 1。

例如，在图 5-1 所示的 B 树中插入关键字 25 时，需将其插入结点 e 中，由于 e 中已经有 3 个关键字，因此将关键字 24 插入结点 e 的父结点 b，并以 24 为分界线将结点 e 分裂为 e1 和 e2 两个结点，结果如图 5-2 所示。

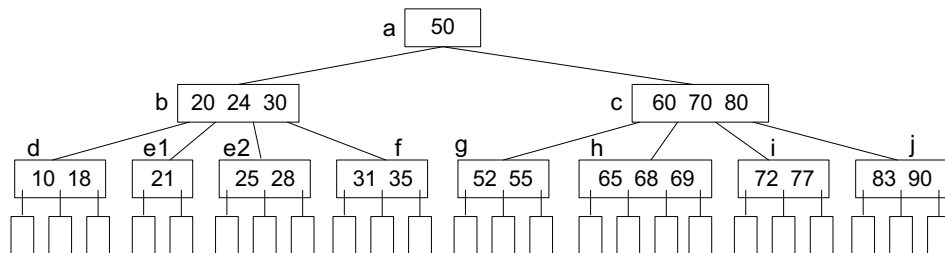


图 5-2 在图 5-1 所示的 4 阶 B 树中插入关键字 25 后的 B 树

函数 `Isgrowing(BTreeNode* root, ElemKeyType akey)` 的功能是：判断在给定的 M 阶 B 树中插入关键字 `akey` 后，该 B 树的高度是否增加，若增加则返回 `TRUE`，否则返回 `FALSE`。其中，`root` 是指向该 M 阶 B 树根结点的指针。

在函数 `Isgrowing` 中，首先调用函数 `SearchBtree`（即函数 5-1）查找关键字 `akey` 是否在给定的 M 阶 B 树中，若在则返回 `FALSE`（表明无需插入关键字 `akey`，树的高度不会增加）；否则，通过判断结点中关键字的数目考察插入关键字 `akey` 后该 B 树的高度是否增加。

【函数 5-2】

`bool Isgrowing(BTreeNode* root, ElemKeyType akey)`

```

{
    BTreeNode *t, *f;
    if ( !SearchBtree(__(4)__) ) {
        t = f;
        while (__(5)__) {
            t = t -> parent;
        }
        if ( !t )
            return TRUE;
    }
    return FALSE;
}

```

试题六（15 分）

阅读下列说明、图和 C++代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某订单管理系统的部分 UML 类图如图 6-1 所示。

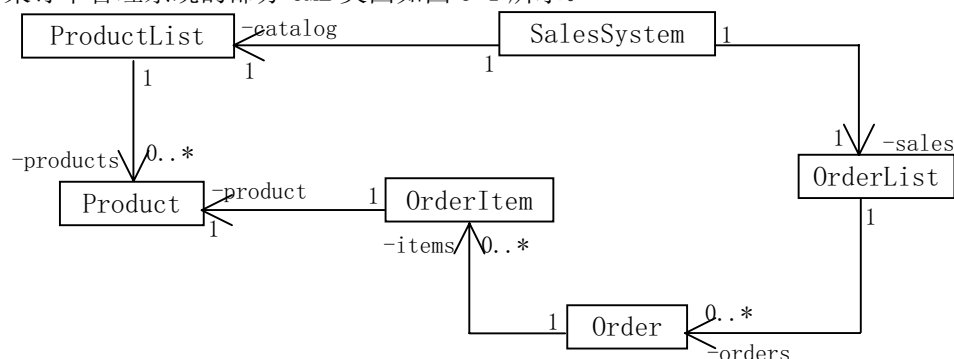


图 6-1 订单管理系统的部分 UML 类图

图 6-1 中，Product 表示产品，ProductList 表示产品目录，Order 表示产品订单，OrderItem 表示产品订单中的一个条目，OrderList 表示订单列表，SalesSystem 提供订单管理系统的操作接口。

请完善类 Order 的成员函数 getOrderedAmount() 和类 SalesSystem 的 statistic() 方法，各个类的属性及部分方法定义参见下面的 C++代码。

【C++代码】

```
class Product {                                //产品类
private:
    string pid;                                //产品识别码
    string description;                        //产品描述
    double price;                              //产品单价
public:
    void setProductPrice(double price);       //设置产品单价
    string getProductId();                     //获取产品识别码
    string getProductDescription ();           //获取产品描述
    double getProductPrice();                  //获取产品单价
    //其他成员省略
};

class ProductList {                            //产品列表类
private:
    vector <Product> products;
public:
    ProductList();
```

```

    Product getProductByIndex(int i);           //获得产品列表中的第 i 件产品
    void addProduct(Product t);                //在产品列表中加入一件产品
    Product * getProductByID(string pid);       //获得识别码为 pid 的产品指针
    unsigned int getProductAmount();            //获得产品列表中的产品数量
};

class OrderItem {                               //订单条目类
private:
    Product *productPtr;                        //指向被订购产品的指针
    int quantity;                              //订购数量
public:
    OrderItem (Product *,int);
    Product * getProductptr ();                //获取指向被订购产品的指针
    int getQuantity ();                        //获取被订购产品的数量
};

class Order {                                   //订单类
private:
    unsigned int orderid;                      //订单识别号
    vector<OrderItem> items;                   //订单内容(订单项)
public:
    Order(unsigned int orderid);
    //获得识别码为 tid 的产品在当前订单中被订购的数量
    int getOrderedAmount(string tid);
    void additem(Product *productPtr,unsigned int n); //在订单中增加一个订单项
    //其他成员省略
};

class OrderList {                             //订单列表类
private:
    vector<Order> orders;
public:
    OrderList();
    //Begin() 返回指向订单列表第一个元素的迭代器(指针)
    virtual vector<Order>::iterator OrderList::Begin();
    //End() 返回指向订单列表最后一个元素之后的迭代器 (指向一个不存在的元素)
    virtual vector<Order>::iterator OrderList::End();
    void addOrder(Order t);                    //在订单列表中加入一份订单
    //其他成员省略
};

```

```

class SalesSystem{
private:
    ProductList catalog;    //产品目录
    OrderList sales;        //订单列表
public:
    SalesSystem();
    void statistic();        //统计所有产品的订购情况
    //其他成员省略
};

//在订单中查找识别码为 tid 的产品的订购数量，若该产品没有被订购，则返回 0
int Order::getOrderedAmount(string tid)
{
    for (int k = 0; k < items.size(); k++) {
        if ( ____ (1) ____ == tid)
            return ____ (2) ____;
    }
    return 0;
}

// 方法 statistic() 依次统计产品目录中每个产品的订购总量，并打印输出
// 每个产品的识别码、描述、订购总量和订购金额
void SalesSystem::statistic()
{
    unsigned int k, t, ordered_qty = 0;
    vector<Order>::iterator it;    Product p;
    cout << "产品识别码\t 描述\t\t 订购数量\t 金额" << endl;

    for (k = 0; k < catalog.getProductAmount(); k++) { //遍历产品列表
        p = ____ (3) ____;    //从产品列表取得一件产品信息存入变量 p
        ordered_qty = 0;
        //通过迭代器变量 it 遍历订单列表中的每一份订单
        for (it = sales.Begin(); ____ (4) ____; it++) {
            //根据产品识别码获得产品 p 在当前订单中被订购的数量
            t = ____ (5) ____ (p.getProductId());
            ordered_qty += t;
        }
        cout << p.getProductId() << "\t\t" << p.getProductDescription() << "\t\t";
        cout << ordered_qty << "\t\t" << p.getProductPrice() * ordered_qty << endl;
    }
}

```


试题七（15 分）

阅读下列说明、图以及 Java 程序，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某订单管理系统的部分 UML 类图如图 7-1 所示。

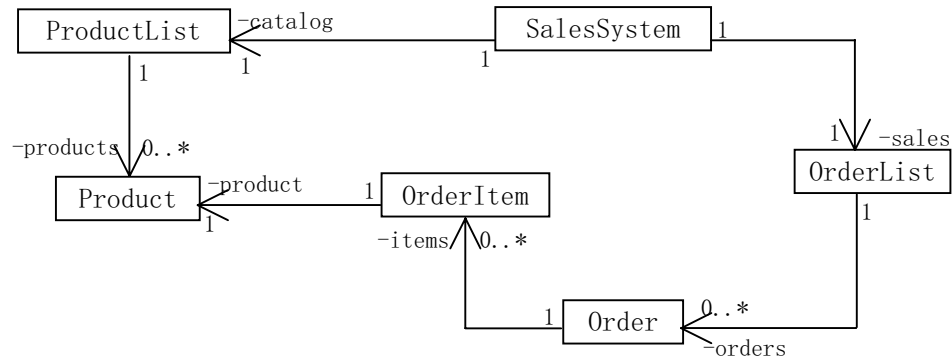


图 7-1 订单管理系统的部分 UML 类图

图 7-1 中，Product 表示产品，ProductList 表示所销售产品的列表，Order 表示产品订单，OrderItem 表示产品订单中的一个条目，OrderList 表示订单列表，SalesSystem 提供订单管理系统的操作接口。各个类的部分属性和方法说明如表 7-1 所示。

表 7-1

类	成员	说明
ProductList	ArrayList<Product> products	
Product	String code	产品编号
	String description	产品描述
	double price	产品单价
	Boolean equals(Object object)	若两个产品相同则返回 true，否则返回 false
OrderItem	Product product	订单项中的产品
	int quantity	产品的订购数量
	Product getProduct()	获取订单项中的产品
Order	ArrayList<OrderItem> items	订单中包含的订单项
OrderList	ArrayList<Order> orders	订单
	void addOrder(Order order)	向订单列表中添加新订单
	int getNumberOfOrders()	获取订单列表中的订单总数
SalesSystem	ProductList catalog	产品目录
	OrderList sales	订单列表
	void statistic()	依次统计产品目录中每个产品的订购总量，并打印出每个产品的编号、说明、订购总量和订购金额

可以使用类 `java.util.ArrayList<E>` 来实现对象的聚集关系，如图 7-1 中 `OrderList` 与 `Order` 之间的聚集关系。

`for-each` 循环提供了一种遍历对象集合的简单方法。在 `for-each` 循环中，可以指定需要遍历的对象集合以及用来接收集合中每个元素的变量，其语法如下：

`for( 用来接收集合中元素的变量 : 需要遍历的对象集合)`

如果要使用 `for-each` 循环来遍历对象集合，那么包含该对象集合的类必须实现接口 `java.util.Iterable<T>`。

Java 程序 7-1 和 Java 程序 7-2 分别给出了类 `OrderList` 和方法 `statistic` 的 Java 代码。

【Java 程序 7-1】

```
import java.util.*;
public class OrderList ____ (1) ____ {
    private ArrayList<Order> orders;
    public OrderList() {
        this.orders = new ArrayList<Order>();
    }
    public void addOrder(Order order) {
        this.orders.add(order);
    }
    public Iterator<Order> iterator() {
        return ____ (2) ____;
    }
    public int getNumberOfOrders() {
        return this.orders.size();
    }
}
```

【Java 程序 7-2】

```
import java.util.*;
public class SalesSystem {
    private ProductList catalog;
    private OrderList sales;
    private static PrintWriter stdOut = new PrintWriter(System.out, true);
    public void statistic() {
        for (Product product :__(3)_) {
            int number = 0;
            for (Order order :__(4)_) {
                for (__(5): order) {
                    if (product.equals(item.getProduct()))
                        number += item.getQuantity();
                }
            }
            stdOut.println(product.getCode() + " "
                            + product.getDescription() + " "
                            + number + " " + number * product.getPrice());
        }
    }
    // 其余的方法未列出
}
```

全国计算机技术与软件专业技术资格（水平）考试

2006 年下半年 软件设计师 上午试卷

（考试时间 9：00～11：30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2006 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
____（88）____月____（89）____日。

（88） A. 9 B. 10 C. 11 D. 12

（89） A. 4 B. 5 C. 6 D. 7

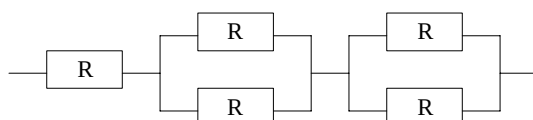
因为考试日期是“11 月 4 日”，故（88）选 C，（89）选 A，应在答题卡序号 88 下对 C 填涂，在序号 89 下对 A 填涂（参看答题卡）。

● 若内存按字节编址，用存储容量为 $32K \times 8$ 比特的存储器芯片构成地址编号 A0000H 至 DFFFFH 的内存空间，则至少需要 (1) 片。

- (1) A. 4 B. 6 C. 8 D. 10

● 某计算机系统由下图所示的部件构成，假定每个部件的千小时可靠度 R 均为 0.9，则该系统的千小时可靠度约为 (2)。

- (2) A. 0.882 B. 0.951 C. 0.9 D. 0.99



● 设指令由取指、分析、执行 3 个子部件完成，每个子部件的工作周期均为 Δt ，采用常规标量单流水线处理机。若连续执行 10 条指令，则共需时间 (3) Δt 。

- (3) A. 8 B. 10 C. 12 D. 14

● 某计算机的时钟频率为 400MHz，测试该计算机的程序使用 4 种类型的指令。每种指令的数量及所需指令时钟数(CPI)如下表所示，则该计算机的指令平均时钟数为 (4)；该计算机的运算速度约为 (5) MIPS。

指令类型	指令数目 (条)	每条指令需时钟数
1	160000	1
2	30000	2
3	24000	4
4	16000	8

- (4) A. 1.85 B. 1.93 C. 2.36 D. 3.75

- (5) A. 106.7 B. 169.5 C. 207.3 D. 216.2

● 某计算机指令字长为 16 位，指令有双操作数、单操作数和无操作数 3 种格式，每个操作数字段均用 6 位二进制表示，该指令系统共有 m 条 ($m < 16$) 双操作数指令，并存在无操作数指令。若采用扩展操作码技术，那么最多还可设计出 (6) 条单操作数指令。

- (6) A. 2^6 B. $(2^4 - m) \times 2^6 - 1$
 C. $(2^4 - m) \times 2^6$ D. $(2^4 - m) \times (2^6 - 1)$

● 以下不属于网络安全控制技术的是 (7)。

- (7) A. 防火墙技术 B. 访问控制技术
 C. 入侵检测技术 D. 差错控制技术

● “冲击波”病毒属于（8）类型的病毒，它利用 Windows 操作系统的（9）漏洞进行快速传播。

- (8) A. 蠕虫 B. 文件 C. 引导区 D. 邮件
(9) A. CGI 脚本 B. RPC C. DNS D. IMAP

● （10） 确定了标准体制和标准化管理体制，规定了制定标准的对象与原则以及实施标准的要求，明确了违法行为的法律责任和处罚办法。

- (10) A. 标准化 B. 标准 C. 标准化法 D. 标准与标准化

● 某开发人员不顾企业有关保守商业秘密的要求，将其参与该企业开发设计的应用软件的核心程序设计技巧和算法通过论文向社会发表，那么该开发人员的行为（11）。

- (11) A. 属于开发人员权利不涉及企业权利 B. 侵犯了企业商业秘密权
 C. 违反了企业的规章制度但不侵权 D. 未侵犯权利人软件著作权

● 计算机要对声音信号进行处理时，必须将它转换成为数字声音信号。最基本的声音信号数字化方法是取样—量化法。若量化后的每个声音样本用 2 个字节表示，则量化分辨率是（12）。

- (12) A. 1/2 B. 1/1024 C. 1/65536 D. 1/131072

● 某幅图像具有 640×480 个像素点，若每个像素具有 8 位的颜色深度，则可表示（13）种不同的颜色，经 5:1 压缩后，其图像数据需占用（14）（Byte）的存储空间。

- (13) A. 8 B. 256 C. 512 D. 1024
(14) A. 61440 B. 307200 C. 384000 D. 3072000

● 常见的软件开发模型有瀑布模型、演化模型、螺旋模型、喷泉模型等。其中（15）模型适用于需求明确或很少变更的项目，（16）模型主要用来描述面向对象的软件开发过程。

- (15) A.瀑布模型 B. 演化模型 C. 螺旋模型 D. 喷泉模型
(16) A.瀑布模型 B. 演化模型 C. 螺旋模型 D. 喷泉模型

● 软件能力成熟度模型（CMM）是目前国际上最流行、最实用的软件生产过程标准和软件企业成熟度的等级认证标准。该模型将软件能力成熟度自低到高依次划分为初始级、可重复级、已定义级、已管理级、优化级。从（17）开始，要求企业建立基本的项目管理过程的政策和管理规程，使项目管理工作有章可循。

- (17) A.初始级 B. 可重复级 C. 已定义级 D. 已管理级

● 软件项目开发成本的估算依据，通常是开发成本估算模型。常用的模型主要有：

- ① IBM 模型 ② Putnam 模型 ③基本 COCOMO 模型

④ 中级 COCOMO 模型 ⑤ 高级 COCOMO 模型
其中 (18) 均是静态单变量模型。

- (18) A. ①② B. ②④⑤ C. ①③ D. ③④⑤

● “通过指明一系列可执行的运算及运算的次序来描述计算过程”是 (19) 语言的特点。

- (19) A. 逻辑式 B. 函数式 C. 交互式 D. 命令式（或过程式）

● “ $X = (A + B) \times (C - D/E)$ ” 的后缀式表示为 (20)。

- (20) A. $XAB+CDE/-\times=$ B. $XAB+C-DE /\times=$
C. $XAB+CDE-/\times=$ D. $XAB+CD-E /\times=$

● 在一个单 CPU 的计算机系统中，采用可剥夺式（也称抢占式）优先级的进程调度方案，且所有任务可以并行使用 I/O 设备。下表列出了三个任务 T1、T2、T3 的优先级、独立运行时占用 CPU 和 I/O 设备的时间。如果操作系统的开销忽略不计，这三个任务从同时启动到全部结束的总时间为 (21) ms，CPU 的空闲时间共有 (22) ms。

任务	优先级	每个任务独立运行时所需的时间
T1	高	对每个任务： 占用 CPU 10ms，I/O 13ms，再占用 CPU 5ms
T2	中	
T3	低	

- (21) A. 28 B. 58 C. 61 D. 64
(22) A. 3 B. 5 C. 8 D. 13

● 从下表关于操作系统存储管理方案 1、方案 2 和方案 3 的相关描述可以看出，它们分别对应 (23) 存储管理方案。

方案	说 明
1	在系统进行初始化的时候就已经将主存储空间划分成大小相等或不等的块，并且这些块的大小在此后是不可以改变的。系统将程序分配在连续的区域中。
2	主存储空间和程序按固定大小单位进行分割，程序可以分配在不连续的区域中。该方案当一个作业的程序地址空间大于主存可以使用的空间时也可以执行。
3	编程时必须划分程序模块和确定程序模块之间的调用关系，不存在调用关系的模块可以占用相同的主存区。

- (23) A. 固定分区、请求分页和覆盖 B. 覆盖、请求分页和固定分区
C. 固定分区、覆盖和请求分页 D. 请求分页、覆盖和固定分区

● 假设系统中有三类互斥资源 R1、R2 和 R3，可用资源数分别为 8、7 和 4。在 T_0 时刻系统中有 P1、P2、P3、P4 和 P5 五个进程，这些进程对资源的最大需求量和已分配资源数如下表所示。在 T_0 时刻系统剩余的可用资源数分别为 (24)。如果进程按 (25) 序列执行，那么系统状态是安全的。

资源 进程	最大需求量			已分配资源数		
	R1	R2	R3	R1	R2	R3
P1	6	4	2	1	1	1
P2	2	2	2	2	1	1
P3	8	1	1	2	1	0
P4	2	2	1	1	2	1
P5	3	4	2	1	1	1

(24) A. 0、1 和 0 B. 0、1 和 1 C. 1、1 和 0 D. 1、1 和 1

(25) A. P1→P2→P4→P5→P3 B. P2→P1→P4→P5→P3
C. P4→P2→P1→P5→P3 D. P4→P2→P5→P1→P3

● 统一过程 (UP) 的基本特征是“用例驱动、以架构为中心的和受控的迭代式增量开发”。UP 将一个周期的开发过程划分为 4 个阶段，其中 (26) 的提交结果包含了系统架构。

(26) A. 先启阶段 B. 精化阶段 C. 构建阶段 D. 提交阶段

● 某软件在应用初期运行在 Windows NT 环境中。现因某种原因，该软件需要在 UNIX 环境中运行，而且必须完成相同的功能。为适应这个要求，软件本身需要进行修改，而所需修改的工作量取决于该软件的 (27)。

(27) A. 可扩充性 B. 可靠性 C. 复用性 D. 可移植性

● 按照 ISO/IEC 9126 软件质量度量模型定义，一个软件的可靠性的子特性包括 (28)。

(28) A. 容错性和安全性 B. 容错性和适应性
C. 容错性和易恢复性 D. 易恢复性和安全性

● (29) 详细描述软件的功能、性能和用户界面，以使用户了解如何使用软件。

(29) A. 概要设计说明书 B. 详细设计说明书
C. 用户手册 D. 用户需求说明书

● 各类软件维护活动中，(30) 维护占整个维护工作的比重最大。

(30) A. 完善性 B. 改正性 C. 适应性 D. 预防性

● 给定 C 语言程序：

```
int foo( int x, int y, int d)
{
    if( x != 0 ) {
        if ( y == 0 ) d = d / x;
        else d = d / (x * y);
    } else {
        if( y == 0 ) d = 0;
        else d = d / y;
    }
    return d ;
}
```

当用路径覆盖法进行测试时，至少需要设计(31)个测试用例。

- (31) A. 3 B. 4 C. 5 D. 8

● 软件的测试通常分单元测试、组装测试、确认测试、系统测试四个阶段进行。(32)属于确认测试阶段的活动。

- (32) A. 设计评审 B. 代码审查 C. 结构测试 D. 可靠性测试

● 面向对象分析的第一步是(33)。

- (33) A. 定义服务 B. 确定附加的系统约束
C. 确定问题域 D. 定义类和对象

● 面向对象程序设计语言为(34)提供支持。

- (34) A. 面向对象用例设计阶段 B. 面向对象分析阶段
C. 面向对象需求分析阶段 D. 面向对象实现阶段

● 下面关于面向对象的描述正确的是(35)。

- (35) A. 针对接口编程，而不是针对实现编程
B. 针对实现编程，而不是针对接口编程
C. 接口与实现不可分割
D. 优先使用继承而非组合

● 下面关于 UML 文档的叙述中正确的是(36)。

- (36) A. UML 文档指导开发人员如何进行面向对象分析
B. UML 文档描述了面向对象分析与设计的结果
C. UML 文档给出了软件的开发过程和设计流程
D. UML 文档指导开发人员如何进行面向对象设计

● UML 的设计视图包含了类、接口和协作，其中，设计视图的静态方面由(37)和(38)表现；动态方面由交互图、(39)表现。

- (37) A. 类图 B. 状态图 C. 活动图 D. 用例图
(38) A. 状态图 B. 顺序图 C. 对象图 D. 活动图
(39) A. 状态图和类图 B. 类图和活动图
 C. 对象图 and 状态图 D. 状态图 and 活动图

● UML 中的构件是遵从一组接口并提供一组接口的实现，下列说法错误的是(40)。

- (40) A. 构件应是可替换的 B. 构件表示的是逻辑模块而不是物理模块
 C. 构件应是组成系统的一部分 D. 构件与类处于不同的抽象层次

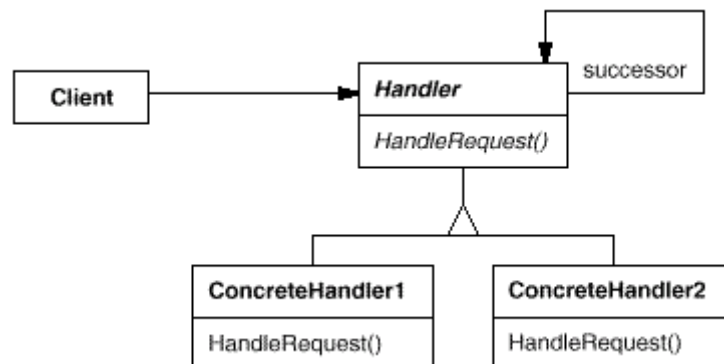
● 设计模式具有(41)的优点。

- (41) A. 适应需求变化 B. 程序易于理解
 C. 减少开发过程中的代码开发工作量 D. 简化软件系统的设计

● 下面的(42)模式将对象组合成树形结构以表示“部分-整体”的层次结构，并使得用户对单个对象和组合对象的使用具有一致性。

- (42) A. 组合 (Composite) B. 桥接 (Bridge)
 C. 修饰 (Decorator) D. 外观 (Facade)

● 下图描述了一种设计模式，该设计模式不可以(43)。

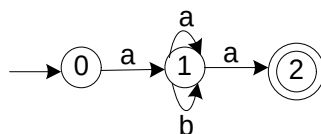


- (43) A. 动态决定由一组对象中某个对象处理该请求
 B. 动态指定处理一个请求的对象集合，并高效率地处理一个请求
 C. 使多个对象都有机会处理请求，避免请求的发送者和接收者间的耦合关系
 D. 将对象连成一条链，并沿着该链传递请求

● 在面向对象程序设计中，常常将接口的定义与接口的实现相分离，可定义不同的类实现相同的接口。在程序运行过程中，对该接口的调用可根据实际的对象类型调用其相应的实现。为达到上述目的，面向对象语言须提供(44)机制。

- (44) A. 继承和过载(overloading) B. 抽象类
C. 继承和重置(overriding) D. 对象自身引用

● 下图是一有限自动机的状态转换图，该自动机所识别语言的特点是(45)，等价的正规式为(46)。



- (45) A. 由符号 a、b 构成且包含偶数个 a 的串
B. 由符号 a、b 构成且开头和结尾符号都为 a 的串
C. 由符号 a、b 构成的任意串
D. 由符号 a、b 构成且 b 的前后必须为 a 的串

- (46) A. $(a|b)^*(aa)^*$ B. $a(a|b)^*a$ C. $(a|b)^*$ D. $a(ba)^*a$

● 关系 R、S 如下图所示，元组演算表达式 $\{t | (\forall u)(R(u) \wedge S(u) \wedge t[3] > u[1])\}$ 的结果为(47)。

A	B	C
1	2	3
4	5	6
7	8	9
10	11	12

R

A	B	C
3	7	11
4	5	6
5	9	13
6	10	14

S

(47) A.	A	B	C	B.	A	B	C
	1	2	3		3	7	11
	4	5	6		4	5	6
C.	A	B	C	D.	A	B	C
	7	8	9		5	9	13
	10	11	12		6	10	14

● 某企业职工和部门的关系模式如下所示，其中部门负责人也是一个职工。职工和部门关系的外键分别是(48)。

职工（职工号，姓名，年龄，月工资，部门号，电话，办公室）
 部门（部门号，部门名，负责人代码，任职时间）

查询每个部门中月工资最高的“职工号”的 SQL 查询语句如下：

Select 职工号 from 职工 as E
 where 月工资= (Select Max (月工资) from 职工 as M (49))。

- (48) A. 职工号和部门号 B. 部门号和负责人代码
 C. 职工号和负责人代码 D. 部门号和职工号
 (49) A. where M.职工号=E.职工号 B. where M.职工号=E.负责人代码
 C. where M.部门号=部门号 D. where M.部门号=E.部门号

● 操作序列 T1、T2、T3 对数据 A、B、C 并发操作如下所示，T1 与 T2 间并发操作 (50) ， T2 与 T3 间并发操作 (51) 。

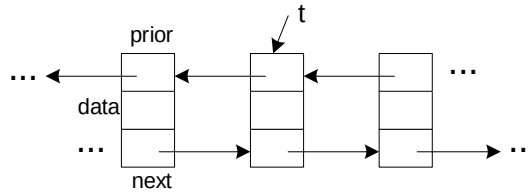
时间	T1	T2	T3
t1	读 A=50		
t2	读 B=200		
t3	X1=A+B		
t4			读 B=200
t5		读 B=200	
t6		B=B-100	
t7		写 B	
t8	读 A=50		
t9	读 B=100		
t10	X1=A+B		
t11	验算不对		B=B+50
t12			写 B

- (50) A. 不存在问题 B. 将丢失修改
 C. 不能重复读 D. 将读“脏”数据
 (51) A. 不存在问题 B. 将丢失修改
 C. 不能重复读 D. 将读“脏”数据

● 结点数目为 n 的二叉查找树（二叉排序树）的最小高度为 (52) 、最大高度为 (53) 。

- (52) A. n B. $\frac{n}{2}$ C. $\lceil \log_2 n \rceil$ D. $\lceil \log_2 (n+1) \rceil$
 (53) A. n B. $\frac{n}{2}$ C. $\lceil \log_2 n \rceil$ D. $\lceil \log_2 (n+1) \rceil$

- 某双向链表中的结点如下图所示，删除 t 所指结点的操作为 (54)。



- (54) A. $t \rightarrow \text{prior} \rightarrow \text{next} = t \rightarrow \text{next}$; $t \rightarrow \text{next} \rightarrow \text{prior} = t \rightarrow \text{prior}$;
 B. $t \rightarrow \text{prior} \rightarrow \text{prior} = t \rightarrow \text{prior}$; $t \rightarrow \text{next} \rightarrow \text{next} = t \rightarrow \text{next}$;
 C. $t \rightarrow \text{prior} \rightarrow \text{next} = t \rightarrow \text{prior}$; $t \rightarrow \text{next} \rightarrow \text{prior} = t \rightarrow \text{next}$;
 D. $t \rightarrow \text{prior} \rightarrow \text{prior} = t \rightarrow \text{next}$; $t \rightarrow \text{next} \rightarrow \text{prior} = t \rightarrow \text{prior}$;

- 对于二维数组 $a[0..4, 1..5]$ ，设每个元素占 1 个存储单元，且以列为主序存储，则元素 $a[2, 2]$ 相对于数组空间起始地址的偏移量是 (55)。

- (55) A. 5 B. 7 C. 10 D. 15

- 对于 n 个元素的关键字序列 $\{k_1, k_2, \dots, k_n\}$ ，当且仅当满足关系 $k_i \leq k_{2i}$ 且 $k_i \leq k_{2i+1}$ ($2i \leq n, 2i+1 \leq n$) 称其为小根堆，反之则为大根堆。以下序列中，(56) 不符合堆的定义。

- (56) A. (4, 10, 15, 72, 39, 23, 18) B. (58, 27, 36, 12, 8, 23, 9)
 C. (4, 10, 18, 72, 39, 23, 15) D. (58, 36, 27, 12, 8, 23, 9)

- 求单源点最短路径的迪杰斯特拉 (Dijkstra) 算法是按 (57) 的顺序求源点到各顶点的最短路径的。

- (57) A. 路径长度递减 B. 路径长度递增
 C. 顶点编号递减 D. 顶点编号递增

- (58) 算法策略与递归技术的联系最弱。

- (58) A. 动态规划 B. 贪心 C. 回溯 D. 分治

- 对于具有 n 个元素的一个数据序列，若只需得到其中第 k 个元素之前的部分排序，最好采用 (59)，使用分治 (Divide and Conquer) 策略的是 (60) 算法。

- (59) A. 希尔排序 B. 直接插入排序 C. 快速排序 D. 堆排序
 (60) A. 冒泡排序 B. 插入排序 C. 快速排序 D. 堆排序

● ARP 协议的作用是__ (61) __，ARP 报文封装在__ (62) __中传送。

- (61) A. 由 IP 地址查找对应的 MAC 地址
B. 由 MAC 地址查找对应的 IP 地址
C. 由 IP 地址查找对应的端口号
D. 由 MAC 地址查找对应的端口号

(62) A. 以太网帧 B. IP 数据报 C. UDP 报文 D. TCP 报文

● 802.11 标准定义的分布式协调功能采用了__ (63) __协议。

(63) A. CSMA/CD B. CSMA/CA C. CDMA/CD D. CDMA/CA

● 设有两个子网 202.118.133.0/24 和 202.118.130.0/24，如果进行路由汇聚，得到的网络地址是__ (64) __。

- (64) A. 202.118.128.0/21 B. 202.118.128.0/22
C. 202.118.130.0/22 D. 202.118.132.0/20

● 路由器收到一个数据包，其目标地址为 195.26.17.4，该地址属于__ (65) __子网。

- (65) A. 195.26.0.0/21 B. 195.26.16.0/20
C. 195.26.8.0/22 D. 195.26.20.0/22

● NAC's (Network Access Control) role is to restrict network access to only compliant endpoints and (66) users. However, NAC is not a complete LAN (67) solution; additional proactive and (68) security measures must be implemented. Nevis is the first and only comprehensive LAN security solution that combines deep security processing of every packet at 10Gbps, ensuring a high level of security plus application availability and performance. Nevis integrates NAC as the first line of LAN security (69). In addition to NAC, enterprises need to implement role-based network access control as well as critical proactive security measures — real-time, multilevel (70) inspection and microsecond threat containment.

- (66) A. automated B. distinguished C. authenticated D. destructed
(67) A. crisis B. security C. favorable D. excellent
(68) A. constructive B. reductive C. reactive D. productive
(69) A. defense B. intrusion C. inbreak D. protection
(70) A. port B. connection C. threat D. insurance

● Virtualization is an approach to IT that pools and shares (71) so that utilization is optimized and supplies automatically meet demand. Traditional IT environments are often silos, where both technology and human (72) are aligned around an application or business function. With a virtualized (73), people, processes, and technology are focused on meeting service levels, (74) is allocated dynamically, resources are optimized, and the entire

infrastructure is simplified and flexible. We offer a broad spectrum of virtualization (75) that allows customers to choose the most appropriate path and optimization focus for their IT infrastructure resources.

- | | | | |
|--------------------|-------------------|---------------|----------------|
| (71) A. advantages | B. resources | C. benefits | D. precedents |
| (72) A. profits | B. costs | C. resources | D. powers |
| (73) A. system | B. infrastructure | C. hardware | D. link |
| (74) A. content | B. position | C. power | D. capacity |
| (75) A. solutions | B. networks | C. interfaces | D. connections |

全国计算机技术与软件专业技术资格（水平）考试

2006 年下半年 软件设计师 下午试卷

（考试时间 14:00~16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2006 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是(1)月(2)日。

因为正确的解答是“11 月 4 日”，故在答题纸的对应栏内写上“11”和“4”（参看下表）。

例题	解答栏
(1)	11
(2)	4

试题一至试题四是必答题

试题一（共 15 分）

阅读以下说明以及数据流图，回答问题1至问题5，将解答填入答题纸的对应栏内。

[说明]

某银行已有一套基于客户机/服务器模式的储蓄系统 A 和一套建帐软件。建帐软件主要用于将储蓄所手工处理的原始数据转换为系统 A 所需的数据格式。该建帐软件具有以下功能：

（1）分户帐录入：手工办理业务时建立的每个分户帐数据均由初录员和复录员分别录入，以确保数据的正确性；

（2）初录/复录比对：将初录员和复录员录入的数据进行一一比较，并标记两套数据是否一致；

（3）数据确认：当上述两套数据完全一致后，将其中任一套作为最终进入系统 A 的原始数据；

（4）汇总核对和打印：对经过确认的数据进行汇总，并和会计账目中的相关数据进行核对，以确保数据的整体正确性，并打印输出经过确认的数据，为以后核查可能的错误提供依据；

（5）数据转换：将经过确认的数据转换为储蓄系统 A 需要的中间格式数据；

（6）数据清除：为加快初录和复录的处理速度，在数据确认之后，可以有选择地清除初录员和复录员录入的数据。

该软件的数据流图如图 1-1～图 1-3 所示。图中部分数据流数据文件的格式如下：

初录分户帐 = 储蓄所号 + 帐号 + 户名 + 开户日 + 开户金额 + 当前余额 + 性质

复录分户帐 = 储蓄所号 + 帐号 + 户名 + 开户日 + 开户金额 + 当前余额 + 性质

初录数据 = 手工分户帐 + 一致性标志

复录数据 = 手工分户帐 + 一致性标志

会计账目 = 储蓄所号 + 总户数 + 总余额

操作结果 = 初录操作结果 + 比对操作结果 + 复录操作结果

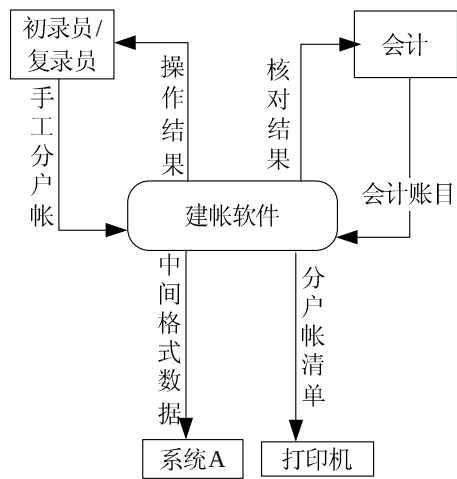


图 1-1 建帐软件顶层数据流图

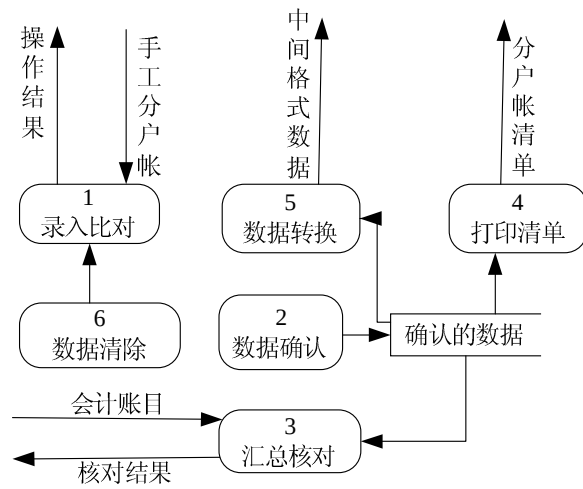


图 1-2 建帐软件第 0 层数据流图

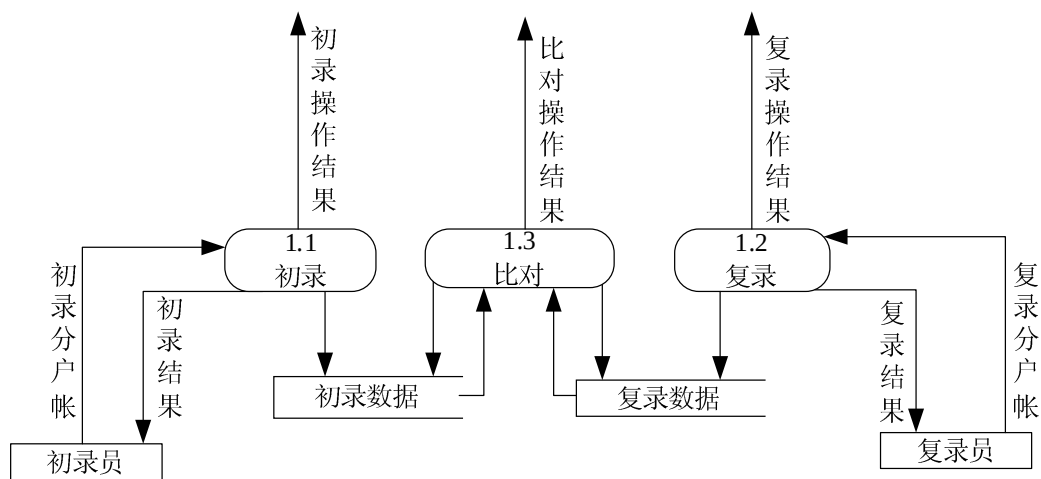


图 1-3 建帐软件第 1 层数据流图

软件需要打印的分户帐清单样式如表 1-1 所示：

表 1-1 分户帐清单样式表

储蓄所	帐号	开户日	户名	其他分户帐数据
储蓄所 1				
	...	...	...	...
储蓄所 1 合计	共 XXX 户，总余额 9999999.99 元			
储蓄所 2				
	...	...	...	...
储蓄所 2 合计	共 XXX 户，总余额 9999999.99 元			

【问题 1】（4 分）

请采用说明中的词汇，给出数据确认处理所需的数据流在第 1 层图中的全部可选起点（第 0 层图和第 1 层图中均未给出）。

【问题 2】（2 分）

不考虑数据确认处理（加工 2），请指出数据流图中存在的错误。

【问题 3】（2 分）

打印分户帐清单时，必须以下列哪一组数据作为关键字进行排序，才能满足需求？请从下面选项中选择，并将对应序号填入答题纸相应栏。

- ① 储蓄所 ② 帐号 ③ 开户日 ④ 总户数和总余额

【问题 4】（4 分）

加工 1（录入比对处理）除能够检查出初录数据和复录数据不一致外，还应当检测出下列哪些错误，请将对应序号填入答题纸对应栏。

- ① 输入的无效字符 ② 输入的半个汉字 ③ 显示器无法显示
④ 初录员重复录入同一帐户 ⑤ 汇总数据与会计账目不符 ⑥ 打印机卡纸

【问题 5】（3 分）

请使用数据字典条目定义形式，给出第 0 层 DFD 中的“手工分户帐”数据流和第 1 层 DFD 中的“初录分户帐”、“复录分户帐”的关系。

试题二(共 15 分)

阅读以下说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

[说明]

某宾馆需要建立一个住房管理系统，部分的需求分析结果如下：

(1) 一个房间有多个床位，同一房间内的床位具有相同的收费标准。不同房间的床位收费标准可能不同。

(2) 每个房间有房间号（如 201、202 等）、收费标准、床位数目等信息。

(3) 每位客人有身份证号码、姓名、性别、出生日期和地址等信息。

(4) 对每位客人的每次住宿，应该记录其入住日期、退房日期和预付款额信息。

(5) 管理系统可查询出客人所住房间号。

根据以上的需求分析结果，设计一种关系模型如图 2-1 所示：



图 2-1 住房管理系统的实体联系图

[问题 1] (1 分)

根据上述说明和实体-联系图，得到该住房管理系统的关系模式如下所示，请补充住宿关系。

房间(房间号，收费标准，床位数目)

客人(身份证号，姓名，性别，出生日期，地址)

住宿(____ (1) _____, 入住日期，退房日期，预付款额)

[问题 2] (4 分)

请给出问题 1 中住宿关系的主键和外键。

[问题 3] (4 分)

若将上述各关系直接实现为对应的物理表，现需查询在 2005 年 1 月 1 日到 2005 年 12 月 31 日期间，在该宾馆住宿次数大于 5 次的客人身份证号，并且按照入住次数进行降序排列。下面是实现该功能的 SQL 语句，请填补语句中的空缺。

```
SELECT 住宿.身份证号, count(入住日期)
FROM 住宿, 客人
WHERE 入住日期 >= '20050101' AND 入住日期 <= '20051231'
      AND 住宿.身份证号 = 客人.身份证号
GROUP BY (2)
(3) count(入住日期) > 5
(4)
```

[问题 4] (6 分)

为加快 SQL 语句的执行效率，可在相应的表上创建索引。根据问题 3 中的 SQL 语句，除主键和外键外，还需要在哪个表的哪些属性上创建索引，应该创建什么类型的索引，请说明原因。

试题三(共 15 分)

阅读以下说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

S 公司开办了在线电子商务网站，主要为各注册的商家提供在线商品销售功能。为更好地吸引用户，S 公司计划为注册的商家提供商品(Commodity)促销(Promotion)功能。商品的分类(Category)不同，促销的方式和内容会有所不同。

注册商家可发布促销信息。商家首先要在自己所销售的商品的分类中，选择促销涉及的某一具体分类，然后选出该分类的一个或多个商品（一种商品仅仅属于一种分类），接着制定出一个比较优惠的折扣政策和促销活动的优惠时间，最后由系统生成促销信息并将该促销信息公布在网站上。

商家发布促销信息后，网站的注册用户便可通过网站购买促销商品。用户可选择参与某一个促销(Promotion)活动，并选择具体的促销商品(Commodity)，输入购买数量等购买信息。系统生成相应的一份促销订单(POrder)。只要用户在优惠活动的时间范围内，通过网站提供的在线支付系统，确认在线支付该促销订单（即完成支付），就可以优惠的价格完成商品的购买活动，否则该促销订单失效。

系统采用面向对象方法开发，系统中的类以及类之间的关系用 UML 类图表示，图 3-1 是该系统类图中的一部分；系统的动态行为采用 UML 序列图表示，图 3-2 是发布促销的序列图。

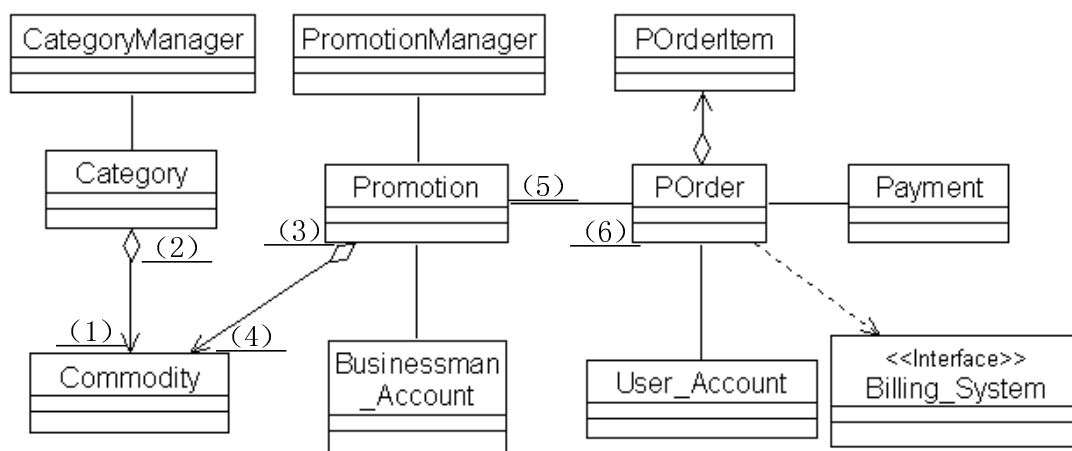


图 3-1 在线促销系统部分类图

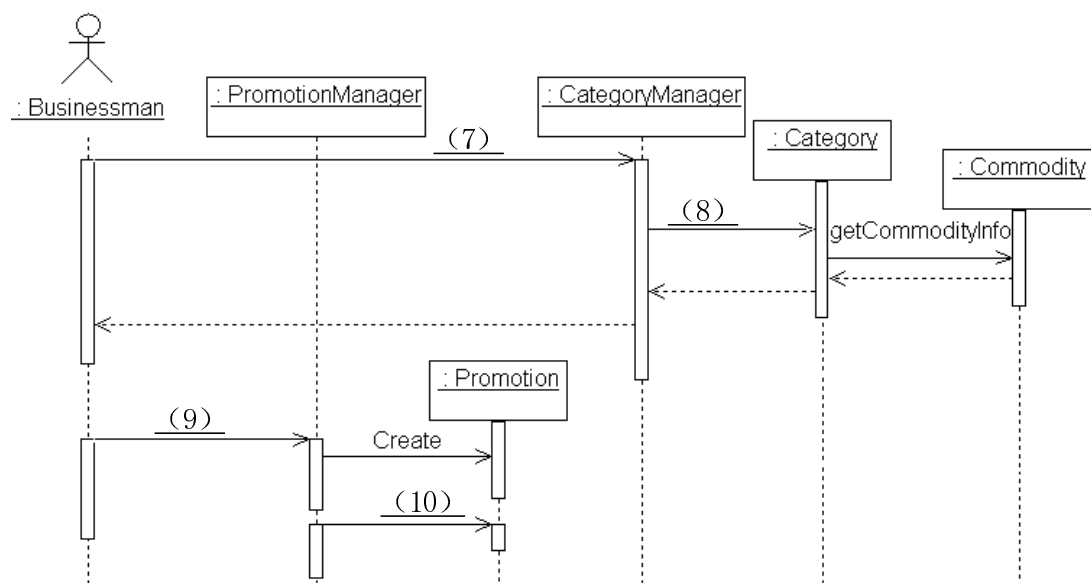


图 3-2 发布促销序列图

[问题 1] (6 分)

识别关联的多重度是面向对象建模过程中的一个重要步骤。根据说明中给出的描述，完成图 3-1 中的(1)~(6)。

[问题 2] (4 分)

请从表 3-1 中选择方法，完成图 3-2 中的(7)~(10)。

表 3-1 可选消息列表

功能描述	方法名
向促销订单中添加所选的商品	buyCommodities
向促销中添加要促销的商品	addCommodities
查找某个促销的所有促销订单信息列表	getPromotionOrders
生成商品信息	createCommodity
查找某个分类中某商家的所有商品信息列表	getCommodities
生成促销信息	createPromotion
生成促销订单信息	createPOrder
查找某个分类的所有促销信息列表	getCategoryPromotion
查找某商家所销售的所有分类列表	getCategories
查找某个促销所涉及的所有商品信息列表	getPromotionCommodities

[问题 3] (5 分)

关联(Association)和聚集 (Aggregation) 是 UML 中两种非常重要的关系。请说明关联和聚集的关系，并说明其不同点。

试题四(共 15 分)

阅读以下说明和图，填补流程图中的空缺，将解答填入答题纸的对应栏内。

[说明]

某汽车制造工厂有两条装配线。汽车装配过程如图 4-1 所示，即汽车底盘进入装配线，零件在多个工位装配，结束时汽车自动完成下线工作。

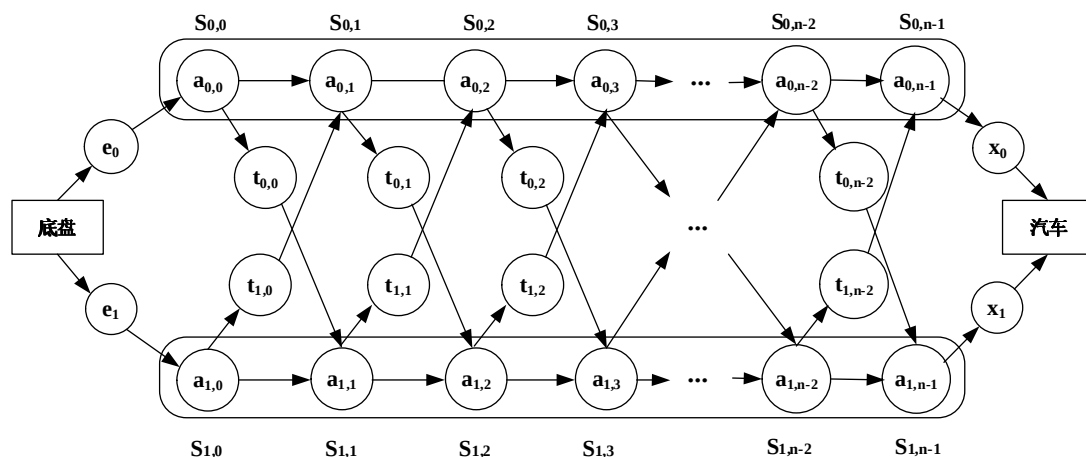


图 4-1 汽车装配线

- (1) e_0 和 e_1 表示底盘分别进入装配线 0 和装配线 1 所需要的时间。
- (2) 每条装配线有 n 个工位，第一条装配线的工位为 $S_{0,0}, S_{0,1}, \dots, S_{0,n-1}$ ，第二条装配线的工位为 $S_{1,0}, S_{1,1}, \dots, S_{1,n-1}$ 。其中 $S_{0,k}$ 和 $S_{1,k}$ ($0 \leq k \leq n-1$) 完成相同的任务，但所需时间可能不同。
- (3) $a_{i,j}$ 表示在工位 $S_{i,j}$ 处的装配时间，其中 i 表示装配线 ($i=0$ 或 $i=1$)， j 表示工位号 ($0 \leq j \leq n-1$)。
- (4) $t_{i,j}$ 表示从 $S_{i,j}$ 处装配完成后转移到另一条装配线下一个工位的时间。
- (5) x_0 和 x_1 表示装配结束后，汽车分别从装配线 0 和装配线 1 下线所需要的时间。
- (6) 在同一条装配线上，底盘从一个工位转移到其下一个工位的时间可以忽略不计。

图 4-2 所示的流程图描述了求最短装配时间的算法，该算法的输入为：

n ：表示装配线上的工位数；

$e[i]$ ：表示 e_0 和 e_1 ， i 取值为 0 或 1；

$a[i][j]$ ：表示 $a_{i,j}$ ， i 的取值为 0 或 1， j 的取值范围为 $0 \sim n-1$ ；

$t[i][j]$ ：表示 $t_{i,j}$ ， i 的取值为 0 或 1， j 的取值范围为 $0 \sim n-1$ ；

$x[i]$ ：表示 x_0 和 x_1 ， i 取值为 0 或 1。

算法的输出为：

f_i ：最短的装配时间；

li ：获得最短装配时间的下线装配线号（0 或者 1）。

算法中使用的 $f[i][j]$ 表示从开始点到 $S_{i,j}$ 处的最短装配时间。

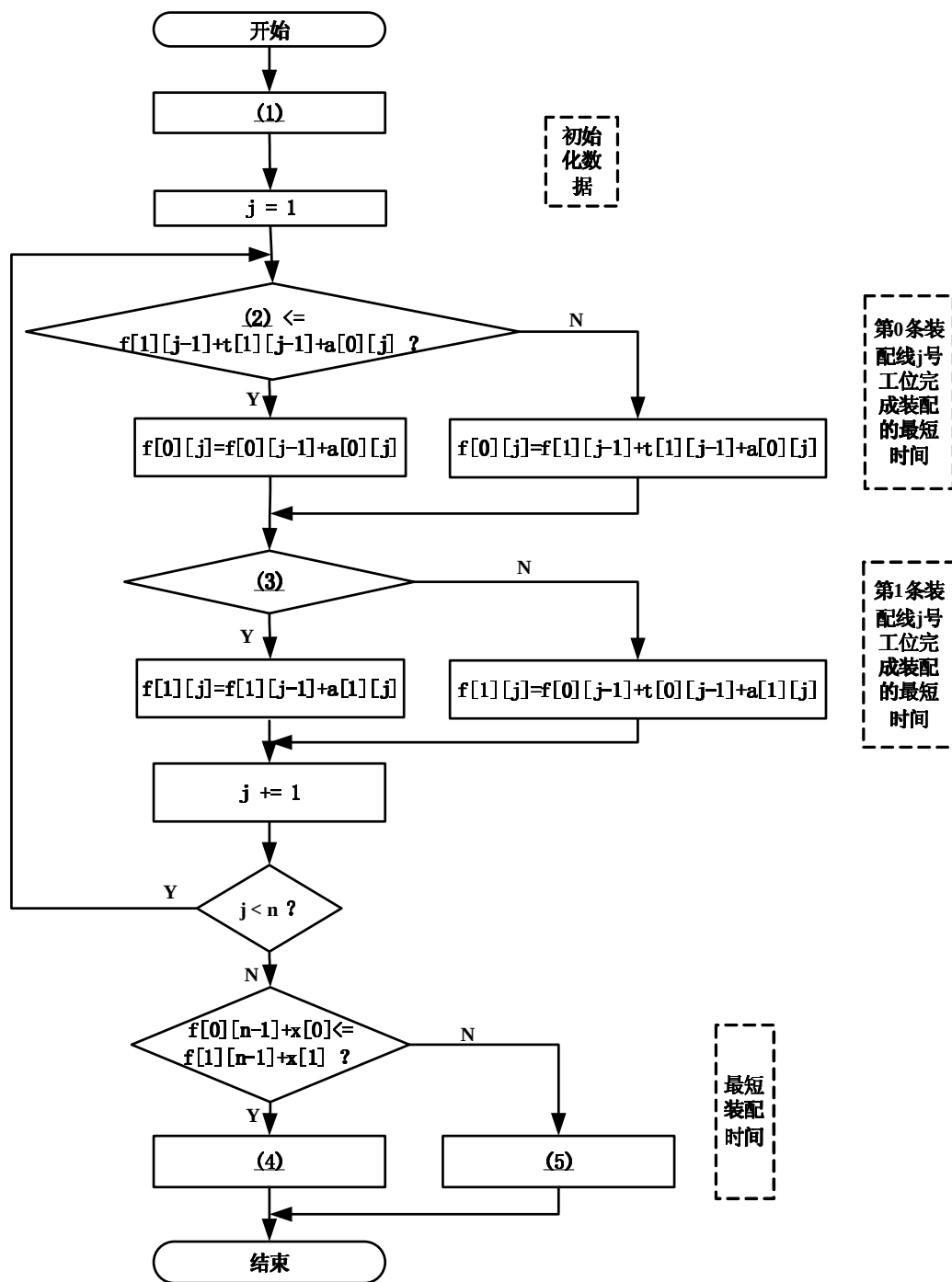


图 4-2 求最短装配时间算法

从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五(15 分)

阅读以下说明、图和 C 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

一般的树结构常采用孩子—兄弟表示法表示，即用二叉链表作树的存储结构，链表中结点的两个链域分别指向该结点的第一个孩子结点和下一个兄弟结点。例如，图 5-1(a) 所示的树的孩子—兄弟表示如图 5-1(b) 所示。

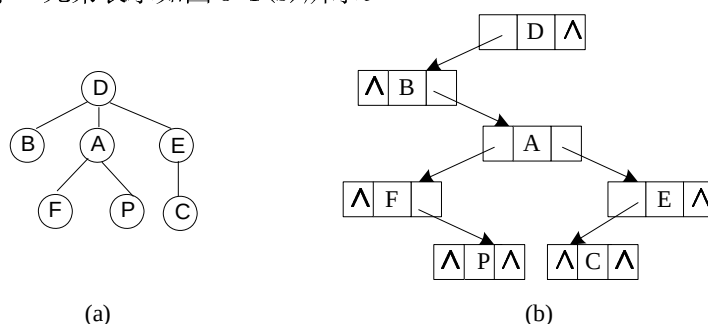


图 5-1 树及其孩子—兄弟表示示意图

函数 LevelTraverse() 的功能是对给定树进行层序遍历。例如，对图 5-1 所示的树进行层序遍历时，结点的访问次序为：D B A E F P C。

对树进行层序遍历时使用了队列结构，实现队列基本操作的函数原型如下表所示：

函数原型	说明
void InitQueue(Queue *Q)	初始化队列
Bool IsEmpty(Queue Q)	判断队列是否为空，若是则返回 TRUE, 否则返回 FALSE
void EnQueue(Queue *Q, TreeNode p)	元素入队列
void DeQueue(Queue *Q, TreeNode *p)	元素出队列

Bool、Status 类型定义如下：

```
typedef enum {FALSE = 0, TRUE = 1} Bool;
```

```
typedef enum {OVERFLOW = -2, UNDERFLOW = -1, ERROR = 0, OK = 1} Status;
```

树的二叉链表结点定义如下：

```
typedef struct Node {
    char data;
    struct Node *firstchild, *nextbrother;
} Node, *TreeNode;
```

[函数]

```
Status LevelTraverse(TreeNode root)
{ /*层序遍历树，树采用孩子-兄弟表示法，root 是树根结点的指针*/
    Queue tempQ;
    TreeNode ptr, brotherptr;

    if (!root)
        return ERROR;

    InitQueue(&tempQ);
    (1);

    brotherptr = root -> nextbrother;
    while (brotherptr) {
        EnQueue(&tempQ, brotherptr);
        (2);
    } /*end-while*/

    while ( (3) ) {
        (4);
        printf("%c\t", ptr->data);

        if ( (5) ) continue;

        (6);
        brotherptr = ptr->firstchild->nextbrother;
        while (brotherptr) {
            EnQueue(&tempQ, brotherptr);
            (7);
        } /*end-while*/

    } /*end-while*/

    return OK;
} /*LevelTraverse*/
```

试题六（共 15 分）

阅读以下说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

传输门是传输系统中的重要装置。传输门具有 Open（打开）、Closed（关闭）、Opening（正在打开）、StayOpen（保持打开）、Closing（正在关闭）五种状态。触发传输门状态转换的事件有 click、complete 和 timeout 三种。事件与其相应的状态转换如图 6-1 所示。

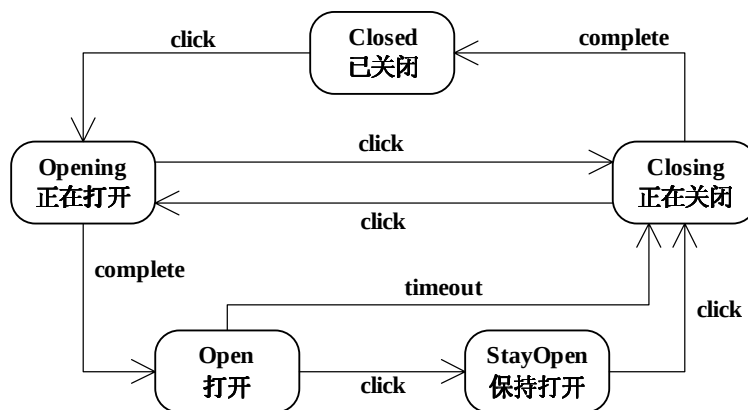


图 6-1 传输门响应事件与其状态转换图

下面的[C++代码 1]与[C++代码 2]分别用两种不同的设计思路对传输门进行状态模拟，请填补代码中的空缺。

[C++代码 1]

```
const int CLOSED = 1;    const int OPENING = 2;
const int OPEN = 3;      const int CLOSING = 4;
const int STAYOPEN = 5;  //定义状态变量，用不同整数表示不同状态

class Door {
private:
    int state; //传输门当前状态
    void setState(int state){ this->state = state; } //设置当前状态
public:
    Door():state(CLOSED) {};
    void getState() { //根据当前状态输出相应的字符串
        switch(state) {
            case OPENING: cout << "OPENING" << endl; break;
            case CLOSED: cout << "CLOSED" << endl; break;
            case OPEN: cout << "OPEN" << endl; break;
            case CLOSING: cout << "CLOSING" << endl; break;
```

```

        case STAYOPEN: cout << "STAYOPEN" << endl; break;
    }
}

void click() {          //发生click事件时进行状态转换
    if (__(1)__) setState(OPENING);
    else if (__(2)__) setState(CLOSING);
    else if (__(3)__) setState(STAYOPEN);
}

void timeout() {        //发生timeout事件时进行状态转换
    if (state == OPEN)  setState(CLOSING);
}

void complete() {       //发生complete事件时进行状态转换
    if (state == OPENING)    setState(OPEN);
    else if (state == CLOSING) setState(CLOSED);
}

};

int main() {
    Door aDoor;
    aDoor.getState();  aDoor.click();  aDoor.getState();  aDoor.complete();
    aDoor.getState();  aDoor.click();  aDoor.getState();  aDoor.click();
    aDoor.getState();  return 0;
}

```

[C++代码 2]

```

class Door {
public:
    DoorState *CLOSED, *OPENING, *OPEN, *CLOSING, *STAYOPEN, *state;
    Door();
    virtual ~Door() { ..... //释放申请的内存, 此处代码省略};
    void setState(DoorState *state) {  this->state = state; }
    void getState() {
        // 此处代码省略, 本方法输出状态字符串,
        // 例如, 当前状态为CLOSED时, 输出字符串为 "CLOSED"
    };
    void click();
    void timeout();
    void complete();
};

```

```

Door::Door() {
    CLOSED = new DoorClosed(this);    OPENING = new DoorOpening(this);
    OPEN = new DoorOpen(this);        CLOSING = new DoorClosing(this);
    STAYOPEN = new DoorStayOpen(this); state = CLOSED;
}
void Door::click() { __ (4) __;}
void Door::timeout() {__ (5) __;}
void Door::complete() {__ (6) __;}

class DoorState //定义一个抽象的状态，它是所有状态类的基类
{
protected: Door *door;
public:
    DoorState(Door *door) { this->door = door; }
    virtual ~DoorState(void);
    virtual void click() {}
    virtual void complete() {}
    virtual void timeout() {}
};

class DoorClosed :public DoorState{ //定义一个基本的 Closed 状态
public:
    DoorClosed(Door *door):DoorState(door) {}
    virtual ~ DoorClosed () {}
    void click();
};

void DoorClosed::click() {__ (7) __;}
// 其它状态类的定义与实现代码省略

int main() {
    Door aDoor;
    aDoor.getState(); aDoor.click();  aDoor.getState(); aDoor.complete();
    aDoor.getState(); aDoor.timeout(); aDoor.getState(); return 0;
}

```

试题七（共 15 分）

阅读以下说明以及 Java 程序，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

传输门是传输系统中的重要装置。传输门具有 Open（打开）、Closed（关闭）、Opening（正在打开）、StayOpen（保持打开）、Closing（正在关闭）五种状态。触发状态的转换事件有 click、complete 和 timeout 三种。事件与其相应的状态转换如图 7-1 所示。

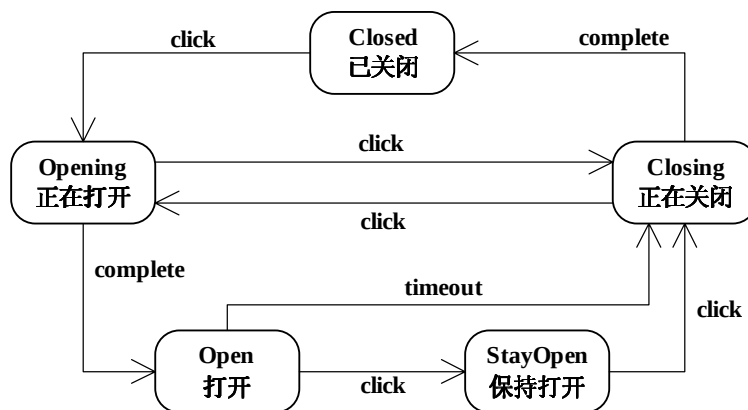


图 7-1 传输门响应事件与其状态转换图

下面的[Java 代码 1]与[Java 代码 2]分别用两种不同的设计思路对传输门进行状态模拟，请填补代码中的空缺。

【Java 代码 1】

```
public class Door {
    public static final int CLOSED = 1;    public static final int OPENING = 2;
    public static final int OPEN = 3;      public static final int CLOSING = 4;
    public static final int STAYOPEN = 5;  private int state = CLOSED;
    //定义状态变量，用不同的整数表示不同状态

    private void setState(int state){this.state = state;} //设置传输门当前状态
    public void getState(){
        // 此处代码省略，本方法输出状态字符串，
        // 例如，当前状态为 CLOSED 时,输出字符串为” CLOSED”
    }

    public void click() { //发生 click 事件时进行状态转换
        if (   (1)   )    setState(OPENING);
        else if (   (2)   )    setState(CLOSING);
        else if (   (3)   )    setState(STAYOPEN);
    }
}
```

```

//发生 timeout 事件时进行状态转换
public void timeout(){ if (state == OPEN)    setState(CLOSING); }

public void complete() { //发生 complete 事件时进行状态转换
    if (state == OPENING)    setState(OPEN);
    else if (state == CLOSING)    setState(CLOSED);
}
public static void main(String [] args){
    Door aDoor = new Door();
    aDoor.getState(); aDoor.click(); aDoor.getState(); aDoor.complete();
    aDoor.getState(); aDoor.click(); aDoor.getState(); aDoor.click();
    aDoor.getState();    return;
}
}

```

[Java 代码 2]

```

public class Door {
    public final DoorState CLOSED = new DoorClosed(this);
    public final DoorState OPENING = new DoorOpening(this);
    public final DoorState OPEN = new DoorOpen(this);
    public final DoorState CLOSING = new DoorClosing(this);
    public final DoorState STAYOPEN = new DoorStayOpen(this);
    private DoorState state = CLOSED;

    //设置传输门当前状态
    public void setState(DoorState state){ this.state = state; }
    public void getState() { //根据当前状态输出对应的状态字符串
        System.out.println(state.getClass().getName());
    }

    public void click() { (4) }; //发生 click 事件时进行状态转换
    public void timeout() { (5) }; //发生 timeout 事件时进行状态转换
    public void complete() { (6) }; //发生 complete 事件时进行状态转换

    public static void main(String[] args){
        Door aDoor = new Door();
        aDoor.getState(); aDoor.click();    aDoor.getState(); aDoor.complete();
        aDoor.getState(); aDoor.timeout(); aDoor.getState(); return;
    }
}

```

```

}

public abstract class DoorState { //定义所有状态类的基类
    protected Door door ;
    public DoorState(Door door) {    this.door = door;    }
    public void click() {}
    public void complete() {}
    public void timeout() {}
}

class DoorClosed extends DoorState{ //定义一个基本的 Closed 状态
    public DoorClosed(Door door) {    super(door);    }
    public void click() {    (7);    }
    //该类定义的其余代码省略

}
//其余代码省略

```


全国计算机技术与软件专业技术资格（水平）考试

2007 年上半年 软件设计师 上午试卷

（考试时间 9：00～11：30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2007 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
____（88）____月____（89）____日。

- | | | | |
|------------|-------|-------|-------|
| （88） A. 4 | B. 5 | C. 6 | D. 7 |
| （89） A. 23 | B. 24 | C. 25 | D. 26 |

因为考试日期是“5 月 26 日”，故（88）选 B，（89）选 D，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 D 填涂（参看答题卡）。

● (1) 不属于计算机控制器中的部件。

- (1) A. 指令寄存器 IR B. 程序计数器 PC
C. 算术逻辑单元 ALU D. 程序状态字寄存器 PSW

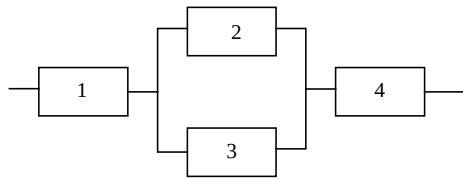
● 在 CPU 与主存之间设置高速缓冲存储器 Cache，其目的是为了 (2)。

- (2) A. 扩大主存的存储容量 B. 提高 CPU 对主存的访问效率
C. 既扩大主存容量又提高存取速度 D. 提高外存储器的速度

● 下面的描述中， (3) 不是 RISC 设计应遵循的设计原则。

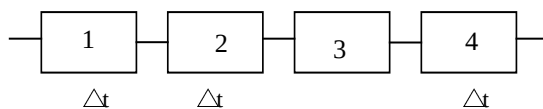
- (3) A. 指令条数应少一些
B. 寻址方式尽可能少
C. 采用变长指令，功能复杂的指令长度长而简单指令长度短
D. 设计尽可能多的通用寄存器

● 某系统的可靠性结构框图如下图所示。该系统由 4 个部件组成，其中 2、3 两部件并联冗余，再与 1、4 部件串联构成。假设部件 1、2、3 的可靠度分别为 0.90、0.70、0.70。若要求该系统的可靠度不低于 0.75，则进行系统设计时，分配给部件 4 的可靠度至少应为 (4)。



- (4) A. $\frac{0.75}{0.9 \times (1 - 0.7)^2}$ B. $\frac{0.75}{0.9 \times (1 - 0.7 \times 0.7)^2}$
C. $\frac{0.75}{0.9 \times (1 - (1 - 0.7)^2)}$ D. $\frac{0.75}{0.9 \times (0.7 + 0.7)}$

● 指令流水线将一条指令的执行过程分为四步，其中第 1、2 和 4 步的经过时间为 Δt ，如下图所示。若该流水线顺序执行 50 条指令共用 $153\Delta t$ ，并且不考虑相关问题，则该流水线的瓶颈第 3 步的时间为 (5) Δt 。



- (5) A. 2 B. 3 C. 4 D. 5

● 系统响应时间和作业吞吐量是衡量计算机系统性能的重要指标。对于一个持续处理业务的系统而言，其(6)。

- (6) A. 响应时间越短，作业吞吐量越小 B. 响应时间越短，作业吞吐量越大
C. 响应时间越长，作业吞吐量越大 D. 响应时间不会影响作业吞吐量

● 下列行为不属于网络攻击的是(7)。

- (7) A. 连续不停 Ping 某台主机
B. 发送带病毒和木马的电子邮件
C. 向多个邮箱群发一封电子邮件
D. 暴力破解服务器密码

● 多形病毒指的是(8)的计算机病毒。

- (8) A. 可在反病毒检测时隐藏自己 B. 每次感染都会改变自己
C. 可以通过不同的渠道进行传播 D. 可以根据不同环境造成不同破坏

● 感染“熊猫烧香”病毒后的计算机不会出现(9)的情况。

- (9) A. 执行文件图标变成熊猫烧香 B. 用户信息被泄漏
C. 系统运行变慢 D. 破坏计算机主板

● 如果两名以上的申请人分别就同样的发明创造申请专利，专利权应授予(10)。

- (10) A. 最先发明的人 B. 最先申请的人 C. 所有申请人 D. 协商后的申请人

● 下列标准代号中，(11)为推荐性行业标准的代号。

- (11) A. SJ/T B. Q/T11 C. GB/T D. DB11/T

● 以下显示器像素点距的规格中，最好的是(12)。

- (12) A. 0.39 B. 0.33 C. 0.31 D. 0.28

● 在彩色喷墨打印机中，将油墨进行混合后得到的颜色称为(13)色。

- (13) A. 相减 B. 相加 C. 互补 D. 比例

● 800×600 的分辨率的图像，若每个像素具有 16 位的颜色深度，则可表示 (14) 种不同的颜色。

- (14) A. 1000 B. 1024 C. 65536 D. 480000

● 结构化开发方法中，数据流图是(15)阶段产生的成果。

- (15) A. 需求分析 B. 总体设计 C. 详细设计 D. 程序编码

● CVS 是一种(16)工具。

- (16) A. 需求分析 B. 编译 C. 程序编码 D. 版本控制

● 以下关于原型化开发方法的叙述中，不正确的是(17)。

- (17) A. 原型化方法适应于需求不明确的软件开发
B. 在开发过程中，可以废弃不用早期构造的软件原型
C. 原型化方法可以直接开发出最终产品
D. 原型化方法利于确认各项系统服务的可用性

● 通常在软件的(18)活动中无需用户参与。

- (18) A. 需求分析 B. 维护 C. 编码 D. 测试

● 进行软件项目的风险分析时，风险避免、风险监控和风险管理及意外事件计划是(19)活动中需要考虑的问题。

- (19) A. 风险识别 B. 风险预测 C. 风险评估 D. 风险控制

● 下面关于编程语言的各种说法中，(20)是正确的。

- (20) A. 由于 C 语言程序是由函数构成的，因此它是一种函数型语言
B. Smalltalk、C++、Java、C#都是面向对象语言
C. 函数型语言适用于编写处理高速计算的程序，常用于超级计算机的模拟计算
D. 逻辑型语言是在 Client/Server 系统中用于实现负载分散的程序语言

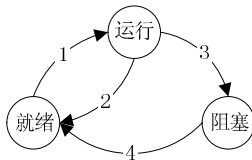
● 在面向对象的语言中，(21)。

- (21) A. 类的实例化是指对类的实例分配存储空间
B. 每个类都必须创建一个实例
C. 每个类只能创建一个实例
D. 类的实例化是指对类进行初始化

● 在统一建模语言 (UML) 中，(22)用于描述系统与外部系统及用户之间的交互。

- (22) A. 类图 B. 用例图 C. 对象图 D. 协作图

● 某系统的进程状态转换如下图所示，图中 1、2、3 和 4 分别表示引起状态转换的不同原因，原因 4 表示(23)；一个进程状态转换会引起另一个进程状态转换的是(24)。



- (23) A. 就绪进程被调度 B. 运行进程执行了 P 操作
C. 发生了阻塞进程等待的事件 D. 运行进程的时间片到了
(24) A. 1→2 B. 2→1 C. 3→2 D. 2→4

● 在操作系统中，虚拟设备通常采用 (25) 设备来提供虚拟设备。

- (25) A. Spooling 技术，利用磁带 B. Spooling 技术，利用磁盘
C. 脱机批处理技术，利用磁盘 D. 通道技术，利用磁带

● 某文件管理系统在磁盘上建立了位示图(bitmap)，记录磁盘的使用情况。若系统中字长为 32 位，磁盘上的物理块依次编号为：0、1、2、...，那么 8192 号物理块的使用情况在位示图中的第 (26) 个字中描述。

- (26) A. 256 B. 257 C. 512 D. 1024

● 某虚拟存储系统采用最近最少使用 (LRU) 页面淘汰算法，假定系统为每个作业分配 3 个页面的主存空间，其中一个页面用来存放程序。现有某作业的部分语句如下：

```
Var A: Array[1..150,1..100] OF integer;
```

```
  i,j: integer;
```

```
  FOR i:=1 to 150 DO
```

```
    FOR j:=1 to 100 DO
```

```
      A[i,j]:=0;
```

设每个页面可存放 150 个整数变量，变量 i、j 放在程序页中。初始时，程序及变量 i、j 已在内存，其余两页为空，矩阵 A 按行序存放。在上述程序片段执行过程中，共产生 (27) 次缺页中断。最后留在内存中的是矩阵 A 的最后 (28)。

- (27) A. 50 B. 100 C. 150 D. 300
(28) A. 2 行 B. 2 列 C. 3 行 D. 3 列

● 统一过程 (UP) 是一种用例驱动的迭代式增量开发过程，每次迭代过程中主要的工作流包括捕获需求、分析、设计、实现和测试等。这种软件过程用例图 (Use Case Diagram) 是通过 (29) 得到的。

- (29) A. 捕获需求 B. 分析 C. 设计 D. 实现

● 序言性注释是指在每个程序或模块开头的一段说明，起辅助理解程序的作用，一般包括：程序的表示、名称和版本号，程序功能描述，接口与界面描述，输入/输出数据说明，开发历史，与运行环境有关的信息等。下列叙述中不属于序言性注释的是 (30)。

- (30) A. 程序对硬件、软件资源的要求
B. 重要变量和参数说明
C. 嵌入在程序中的 SQL 语句
D. 程序开发的原作者、审查者、修改者、编程日期等

● ISO/IEC 9126 软件质量模型中第一层定义了六个质量特性，并为各质量特性定义了相应的质量子特性，其中易分析子特性属于软件的质量特性。

- (31) A. 可靠性 B. 效率 C. 可维护性 D. 功能性

● 软件 (32) 的提高，有利于软件可靠性的提高。

(32) A. 存储效率 B. 执行效率 C. 容错性 D. 可移植性

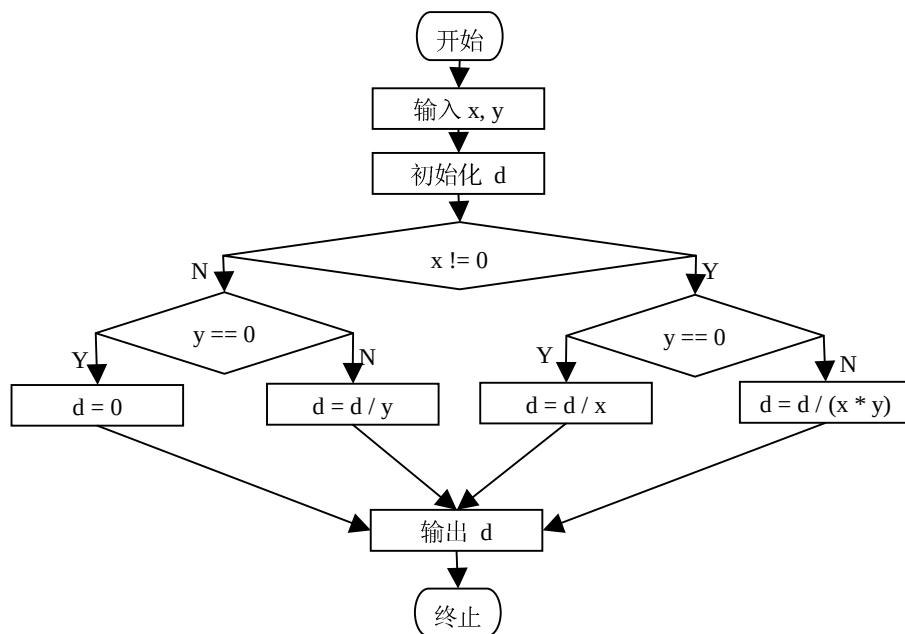
● 软件能力成熟度模型将软件能力成熟度自低到高依次划分为初始级、可重复级、定义级、管理级和优化级，并且高级别成熟度一定可以达到低级别成熟度的要求。其中 (33) 中的开发过程及相应的管理工作均已标准化、文档化，并已建立完善的培训制度和专家评审制度。

(33) A. 可重复级和定义级 B. 定义级和管理级
C. 管理级和优化级 D. 定义级、管理级和优化级

● LOC 是软件规模的一种量度，它表示 (34)。

(34) A. 软件功能数 B. 源代码行数 C. 每单位成本数 D. 工作量

● 阅读下列流程图：



当用判定覆盖法进行测试时，至少需要设计 (35) 个测试用例。

(35) A. 2 B. 4 C. 6 D. 8

● 面向对象分析与设计是面向对象软件开发过程中的两个重要阶段，下列活动中，(36) 不属于面向对象分析阶段。

(36) A. 构建分析模型 B. 识别分析类 C. 确定接口规格 D. 评估分析模型

● 在某大学学生学籍管理信息系统中，假设学生年龄的输入范围为 16~40，则根据黑盒测试中的等价类划分技术，下面划分正确的是(37)。

- (37) A. 可划分为 2 个有效等价类，2 个无效等价类
B. 可划分为 1 个有效等价类，2 个无效等价类
C. 可划分为 2 个有效等价类，1 个无效等价类
D. 可划分为 1 个有效等价类，1 个无效等价类

● 面向对象分析需要找出软件需求中客观存在的所有实体对象（概念），然后归纳、抽象出实体类。(38)是寻找实体对象的有效方法之一。

- (38) A. 会议调查 B. 问卷调查 C. 电话调查 D. 名词分析

● 在“模型—视图—控制器”（MVC）模式中，(39)主要表现用户界面，(40)用来描述核心业务逻辑。

- (39) A. 视图 B. 模型 C. 控制器 D. 视图和控制器
(40) A. 视图 B. 模型 C. 控制器 D. 视图和控制器

● 在进行面向对象设计时，采用设计模式能够(41)。

- (41) A. 复用相似问题的相同解决方案
B. 改善代码的平台可移植性
C. 改善代码的可理解性
D. 增强软件的易安装性

● 下面给出了四种设计模式的作用：

外观（Facade）：为子系统的一组功能调用提供一个一致的接口，这个接口使得这一子系统更加容易使用；

装饰(Decorate)：当不能采用生成子类的方法进行扩充时，动态地给一个对象添加一些额外的功能；

单件（Singleton）：保证一个类仅有一个实例，并提供一个访问它的全局访问点；

模板方法（Template Method）：在方法中定义算法的框架，而将算法中的一些操作步骤延迟到子类中实现。

请根据下面叙述的场景选用适当的设计模式。若某面向对象系统中的某些类有且只有一个实例，那么采用(42)设计模式能够有效达到该目的；该系统中的某子模块需要为其它模块提供访问不同数据库系统（Oracle、SQL Server、DB2 UDB 等）的功能，这些数据库系统提供的访问接口有一定的差异，但访问过程却都是相同的，例如，先连接数据库，再打开数据库，最后对数据进行查询，(43)设计模式可抽象出相同的数据库访问过程；系统中的文本显示类（TextView）和图片显示类（PictureView）都继承了组件类（Component），分别显示文本和图片内容，现需要构造带有滚动条、或者带有黑色边框、或者既有滚动条又有黑色边框的文本显示控件和图片显示控件，但希望最多只增加三个

类, (44) 设计模式可以实现该目的。

- (42) A. 外观 B. 装饰 C. 单件 D. 模板方法
(43) A. 外观 B. 装饰 C. 单件 D. 模板方法
(44) A. 外观 B. 装饰 C. 单件 D. 模板方法

● 在采用标准 UML 构建的用例模型 (Use-Case Model) 中, 参与者 (Actor) 与用例 (Use Case) 是模型中的主要元素, 其中参与者与用例之间可以具有 (45) 关系。

- (45) A. 包含 (include) B. 递归 (Recursive)
C. 关联 (Association) D. 组合 (Composite)

● 当采用标准 UML 构建系统类模型 (Class Model) 时, 若类 B 除具有类 A 的全部特性外, 类 B 还可定义新的特性以及置换类 A 的部分特性, 那么类 B 与类 A 具有 (46) 关系; 若类 A 的对象维持类 B 对象的引用或指针, 并可与类 C 的对象共享相同的类 B 的对象, 那么类 A 与类 B 具有 (47) 关系。

- (46) A. 聚合 B. 泛化 C. 传递 D. 迭代
(47) A. 聚合 B. 泛化 C. 传递 D. 迭代

● 表达式 “(a+b)*(c-d)” 的后缀表示为 (48)。

- (48) A. ab+cd-* B. abcd+ -* C. ab+*cd- D. abcd*+-

● 函数 t()、f() 的定义如下所示, 若调用函数 t 时传递给 x 的值为 3, 并且调用函数 f() 时, 第一个参数采用传值 (call by value) 方式, 第二个参数采用传引用 (call by reference) 方式, 则函数 t 的返回值为 (49)。

t(int x)

```
int a;  
a = 3*x - 1;  
f(x, a);  
return a+x;
```

f(int r, int s)

```
int x;  
x = 2*r + 1 ; s = x*r;  
r = s - x;  
return;
```

- (49) A. 35 B. 24 C. 22 D. 11

● 程序设计语言中 (50)。

- (50) A. while 循环语句的执行效率比 do-while 循环语句的执行效率高
B. while 循环语句的循环体执行次数比循环条件的判断次数多 1, 而 do-while 语句的循环体执行次数比循环条件的判断次数少 1
C. while 语句的循环体执行次数比循环条件的判断次数少 1, 而 do-while 语句的循环体执行次数比循环条件的判断次数多 1
D. while 语句的循环体执行次数比循环条件的判断次数少 1, 而 do-while 语句的循环体执行次数等于循环条件的判断次数

- 关系 R、S 如下图所示，关系代数表达式 $\pi_{1,5,6}(\sigma_{1>5}(R \times S)) =$ (51)。

A	B	C	A	B	C
1	2	3	3	7	11
4	5	6	4	7	6
7	8	9	5	12	13
10	11	12	6	10	14

R

S

(51) A.	A	B	C	B.	A	B	C
	1	12	13		10	7	11
	1	10	14		10	7	6
C.	A	B	C	D.	A	B	C
	7	12	13		4	7	6
	7	10	14		4	7	11

- 在数据库管理系统中，(52) 不属于安全性控制机制。

(52) A. 完整性约束 B. 视图 C. 密码验证 D. 用户授权

- 给定供应关系 SPJ (供应商号, 零件号, 工程号, 数量), 查询某工程至少用了 3 家供应商 (包含 3 家) 供应的零件的平均数量, 并按工程号的降序排列。

SELECT 工程号, (53) FROM SPJ

GROUP BY 工程号

(54)

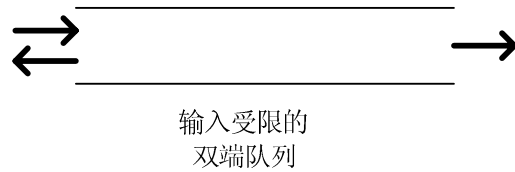
ORDER BY 工程号 DESC;

- (53) A. AVG (数量) At 平均数量 B. AVG (数量) AS 平均数量
C. 平均数量 At AVG (数量) D. 平均数量 AS AVG (数量)
- (54) A. HAVING COUNT (DISTINCT (供应商号)) >2
B. Where COUNT (供应商号) >2
C. HAVING (DISTINCT (供应商号)) >2
D. Where 供应商号 2

- 设关系模式 R(A, B, C), 传递依赖指的是 (55); 下列结论错误的是 (56)。

- (55) A. 若 $A \rightarrow B, B \rightarrow C$, 则 $A \rightarrow C$ B. 若 $A \rightarrow B, A \rightarrow C$, 则 $A \rightarrow BC$
C. 若 $A \rightarrow C$, 则 $AB \rightarrow C$ D. 若 $A \rightarrow BC$, 则 $A \rightarrow B, A \rightarrow C$
- (56) A. 若 $A \rightarrow BC$, 则 $A \rightarrow B, A \rightarrow C$ B. 若 $A \rightarrow B, A \rightarrow C$, 则 $A \rightarrow BC$
C. 若 $A \rightarrow C$, 则 $AB \rightarrow C$ D. 若 $AB \rightarrow C$, 则 $A \rightarrow C, B \rightarrow C$

● 输入受限的双端队列是指元素只能从队列的一端输入、但可以从队列的两端输出，如下图所示。若有 8、1、4、2 依次进入输入受限的双端队列，则得不到输出序列 (57)。

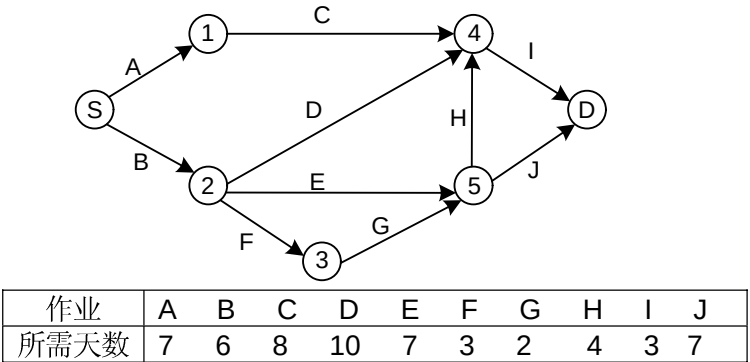


(57) A. 2、8、1、4 B. 1、4、8、2 C. 4、2、1、8 D. 2、1、4、8

● 已知某二叉树的中序序列为 CBDAEFI、先序序列为 ABCDEFI，则该二叉树的高度为 (58)。

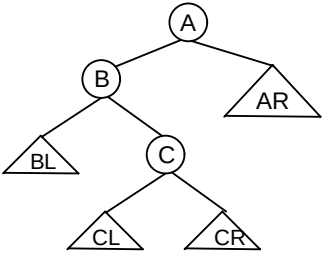
(58) A. 2 B. 3 C. 4 D. 5

● 某工程计划如下图所示，各个作业所需的天数如下表所示，设该工程从第 0 天开工，则该工程的最短工期是 (59) 天，作业 J 最迟应在第 (60) 天开工。



(59) A. 17 B. 18 C. 19 D. 20
 (60) A. 11 B. 13 C. 14 D. 16

● 下图所示平衡二叉树（树中任一结点的左右子树高度之差不超过 1）中，结点 A 的右子树 AR 高度为 h，结点 B 的左子树 BL 高度为 h，结点 C 的左子树 CL、右子树 CR 高度都为 h-1。若在 CR 中插入一个结点并使得 CR 的高度增加 1，则该二叉树 (61)。



- (61) A. 以 B 为根的子二叉树变为不平衡 B. 以 C 为根的子二叉树变为不平衡
C. 以 A 为根的子二叉树变为不平衡 D. 仍然是平衡二叉树

● 设商店有 10 元、5 元、2 元和 1 元的零币，每种零币的数量充足。售货员给顾客找零钱时，零币的数量越少越好。例如给顾客找零 29 元：先选 2 张 10 元币，然后选择 1 张 5 元币，再选择两张 2 元币。以上的找零钱方法采用了 (62) 策略。

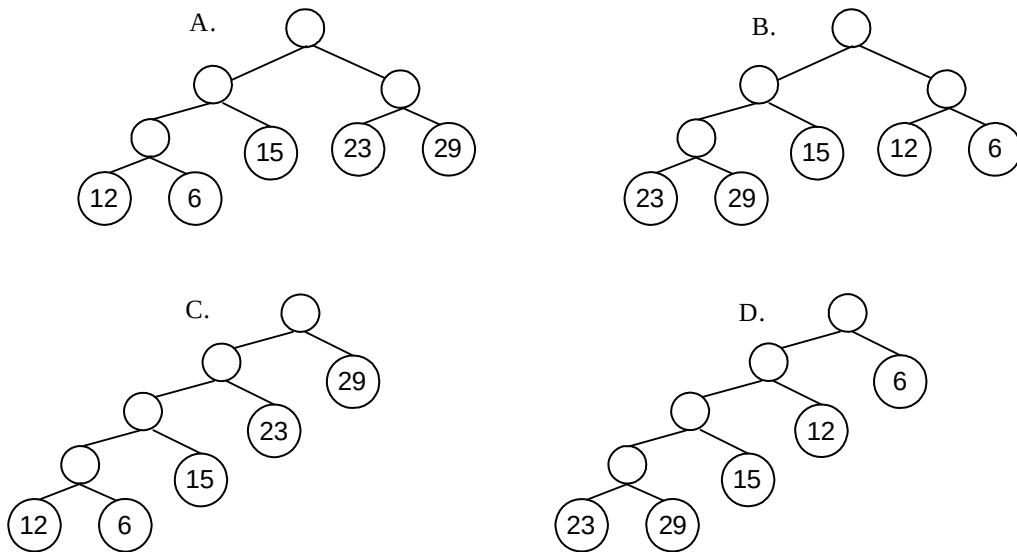
- (62) A. 分治 B. 贪心 C. 动态规划 D. 回溯

● 对 n 个元素的数组进行 (63)，其平均时间复杂度和最坏情况下的时间复杂度都是 $O(n \log n)$ 。

- (63) A. 希尔排序 B. 快速排序 C. 堆排序 D. 选择排序

● 由权值为 29、12、15、6、23 的五个叶子结点构造的哈夫曼树为 (64)，其带权路径长度为 (65)。

(64)



- (65) A. 85 B. 188 C. 192 D. 222

● 关于路由器，下列说法中错误的是 (66)。

- (66) A. 路由器可以隔离子网，抑制广播风暴
B. 路由器可以实现网络地址转换
C. 路由器可以提供可靠性不同的多条路由选择
D. 路由器只能实现点对点的传输

● 关于 ARP 表，以下描述中正确的是 (67)。

- (67) A. 提供常用目标地址的快捷方式来减少网络流量
B. 用于建立 IP 地址到 MAC 地址的映射
C. 用于在各个子网之间进行路由选择
D. 用于进行应用层信息的转换

● 分配给某校园网的地址块是 202.105.192.0/18，该校园网包含 (68) 个 C 类网络。

- (68) A. 6 B. 14 C. 30 D. 62

● 在 Windows 操作系统中，采用 (69) 命令来测试到达目标所经过的路由器数目及 IP 地址。

- (69) A. ping B. tracert C. arp D. nslookup

● 以下关于 DHCP 服务的说法中正确的是 (70)。

- (70) A. 在一个子网内只能设置一台 DHCP 服务器，以防止冲突
B. 在默认情况下，客户机采用最先到达的 DHCP 服务器分配的 IP 地址
C. 使用 DHCP 服务，无法保证某台计算机使用固定 IP 地址
D. 客户端在配置时必须指明 DHCP 服务器 IP 地址，才能获得 DHCP 服务

● (71) analysis emphasizes the drawing of pictorial system models to document and validate both existing and/or proposed systems. Ultimately, the system models become the (72) for designing and constructing an improved system. (73) is such a technique. The emphasis in this technique is process-centered. Systems analysts draw a series of process models called (74). (75) is another such technique that integrates data and process concerns into constructs called objects.

- | | | | |
|-----------------------------|----------------------------|-----------------------------|--------------|
| (71) A. Prototyping | B. Accelerated | C. Model-driven | D. Iterative |
| (72) A. image | B. picture | C. layout | D. blueprint |
| (73) A. Structured analysis | B. Information Engineering | | |
| | C. Discovery Prototyping | D. Object-Oriented analysis | |
| (74) A. PERT | B. DFD | C. ERD | D. UML |
| (75) A. Structured analysis | B. Information Engineering | | |
| | C. Discovery Prototyping | D. Object-Oriented analysis | |

全国计算机技术与软件专业技术资格（水平）考试

2007 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2007 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是(1)月(2)日。

因为正确的解答是“5 月 26 日”，故在答题纸的对应栏内写上“5”和“26”（参看下表）。

例题	解答栏
(1)	5
(2)	26

试题一（共 15 分）

阅读以下说明和图，回答问题1至问题3，将解答填入答题纸的对应栏内。

【说明】

某房屋租赁公司欲建立一个房屋租赁服务系统，统一管理房主和租赁者的信息，从而快速地提供租赁服务。该系统具有以下功能：

1. 登记房主信息。对于每名房主，系统需登记其姓名、住址和联系电话，并将这些信息写入房主信息文件。

2. 登记房屋信息。所有在系统中登记的房屋都有一个唯一的识别号（对于新增加的房屋，系统会自动为其分配一个识别号）。除此之外，还需登记该房屋的地址、房型（如平房、带阳台的楼房、独立式住宅等）、最多能够容纳的房客数、租金及房屋状态（待租赁、已出租）。这些信息都保存在房屋信息文件中。一名房主可以在系统中登记多个待租赁的房屋。

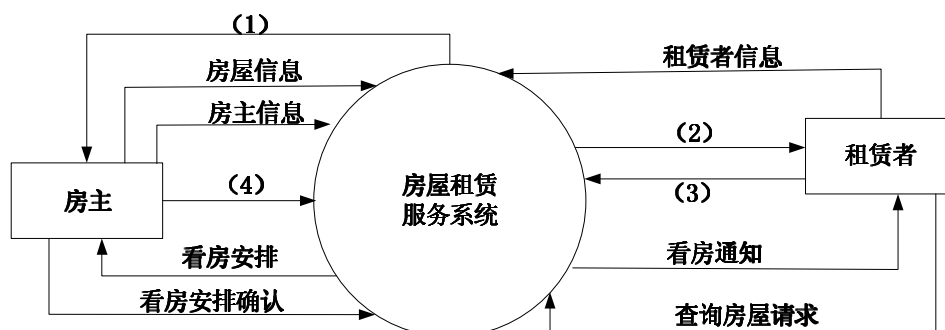
3. 登记租赁者信息。所有想通过该系统租赁房屋的租赁者，必须首先在系统中登记个人信息，包括：姓名、住址、电话号码、出生年月和性别。这些信息都保存在租赁者信息文件中。

4. 租赁房屋。已经登记在系统中的租赁者，可以得到一份系统提供的待租赁房屋列表。一旦租赁者从中找到合适的房屋，就可以提出看房请求。系统会安排租赁者与房主见面。对于每次看房，系统会生成一条看房记录并将其写入看房记录文件中。

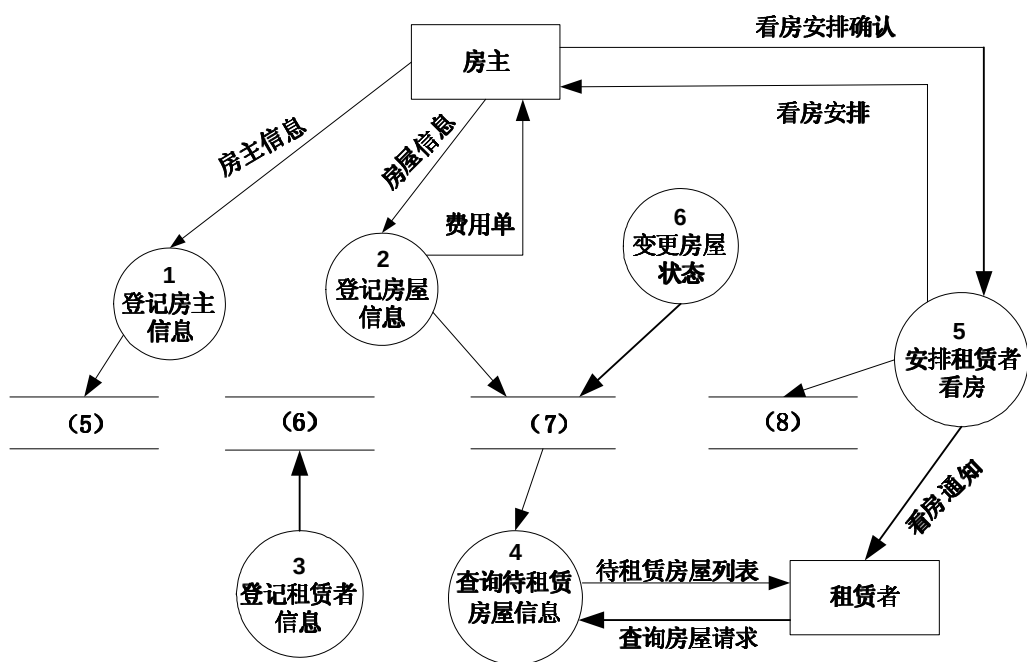
5. 收取手续费。房主登记完房屋后，系统会生成一份费用单，房主根据费用单交纳相应的费用。

6. 变更房屋状态。当租赁者与房主达成租房或退房协议后，房主向系统提交变更房屋状态的请求。系统将根据房主的请求，修改房屋信息文件。

数据流图 1-1 和 1-2 分别给出了该系统的顶层数据流图和 0 层数据流图。



数据流图 1-1



数据流图 1-2

[问题 1] (4 分)

使用[说明]中给出的词汇，将数据流图 1-1 中 (1) ~ (4) 处的数据流补充完整。

[问题 2] (4 分)

使用[说明]中给出的词汇，将数据流图 1-2 中的 (5) ~ (8) 补充完整。

[问题 3] (7 分)

数据流程图 1-2 中缺失了三条数据流，请指出这三条数据流的起点、终点和数据流名称。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

某医院的门诊管理系统实现了为患者提供挂号、处方药品收费的功能。具体的需求及设计如下：

1. 医院医师具有编号，姓名，科室，职称，出诊类型和出诊费用，其中出诊类型分为专家门诊和普通门诊，与医师职称无关；各个医师可以具有不同的出诊费用，与职称和出诊类型无关。

2. 患者首先在门诊挂号处挂号，选择科室和医师，根据选择的医师缴纳挂号费（医师出诊费）。收银员为患者生成挂号单，如表 2-1 所示，其中，就诊类型为医师的出诊类型。

表 2-1 XX 医院门诊挂号单

收银员：13011

时间：2007 年 2 月 1 日 08:58

就诊号	姓名	科室	医师	就诊类型	挂号费
20070205015	叶萌	内科	杨玉明	专家门诊	5 元

3. 患者在医师处就诊后，凭借挂号单和医师手写处方到门诊药房交费买药。收银员根据就诊号和医师处方中开列的药品信息，查询药品库（如表 2-2 所示）并生成门诊处方单（如表 2-3 所示）。

表 2-2 药品库

药品编码	药品名称	类型	库存	货架编号	单位	规格	单价
12007	牛蒡子	中药	51590	B1401	G	炒	0.0340
11090	百部	中药	36950	B1523	G	片	0.0313

表 2-3 XX 医院门诊处方单

时间：2007 年 2 月 1 日 10:31

就诊号	20070205015	病人姓名	叶萌	医师姓名	杨玉明
金额总计	0.65	项目总计	2	收银员	21081
药品编码	药品名称	数量	单位	单价	金额(元)
12007	牛蒡子	10	G	0.0340	0.34
11090	百部	10	G	0.0313	0.31

4. 由于药品价格会发生变化，因此，门诊管理系统必须记录处方单上药品的单价。

根据需求阶段收集的信息，设计的实体联系图和关系模式（不完整）如下所示：

1. 实体联系图

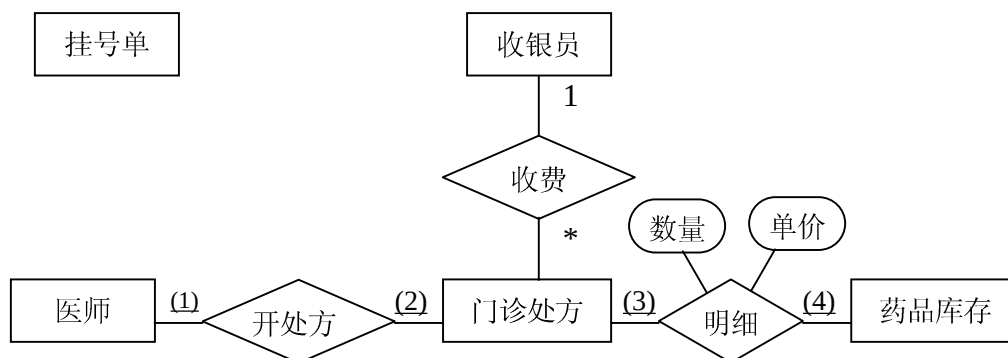


图 2-1 实体联系图

2. 关系模式

挂号单（就诊号，病患姓名，医师编号，时间，(5)）

收银员（编号，姓名，级别）

医师（编号，姓名，科室，职称，出诊类型，出诊费用）

门诊处方（(6)，收银员，时间）

处方明细（就诊号，(7)）

药品库（药品编码，药品名称，(8)）

[问题 1]（4 分）

根据问题描述，填写 2-1 实体联系图中(1)~(4)处联系的类型。

[问题 2]（4 分）

图 2-1 中还缺少几个联系？请指出每个联系两端的实体名，格式如下：

实体 1：实体 2

例如，收银员与门诊处方之间存在联系，表示为：

收银员：门诊处方 或 门诊处方：收银员

[问题 3]（7 分）

根据实体联系图 2-1，填写挂号单、门诊处方、处方明细和药品库关系模式中的空（5）~（8）处，并指出挂号单、门诊处方和处方明细关系模式的主键。

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

[说明]

某图书管理系统的主要功能如下：

- 1. 图书管理系统的资源目录中记录着所有可供读者借阅的资源，每项资源都有一个唯一的索引号。系统需登记每项资源的名称、出版时间和资源状态（可借阅或已借出）。
- 2. 资源可以分为两类：图书和唱片。对于图书，系统还需登记作者和页数；对于唱片，还需登记演唱者和介质类型（CD 或者磁带）。
- 3. 读者信息保存在图书管理系统的读者信息数据库中，记录的信息包括：读者的识别码和读者姓名。系统为每个读者创建了一个借书记录文件，用来保存读者所借资源的相关信息。

现采用面向对象方法开发该图书管理系统。识别类是面向对象分析的第一步。比较常用的识别类的方法是寻找问题描述中的名词，再根据相关规则从这些名词中删除不可能成为类的名词，最终得到构成该系统的类。表 3-1 给出了[说明]中出现的所有名词。

表 3-1

图书管理系统	资源目录	读者	资源
索引号	系统	名称	出版时间
资源状态	图书	唱片	作者
页数	演唱者	介质类型	CD
磁带	读者信息	读者信息数据库	识别码
姓名	借书记录文件	信息	

通过对表 3-1 中的名词进行分析，最终得到了图 3-1 所示的 UML 类图（类的说明如表 3-2 所示）。

表 3-2

类 名	说 明
LibrarySystem	图书管理系统
BorrowerDB	保存读者信息的数据库
CatalogItem	资源目录中保存的每项资源
Borrower	读者
BorrowerItems	为每个读者创建的借书记录文件

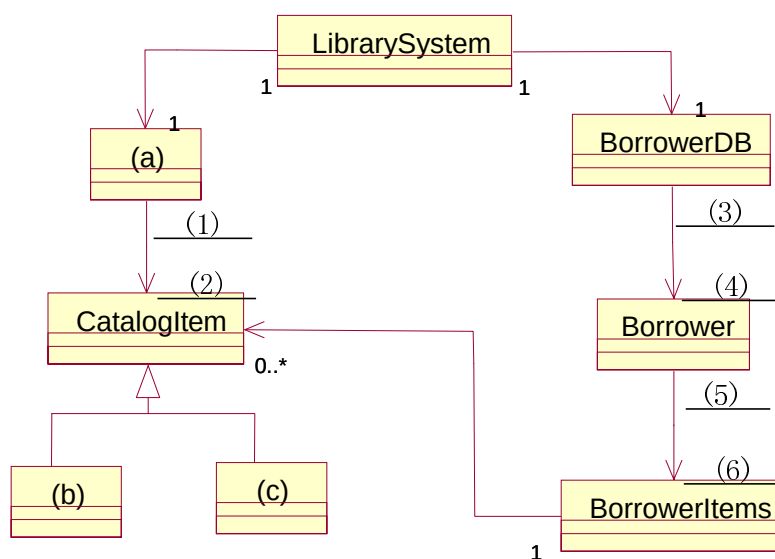


图 3-1

[问题 1] (3 分)

表 3-2 所给出的类并不完整，根据[说明]和表 3-1，将图 3-1 中的 (a) ~ (c) 处补充完整。

[问题 2] (6 分)

根据[说明]中的描述，给出图 3-1 中的类 CatalogItem 以及 (b)、(c) 处所对应的类的关键属性（使用表 3-1 中给出的词汇），其中，CatalogItem 有 4 个关键属性；(b)、(c) 处对应的类各有 2 个关键属性。

[问题 3] (6 分)

识别关联的多重度是面向对象建模过程中的一个重要步骤。根据[说明]中给出的描述，完成图 3-1 中的 (1)~(6)。

试题四(共 15 分)

阅读以下说明和图，填补流程图中的空缺，将解答填入答题纸的对应栏内。

[说明]

在一条农村公路的一边稀疏地分布着房子，其分布如图 4-1 所示。某电信公司需要在某些位置放置蜂窝电话基站，由于基站的覆盖范围是 6 公里，因此必须使得每栋房子到某个基站的直线距离不超过 6 公里。为简化问题，假设所有房子在同一直线上，并且基站沿该直线放置。现采用贪心策略实现用尽可能少的基站覆盖所有的房子。

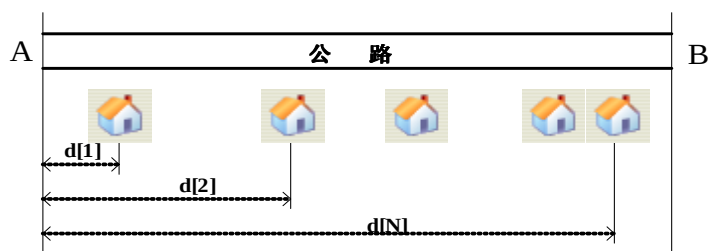


图 4-1

实现贪心算法的流程如图 4-2 所示，请填充其中空白并计算该算法的时间复杂度，其中：

1. $d[i](1 \leq i \leq N)$ 表示第 i 个房子到公路 A 端的距离， N 表示房子的总数，房子的编号按照房子到公路 A 端的距离从小到大进行编号。

2. $s[k]$ 表示第 k ($k \geq 1$) 个基站到公路 A 端的距离，算法结束后 k 的值为基站的总数。

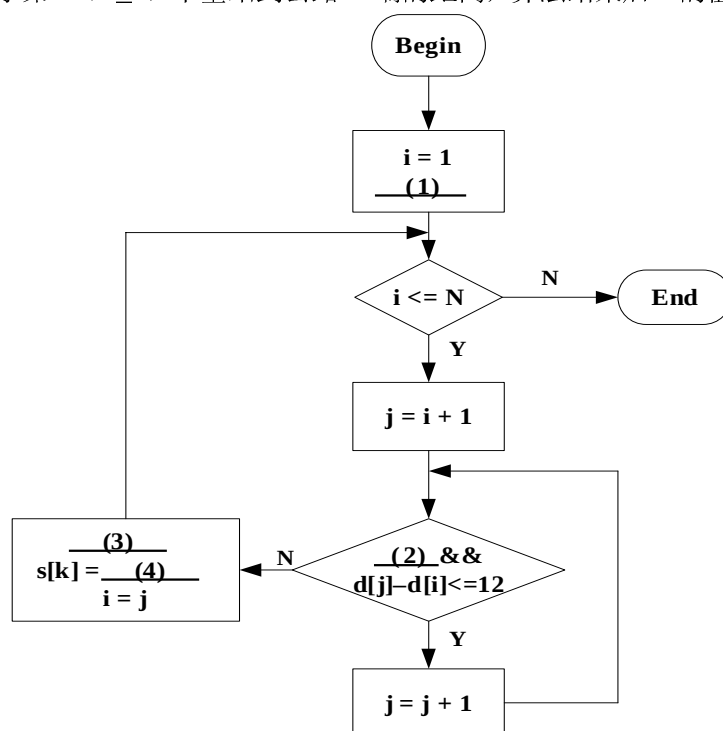


图 4-2

该算法的时间复杂度为 (5)。

从下列的3道试题（试题五至试题七）中任选1道解答。
如果解答的试题数超过1道，则题号小的1道解答有效。

试题五（共15分）

阅读以下说明和C语言函数，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

在一个分布网络中，资源（石油、天然气、电力等）可从生产地送往其他地方。在传输过程中，资源会有损耗。例如，天然气的气压会减少，电压会降低。我们将需要输送的资源信息称为信号。在信号从信源地送往消耗地的过程中，仅能容忍一定范围的信号衰减，称为容忍值。分布网络可表示为一个树型结构，如图5-1所示。信号源是树根，树中的每个节点（除了根）表示一个可以放置放大器的子节点，其中某些节点同时也是信号消耗点，信号从一个节点流向其子节点。

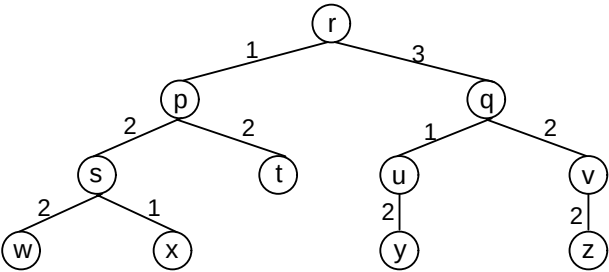


图 5-1

每个节点有一个 d 值，表示从其父节点到该节点的信号衰减量。例如，在图 5-1 中，节点 w 、 p 、 q 的 d 值分别为 2、1、3，树根节点表示信号源，其 d 值为 0。

每个节点有一个 M 值，表示从该节点出发到其所有叶子的信号衰减量的最大值。显然，叶子节点的 M 值为 0。对于非叶子节点 j ， $M(j) = \max\{M(k) + d(k) \mid k \text{ 是 } j \text{ 的孩子节点}\}$ 。在此公式中，要计算节点的 M 值，必须先算出其所有子节点的 M 值。

在计算 M 值的过程中，对于某个节点 i ，其有一个子节点 k 满足 $d(k) + M(k)$ 大于容忍值，则应在 k 处放置放大器，否则，从节点 i 到某叶子节点的信号衰减量会超过容忍值，使得到达该叶子节点时信号不可用，而在节点 i 处放置放大器并不能解决到达叶子节点的信号衰减问题。

例如，在图 5-1 中，从节点 p 到其所有叶子节点的最大衰减值为 4。若容忍值为 3，则必须在 s 处放置信号放大器，这样可使得节点 p 的 M 值为 2。同样，需要在节点 q 、 v 处放置信号放大器，如图 5-2 阴影节点所示。若在某节点放置了信号放大器，则从该节点输出的信号与信号源输出的信号等价。

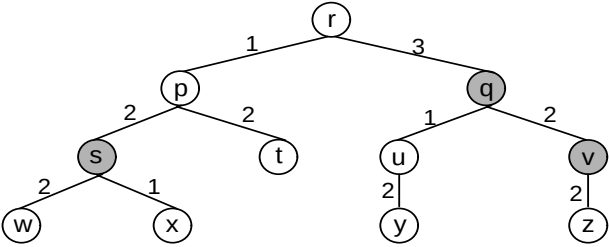


图 5-2

函数 placeBoosters(TreeNode *root)的功能是：对于给定树型分布网络中各个节点，计算其信号衰减量的最大值，并确定应在树中的哪些节点放置信号放大器。

全局变量 Tolerance 保存信号衰减容忍值。

树的节点类型定义如下：

```
typedef struct TreeNode {
    int id;                        /*当前节点的识别号*/
    int ChildNum;                 /*当前节点的子节点数目*/
    int d;                       /*父节点到当前节点的信号衰减值*/
    struct TreeNode **childptr; /*向量，存放当前节点到其所有子节点的指针*/
    int M;                       /*当前节点到其所有子节点的信号衰减值中的最大值*/
    bool boost;                  /*是否在当前节点放置信号放大器的标志*/
}TreeNode;
```

[C 语言函数]

```
void placeBoosters(TreeNode *root )
{ /* 计算 root 所指节点处的衰减量，如果衰减量超出了容忍值，则放置放大器 */
    TreeNode *p;
    int i, degradation;
    if (__(1)__) {
        degradation = 0; root->M = 0;
        i = 0;
        if (i >= root->ChildNum)
            return;
        p = __(2)__;
        for(; i < root->ChildNum && p; i++, p = __(3)__) {
            p->M = 0;
            __(4)__;
            if (p->d + p->M > Tolerance) { /*在 p 所指节点中放置信号放大器*/
                p->boost = true;
                p->M = 0;
            }
            if (p->d + p->M > degradation)
                degradation = p->d + p->M;
        }
        root -> M = __(5)__;
    }
}
```

试题六（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。
[说明]

某游戏公司现欲开发一款面向儿童的模拟游戏，该游戏主要模拟现实世界中各种鸭子的发声特征、飞行特征和外观特征。游戏需要模拟的鸭子种类及其特征如表 6-1 所示：

表 6-1

鸭子种类	发声特征	飞行特征	外观特征
灰鸭（MallardDuck）	发出“嘎嘎”声（Quack）	用翅膀飞行（FlyWithWings）	灰色羽毛
红头鸭（RedHeadDuck）	发出“嘎嘎”声（Quack）	用翅膀飞行（FlyWithWings）	灰色羽毛、头部红色
棉花鸭（CottonDuck）	不发声（QuackNoWay）	不能飞行（FlyNoWay）	白色
橡皮鸭（RubberDuck）	发出橡皮与空气摩擦的声（Squeak）	不能飞行（FlyNoWay）	黑白橡皮颜色

为支持将来能够模拟更多种类鸭子的特征，采用策略设计模式(Strategy)设计的类图如图 6-1 所示：

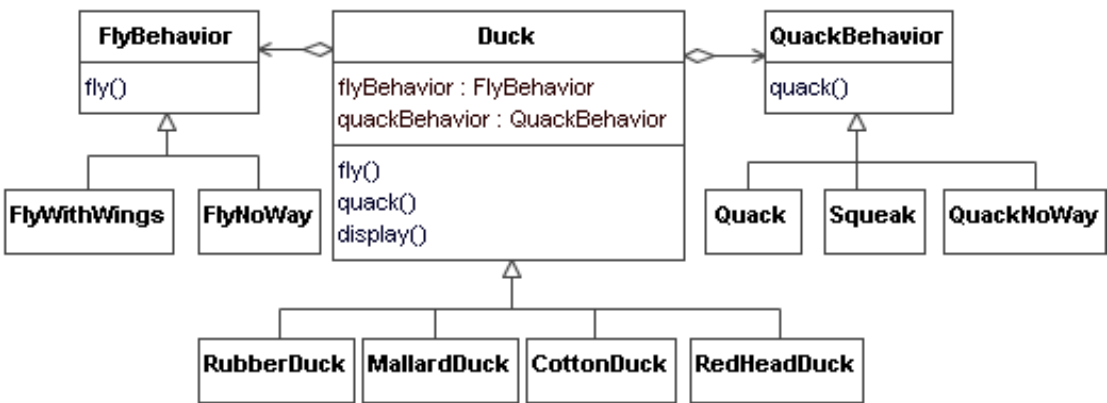


图 6-1

其中，Duck 为抽象类，描述了抽象的鸭子，而类 RubberDuck、MallardDuck、CottonDuck 和 RedHeadDuck 分别描述具体的鸭子种类，方法 fly()、quack()和 display()分别表示不同种类的鸭子都具有飞行特征、发声特征和外观特征；类 FlyBehavior 与 QuackBehavior 为抽象类，分别用于表示抽象的飞行行为与发声行为；类 FlyNoWay 与 FlyWithWings 分别描述不能飞行的行为和用翅膀飞行的行为；类 Quack、Squeak 与 QuackNoWay 分别描述发出“嘎嘎”声的行为、发出橡皮与空气摩擦声的行为与不发声的行为。请填补以下代码中的空缺。

[C++代码]

```

#include<iostream>
using namespace (1);
class FlyBehavior {

```

```

        public : __ (2) __ fly() = 0;
    };
    class QuackBehavior {
        public: __ (3) __ quack() = 0;
    };
    class FlyWithWings:public FlyBehavior{
        public: void fly() { cout << "使用翅膀飞行 !" << endl; }
    };
    class FlyNoWay:public FlyBehavior{
        public: void fly() { cout << "不能飞行 !" << endl; }
    };
    class Quack:public QuackBehavior{
        public: void quack() { cout << "发出\`嘎嘎\`声 !" << endl; }
    };
    class Squeak:public QuackBehavior{
        public: void quack() { cout << "发出空气与橡皮摩擦声 !" << endl; }
    };
    class QuackNoWay:public QuackBehavior{
        public: void quack () { cout << "不能发声 !" << endl; }
    };
    class Duck {
    protected:
        FlyBehavior * __ (4) __;
        QuackBehavior * __ (5) __;
    public:
        void fly() { __ (6) __; }
        void quack() { __ (7) __; };
        virtual void display()=0;
    };
    class RubberDuck: public Duck {
    public:
        RubberDuck() {
            flyBehavior = new __ (8) __;
            quackBehavior = new __ (9) __;
        }
        ~RubberDuck() {
            if(!flyBehavior) delete flyBehavior;
            if(!quackBehavior) delete quackBehavior;
        }
        void display() { /*此处省略显示橡皮鸭的代码 */ }
    };
    //其它代码省略

```


试题七（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

[说明]

某游戏公司现欲开发一款面向儿童的模拟游戏，该游戏主要模拟现实世界中各种鸭子的发声特征、飞行特征和外观特征。游戏需要模拟的鸭子种类及其特征如表 7-1 所示：

表 7-1

鸭子种类	发声特征	飞行特征	外观特征
灰鸭（MallardDuck）	发出“嘎嘎”声（Quack）	用翅膀飞行（FlyWithWings）	灰色羽毛
红头鸭（RedHeadDuck）	发出“嘎嘎”声（Quack）	用翅膀飞行（FlyWithWings）	灰色羽毛、头部红色
棉花鸭（CottonDuck）	不发声（QuackNoWay）	不能飞行（FlyNoWay）	白色
橡皮鸭（RubberDuck）	发出橡皮与空气摩擦的声（Squeak）	不能飞行（FlyNoWay）	黑白橡皮颜色

为支持将来能够模拟更多种类鸭子的特征，采用策略设计模式(Strategy)设计的类图如图 7-1 所示：

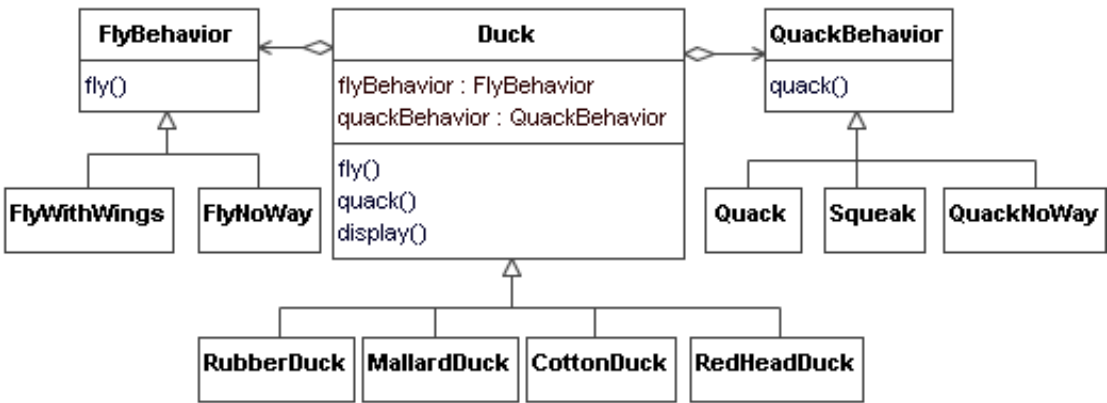


图 7-1

其中，Duck 为抽象类，描述了抽象的鸭子，而类 RubberDuck、MallardDuck、CottonDuck 和 RedHeadDuck 分别描述具体的鸭子种类，方法 fly()、quack()和 display()分别表示不同种类的鸭子都具有飞行特征、发声特征和外观特征；接口 FlyBehavior 与 QuackBehavior 分别用于表示抽象的飞行行为与发声行为；类 FlyNoWay 与 FlyWithWings 分别描述不能飞行的行为和用翅膀飞行的行为；类 Quack、Squeak 与 QuackNoWay 分别描述发出“嘎嘎”声的行为、发出橡皮与空气摩擦声的行为与不发声的行为。请填补以下代码中的空缺。

[Java 代码]

(1) FlyBehavior {
 public void fly();

```

};
(2) QuackBehavior {
    public void quack();
};
class FlyWithWings implements FlyBehavior{
    public void fly(){ System.out.println("使用翅膀飞行 ! "); }
};
class FlyNoWay implements FlyBehavior{
    public void fly(){ System.out.println("不能飞行 ! "); }
};
class Quack implements QuackBehavior{
    public void quack(){ System.out.println("发出\`嘎嘎\`声 ! "); }
};
class Squeak implements QuackBehavior{
    public void quack(){ System.out.println("发出空气与橡皮摩擦声 ! "); }
};
class QuackNoWay implements QuackBehavior{
    public void quack () { System.out.println("不能发声 ! "); }
};
abstract class Duck {
    protected FlyBehavior (3);
    protected QuackBehavior (4);
    public void fly(){ (5); }
    public void quack() { (6); };
    public (7) void display();
};
class RubberDuck extends Duck {
    public RubberDuck(){
        flyBehavior = new (8);
        quackBehavior = new (9);
    }
    public void display(){ /* 此处省略显示橡皮鸭的代码 */ }
};
//其它代码省略

```

全国计算机技术与软件专业技术资格（水平）考试

2007 年下半年 软件设计师 上午试卷

（考试时间 9:00~11:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2007 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
____（88）____月____（89）____日。

（88） A. 12 B. 11 C. 10 D. 9

（89） A. 6 B. 5 C. 4 D. 3

因为考试日期是“11 月 3 日”，故（88）选 B，（89）选 D，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 D 填涂（参看答题卡）。

● 在指令系统的各种寻址方式中, 获取操作数最快的方式是 (1)。若操作数的地址包含在指令中, 则属于 (2) 方式。

- (1) A. 直接寻址 B. 立即寻址 C. 寄存器寻址 D. 间接寻址
(2) A. 直接寻址 B. 立即寻址 C. 寄存器寻址 D. 间接寻址

● 系统响应时间和作业吞吐量是衡量计算机系统性能的重要指标。对于一个持续处理业务的系统而言, (3), 表明其性能越好。

- (3) A. 响应时间越短, 作业吞吐量越小 B. 响应时间越短, 作业吞吐量越大
C. 响应时间越长, 作业吞吐量越大 D. 响应时间不会影响作业吞吐量

● 若每一条指令都可以分解为取指、分析和执行三步。已知取指时间 $t_{\text{取指}}=4\Delta t$, 分析时间 $t_{\text{分析}}=3\Delta t$, 执行时间 $t_{\text{执行}}=5\Delta t$ 。如果按串行方式执行完 100 条指令需要 (4) Δt 。如果按照流水方式执行, 执行完 100 条指令需要 (5) Δt 。

- (4) A. 1190 B. 1195 C. 1200 D. 1205
(5) A. 504 B. 507 C. 508 D. 510

● 若内存地址区间为 4000H~43FFH, 每个存储单元可存储 16 位二进制数, 该内存区域用 4 片存储器芯片构成, 则构成该内存所用的存储器芯片的容量是 (6)。

- (6) A. $512 \times 16\text{bit}$ B. $256 \times 8\text{bit}$ C. $256 \times 16\text{bit}$ D. $1024 \times 8\text{bit}$

● 某 Web 网站向 CA 申请了数字证书。用户登录该网站时, 通过验证 (7), 可确认该数字证书的有效性, 从而 (8)。

- (7) A. CA 的签名 B. 网站的签名 C. 会话密钥 D. DES 密码
(8) A. 向网站确认自己的身份 B. 获取访问网站的权限
C. 和网站进行双向认证 D. 验证该网站的真伪

● 实现 VPN 的关键技术主要有隧道技术、加解密技术、(9) 和身份认证技术。

- (9) A. 入侵检测技术 B. 病毒防治技术
C. 安全审计技术 D. 密钥管理技术

● 若某人持有盗版软件, 但他本人确实不知道该软件是盗版的, 则 (10) 承担侵权责任。

- (10) A. 应由该软件的持有者 B. 应由该软件的提供者
C. 应由该软件的提供者和持有者共同 D. 该软件的提供者和持有者都不

● (11) 不属于知识产权的范围。

- (11) A. 地理标志权 B. 物权 C. 邻接权 D. 商业秘密权

● W3C 制定了同步多媒体集成语言规范，称为 (12) 规范。

(12) A. XML B. SMIL C. VRML D. SGML

● 对同一段音乐可以选用 MIDI 格式或 WAV 格式来记录存储。以下叙述中 (13) 是不正确的。

- (13) A. WAV 格式的音乐数据量比 MIDI 格式的音乐数据量大
B. 记录演唱会实况不能采用 MIDI 格式的音乐数据
C. WAV 格式的音乐数据没有体现音乐的曲谱信息
D. WAV 格式的音乐数据和 MIDI 格式的音乐数据都能记录音乐波形信息

● 设计制作一个多媒体地图导航系统，使其能根据用户需求缩放地图并自动搜索路径，最适合的地图数据应该是 (14)。

(14) A. 真彩色图像 B. 航拍图像 C. 矢量化图形 D. 高清晰灰度图像

● 给定 C 语言的数据结构

```
struct T {  
    int w;  
    union T { char c; int i; double d; } U;  
};
```

假设 char 类型变量的存储区大小是 1 字节，int 类型变量的存储区大小是 4 字节，double 类型变量的存储区大小是 8 字节，则在不考虑字对齐方式的情况下，为存储一个 struct T 类型变量所需要的存储区域至少应为 (15) 字节。

(15) A. 4 B. 8 C. 12 D. 17

● 在过程式程序设计 (①)、数据抽象程序设计 (②)、面向对象程序设计 (③)、泛型 (通用) 程序设计 (④) 中，C++ 语言支持 (16)，C 语言支持 (17)。

(16) A. ① B. ②③ C. ③④ D. ①②③④

(17) A. ① B. ①③ C. ②③ D. ①②③④

● C 语言是一种 (18) 语言。

(18) A. 编译型 B. 解释型 C. 编译、解释混合型 D. 脚本

● 采用 UML 进行软件建模过程中，类图是系统的一种静态视图，用 (19) 可明确表示两类事物之间存在的整体/部分形式的关联关系。

(19) A. 依赖关系 B. 聚合关系 C. 泛化关系 D. 实现关系

● 若程序运行时系统报告除数为 0，这属于 (20) 错误。

(20) A. 语法 B. 语用 C. 语义 D. 语境

● 集合 $L = \{a^m b^m \mid m \geq 0\}$ (21)。

- (21) A. 可用正规式 “ $a^* b^*$ ” 表示
 B. 不能用正规式表示, 但可用非确定的有限自动机识别
 C. 可用正规式 “ $a^m b^m$ ” 表示
 D. 不能用正规式表示, 但可用上下文无关文法表示

● 表达式 “ $X = A + B \times (C - D) / E$ ” 的后缀表示形式可以为 (22) (运算符优先级相同时, 遵循左结合的原则)。

- (22) A. $XAB + CDE / - \times =$ B. $XA + BC - DE / \times =$
 C. $XABCD - \times E / + =$ D. $XABCDE + \times - / =$

● 设备驱动程序是直接与 (23) 打交道的软件模块。一般而言, 设备驱动程序的任务是接受来自与设备 (24)。

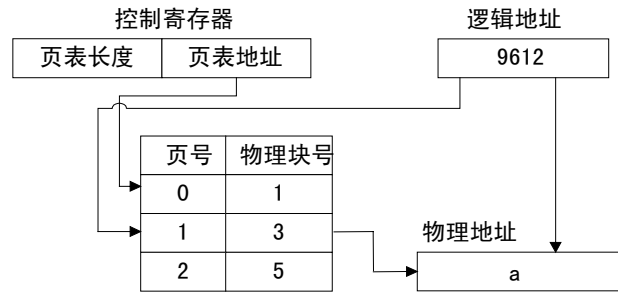
- (23) A. 硬件 B. 办公软件 C. 编译程序 D. 连接程序
 (24) A. 有关的上层软件的抽象请求, 进行与设备相关的处理
 B. 无关的上层软件的抽象请求, 进行与设备相关的处理
 C. 有关的上层软件的抽象请求, 进行与设备无关的处理
 D. 无关的上层软件的抽象请求, 进行与设备无关的处理

● 某系统中有四种互斥资源 R1、R2、R3 和 R4, 可用资源数分别为 3、5、6 和 8。假设在 T_0 时刻有 P1、P2、P3 和 P4 四个进程, 并且这些进程对资源的最大需求量和已分配资源数如下表所示, 那么在 T_0 时刻系统中 R1、R2、R3 和 R4 的剩余资源数分别为 (25)。如果从 T_0 时刻开始进程按 (26) 顺序逐个调度执行, 那么系统状态是安全的。

资源 进程	最大需求量				已分配资源数			
	R1	R2	R3	R4	R1	R2	R3	R4
P1	1	2	3	6	1	1	2	4
P2	1	1	2	2	0	1	2	2
P3	1	2	1	1	1	1	1	0
P4	1	1	2	3	1	1	1	1

- (25) A. 3、5、6 和 8 B. 3、4、2 和 2 C. 0、1、2 和 1 D. 0、1、0 和 1
 (26) A. $P1 \rightarrow P2 \rightarrow P4 \rightarrow P3$ B. $P2 \rightarrow P1 \rightarrow P4 \rightarrow P3$
 C. $P3 \rightarrow P2 \rightarrow P1 \rightarrow P4$ D. $P4 \rightarrow P2 \rightarrow P3 \rightarrow P1$

● 页式存储系统的逻辑地址是由页号和页内地址两部分组成, 地址变换过程如下图所示。假定页面的大小为 8K, 图中所示的十进制逻辑地址 9612 经过地址变换后, 形成的物理地址 a 应为十进制 (27)。



- (27) A. 42380 B. 25996 C. 9612 D. 8192

● 若文件系统容许不同用户的文件可以具有相同的文件名，则操作系统应采用(28)来实现。

- (28) A. 索引表 B. 索引文件 C. 指针 D. 多级目录

● 在软件开发中，(29)不能用来描述项目开发的进度安排。在其他三种图中，可用(30)动态地反映项目开发进展情况。

- (29) A. 甘特图 B. PERT 图 C. PERT/CPM 图 D. 鱼骨图
(30) A. 甘特图 B. PERT 图 C. PERT/CPM 图 D. 鱼骨图

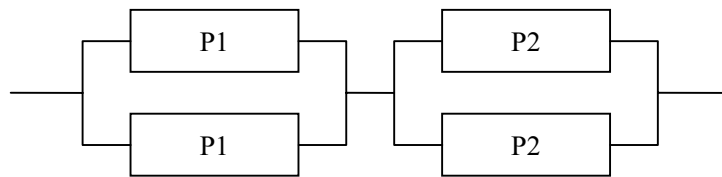
● 选择软件开发工具时，应考虑功能、(31)、稳健性、硬件要求和性能、服务和支持。

- (31) A. 易用性 B. 易维护性 C. 可移植性 D. 可扩充性

● 内聚性和耦合性是度量软件模块独立性的重要准则，软件设计时应力求(32)。

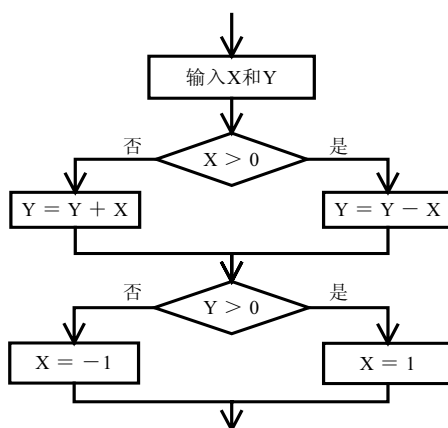
- (32) A. 高内聚，高耦合 B. 高内聚，低耦合
C. 低内聚，高耦合 D. 低内聚，低耦合

● 某大型软件系统按功能可划分为 2 段 P1 和 P2。为提高系统可靠性，软件应用单位设计了如下图给出的软件冗余容错结构，其中 P1 和 P2 均有一个与其完全相同的冗余备份。若 P1 的可靠度为 0.9，P2 的可靠度为 0.9，则整个系统的可靠度是(33)。



- (33) A. 0.6561 B. 0.81 C. 0.9801 D. 0.9

● 对于如下的程序流程，当采用语句覆盖法设计测试案例时，至少需要设计 (34) 个测试案例。



(34) A. 1 B. 2 C. 3 D. 4

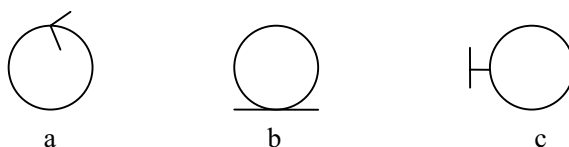
● 为验证程序模块 A 是否正确实现了规定的功能，需要进行 (35)；为验证模块 A 能否与其他模块按照规定方式正确工作，需要进行 (36)。

(35) A. 单元测试 B. 集成测试 C. 确认测试 D. 系统测试
 (36) A. 单元测试 B. 集成测试 C. 确认测试 D. 系统测试

● (37) 表示了系统与参与者之间的接口。在每一个用例中，该对象从参与者处收集信息，并将之转换为一种被实体对象和控制对象使用的形式。

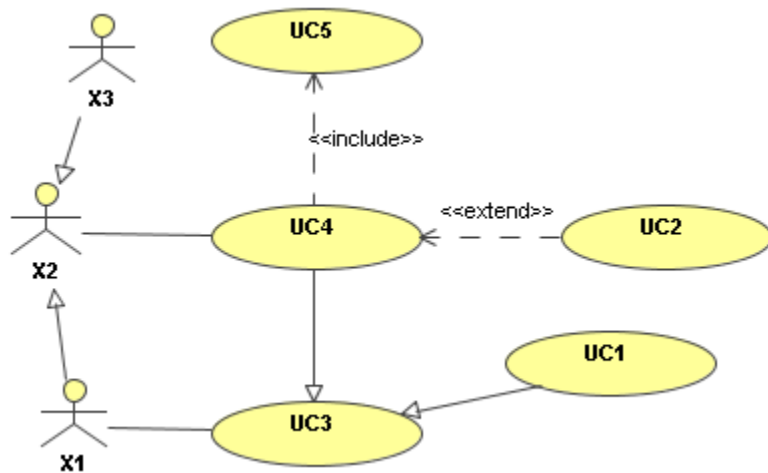
(37) A. 边界对象 B. 可视化对象 C. 抽象对象 D. 实体对象

● 在 UML 语言中，下图中的 a、b、c 三种图形符号按照顺序分别表示 (38)。



(38) A. 边界对象、实体对象、控制对象 B. 实体对象 、边界对象、控制对象
 C. 控制对象、实体对象、边界对象 D. 边界对象 、控制对象、实体对象

● 在下面的用例图 (UseCase Diagram) 中，X1、X2 和 X3 表示 (39)，已知 UC3 是抽象用例，那么 X1 可通过 (40) 用例与系统进行交互。并且，用例 (41) 是 UC4 的可选部分，用例 (42) 是 UC4 的必须部分。



- (39) A. 人 B. 系统 C. 参与者 D. 外部软件
- (40) A. UC4、UC1 B. UC5、UC1 C. UC5、UC2 D. UC1、UC2
- (41) A. UC1 B. UC2 C. UC3 D. UC5
- (42) A. UC1 B. UC2 C. UC3 D. UC5

● (43) 设计模式定义了对象间的一种一对多的依赖关系，以便当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并自动刷新。

- (43) A. Adapter (适配器) B. Iterator (迭代器)
- C. Prototype (原型) D. Observer (观察者)

● UML 中有多种类型的图，其中，(44) 对系统的使用方式进行分类，(45) 显示了类及其相互关系，(46) 显示人或对象的活动，其方式类似于流程图，通信图显示在某种情况下对象之间发送的消息，(47) 与通信图类似，但强调的是顺序而不是连接。

- (44) A. 用例图 B. 顺序图 C. 类图 D. 活动图
- (45) A. 用例图 B. 顺序图 C. 类图 D. 活动图
- (46) A. 用例图 B. 顺序图 C. 类图 D. 活动图
- (47) A. 用例图 B. 顺序图 C. 类图 D. 活动图

● 正则表达式 $1^*(0|01)^*$ 表示的集合元素的特点是 (48)。

- (48) A. 长度为奇数的 0、1 串
- B. 开始和结尾字符必须为 1 的 0、1 串
- C. 串的长度为偶数的 0、1 串
- D. 不包含子串 011 的 0、1 串

● 设某程序中定义了全局整型变量 x 和 r ，且函数 $f()$ 的定义如下所示，则在语句 “ $x = r * r + 1$;” 中 (49)。

```

int f(int r)
{
    int x;
    x = r*r + 1;
    return x;
}

```

- (49) A. x 和 r 均是全局变量 B. x 是全局变量、r 是形式参数
 C. x 是局部变量、r 是形式参数 D. x 是局部变量、r 是全局变量

● 程序语言的大多数语法现象可用上下文无关文法描述。对于一个上下文无关文法 $G=(N, T, P, S)$ ，其中 N 是非终结符号的集合， T 是终结符号的集合， P 是产生式集合， S 是开始符号。令集合 $V=N \cup T$ ，那么 G 所描述的语言是 (50) 的集合。

- (50) A. 从 S 出发推导出的包含 V 中所有符号的串
 B. 从 S 出发推导出的仅包含 T 中符号的串
 C. N 中所有符号组成的串
 D. T 中所有符号组成的串

● 在数据库系统中，数据的完整性约束的建立需要通过数据库管理系统提供的 (51) 语言来实现。

- (51) A. 数据定义 B. 数据操作 C. 数据查询 D. 数据控制

● 若某个关系的主码为全码，则该主码应包含 (52)。

- (52) A. 单个属性 B. 两个属性 C. 多个属性 D. 全部属性

● 建立一个供应商、零件数据库。其中“供应商”表 $S(Sno, Sname, Zip, City)$ 分别表示：供应商代码、供应商名、供应商邮编、供应商所在城市，其函数依赖为： $Sno \rightarrow (Sname, Zip, City)$ ， $Zip \rightarrow City$ 。“供应商”表 S 属于 (53)。

- (53) A. 1NF B. 2NF C. 3NF D. BCNF

● 关系 R, S 如下图所示， $R \bowtie S$ 可由 (54) 基本的关系运算组成， $R \bowtie S =$ (55)。

A	B	C
a	b	c
b	a	d
c	d	e
d	f	g

R

A	C	D
a	c	d
d	f	g
b	d	g

S

- (54) A. π 、 σ 和 $\times$ B. $-$ 、 σ 和 $\times$ C. $\cap$ 、 σ 和 $\times$ D. π 、 σ 和 $\cap$

(55) A.	A	B	C
	a	b	c
	b	a	d
	c	d	e

B.	A	B	C	D
	a	b	c	d
	b	a	d	g
	d	f	g	g

C.	A	B	C
	a	b	c
	b	a	d

D.	A	B	C	D
	a	b	c	d
	b	a	d	g

● 若事务 T1 对数据 A 已加排它锁，那么其它事务对数据 A (56)。

- (56) A. 加共享锁成功，加排它锁失败 B. 加排它锁成功，加共享锁失败
C. 加共享锁、加排它锁都成功 D. 加共享锁、加排它锁都失败

● 拓扑排序是指有向图中的所有顶点排成一个线性序列的过程，若在有向图中从顶点 v_i 到 v_j 有一条路径，则在该线性序列中，顶点 v_i 必然在顶点 v_j 之前。因此，若不能得到全部顶点的拓扑排序序列，则说明该有向图一定 (57)。

- (57) A. 包含回路 B. 是强连通图
C. 是完全图 D. 是有向树

● 设栈 S 和队列 Q 的初始状态为空，元素按照 a、b、c、d、e 的次序进入栈 S，当一个元素从栈中出来后立即进入队列 Q。若队列的输出元素序列是 c、d、b、a、e，则元素的出栈顺序是 (58)，栈 S 的容量至少为 (59)。

- (58) A. a、b、c、d、e B. e、d、c、b、a
C. c、d、b、a、e D. e、a、b、d、c
(59) A. 2 B. 3 C. 4 D. 5

● 对于 n ($n \geq 0$) 个元素构成的线性序列 L，在 (60) 时适合采用链式存储结构。

- (60) A. 需要频繁修改 L 中元素的值 B. 需要频繁地对 L 进行随机查找
C. 需要频繁地对 L 进行删除和插入操作 D. 要求 L 存储密度高

● 对于二叉查找树 (Binary Search Tree)，若其左子树非空，则左子树上所有结点的值均小于根结点的值；若其右子树非空，则右子树上所有结点的值均大于根结点的值；左、右子树本身就是两棵二叉查找树。因此，对任意一棵二叉查找树进行 (61) 遍历可以得到一个结点元素的递增序列。在具有 n 个结点的二叉查找树上进行查找运算，最坏情况下的算法复杂度为 (62)。

- (61) A. 先序 B. 中序 C. 后序 D. 层序
(62) A. $O(n^2)$ B. $O(n \log_2 n)$ C. $O(\log_2 n)$ D. $O(n)$

● 迪杰斯特拉 (Dijkstra) 算法按照路径长度递增的方式求解单源点最短路径问题, 该算法运用了 (63) 算法策略。

- (63) A. 贪心 B. 分而治之 C. 动态规划 D. 试探+回溯

● 关于算法与数据结构的关系, (64) 是正确的。

- (64) A. 算法的实现依赖于数据结构的设计
B. 算法的效率与数据结构无关
C. 数据结构越复杂, 算法的效率越高
D. 数据结构越简单, 算法的效率越高

● 若一个问题既可以用迭代方式也可以用递归方式求解, 则 (65) 方法具有更高的时空效率。

- (65) A. 迭代 B. 递归
C. 先递归后迭代 D. 先迭代后递归

● 在 FTP 协议中, 控制连接是由 (66) 主动建立的。

- (66) A. 服务器端 B. 客户端 C. 操作系统 D. 服务提供商

● 网页中代码 <input type="text" name="foo" size=20> 定义了 (67)。

- (67) A. 一个单选框
B. 一个单行文本输入框
C. 一个提交按钮
D. 一个使用图像的提交按钮

● 电子邮件应用程序利用 POP3 协议 (68)。

- (68) A. 创建邮件 B. 加密邮件
C. 发送邮件 D. 接收邮件

● 在进行金融业务系统的网络设计时, 应该优先考虑 (69) 原则。在进行企业网络的需求分析时, 应该首先进行 (70)。

- (69) A. 先进性 B. 开放性 C. 经济性 D. 高可用性
(70) A. 企业应用分析 B. 网络流量分析
C. 外部通信环境调研 D. 数据流向图分析

● The Rational Unified Process (RUP) is a software engineering process, which captures many of best practices in modern software development. The notions of (71) and scenarios have been proven to be an excellent way to capture function requirements. RUP can be described in two dimensions – time and content. In the time dimension, the software lifecycle is broken

into cycles. Each cycle is divided into four consecutive (72) which is concluded with a well-defined (73) and can be further broken down into (74) – a complete development loop resulting in a release of an executable product, a subset of the final product under development, which grows incrementally to become the final system. The content structure refers to the disciplines, which group (75) logically by nature.

- | | | | | |
|------|-----------------|--------------|---------------|------------------|
| (71) | A. artifacts | B. use-cases | C. actors | D. workers |
| (72) | A. orientations | B. views | C. aspects | D. phases |
| (73) | A. milestone | B. end-mark | C. measure | D. criteria |
| (74) | A. rounds | B. loops | C. iterations | D. circularities |
| (75) | A. functions | B. workflows | C. actions | D. activities |

全国计算机技术与软件专业技术资格（水平）考试

2007 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2007 年下半年全国计算机技术与软件专业技术资格(水平)考试日期是(1)月(2)日。

因为正确的解答是“11 月 3 日”，故在答题纸的对应栏内写上“11”和“3”（参看下表）。

例题	解答栏
(1)	11
(2)	3

试题一（共 15 分）

阅读以下说明和图，回答问题1至问题4，将解答填入答题纸的对应栏内。

【说明】

某高校欲开发一个成绩管理系统，记录并管理所有选修课程的学生的平时成绩和考试成绩，其主要功能描述如下：

1. 每门课程都有 3 到 6 个单元构成，每个单元结束后会进行一次测试，其成绩作为这门课程的平时成绩。课程结束后进行期末考试，其成绩作为这门课程的考试成绩。
2. 学生的平时成绩和考试成绩均由每门课程的主讲教师上传给成绩管理系统。
3. 在记录学生成绩之前，系统需要验证这些成绩是否有效。首先，根据学生信息文件来确认该学生是否选修这门课程，若没有，那么这些成绩是无效的；如果他的确选修了这门课程，再根据课程信息文件和课程单元信息文件来验证平时成绩是否与这门课程所包含的单元相对应，如果是，那么这些成绩是有效的，否则无效。
4. 对于有效成绩，系统将其保存在课程成绩文件中。对于无效成绩，系统会单独将其保存在无效成绩文件中，并将详细情况提交给教务处。在教务处没有给出具体处理意见之前，系统不会处理这些成绩。
5. 若一门课程的所有有效的平时成绩和考试成绩都已经被系统记录，系统会发送课程完成通知给教务处，告知该门课程的成绩已经齐全。教务处根据需要，请求系统生成相应的成绩列表，用来提交考试委员会审查。
6. 在生成成绩列表之前，系统会生成一份成绩报告给主讲教师，以便核对是否存在错误。主讲教师须将核对之后的成绩报告返还系统。
7. 根据主讲教师核对后的成绩报告，系统生成相应的成绩列表，递交考试委员会进行审查。考试委员会在审查之后，上交一份成绩审查结果给系统。对于所有通过审查的成绩，系统将会生成最终的成绩单，并通知每个选课学生。

现采用结构化方法对这个系统进行分析与设计，得到如图 1-1 所示的顶层数据流图和图 1-2 所示的 0 层数据流图。

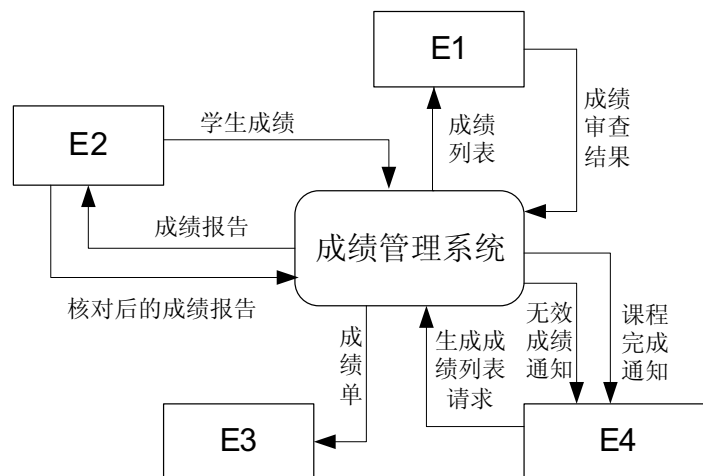


图 1-1 顶层数据流图

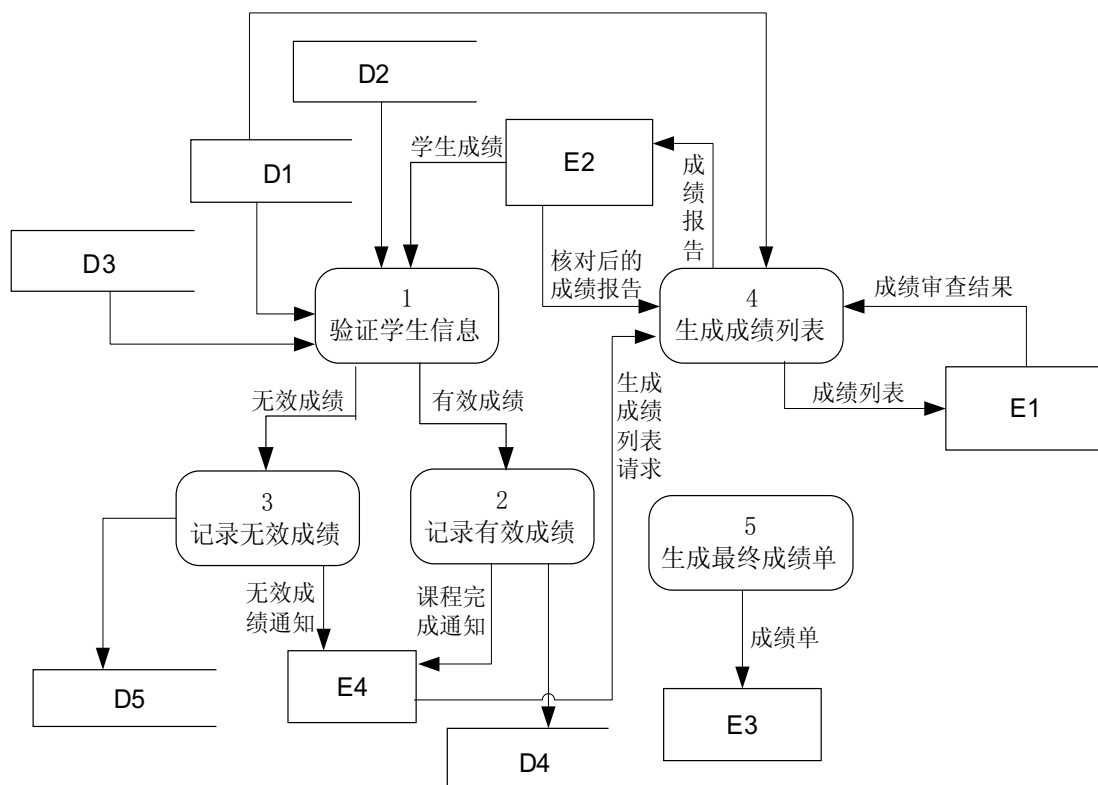


图 1-2 0 层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的外部实体 E1~E4 的名称。

【问题 2】（3 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D5 的名称。

【问题 3】（6 分）

数据流图 1-2 缺少了三条数据流，根据说明及数据流图 1-1 提供的信息，分别指出这三条数据流的起点和终点。

起 点	终 点

【问题 4】（2 分）

数据流图是在系统分析与总体设计阶段宏观地描述系统功能需求的重要图形化工具，程序流程图也是软件开发过程中比较常用的图形化工具。简要说明程序流程图的适用场合与作用。

试题二（15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某汽车维修站拟开发一套小型汽车维修管理系统，对车辆的维修情况进行管理。

1. 对于新客户及车辆，汽车维修管理系统首先登记客户信息，包括：客户编号、客户名称、客户性质（个人、单位）、折扣率、联系人、联系电话等信息；还要记录客户的车辆信息，包括：车牌号、车型、颜色等信息。一个客户至少有一台车。客户及车辆信息如表 2-1 所示。

表 2-1 客户及车辆信息

客户编号	GS0051	客户名称	××公司	客户性质	单位
折扣率	95%	联系人	杨浩东	联系电话	82638779
车牌号		颜色	车型	车辆类别	
**0761		白色	帕萨特	微型车	

2. 记录维修车辆的故障信息。包括：维修类型（普通、加急）、作业分类（大、中、小修）、结算方式（自付、三包、索赔）等信息。维修厂的员工分为：维修员和业务员。车辆维修首先委托给业务员。业务员对车辆进行检查和故障分析后，与客户磋商，确定故障现象，生成维修委托书。如表 2-2 所示。

表 2-2 维修委托书

No.20070702003

登记日期：2007-07-02

车牌号	**0765	客户编号	GS0051	维修类型	普通
作业类别	中修	结算方式	自付	进厂时间	20070702 11:09
业务员	张小江	业务员编号	012	预计完工时间	
故障描述					
车头损坏，水箱漏水					

3. 维修车间根据维修委托书和车辆的故障现象，在已有的维修项目中选择并确定一个或多个具体维修项目，安排相关的维修工及工时，生成维修派工单。维修派工单如表 2-3 所示。

表 2-3 维修派工单

No.20070702003

维修项目编号	维修项目	工时	维修工编号	维修工工种
12	维修车头	5.00	0 2	机修
12	维修车头	2.00	0 3	漆工
15	水箱焊接补漏	1.00	0 6	焊工
17	更换车灯	1.00	0 2	机修

4. 客户车辆在车间修理完毕后，根据维修项目单价和维修派工单中的工时计算车辆此

次维修的总费用，记录在委托书中。

根据需求阶段收集的信息，设计的实体联系图（图 2-1）和关系模式（不完整）如下所示。图 2-1 中业务员和维修工是员工的子实体。

【概念结构设计】

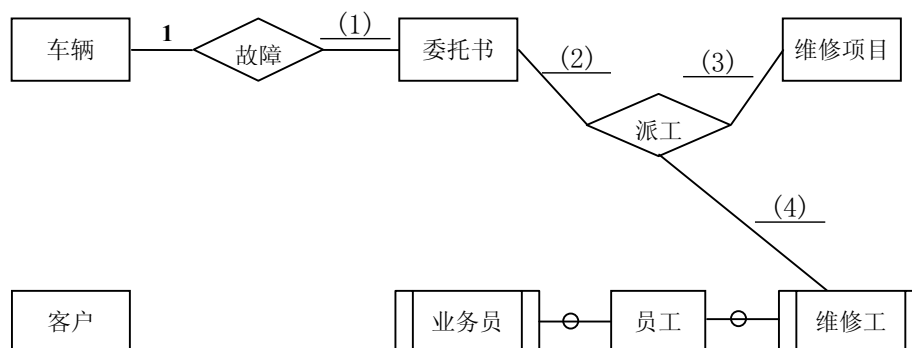


图 2-1 实体联系图

【逻辑结构设计】

客户(_____(5)_____, 折扣率, 联系人, 联系电话)

车辆(车牌号, 客户编号, 车型, 颜色, 车辆类别)

委托书(_____(6)_____, 维修类型, 作业分类, 结算方式, 进厂时间,
预计完工时间, 登记日期, 故障描述, 总费用)

维修项目(维修项目编号, 维修项目, 单价)

派工单(_____(7)_____, 工时)

员工(_____(8)_____, 工种, 员工类型, 级别)

【问题 1】(4 分)

根据问题描述，填写图 2-1 中(1)~(4)处联系的类型。联系类型分为一对一、一对多和多对多三种，分别使用 1:1, 1:n 或 1:*, m:n 或 *: *表示。

【问题 2】(4 分)

补充图 2-1 中的联系并指明其联系类型。联系名可为：联系 1，联系 2，…。

【问题 3】(4 分)

根据图 2-1 和说明，将逻辑结构设计阶段生成的关系模式中的空(5)~(8)补充完整。

【问题 4】(3 分)

根据问题描述，写出客户、委托书和派工单这三个关系的主键。

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

已知某唱片播放器不仅可以播放唱片，而且可以连接电脑并把电脑中的歌曲刻录到唱片上（同步歌曲）。连接电脑的过程中还可自动完成充电。

关于唱片，还有以下描述信息：

1. 每首歌曲的描述信息包括：歌曲的名字、谱写这首歌曲的艺术家以及演奏这首歌曲的艺术家。只有两首歌曲的这三部分信息完全相同时，才认为它们是同一首歌曲。艺术家可能是一名歌手或一支由 2 名或 2 名以上的歌手所组成的乐队。一名歌手可以不属于任何乐队，也可以属于一个或多个乐队。

2. 每张唱片由多条音轨构成；一条音轨中只包含一首歌曲或为空，一首歌曲可分布在多条音轨上；同一首歌曲在一张唱片中最多只能出现一次。

3. 每条音轨都有一个开始位置和持续时间。一张唱片上音轨的次序是非常重要的，因此对于任意一条音轨，播放器需要准确地知道，它的下一条音轨和上一条音轨是什么（如果存在的话）。

根据上述描述，采用面向对象方法对其进行分析与设计，得到了如表 3-1 所示的类列表、如图 3-1 所示的初始类图以及如图 3-2 所示的描述播放器行为的 UML 状态图。

表 3-1 类列表

类 名	说 明
Artist	艺术家
Song	歌曲
Band	乐队
Musician	歌手
Track	音轨
Album	唱片

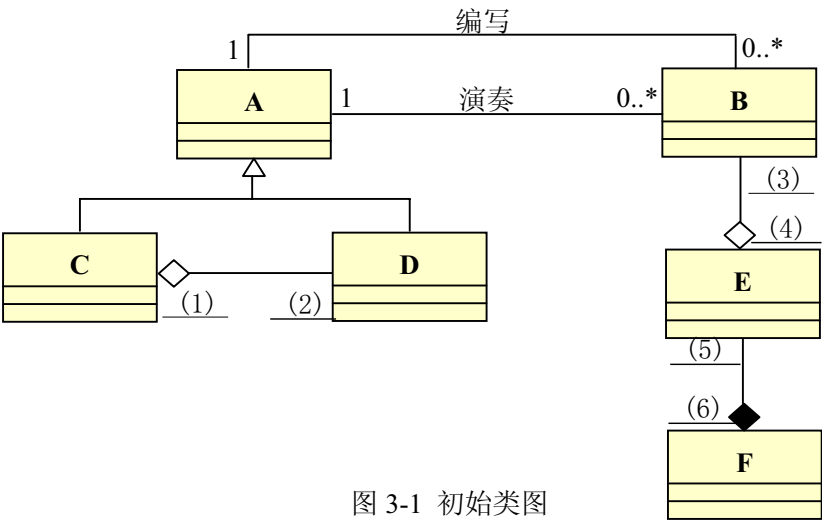


图 3-1 初始类图

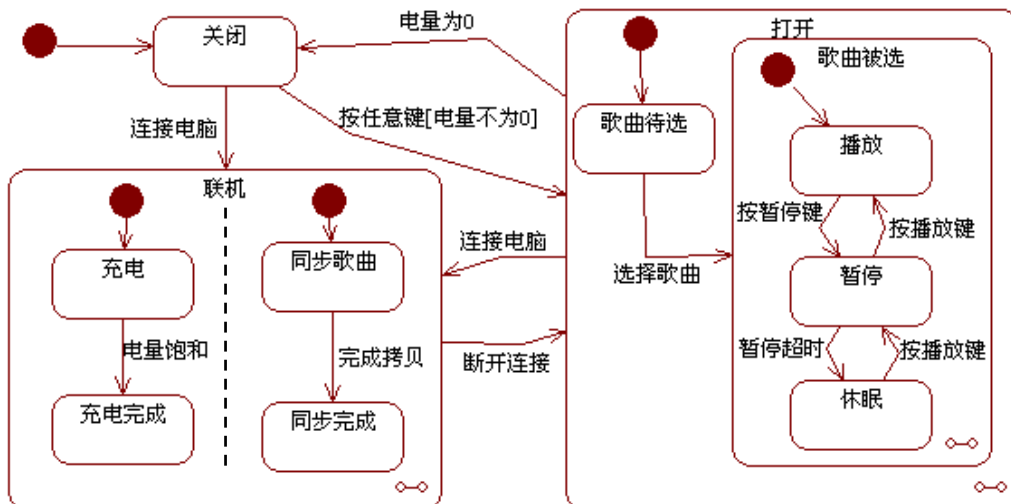


图 3-2 播放器行为 UML 状态图

【问题 1】（3 分）

根据说明中的描述，使用表 3-1 给出的类的名称，给出图 3-1 中的 A~F 所对应的类。

【问题 2】（6 分）

根据说明中的描述，给出图 3-1 中（1）~（6）处的多重度。

【问题 3】（4 分）

图 3-1 中缺少了一条关联，请指出这条关联两端所对应的类以及每一端的多重度。

类	多重度

【问题 4】（2 分）

根据图 3-2 所示的播放器行为 UML 状态图，给出从“关闭”状态到“播放”状态所经过的最短事件序列（假设电池一开始就是有电的）。

试题四(共 15 分)

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某机器上需要处理 n 个作业 $job_1, job_2, \dots, job_n$ ，其中：

- (1) 每个作业 $job_i (1 \leq i \leq n)$ 的编号为 i ， job_i 有一个收益值 $p[i]$ 和最后期限值 $d[i]$ ；
- (2) 机器在一个时刻只能处理一个作业，而且每个作业需要一个单位时间进行处理，一旦作业开始就不可中断，每个作业的最后期限值为单位时间的正整数倍；
- (3) $job_1 \sim job_n$ 的收益值呈非递增顺序排列，即 $p[1] \geq p[2] \geq \dots \geq p[n]$ ；
- (4) 如果作业 job_i 在其期限之内完成，则获得收益 $p[i]$ ；如果在其期限之后完成，则没有收益。

为获得较高的收益，采用贪心策略求解在期限之内完成的作业序列。图 4-1 是基于贪心策略求解该问题的流程图。

(1) 整型数组 $J[]$ 有 n 个存储单元，变量 k 表示在期限之内完成的作业数， $J[1..k]$ 存储所有能够在期限内完成的作业编号，数组 $J[1..k]$ 里的作业按其最后期限非递减排序，即 $d[J[1]] \leq \dots \leq d[J[k]]$ 。

(2) 为了方便于在数组 J 中加入作业，增加一个虚拟作业 job_0 ，并令 $d[0] = 0$ ， $J[0] = 0$ 。

(3) 算法大致思想：先将作业 job_1 的编号 1 放入 $J[1]$ ，然后，依次对每个作业 $job_i (2 \leq i \leq n)$ 进行判定，看其能否插入到数组 J 中，若能，则将其编号插入到数组 J 的适当位置，并保证 J 中作业按其最后期限非递减排列，否则不插入。

job_i 能插入数组 J 的充要条件是： job_i 和数组 J 中已有作业均能在其期限之内完成。

(4) 流程图中的主要变量说明如下：

- i ：循环控制变量，表示作业的编号；
- k ：表示在期限内完成的作业数；
- r ：若 job_i 能插入数组 J ，则其在数组 J 中的位置为 $r+1$ ；
- q ：循环控制变量，用于移动数组 J 中的元素。

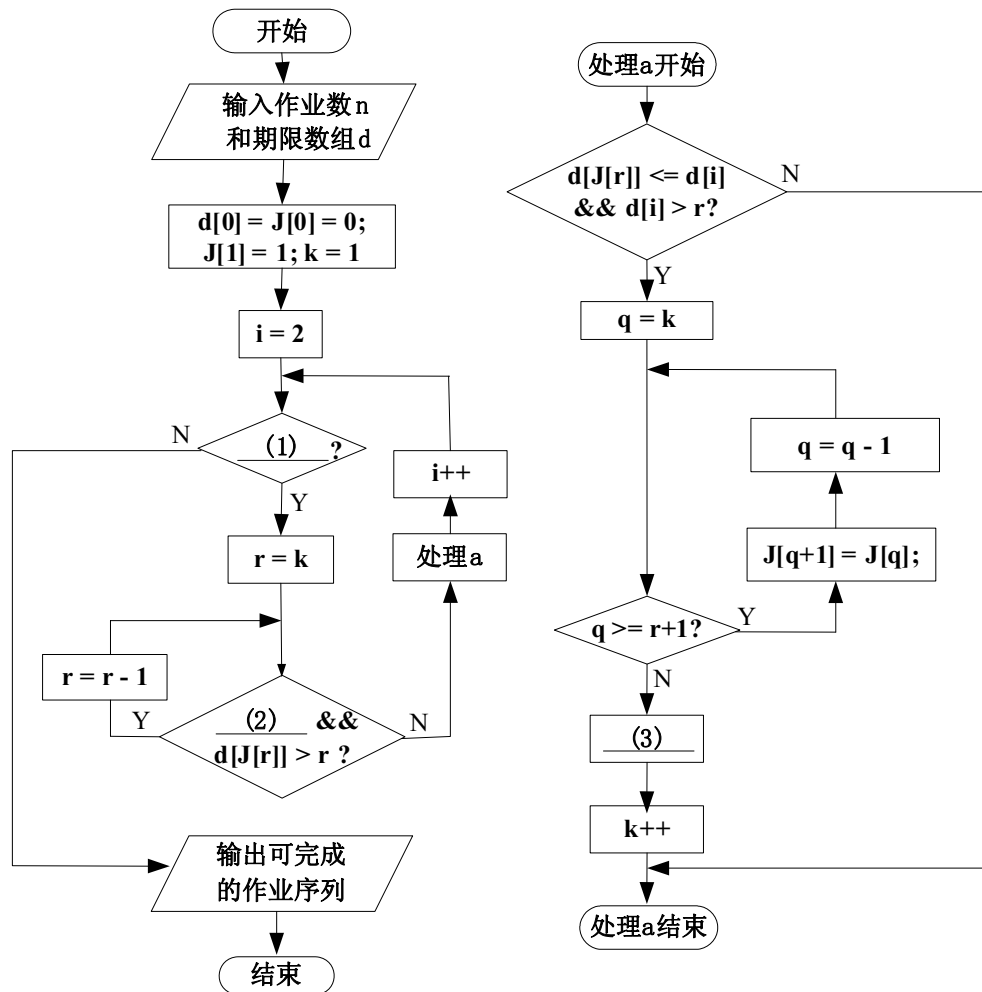


图 4-1 贪心策略流程图

【问题 1】(9 分)

请填充图 4-1 中的空缺(1)、(2)和(3)处。

【问题 2】(4 分)

假设有 6 个作业 $job_1, job_2, \dots, job_6$;

完成作业的收益数组 $p=(p[1], p[2], p[3], p[4], p[5], p[6]) = (90, 80, 50, 30, 20, 10)$;

每个作业的处理期限数组 $d=(d[1], d[2], d[3], d[4], d[5], d[6]) = (1, 2, 1, 3, 4, 3)$ 。

请应用试题中描述的贪心策略算法，给出在期限之内处理的作业编号序列____(4)____
(按作业处理的顺序给出)，得到的总收益为____(5)____。

【问题 3】(2 分)

对于本题的作业处理问题，用图 4-1 的贪心算法策略，能否求得最高收益？____(6)____。

用贪心算法求解任意给定问题时，是否一定能得到最优解？____(7)____。

从下列 3 道试题（试题五至试题七）中任选 1 道解答。如
果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五（共 15 分）

阅读以下说明和 C 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

在一个简化的绘图程序中，支持的图形种类有点(point)和圆(circle)，在设计过程中采用面向对象思想，认为所有的点和圆都是一种图形(shape)，并定义了类型 shape_t、point_t 和 circle_t 分别表示基本图形、点和圆，并且点和圆具有基本图形的所有特征。

[C 代码]

```
typedef enum { point, circle } shape_type; /* 程序中的两种图形：点和圆 */
typedef struct {
    shape_type type; /* 图形种类标识：点或者圆 */
    void (*destroy)(); /* 销毁图形操作的函数指针 */
    void (*draw)(); /* 绘制图形操作的函数指针 */
} shape_t;
typedef struct { shape_t common; int x; int y; } point_t; /* 定义点类型，x、y 为点坐标 */
void destroyPoint(point_t* this) { free(this); printf("Point destroyed!\n"); } /* 销毁点对象 */
void drawPoint(point_t* this) { printf("P(%d,%d)", this->x, this->y); } /* 绘制点对象 */
shape_t* createPoint(va_list* ap) { /* 创建点对象，并设置其属性 */
    point_t* p_point;
    if( (p_point = (point_t*)malloc(sizeof(point_t))) == NULL ) return NULL;
    p_point->common.type = point; p_point->common.destroy = destroyPoint;
    p_point->common.draw = drawPoint;
    p_point->x = va_arg(*ap, int); /* 设置点的横坐标 */
    p_point->y = va_arg(*ap, int); /* 设置点的纵坐标 */
    return (shape_t*)p_point; /* 返回点对象指针 */
}
typedef struct { /* 定义圆类型 */
    shape_t common;
    point_t *center; /* 圆心点 */
    int radius; /* 圆半径 */
} circle_t;
void destroyCircle(circle_t* this){
    free(___(1)); free(this); printf("Circle destroyed!\n");
}
void drawCircle(circle_t* this) {
```

```

    printf("C");
    (2)____.draw( this->center );          /* 绘制圆心 */
    printf("%d", this->radius);
}
shape_t* createCircle(va_list* ap) {      /* 创建一个圆，并设置其属性 */
    circle_t* p_circle;
    if( (p_circle = (circle_t*)malloc(sizeof(circle_t))) == NULL ) return NULL;
    p_circle->common.type = circle;      p_circle->common.destroy = destroyCircle;
    p_circle->common.draw = drawCircle;
    (3)____ = createPoint(ap);          /* 设置圆心 */
    p_circle->radius = va_arg(*ap, int); /* 设置圆半径 */
    return p_circle;
}
shape_t* createShape(shape_type st, ...) { /* 创建某一种具体的图形 */
    va_list ap;                          /* 可变参数列表 */
    shape_t* p_shape = NULL;
    (4)____(ap, st);
    if( st == point ) p_shape = createPoint( &ap); /* 创建点对象 */
    if( st == circle ) p_shape = createCircle(&ap); /* 创建圆对象 */
    va_end(ap);
    return p_shape;
}
int main() {
    int i;                               /* 循环控制变量，用于循环计数 */
    shape_t* shapes[2]; /* 图形指针数组，存储图形的地址 */
    shapes[0] = createShape( point, 2, 3); /* 横坐标为 2，纵坐标为 3 */
    shapes[1] = createShape( circle, 20, 40, 10); /* 圆心坐标(20,40)，半径为 10 */
    for(i=0; i<2; i++) { shapes[i]->draw(shapes[i]); printf("\n"); } /* 绘制数组中图形 */
    for( i = 1; i >= 0; i-- ) shapes[i]->destroy(shapes[i]); /* 销毁数组中图形 */
    return 0;
}

```

[运行结果]

```

P(2,3)
(5)____
Circle destroyed!
Point destroyed!

```


试题六（共 15 分）

阅读下列说明和 C++ 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

已知某企业的采购审批是分级进行的，即根据采购金额的不同由不同层次的主管人员来审批，主任可以审批 5 万元以下（不包括 5 万元）的采购单，副董事长可以审批 5 万元至 10 万元（不包括 10 万元）的采购单，董事长可以审批 10 万元至 50 万元（不包括 50 万元）的采购单，50 万元及以上的采购单就需要开会讨论决定。

采用责任链设计模式（Chain of Responsibility）对上述过程进行设计后得到的类图如图 6-1 所示。

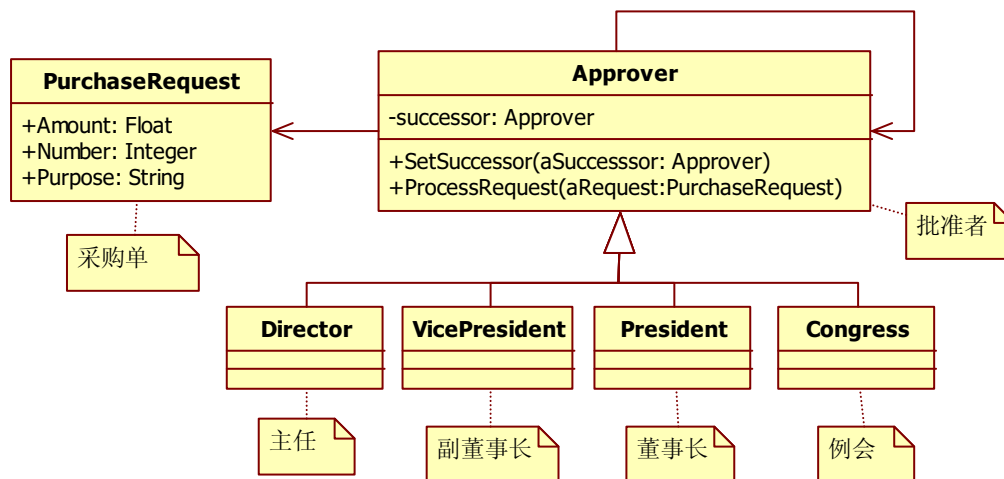


图 6-1 设计类图

[C++代码]

```
#include <string>
#include <iostream>
using namespace std;
class PurchaseRequest {
public:
    double Amount;        // 一个采购的金额
    int Number;           // 采购单编号
    string Purpose;       // 采购目的
};
class Approver {          // 审批者类
public:
    Approver() { successor = NULL; }
    virtual void ProcessRequest(PurchaseRequest aRequest) {
        if (successor != NULL) { successor->__(1)__; }
    }
}
```

```

        void SetSuccessor(Approver *aSuccessor){ successor = aSuccessor; }
private:
    ____ (2) ____ successor;
};
class Congress : public Approver {
public:
    void ProcessRequest(PurchaseRequest aRequest){
        if(aRequest.Amount >= 500000){ /* 决定是否审批的代码省略 */ }
        else ____ (3) ____ ProcessRequest(aRequest);
    }
};
class Director : public Approver {
public:
    void ProcessRequest(PurchaseRequest aRequest){ /* 此处代码省略 */ }
};
class President : public Approver {
public:
    void ProcessRequest(PurchaseRequest aRequest){ /* 此处代码省略 */ }
};
class VicePresident : public Approver {
public:
    void ProcessRequest(PurchaseRequest aRequest){ /* 此处代码省略 */ }
};
void main(){
    Congress Meeting; VicePresident Sam; Director Larry ; President Tammy;
    // 构造责任链
    Meeting.SetSuccessor(NULL);          Sam.SetSuccessor(____ (4) ____);
    Tammy.SetSuccessor(____ (5) ____);    Larry.SetSuccessor(____ (6) ____);

    PurchaseRequest aRequest; // 构造一采购审批请求
    cin >> aRequest.Amount;   // 输入采购请求的金额

    ____ (7) ____ .ProcessRequest(aRequest);    // 开始审批
    return ;
}

```

试题七（共 15 分）

阅读下列说明和 Java 代码，将应填入__(n)__处的字句写在答题纸的对应栏内。

【说明】

已知某企业的采购审批是分级进行的，即根据采购金额的不同由不同层次的主管人员来审批，主任可以审批 5 万元以下（不包括 5 万元）的采购单，副董事长可以审批 5 万元至 10 万元（不包括 10 万元）的采购单，董事长可以审批 10 万元至 50 万元（不包括 50 万元）的采购单，50 万元及以上的采购单就需要开会讨论决定。

采用责任链设计模式（Chain of Responsibility）对上述过程进行设计后得到的类图如图 7-1 所示。

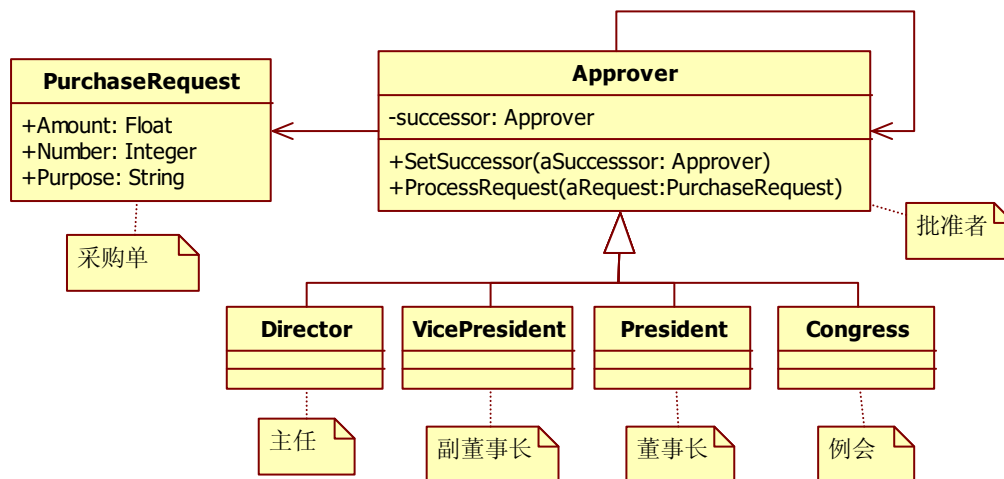


图 7-1 设计类图

[Java 代码]

```
class PurchaseRequest {
    public double Amount; // 一个采购的金额
    public int Number; // 采购单编号
    public String Purpose; // 采购目的
};

class Approver { // 审批者类
    public Approver() { successor = null; }
    public void ProcessRequest(PurchaseRequest aRequest) {
        if (successor != null) { successor.__(1)__; }
    }
    public void SetSuccessor(Approver aSuccessor) { successor = aSuccessor; }
    private __(2)__ successor;
};

class Congress extends Approver {
```

```

    public void ProcessRequest(PurchaseRequest aRequest){
        if(aRequest.Amount >= 500000){ /* 决定是否审批的代码省略 */ }
        else ____ (3) ____ .ProcessRequest(aRequest);
    }
};
class Director extends Approver {
    public void ProcessRequest(PurchaseRequest aRequest){ /* 此处代码省略 */ }
};
class President extends Approver {
    public void ProcessRequest(PurchaseRequest aRequest){ /* 此处代码省略 */ }
};
class VicePresident extends Approver {
    public void ProcessRequest(PurchaseRequest aRequest){ /* 此处代码省略 */ }
};

public class rs {
    public static void main(String[] args) throws IOException {
        Congress Meeting = new Congress();
        VicePresident Sam = new VicePresident();
        Director Larry = new Director();
        President Tammy = new President();
        // 构造责任链
        Meeting.SetSuccessor(null);    Sam.SetSuccessor(____ (4) ____);
        Tammy.SetSuccessor(____ (5) ____);    Larry.SetSuccessor(____ (6) ____);

        // 构造一采购审批请求
        PurchaseRequest aRequest = new PurchaseRequest();
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));
        aRequest.Amount = Double.parseDouble(br.readLine());

        ____ (7) ____ .ProcessRequest(aRequest);    // 开始审批
        return ;
    }
}

```

全国计算机技术与软件专业技术资格（水平）考试

2008 年上半年 软件设计师 上午试卷

（考试时间 9：00～11：30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2008 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是____（88）____月____（89）____日。

- | | | | |
|------------|-------|-------|-------|
| （88） A. 4 | B. 5 | C. 6 | D. 7 |
| （89） A. 21 | B. 22 | C. 23 | D. 24 |

因为考试日期是“5 月 24 日”，故（88）选 B，（89）选 D，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 D 填涂（参看答题卡）。

● 在计算机体系结构中，CPU 内部包括程序计数器 PC、存储器数据寄存器 MDR、指令寄存器 IR 和存储器地址寄存器 MAR 等。若 CPU 要执行的指令为：MOV R0, #100（即将数值 100 传送到寄存器 R0 中），则 CPU 首先要完成的操作是（1）。

- (1) A. 100→R0 B. 100→MDR C. PC→MAR D. PC→IR

● 现有四级指令流水线，分别完成取指、取数、运算、传送结果四步操作。若完成上述操作的时间依次为 9ns、10ns、6ns、8ns，则流水线的操作周期应设计为（2）ns。

- (2) A. 6 B. 8 C. 9 D. 10

● 内存按字节编址，地址从 90000H 到 CFFFFH，若用存储容量为 16K×8bit 的存储器芯片构成该内存，至少需要（3）片。

- (3) A. 2 B. 4 C. 8 D. 16

● CPU 中的数据总线宽度会影响（4）。

- (4) A. 内存容量的大小 B. 系统的运算速度
C. 指令系统的指令数量 D. 寄存器的宽度

● 利用高速通信网络将多台高性能工作站或微型机互连构成机群系统，其系统结构形式属于（5）计算机。

- (5) A. 单指令流单数据流（SISD） B. 多指令流单数据流（MISD）
C. 单指令流多数据流（SIMD） D. 多指令流多数据流（MIMD）

● 内存采用段式存储管理有许多优点，但“（6）”不是其优点。

- (6) A. 分段是信息的逻辑单位，用户不可见
B. 各段程序的修改互不影响
C. 地址变换速度快、内存碎片少
D. 便于多道程序共享主存的某些段

● 如果希望别的计算机不能通过 ping 命令测试服务器的连通情况，可以（7）。如果希望通过默认的 Telnet 端口连接服务器，则下面对防火墙配置正确的是（8）。

- (7) A. 删除服务器中的 ping.exe 文件
B. 删除服务器中的 cmd.exe 文件
C. 关闭服务器中 ICMP 端口
D. 关闭服务器中的 Net Logon 服务

(8)

A.

服务设置

服务描述 (D):
Telnet

在您的网络上主持此服务的计算机的名称或 IP 地址 (H) (例如, 192.168.0.12):
202.1.1.1

此服务的外部端口号 (E):
23

此服务的内部端口号 (I):
23

☒ TCP (T) ☐ UDP (U)

确定 取消

B.

服务设置

服务描述 (D):
Telnet

在您的网络上主持此服务的计算机的名称或 IP 地址 (H) (例如, 192.168.0.12):
202.1.1.1

此服务的外部端口号 (E):
23

此服务的内部端口号 (I):
23

☐ TCP (T) ☒ UDP (U)

确定 取消

C.

服务设置

服务描述 (D):
Telnet

在您的网络上主持此服务的计算机的名称或 IP 地址 (H) (例如, 192.168.0.12):
202.1.1.1

此服务的外部端口号 (E):
110

此服务的内部端口号 (I):
110

☒ TCP (T) ☐ UDP (U)

确定 取消

D.

服务设置

服务描述 (D):
Telnet

在您的网络上主持此服务的计算机的名称或 IP 地址 (H) (例如, 192.168.0.12):
202.1.1.1

此服务的外部端口号 (E):
110

此服务的内部端口号 (I):
110

☐ TCP (T) ☒ UDP (U)

确定 取消

● 某银行为用户提供网上服务, 允许用户通过浏览器管理自己的银行账户信息。为保障通信的安全性, 该 Web 服务器可选的协议是 (9)。

(9) A. POP B. SNMP C. HTTP D. HTTPS

● 关于软件著作权产生的时间, 表述正确的是 (10)。

(10) A. 自软件首次公开发表时
B. 自开发者有开发意图时
C. 自软件得到国家著作权行政管理部门认可时
D. 自软件完成创作之日起

● 李某大学毕业后在 M 公司销售部门工作，后由于该公司软件开发部门人手较紧，李某被暂调到该公司软件开发部开发新产品，2 周后，李某开发出一种新软件。该软件著作权应归(11)所有。

- (11) A. 李某 B. M 公司 C. 李某和 M 公司 D. 软件开发部

● 一幅灰度图像，若每个像素有 8 位像素深度，则最大灰度数目为(12)。

- (12) A. 128 B. 256 C. 512 D. 1024

● 当图像分辨率为 800×600 ，屏幕分辨率为 640×480 时，(13)。

- (13) A. 屏幕上显示一幅图像的 64%左右 B. 图像正好占满屏幕
C. 屏幕上显示一幅完整的图像 D. 图像只占屏幕的一部分

● 若视频图像每帧的数据量为 6.4MB，帧速率为 30 帧/秒，则显示 10 秒的视频信息，其原始数据量为(14)MB。

- (14) A. 64 B. 192 C. 640 D. 1920

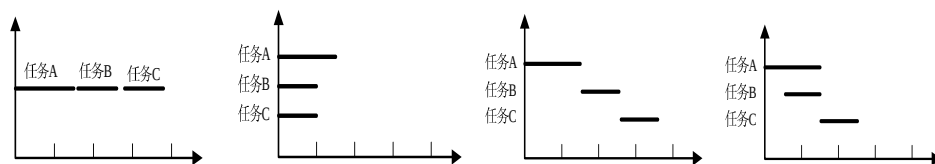
● (15) 是一种面向数据流的开发方法，其基本思想是软件功能的分解和抽象。

- (15) A. 结构化开发方法 B. Jackson 系统开发方法
C. Booch 方法 D. UML (统一建模语言)

● 采用 UML 进行软件设计时，可用(16)关系表示两类事物之间存在的特殊/一般关系，用聚集关系表示事物之间存在的整体/部分关系。

- (16) A. 依赖 B. 聚集 C. 泛化 D. 实现

● 某项目制定的开发计划中定义了三个任务，其中任务 A 首先开始，且需要 3 周完成，任务 B 必须在任务 A 启动 1 周后开始，且需要 2 周完成，任务 C 必须在任务 A 完成后才能开始，且需要 2 周完成。该项目的进度安排可用下面的甘特图(17)来描述。



- (17) A. B. C. D.

● 风险分析在软件项目开发中具有重要作用，包括风险识别、风险预测、风险评估和风险控制等。“建立风险条目检查表”是(18)时的活动，“描述风险的结果”是(19)时的活动。

- (18) A. 风险识别 B. 风险预测 C. 风险评估 D. 风险控制
(19) A. 风险识别 B. 风险预测 C. 风险评估 D. 风险控制

● 编译器对高级语言源程序的处理过程可以划分为词法分析、语法分析、语义分析、中间代码生成、代码优化、目标代码生成等几个阶段，其中，(20)并不是每种编译器都必需的。

- (20) A. 词法分析和语法分析 B. 语义分析和中间代码生成
C. 中间代码生成和代码优化 D. 代码优化和目标代码生成

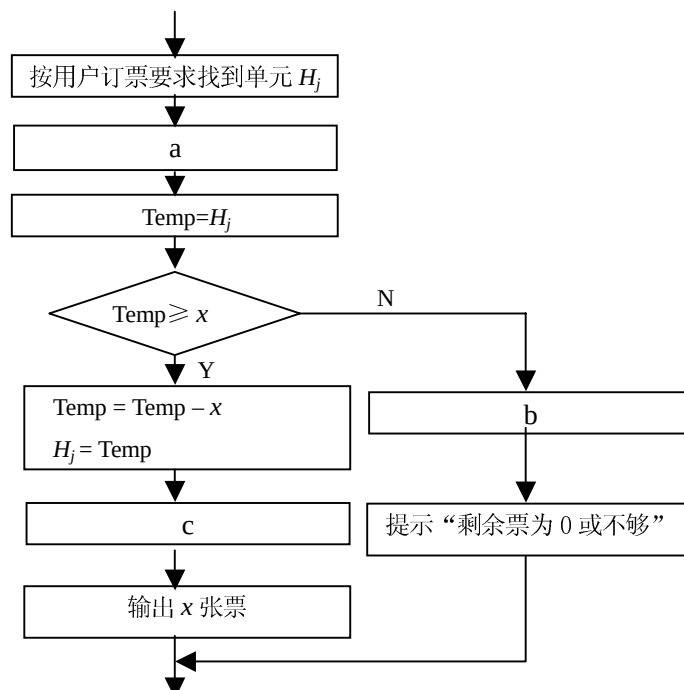
● 已知某文法 $G[S]: S \rightarrow 0S0 \quad S \rightarrow 1$ ，从 S 推导出的符号串可用 (21) ($n \geq 0$) 描述。

- (21) A. $(010)^n$ B. $0^n 10^n$ C. 1^n D. $01^n 0$

● 下列叙述中错误的是 (22)。

- (22) A. 面向对象程序设计语言可支持过程化的程序设计
B. 给定算法的时间复杂性与实现该算法所采用的程序设计语言无关
C. 与汇编语言相比，采用脚本语言编程可获得更高的运行效率
D. 面向对象程序设计语言不支持对一个对象的成员变量进行直接访问

● 某火车票销售系统有 n 个售票点，该系统为每个售票点创建一个进程 $P_i (i=1, 2, \dots, n)$ 。假设 $H_j (j=1, 2, \dots, m)$ 单元存放某日某车次的剩余票数，Temp 为 P_i 进程的临时工作单元， x 为某用户的订票张数。初始化时系统应将信号量 S 赋值为 (23)。 P_i 进程的工作流程如下，若用 P 操作和 V 操作实现进程间的同步与互斥，则图中 a、b 和 c 应分别填入 (24)。



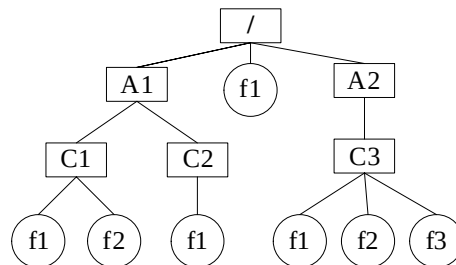
- (23) A. 0 B. 1 C. 2 D. 3
 (24) A. P(S)、V(S) 和 V(S) B. P(S)、P(S) 和 V(S)
 C. V(S)、P(S) 和 P(S) D. V(S)、V(S) 和 P(S)

● 在下图所示的树型文件系统中，方框表示目录，圆圈表示文件，“/”表示路径中的分隔符，“/”在路径之首时表示根目录。图中，(25)。假设当前目录是 A2，若进程 A 以如下两种方式打开文件 f2：

方式① `fd1=open("_(26) /f2",o_RDONLY);`

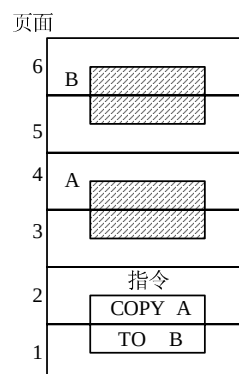
方式② `fd1=open("/A2/C3/f2",o_RDONLY);`

那么，采用方式①的工作效率比方式②的工作效率高。



- (25) A. 根目录中文件 f1 与子目录 C1、C2 和 C3 中文件 f1 一定相同
 B. 子目录 C1 中文件 f2 与子目录 C3 中文件 f2 一定相同
 C. 子目录 C1 中文件 f2 与子目录 C3 中文件 f2 一定不同
 D. 子目录 C1 中文件 f2 与子目录 C3 中文件 f2 是可能相同也可能不相同
 (26) A. /A2/C3 B. A2/C3 C. C3 D. f2

● 在某计算机中，假设某程序的 6 个页面如下图所示，其中某指令“COPY A TO B”跨两个页面，且源地址 A 和目标地址 B 所涉及的区域也跨两个页面。若地址为 A 和 B 的



操作数均不在内存，计算机执行该 COPY 指令时，系统将产生(27)次缺页中断；若系统产生三次缺页中断，那么该程序应有(28)个页面在内存。

- (27) A. 2 B. 3 C. 4 D. 5
(28) A. 2 B. 3 C. 4 D. 5

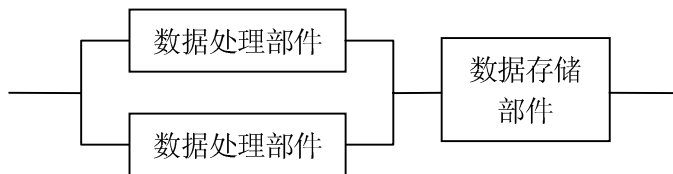
● 极限编程(eXtreme Programming)是一种轻量级软件开发方法, (29)不是它强调的准则。

- (29) A. 持续的交流和沟通 B. 用最简单的设计实现用户需求
C. 用测试驱动开发 D. 关注用户反馈

● 某公司采用的软件开发过程通过了 CMM2 认证, 表明该公司(30)。

- (30) A. 开发项目成效不稳定, 管理混乱
B. 对软件过程和产品质量建立了定量的质量目标
C. 建立了基本的项目级管理制度和规程, 可对项目的成本、进度进行跟踪和控制
D. 可集中精力采用新技术新方法, 优化软件过程

● 某数据处理软件包括 2 个完全相同的数据处理部件和 1 个数据存储部件, 且采用下图给出的容错方案。当数据处理部件的可靠性为 0.6 时, 为使整个软件系统的可靠性不小于 0.66, 则数据存储部件的可靠性至少应为(31)。



- (31) A. 0.6 B. 0.66 C. 0.79 D. 1.0

● 在软件设计和编码过程中, 采取 “(32)” 的做法将使软件更加容易理解和维护。

- (32) A. 良好的程序结构, 有无文档均可
B. 使用标准或规定之外的语句
C. 编写详细正确的文档, 采用良好的程序结构
D. 尽量减少程序中的注释

● 软件维护成本在软件成本中占较大比重。为降低维护的难度, 可采取的措施有(33)。

- (33) A. 设计并实现没有错误的软件
B. 限制可修改的范围
C. 增加维护人员数量
D. 在开发过程中就采取有利于维护的措施, 并加强维护管理

● 软件文档按照其产生和使用的范围可分为开发文档、管理文档和用户文档。其中开发文档不包括(34)。

- (34) A. 软件需求说明 B. 可行性研究报告
C. 维护修改建议 D. 项目开发计划

● 软件测试是软件开发中不可缺少的活动，通常(35)在代码编写阶段进行。检查软件的功能是否与用户要求一致是(36)的任务。

- (35) A. 验收测试 B. 系统测试 C. 单元测试 D. 集成测试
(36) A. 验收测试 B. 系统测试 C. 单元测试 D. 集成测试

● (37)是指把数据以及操作数据的相关方法组合在同一个单元中，使我们可以把类作为软件中的基本复用单元，提高其内聚度，降低其耦合度。面向对象中的(38)机制是对现实世界中遗传现象的模拟，通过该机制，基类的属性和方法被遗传给派生类。

- (37) A. 封装 B. 多态 C. 继承 D. 变异
(38) A. 封装 B. 多态 C. 继承 D. 变异

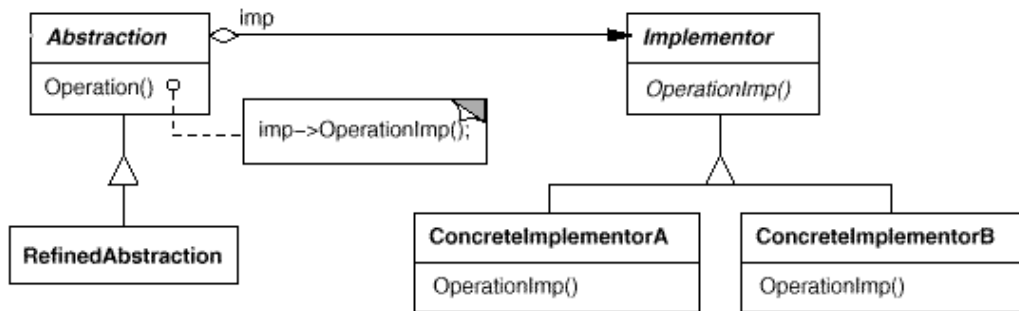
● (39)以静态或动态的连接方式，为应用程序提供一组可使用的类。(40)除了提供可被应用程序调用的类以外，还基本实现了一个可执行的架构。

- (39) A. 函数库 B. 类库 C. 框架 D. 类属
(40) A. 函数库 B. 类库 C. 框架 D. 类属

● 已知某子系统为外界提供功能服务，但该子系统中存在很多粒度十分小的类，不便被外界系统直接使用，采用(41)设计模式可以定义一个高层接口，这个接口使得这一子系统更加容易使用；当不能采用生成子类的方法进行扩充时，可采用(42)设计模式动态地给一个对象添加一些额外的职责。

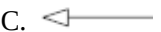
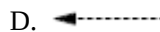
- (41) A. Facade (外观) B. Singleton (单件)
C. Participant (参与者) D. Decorator (装饰)
(42) A. Facade (外观) B. Singleton (单件)
C. Participant (参与者) D. Decorator (装饰)

● (43)设计模式将抽象部分与它的实现部分相分离，使它们都可以独立地变化。下图为该设计模式的类图，其中，(44)用于定义实现部分的接口。



- (43) A. Singleton (单件) B. Bridge (桥接)
 C. Composite (组合) D. Facade (外观)
- (44) A. Abstraction B. ConcreteImplementorA
 C. ConcreteImplementorB D. Implementor

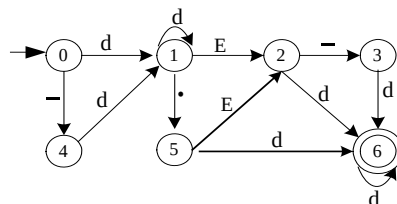
● 在 UML 类图中，类与类之间存在依赖 (Dependency)、关联 (Association)、聚合 (Aggregation)、组合 (Composition) 和继承 (Inheritance) 五种关系，其中，(45) 关系表明类之间的相互联系最弱，(46) 关系表明类之间的相互联系最强，聚合 (Aggregation) 的标准 UML 图形表示是 (47)。

- (45) A. 依赖 B. 聚合 C. 组合 D. 继承
 (46) A. 依赖 B. 聚合 C. 组合 D. 继承
 (47) A.  B.  C.  D. 

● 有限自动机 (FA) 可用于识别高级语言源程序中的记号 (单词)，FA 可分为确定的有限自动机 (DFA) 和不确定的有限自动机 (NFA)。若某 DFA D 与某 NFA M 等价，则 (48)。

- (48) A. DFA D 与 NFA M 的状态数一定相等
 B. DFA D 与 NFA M 可识别的记号相同
 C. NFA M 能识别的正规集是 DFA D 所识别正规集的真子集
 D. DFA D 能识别的正规集是 NFA M 所识别正规集的真子集

● 某确定性有限自动机 (DFA) 的状态转换图如下图所示，令 $d=0|1|2|\dots|9$ ，则以下字符串中，能被该 DFA 接受的是 (49)。



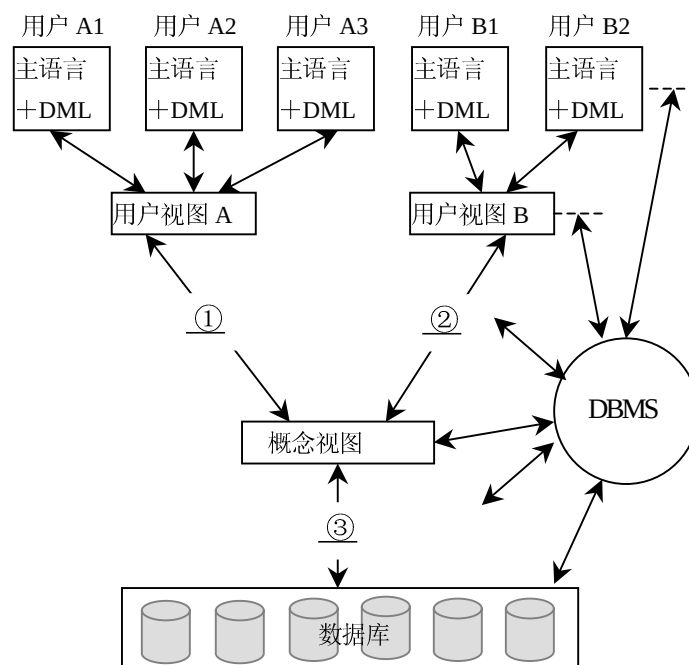
- (49) A. 3857 B. 1.2E+5 C. -123.67 D. 0.576E10

● 若有数组声明 $a[0..3, 0..2, 1..4]$ ，设编译时为 a 分配的存储空间首地址为 $base_a$ ，且每个数组元素占据一个存储单元。当元素以行为序存放（即按 $a[0, 0, 1]$ ， $a[0, 0, 2]$ ， $a[0, 0, 3]$ ， $a[0, 0, 4]$ ， $a[0, 1, 1]$ ， $a[0, 1, 2]$ ， $\dots$ ， $a[3, 2, 4]$ 顺序存储），则数组元素 $a[2, 2, 2]$ 在其存储空间中相对 $base_a$ 的偏移量是 (50)。

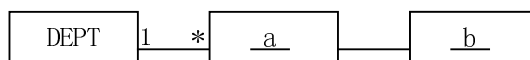
- (50) A. 8 B. 12 C. 33 D. 48

● 从数据库管理系统的角度看，数据库系统一般采用如下图所示的三级模式结构。图中①②处应填写 (51)，③处应填写 (52)。

- (51) A. 外模式 / 概念模式 B. 概念模式 / 内模式
C. 外模式 / 概念模式映象 D. 概念模式 / 内模式映象
(52) A. 外模式 / 概念模式 B. 概念模式 / 内模式
C. 外模式 / 概念模式映象 D. 概念模式 / 内模式映象



● 设有职工 EMP（职工号，姓名，性别，部门号，职务，进单位时间，电话），职务 JOB（职务，月薪）和部门 DEPT（部门号，部门名称，部门电话，负责人）实体集。一个职务可以由多个职工担任，但一个职工只能担任一个职务，并属于一个部门，部门负责人是一个职工。下图所示的 a、b 处的实体名分别为 (53)；图中 a、b 之间为 (54) 联系。



- (53) A. DEPT、EMP B. EMP、DEPT C. JOB、EMP D. EMP、JOB

- (54) A. 1 1 B. 1 * C. * 1 D. * *

● 若关系 R 、 S 如下图所示，则 R 与 S 自然连接后的属性列数和元组个数分别为 (55)； $\pi_{1,4}(\sigma_{3=6}(R \times S)) =$ (56)。

A	B	C	D	C	D
a	b	c	d	c	d
a	c	d	c	g	f
a	d	g	f		
a	b	g	f		

R

S

- (55) A. 4 和 3 B. 4 和 6 C. 6 和 3 D. 6 和 6
- (56) A. $\pi_{A,D}(\sigma_{C=D}(R \times S))$ B. $\pi_{A,R,D}(\sigma_{S.C=R.D}(R \times S))$
 C. $\pi_{A,R,D}(\sigma_{R.C=S.D}(R \times S))$ D. $\pi_{R,A,R,D}(\sigma_{S.C=S.D}(R \times S))$

● 已知一个线性表 (16, 25, 35, 43, 51, 62, 87, 93)，采用散列函数 $H(\text{Key}) = \text{Key} \bmod 7$ 将元素散列到表长为 9 的散列表中。若采用线性探测的开放定址法解决冲突（顺序地探查可用存储单元），则构造的哈希表为 (57)，在该散列表上进行等概率成功查找的平均查找长度为 (58)（为确定记录在查找表中的位置，需和给定关键字值进行比较的次数的期望值称为查找算法在查找成功时的平均查找长度）。

(57) A.

0	1	2	3	4	5	6	7	8
35	43	16	51	25		62	87	93

B.

0	1	2	3	4	5	6	7	8
35	43	16	93	25	51	62	87	

C.

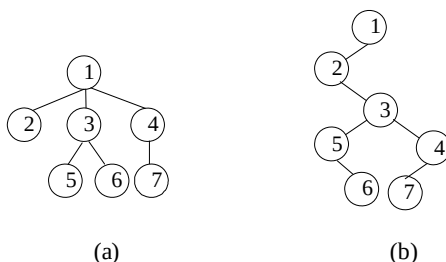
0	1	2	3	4	5	6	7	8
35	43	16	51	25	87	62	93	

D.

0	1	2	3	4	5	6	7	8
35	43	16	51	25	87	62		93

- (58) A. $(5*1+2+3+6)/8$ B. $(5*1+2+3+6)/9$
 C. $(8*1)/8$ D. $(8*1)/9$

● 若将某有序树 T 转换为二叉树 T_1 ，则 T 中结点的后（根）序序列就是 T_1 中结点的 (59) 遍历序列。例如，下图(a)所示的有序树转化为二叉树后如图(b)所示。



- (59) A. 先序 B. 中序 C. 后序 D. 层序

● 设一个包含 N 个顶点、 E 条边的简单有向图采用邻接矩阵存储结构(矩阵元素 $A[i][j]$ 等于 1/0 分别表示顶点 i 与顶点 j 之间有/无弧), 则该矩阵的元素数目为 (60), 其中非零元素数目为 (61)。

- (60) A. E^2 B. N^2 C. $N^2 - E^2$ D. $N^2 + E^2$
 (61) A. N B. $N + E$ C. E D. $N - E$

● 一个算法是对某类给定问题求解过程的精确描述, 算法中描述的操作都可以通过将已经实现的基本操作执行有限次来实现, 这句话说明算法具有 (62) 特性。

- (62) A. 有穷性 B. 可行性 C. 确定性 D. 健壮性

● 斐波那契(Fibonacci)数列可以递归地定义为:

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

用递归算法求解 $F(5)$ 时需要执行 (63) 次“+”运算, 该方法采用的算法策略是 (64)。

- (63) A. 5 B. 6 C. 7 D. 8
 (64) A. 动态规划 B. 分治 C. 回溯 D. 分支限界

● 若总是以待排序列的第一个元素作为基准元素进行快速排序, 那么最好情况下的时间复杂度为 (65)。

- (65) A. $O(\log_2 n)$ B. $O(n)$ C. $O(n \log_2 n)$ D. $O(n^2)$

● 运行 Web 浏览器的计算机与网页所在的计算机要建立 (66) 连接, 采用 (67) 协议传输网页文件。

- (66) A. UDP B. TCP C. IP D. RIP
 (67) A. HTTP B. HTML C. ASP D. RPC

● (68) 不属于电子邮件协议。

(68) A. POP3 B. SMTP C. IMAP D. MPLS

● 某客户端在采用 ping 命令检测网络连接故障时,发现可以 ping 通 127.0.0.1 及本机的 IP 地址,但无法 ping 通同一网段内其他工作正常的计算机的 IP 地址,说明该客户端的故障是 (69) 。

(69) A. TCP/IP 协议不能正常工作 B. 本机网卡不能正常工作
C. 本机网络接口故障 D. 本机 DNS 服务器地址设置错误

● 用户可以通过 http://www.a.com 和 http://www.b.com 访问在同一台服务器上 (70) 不同的两个 Web 站点。

(70) A. IP 地址 B. 端口号
C. 协议 D. 虚拟目录

● Object-oriented analysis (OOA) is a semiformal specification technique for the object-oriented paradigm. Object-oriented analysis consists of three steps. The first step is (71). It determines how the various results are computed by the product and presents this information in the form of a (72) and associated scenarios. The second is (73), which determines the classes and their attributes, then determines the interrelationships and interaction among the classes. The last step is (74), which determines the actions performed by or to each class or subclass and presents this information in the form of (75).

(71) A. use-case modeling	B. class modeling
C. dynamic modeling	D. behavioral modeling
(72) A. collaboration diagram	B. sequence diagram
C. use-case diagram	D. activity diagram
(73) A. use-case modeling	B. class modeling
C. dynamic modeling	D. behavioral modeling
(74) A. use-case modeling	B. class modeling
C. dynamic modeling	D. behavioral modeling
(75) A. activity diagram	B. component diagram
C. sequence diagram	D. state diagram

全国计算机技术与软件专业技术资格（水平）考试

2008 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2008 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是(1)月(2)日。

因为正确的解答是“5 月 24 日”，故在答题纸的对应栏内写上“5”和“24”（参看下表）。

例题	解答栏
(1)	5
(2)	24

试题一（共 15 分）

阅读以下说明和图，回答问题1至问题4，将解答填入答题纸的对应栏内。

【说明】

某音像制品出租商店欲开发一个音像管理信息系统，管理音像制品的租借业务。需求如下：

1. 系统中的客户信息文件保存了该商店的所有客户的用户名、密码等信息。对于首次来租借的客户，系统会为其生成用户名和初始密码。
2. 系统中音像制品信息文件记录了商店中所有音像制品的详细信息及其库存数量。
3. 根据客户所租借的音像制品的品种，会按天收取相应的费用。音像制品的最长租借周期为一周，每位客户每次最多只能租借 6 件音像制品。
4. 客户租借某种音像制品的具体流程为：
 - （1）根据客户提供的用户名和密码，验证客户身份。
 - （2）若该客户是合法客户，查询音像制品信息文件，查看商店中是否还有这种音像制品。
 - （3）若还有该音像制品，且客户所要租借的音像制品数小于等于 6 个，就可以将该音像制品租借给客户。这时，系统给出相应的租借确认信息，生成一条新的租借记录并将其保存在租借记录文件中。
 - （4）系统计算租借费用，将费用信息保存在租借记录文件中并告知客户。
 - （5）客户付清租借费用之后，系统接收客户付款信息，将音像制品租借给该客户。
5. 当库存中某音像制品数量不能满足客户的租借请求数量时，系统可以接受客户网上预约租借某种音像制品。系统接收到预约请求后，检查库存信息，验证用户身份，创建相应的预约记录，生成预约流水号给该客户，并将信息保存在预约记录文件中。
6. 客户归还到期的音像制品，系统修改租借记录文件，并查询预约记录文件和客户信息文件，判定是否有客户预约了这些音像制品。若有，则生成预约提示信息，通知系统履行预约服务，系统查询客户信息文件和预约记录文件，通知相关客户前来租借音像制品。

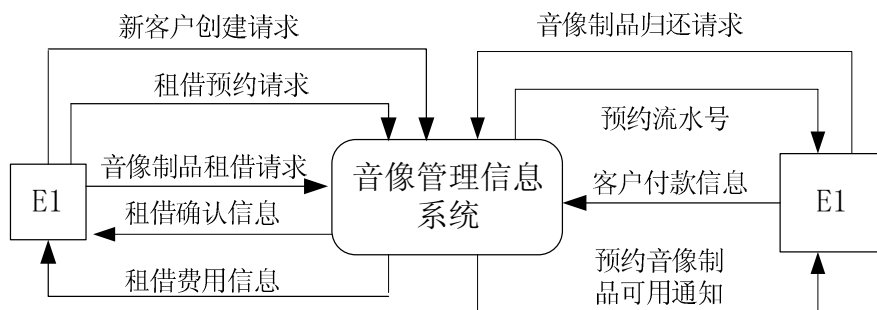


图 1-1 顶层数据流图

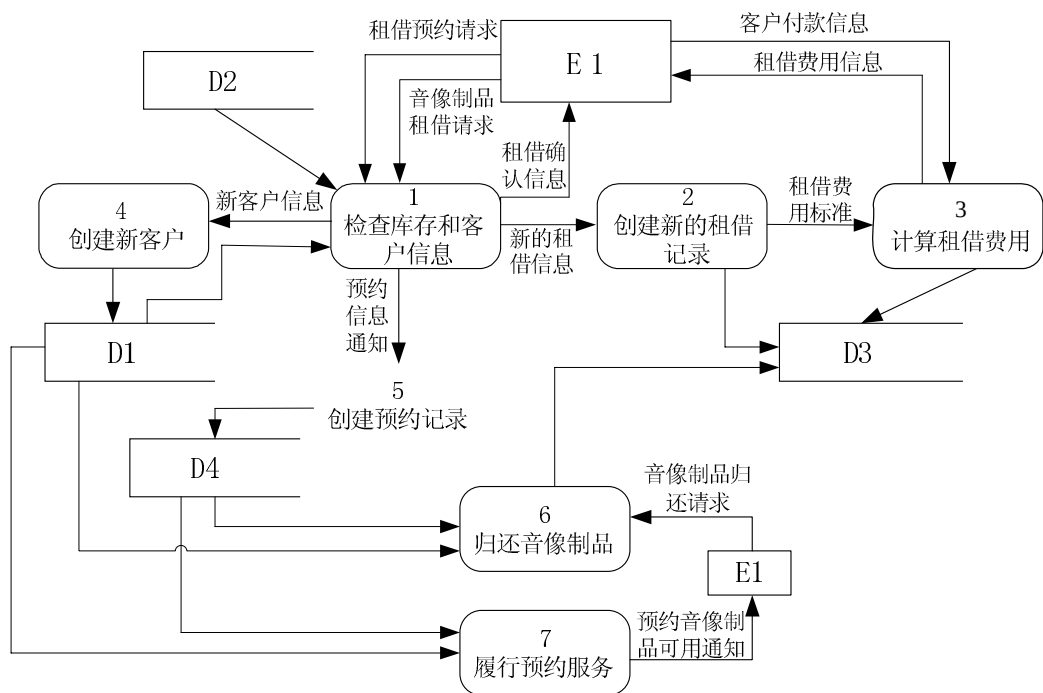


图 1-2 0 层数据流图

【问题 1】（1 分）

图 1-1 中只有一个外部实体 E1。使用【说明】中的词语，给出 E1 的名称。

【问题 2】（6 分）

使用【说明】中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（6 分）

数据流图 1-2 缺少了三条数据流，根据说明及数据流图 1-1 提供的信息，分别指出这三条数据流的起点和终点。

起 点	终 点

【问题 4】（2 分）

在进行系统分析与设计时，面向数据结构的设计方法（如 Jackson 方法）也被广泛应用。简要说明面向数据结构设计方法的基本思想及其适用场合。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某地区举行篮球比赛，需要开发一个比赛信息管理系统来记录比赛的相关信息。

【需求分析结果】

1. 登记参赛球队的信息。记录球队的名称、代表地区、成立时间等信息。系统记录球队每个队员的姓名、年龄、身高、体重等信息。每个球队有一个教练负责管理球队，一个教练仅负责一个球队。系统记录教练的姓名、年龄等信息。
2. 安排球队的训练信息。比赛组织者为球队提供了若干个场地，供球队进行适应性训练。系统记录现有的场地信息，包括：场地名称、场地规模、位置等信息。系统可为每个球队安排不同的训练场地，如表 2-1 所示。系统记录训练场地安排的信息。

表 2-1 训练安排表

球队名称	场地名称	训练时间
解放军	一号球场	2008-06-09 14:00—18:00
解放军	一号球场	2008-06-12 09:00—12:00
解放军	二号球场	2008-06-11 14:00—18:00
山西	一号球场	2008-06-10 09:00—12:00

3. 安排比赛。该赛事聘请专职裁判，每场比赛只安排一个裁判。系统记录裁判的姓名、年龄、级别等信息。系统按照一定的规则，首先分组，然后根据球队、场地和裁判情况，安排比赛（每场比赛的对阵双方分别称为甲队和乙队）。记录参赛球队名称、比赛时间、比分、比赛场地等信息，如表 2-2 所示。
4. 所有球员、教练和裁判可能出现重名情况。

表 2-2 比赛安排表

A 组：

甲队——乙队	场地名称	比赛时间	裁判	比分
解放军——北京	一号球场	2008-06-17 15:00	李大明	
天津——山西	一号球场	2008-06-17 19:00	胡学梅	

B 组：

甲队——乙队	场地名称	比赛时间	裁判	比分
上海——安徽	二号球场	2008-06-17 15:00	丁鸿平	
山东——辽宁	二号球场	2008-06-17 19:00	郭爱琪	

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图和关系模式(不完整)如下：

1. 实体联系图

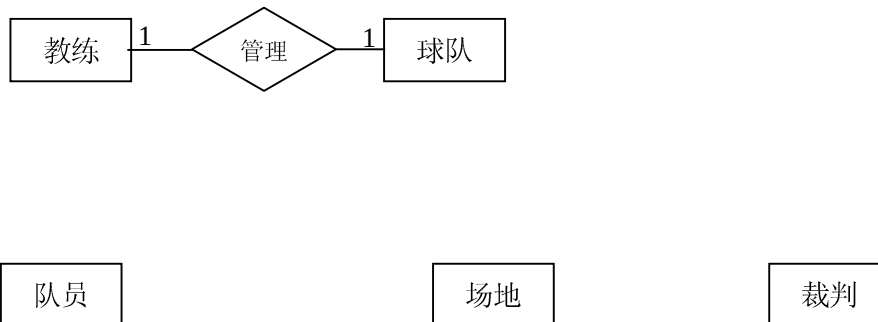


图 2-1 实体联系图

2. 关系模式

教练(教练编号, 姓名, 年龄)

队员(队员编号, 姓名, 年龄, 身高, 体重, _____(a)_____)

球队(球队名称, 代表地区, 成立时间, _____(b)_____)

场地(场地名称, 场地规模, 位置)

训练记录(_____(c)_____)

裁判(裁判编号, 姓名, 年龄, 级别)

比赛记录(_____(d)_____)

【问题 1】(4 分)

根据问题描述，补充联系及其类型，完善实体联系图 2-1。（联系及其类型的书写格式参照教练与球队之间的联系描述，联系名称也可使用**联系1**、**联系2**、...）

【问题 2】(8 分)

根据实体联系图 2-1，填充关系模式中的 (a)、(b)、(c) 和 (d)，并给出训练记录和比赛记录关系模式的主键和外键。

【问题 3】(3 分)

如果考虑记录一些特别资深的热心球迷的情况，每个热心球迷可能支持多个球队。热心球迷包括：姓名、住址和喜欢的俱乐部等基本信息。根据这一要求修改图 2-1 的实体联系图，给出修改后的关系模式。（仅给出增加的关系模式描述）

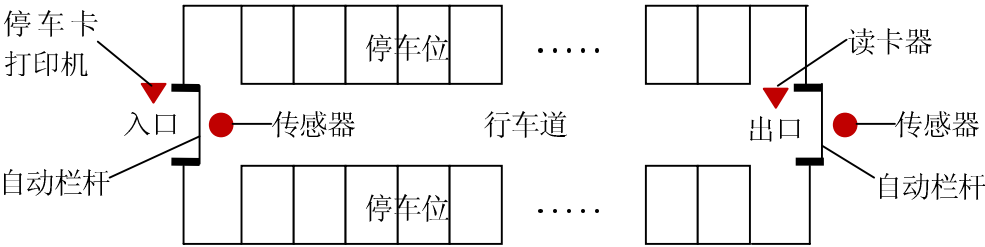
试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某汽车停车场欲建立一个信息系统，已经调查到的需求如下：

1. 在停车场的入口和出口分别安装一个自动栏杆、一台停车卡打印机、一台读卡器和一个车辆通过传感器，示意图如下：



2. 当汽车到达入口时，驾驶员按下停车卡打印机的按钮获取停车卡。当驾驶员拿走停车卡后，系统命令栏杆自动抬起；汽车通过入口后，入口处的传感器通知系统发出命令，栏杆自动放下。

3. 在停车场内分布着若干个付款机器。驾驶员将在入口处获取的停车卡插入付款机器，并缴纳停车费。付清停车费之后，将获得一张出场卡，用于离开停车场。

4. 当汽车到达出口时，驾驶员将出场卡插入出口处的读卡器。如果这张卡是有效的，系统命令栏杆自动抬起；汽车通过出口后，出口传感器通知系统发出命令，栏杆自动放下。若这张卡是无效的，系统不发出栏杆抬起命令而发出告警信号。

5. 系统自动记录停车场内空闲的停车位的数量。若停车场当前没有车位，系统将在入口处显示“车位已满”信息。这时，停车卡打印机将不再出卡，只允许场内汽车出场。

根据上述描述，采用面向对象方法对其进行分析与设计，得到了表 3-1 所示的类/用例/状态列表、图 3-1 所示的用例图、图 3-2 所示的初始类图以及图 3-3 所示的描述入口自动栏杆行为的 UML 状态图。

表 3-1 类/用例/状态列表

用例名	说明	类名	说明	状态名	说明
Car entry	汽车进入停车场	CentralComputer	停车场信息系统	Idle	空闲状态，汽车可以进入停车场
Car exit	汽车离开停车场	PaymentMachine	付款机器	Disable	没有车位
Report Statistics	记录停车场的相关信息	CarPark	停车场，保存车位信息	Await Entry	等待汽车进入
		Barrier	自动护栏	Await Ticket Take	等待打印停车卡
Car entry when full	没有车位时，汽车请求进入停车场	EntryBarrier	入口的护栏	Await Enable	等待停车场内有空闲车位
		ExitBarrier	出口的护栏		

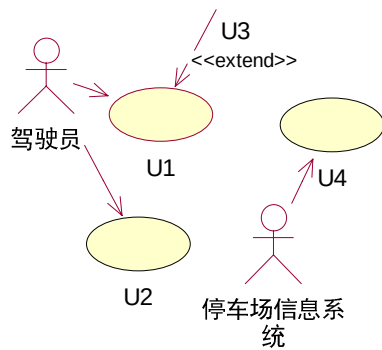


图 3-1 用例图

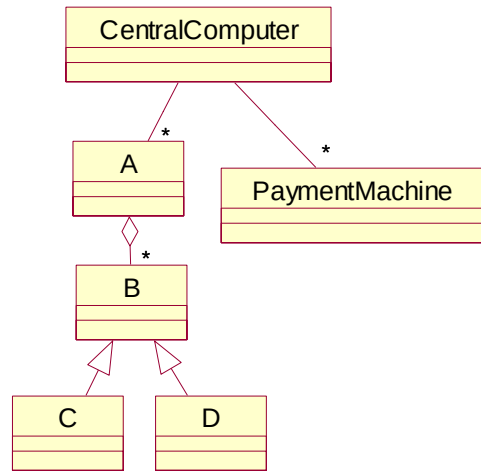


图 3-2 初始类图

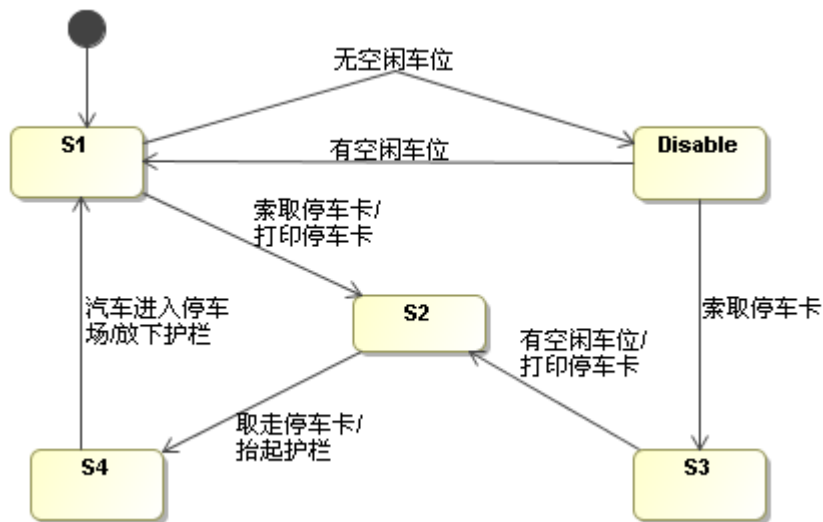


图 3-3 入口护栏的状态图

【问题 1】(3 分)

根据说明中的描述，使用表 3-1 给出的用例名称，给出图 3-1 中 U1、U2 和 U3 所对应的用例。

【问题 2】(5 分)

根据说明中的描述，使用表 3-1 给出的类的名称，给出图 3-2 中的 A~D 所对应的类。

【问题 3】(4 分)

根据说明中的描述，使用表 3-1 给出的状态名称，给出图 3-3 中 S1~S4 所对应的状态。

【问题 4】(3 分)

简要解释图 3-1 中用例 U1 和 U3 之间的 extend 关系的内涵。

试题四（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

快速排序是一种典型的分治算法。采用快速排序对数组 $A[p..r]$ 排序的三个步骤如下：

分解： 选择一个枢轴(pivot)元素划分数组。将数组 $A[p..r]$ 划分为两个子数组（可能为空） $A[p..q-1]$ 和 $A[q+1..r]$ ，使得 $A[q]$ 大于等于 $A[p..q-1]$ 中的每个元素，小于 $A[q+1..r]$ 中的每个元素。 q 的值在划分过程中计算。

递归求解： 通过递归的调用快速排序，对子数组 $A[p..q-1]$ 和 $A[q+1..r]$ 分别排序。

合并： 快速排序在原地排序，故不需合并操作。

【问题 1】（6 分）

下面是快速排序的伪代码，请填补其中的空缺。伪代码中的主要变量说明如下：

A: 待排序数组

p, r: 数组元素下标，从 p 到 r

q: 划分的位置

x: 枢轴元素

i: 整型变量，用于描述数组下标。下标小于或等于 i 的元素的值小于或等于枢轴元素的值

j: 循环控制变量，表示数组元素下标

```
QUICKSORT(A, p, r){
    if (p < r){
        q = PARTITION(A,p,r) ;
        QUICKSORT(A, p, q-1);
        QUICKSORT(A, q+1, r);
    }
}

PARTITION(A, p, r){
    x = A[r]; i = p - 1;
    for (j = p; j ≤ r - 1; j++){
        if (A[j] ≤ x){
            i = i + 1;
            交换 A[i] 和 A[j]
        }
    }
    交换 (1) 和 (2) //注：空 (1) 和空 (2) 答案可互换，但两空全部答对方可得分
    return (3)
}
```

【问题2】(5分)

(1) 假设要排序包含 n 个元素的数组，请给出在各种不同的划分情况下，快速排序的时间复杂度，用 O 记号。最佳情况为 (4)，平均情况为 (5)，最坏情况为 (6)。

(2) 假设要排序的 n 个元素都具有相同值时，快速排序的运行时间复杂度属于哪种情况？(7)。(最佳、平均、最坏)

【问题3】(4分)

(1) 待排序数组是否能被较均匀地划分对快速排序的性能有重要影响，因此枢轴元素的选取非常重要。有人提出从待排序的数组元素中随机地取出一个元素作为枢轴元素，下面是随机化快速排序划分的伪代码——利用原有的快速排序的划分操作，请填写其中的空缺处。其中， $\text{RANDOM}(i, j)$ 表示随机取 i 到 j 之间的一个数，包括 i 和 j 。

```
RANDOMIZED-PARTITION(A,p,r){  
    i = RANDOM(p,r);  
    交换 (8) 和 (9) ;//注：空(8)和空(9)答案可互换，但两空全部答对方可得分  
    return PARTITION(A,p,r);  
}
```

(2) 随机化快速排序是否能够消除最坏情况的发生？(10)。(是或否)

从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五（共 15 分）

阅读下列说明和 C 代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

栈(Stack)结构是计算机语言实现中的一种重要数据结构。对于任意栈，进行插入和删除操作的一端称为栈顶(Stack Top)，而另一端称为栈底(Stack Bottom)。栈的基本操作包括：创建栈(NewStack)、判断栈是否为空(IsEmpty)、判断栈是否已满(IsFull)、获取栈顶数据(Top)、压栈/入栈(Push)、弹栈/出栈(Pop)。

当设计栈的存储结构时，可以采取多种方式。其中，采用链式存储结构实现的栈中各数据项不必连续存储（如图 5-1）。

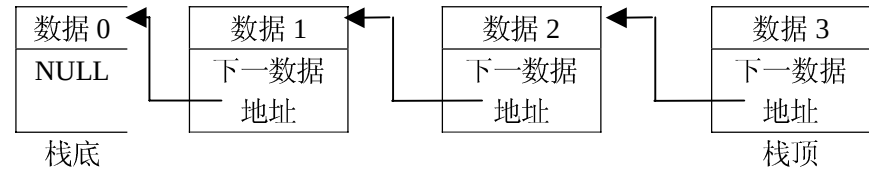


图 5-1 栈的链式存储结构示意图

以下 C 代码采用链式存储结构实现一个整数栈操作。

【C 代码】

```
typedef struct List {
    int data;           // 栈数据
    struct List* next;  // 上次入栈的数据地址
}List;

typedef struct Stack {
    List* pTop; // 当前栈顶指针
}Stack;

Stack* NewStack() { return (Stack*)calloc(1, sizeof(Stack)); }

int IsEmpty(Stack* S) { //判断栈S是否为空栈
    if( (1) ) return 1;
    return 0;
}
```

```

int Top(Stack* S){ //获取栈顶数据。若栈为空，则返回机器可表示的最小整数
    if( IsEmpty(S) ) return INT_MIN;
    return (2);
}

void Push(Stack* S, int theData) { //将数据theData压栈
    List* newNode;
    newNode = (List*)calloc(1, sizeof(List));
    newNode->data = theData;
    newNode->next = S->pTop;
    S->pTop = (3);
}

void Pop(Stack* S)  { //弹栈
    List* lastTop;
    if( IsEmpty(S) ) return;
    lastTop = S->pTop;
    S->pTop = (4);
    free(lastTop);
}

#define MD(a)  a<<2

int main() {
    int i;
    Stack* myStack;
    myStack = NewStack();
    Push(myStack, MD(1));
    Push(myStack, MD(2));
    Pop(myStack);
    Push(myStack, MD(3)+1);
    while( !IsEmpty(myStack) ) {
        printf("%d ", Top(myStack));
        Pop(myStack);
    }
    return 0;
}

```

以上程序运行时的输出结果为: (5)

试题六（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

已知某企业欲开发一家用电器遥控系统，即用户使用一个遥控器即可控制某些家用电器的开与关。遥控器如图 6-1 所示。该遥控器共有 4 个按钮，编号分别是 0 至 3，按钮 0 和 2 能够遥控打开电器 1 和电器 2，按钮 1 和 3 则能遥控关闭电器 1 和电器 2。由于遥控系统需要支持形式多样的电器，因此，该系统的设计要求具有较高的扩展性。现假设需要控制客厅电视和卧室电灯，对该遥控系统进行设计所得类图如 6-2 所示。

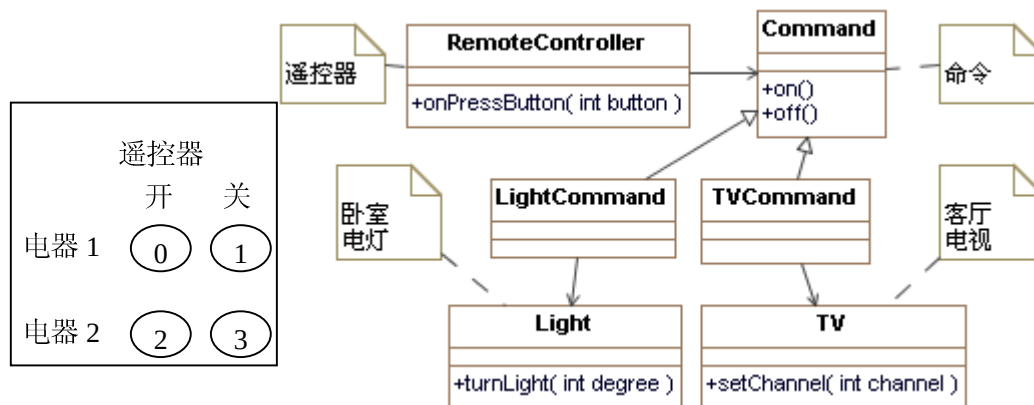


图 6-1 遥控器

图 6-2 设计类图

图 6-2 中，类 RemoteController 的方法 onPressButton(int button) 表示当遥控器按键按下时调用的方法，参数为按键的编号；Command 接口中 on 和 off 方法分别用于控制电器的开与关；Light 中 turnLight(int degree) 方法用于调整电灯灯光的强弱，参数 degree 值为 0 时表示关灯，值为 100 时表示开灯并且将灯光亮度调整到最大；TV 中 setChannel(int channel) 方法表示设置电视播放的频道，参数 channel 值为 0 时表示关闭电视，为 1 时表示开机并将频道切换为第 1 频道。

【C++ 代码】

```
class Light{ //电灯类
public:
    void turnLight(int degree){ //调整灯光亮度，0 表示关灯，100 表示亮度最大};
};
class TV{ //电视机类
public:
    void setChannel(int channel){ //调整电视频道，0 表示关机，1 表示开机并切换到 1 频道};
};
class Command{ //抽象命令类
public:
    virtual void on()=0;
    virtual void off()=0;
};
```

```

class RemoteController{ //遥控器类
protected:
    Command *commands[4]; //遥控器有 4 个按钮，按照编号分别对应 4 个 Command 对象
public:
    void onPressedButton(int button){ //按钮被按下时执行命令对象中的命令
        if(button % 2 == 0)commands[button]->on();
        else commands[button]->off();
    }
    void setCommand(int button,Command * command){
        (1) = command; //设置每个按钮对应的命令对象
    }
};

class LightCommand : public Command{ //电灯命令类
protected:    Light *light; //指向要控制的电灯对象
public:
    void on(){light->turnLight(100);};
    void off(){light->(2);};
    LightCommand(Light * light){this->light = light;};
};

class TVCommand : public Command{ //电视机命令类
protected:    TV * tv; //指向要控制的电视机对象
public:
    void on(){tv->(3);};
    void off(){tv->setChannel(0);};
    TVCommand(TV * tv){ this->tv = tv; };
};

void main(){
    Light light;    TV tv; //创建电灯和电视对象
    LightCommand lightCommand(&light);
    TVCommand tvCommand(&tv);
    RemoteController remoteController;
    remoteController.setCommand(0,(4)); //设置按钮 0 的命令对象
    ...//此处省略设置按钮 1、按钮 2 和按钮 3 的命令对象代码
}

```

本题中，应用命令模式能够有效让类(5)和类(6)、类(7)之间的耦合性降至最小。

试题七（共 15 分）

阅读下列说明和 Java 代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

已知某企业欲开发一家用电器遥控系统，即用户使用一个遥控器即可控制某些家用电器的开与关。遥控器如图 7-1 所示。该遥控器共有 4 个按钮，编号分别是 0 至 3，按钮 0 和 2 能够遥控打开电器 1 和电器 2，按钮 1 和 3 则能遥控关闭电器 1 和电器 2。由于遥控系统需要支持形式多样的电器，因此，该系统的设计要求具有较高的扩展性。现假设需要控制客厅电视和卧室电灯，对该遥控系统进行设计所得类图如 7-2 所示。

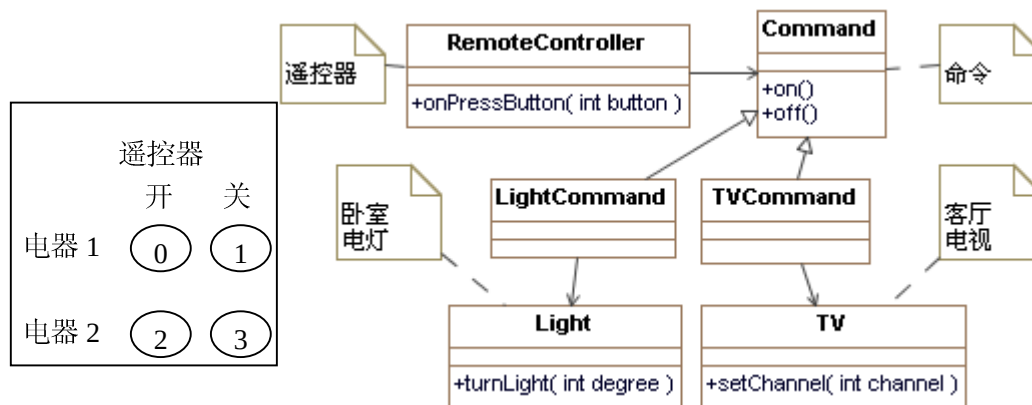


图 7-1 遥控器

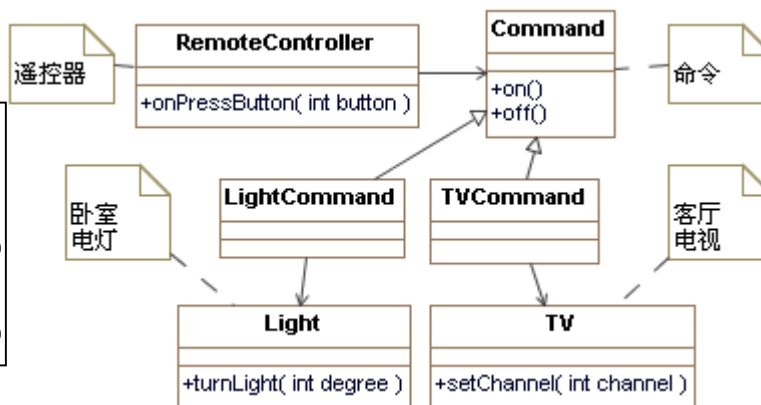


图 7-2 设计类图

图 7-2 中，类 RemoteController 的方法 onPressButton(int button)表示当遥控器按键按下时调用的方法，参数为按键的编号；Command 接口中 on 和 off 方法分别用于控制电器的开与关；Light 中 turnLight(int degree)方法用于调整电灯灯光的强弱，参数 degree 值为 0 时表示关灯，值为 100 时表示开灯并且将灯光亮度调整到最大；TV 中 setChannel(int channel)方法表示设置电视播放的频道，参数 channel 值为 0 时表示关闭电视，为 1 时表示开机并将频道切换为第 1 频道。

【Java 代码】

```
class Light{ //电灯类
    public void turnLight(int degree){ //调整灯光亮度，0表示关灯，100表示亮度最大}
};
class TV{ //电视机类
    public void setChannel(int channel){ // 0表示关机，1表示开机并切换到1频道 }
};
interface Command{ //抽象命令类
    void on();
    void off();
};
```

```

class RemoteController{ //遥控器类
    protected Command []commands = new Command[4];
    //遥控器有4个按钮，按照编号分别对应4个Command对象
    public void onPressButton(int button){
        //按钮被按下时执行命令对象中的命令
        if(button % 2 == 0)commands[button].on();
        else commands[button].off();
    }
    public void setCommand(int button, Command command){
        (1) = command; //设置每个按钮对应的命令对象
    }
};

class LightCommand implements Command{ //电灯命令类
    protected Light light; //指向要控制的电灯对象
    public void on(){light.turnLight(100);};
    public void off(){light.(2) };};
    public LightCommand(Light light){this.light = light;};
};

class TVCommand implements Command{ //电视机命令类
    protected TV tv; //指向要控制的电视机对象
    public void on(){tv.(3) };};
    public void off(){tv.setChannel(0);};
    public TVCommand(TV tv){this.tv = tv;};
};

public class rs{
    public static void main(String []args){
        Light light = new Light();    TV tv = new TV();//创建电灯和电视对象
        LightCommand lightCommand = new LightCommand(light);
        TVCommand tvCommand = new TVCommand(tv);
        RemoteController remoteController = new RemoteController();
        //设置按钮和命令对象
        remoteController.setCommand(0,(4) );
        ...//此处省略设置按钮 1、按钮 2 和按钮 3 的命令对象代码
    }
}

```

本题中，应用命令模式能够有效让类 (5) 和类 (6)、类 (7) 之间的耦合性降至最小。

全国计算机技术与软件专业技术资格（水平）考试

2008 年下半年 软件设计师 上午试卷（B）

（考试时间 9：00～11：30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2008 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
____（88）____月____（89）____日。

（88） A. 11 B. 12 C. 10 D. 9

（89） A. 18 B. 19 C. 20 D. 21

因为考试日期是“12 月 21 日”，故（88）选 B，（89）选 D，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 D 填涂（参看答题卡）。

● 计算机内存一般分为静态数据区、代码区、栈区和堆区，若某指令的操作数之一采用立即数寻址方式，则该操作数位于(1)。

- (1) A. 静态数据区 B. 代码区 C. 栈区 D. 堆区

● 计算机在进行浮点数的相加（减）运算之前先进行对阶操作，若 x 的阶码大于 y 的阶码，则应将(2)。

- (2) A. x 的阶码缩小至与 y 的阶码相同，且使 x 的尾数部分进行算术左移
B. x 的阶码缩小至与 y 的阶码相同，且使 x 的尾数部分进行算术右移
C. y 的阶码扩大至与 x 的阶码相同，且使 y 的尾数部分进行算术左移
D. y 的阶码扩大至与 x 的阶码相同，且使 y 的尾数部分进行算术右移

● 在 CPU 中，(3) 可用于传送和暂存用户数据，为 ALU 执行算术逻辑运算提供工作区。

- (3) A. 程序计数器 B. 累加寄存器 C. 程序状态寄存器 D. 地址寄存器

● 下面关于在 I/O 设备与主机间交换数据的叙述，(4) 是错误的。

- (4) A. 中断方式下，CPU 需要执行程序来实现数据传送任务
B. 中断方式和 DMA 方式下，CPU 与 I/O 设备都可同步工作
C. 中断方式和 DMA 方式中，快速 I/O 设备更适合采用中断方式传递数据
D. 若同时接到 DMA 请求和中断请求，CPU 优先响应 DMA 请求

● 下面关于校验方法的叙述，(5) 是正确的。

- (5) A. 采用奇偶校验可检测数据传输过程中出现一位数据错误的位置并加以纠正
B. 采用海明校验可检测数据传输过程中出现一位数据错误的位置并加以纠正
C. 采用海明校验，校验码的长度和位置可随机设定
D. 采用 CRC 校验，需要将校验码分散开并插入数据的指定位置中

● Cache 用于存放主存数据的部分拷贝，主存单元地址与 Cache 单元地址之间的转换工作由(6) 完成。

- (6) A. 硬件 B. 软件 C. 用户 D. 程序员

● 在 Windows Server 2003 下若选择安全登录，则首先需要按(7) 组合键。

- (7) A. Shift+Alt+Esc B. Ctrl+Alt+Tab C. Ctrl+Shift D. Ctrl+Alt+Del

● 为了防止电子邮件中的恶意代码，应该用(8) 方式阅读电子邮件。

- (8) A. 文本 B. 网页 C. 程序 D. 会话

● TCP/IP 在多个层引入了安全机制，其中 TLS 协议位于(9)。

- (9) A. 数据链路层 B. 网络层 C. 传输层 D. 应用层

● 我国专利申请的原则之一是 (10) 。

- (10) A. 申请在先
B. 申请在先与使用在先相结合
C. 使用在先
D. 申请在先、使用在先或者二者结合

● 李某在《电脑与编程》杂志上看到张某发表的一组程序，颇为欣赏，就复印了一百份作为程序设计辅导材料发给了学生。李某又将这组程序逐段加以评析，写成评论文章后投到《电脑编程技巧》杂志上发表。李某的行为（11）。

- (11) A. 侵犯了张某的著作权，因为其未经许可，擅自复印张某的程序
B. 侵犯了张某的著作权，因为在评论文章中全文引用了发表的程序
C. 不侵犯张某的著作权，其行为属于合理使用
D. 侵犯了张某的著作权，因为其擅自复印，又在其发表的文章中全文引用了张某的程序

● MP3 是目前最流行的数字音乐压缩编码格式之一，其命名中“MP”是指 (12)，
“3”是指 (13)。

- (12) A. media player B. multiple parts C. music player D. MPEG-1 Audio
(13) A. MPEG-3 B. version 3 C. part 3 D. layer 3

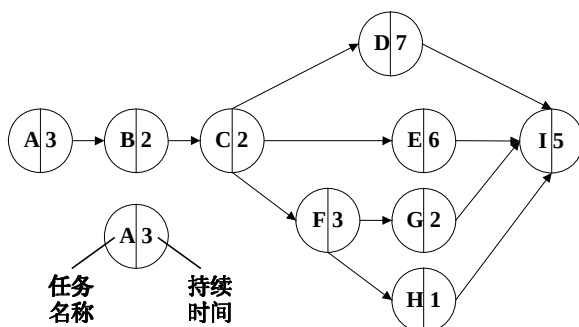
● 某数码相机内置 128MB 的存储空间，拍摄分辨率设定为 1600×1200 像素，颜色深度为 24 位，若不采用压缩存储技术，使用内部存储器最多可以存储 (14) 张照片。

- (14) A. 12 B. 22 C. 13 D. 23

● 在面向对象系统中，用(15)关系表示一个较大的“整体”类包含一个或多个较小的“部分”类。

- (15) A. 泛化 B. 聚合 C. 概化 D. 合成

● 若一个项目由 9 个主要任务构成，其计划图（如下图所示）展示了任务之间的前后关系以及每个任务所需天数，该项目的关键路径是（16），完成项目所需的最短时间是（17）天。



- (16) A. $A \rightarrow B \rightarrow C \rightarrow D \rightarrow I$ B. $A \rightarrow B \rightarrow C \rightarrow E \rightarrow I$
 C. $A \rightarrow B \rightarrow C \rightarrow F \rightarrow G \rightarrow I$ D. $A \rightarrow B \rightarrow C \rightarrow F \rightarrow H \rightarrow I$
- (17) A. 16 B. 17 C. 18 D. 19

● RUP (Rational Unified Process) 分为 4 个阶段，每个阶段结束时都有重要的里程碑，其中生命周期架构是在 (18) 结束时的里程碑。

- (18) A. 初启阶段 B. 精化阶段 C. 构建阶段 D. 移交阶段

● 在软件工程环境中进行风险识别时，常见的、已知的及可预测的风险类包括产品规模、商业影响等，与开发工具的可用性及质量相关的风险是 (19)。

- (19) A. 客户特性 B. 过程定义 C. 开发环境 D. 构建技术

● 编译程序对高级语言源程序进行翻译时，需要在该程序的地址空间中为变量指定地址，这种地址称为 (20)。

- (20) A. 逻辑地址 B. 物理地址 C. 接口地址 D. 线性地址

● 程序设计语言一般都提供多种循环语句，例如实现先判断循环条件再执行循环体的 while 语句和先执行循环体再判断循环条件的 do-while 语句。关于这两种循环语句，在不改变循环体的条件下，(21) 是正确的。

- (21) A. while 语句的功能可由 do-while 语句实现
B. do-while 语句的功能可由 while 语句实现
C. 若已知循环体的次数，则只能使用 while 语句
D. 循环条件相同时，do-while 语句的执行效率更高

● 表达式 $(a-b)*(c+5)$ 的后缀式是 (22)。

- (22) A. $a\ b\ c\ 5\ +\ * -$ B. $a\ b -\ c\ +\ 5\ *$
C. $a\ b\ c -\ * 5\ +$ D. $a\ b -\ c\ 5\ +\ *$

● 假设系统中有四类互斥资源 R1、R2、R3 和 R4，可用资源数分别为 9、6、3 和 3。在 T_0 时刻系统中有 P1、P2、P3 和 P4 四个进程，这些进程对资源的最大需求量和已分配资源数如下表所示。在 T_0 时刻系统剩余的可用资源数分别为 (23)。如果 P1、P2、P3 和 P4 进程按 (24) 序列执行，那么系统状态是安全的。

资源 进程	最大需求量				已分配资源数			
	R1	R2	R3	R4	R1	R2	R3	R4
P1	6	4	2	1	1	1	1	1
P2	2	2	2	1	2	1	1	1
P3	8	1	1	1	2	1	0	0
P4	2	2	1	1	1	2	1	1

- (23) A. 2、1、0 和 1 B. 3、1、0 和 0
C. 3、1、1 和 1 D. 3、0、1 和 1
(24) A. P1→P2→P4→P3 B. P2→P1→P4→P3
C. P3→P4→P1→P2 D. P4→P2→P1→P3

● 某文件管理系统为了记录磁盘的使用情况，在磁盘上建立了位示图(bitmap)。若系统中字长为 16 位，磁盘上的物理块依次编号为：0、1、2、...，那么 8192 号物理块的使用情况在位示图中的第 (25) 个字中描述。

(25) A. 256 B. 257 C. 512 D. 513

● 在操作系统设备管理中，通常临界资源不能采用 (26) 分配算法。

(26) A. 静态优先级 B. 动态优先级 C. 时间片轮转 D. 先来先服务

● 某虚拟存储系统采用最近最少使用 (LRU) 页面淘汰算法。假定系统为每个作业分配 3 个页面的主存空间，其中一个页面用来存放程序。现有某作业的部分语句如下：

```
Var A: Array[1..128,1..128] OF integer;
```

```
  i,j: integer;
```

```
  FOR i:=1 to 128 DO
```

```
    FOR j:=1 to 128 DO
```

```
      A[i,j]:=0;
```

设每个页面可存放 128 个整数变量，变量 i、j 放在程序页中，矩阵 A 按行序存放。初始时，程序及变量 i、j 已在内存，其余两页为空。在上述程序片段执行过程中，共产生 (27) 次缺页中断。最后留在内存中的是矩阵 A 的最后 (28)。

(27) A. 64 B. 128 C. 256 D. 512

(28) A. 2 行 B. 2 列 C. 1 行 D. 1 列

● 软件能力成熟度模型 (CMM) 将软件能力成熟度自低到高依次划分为初始级、可重复级、定义级、管理级和优化级。其中 (29) 对软件过程和产品都有定量的理解与控制。

(29) A. 可重复级和定义级 B. 定义级和管理级
 C. 管理级和优化级 D. 定义级、管理级和优化级

● 极限编程 (XP) 包含了策划、设计、编程和测试四个活动，其十二个最佳实践中的“持续集成”实践在 (30) 活动中进行。

(30) A. 策划和设计 B. 设计和编程
 C. 设计和测试 D. 编程和测试

● ISO/IEC 9126 软件质量模型中第一层定义了六个质量特性，并为各质量特性定义了相应的质量子特性。子特性 (31) 属于可靠性质量特性。

(31) A. 准确性 B. 易理解性 C. 成熟性 D. 易学性

● UP (统一过程) 是用例驱动的、以架构为核心、迭代和增量的软件过程框架，它提供了一种 (32) 的特性。

(32) A. 演进 B. 敏捷 C. 测试驱动 D. 持续集成

● 系统测试人员与系统开发人员需要通过文档进行沟通，系统测试人员应根据一系列文档对系统进行测试，然后将工作结果撰写成(33)，交给系统开发人员。

- (33) A. 系统开发合同 B. 系统设计说明书
C. 测试计划 D. 系统测试报告

● 系统的可维护性可以用系统的可维护性评价指标来衡量。系统的可维护性评价指标不包括(34)。

- (34) A. 可理解性 B. 可修改性 C. 准确性 D. 可测试性

● 某程序根据输入的三条线段长度，判断这三条线段能否构成三角形。以下 6 个测试用例中，(35) 两个用例属于同一个等价类。

- ①6、7、13; ②4、7、10; ③9、20、35;
④9、11、21; ⑤5、5、4; ⑥4、4、4。
(35) A. ①② B. ③④ C. ⑤⑥ D. ①④

● 在模拟环境下，常采用黑盒测试检验所开发的软件是否与需求规格说明书一致。其中有效性测试属于(36)中的一个步骤。

- (36) A. 单元测试 B. 集成测试 C. 确认测试 D. 系统测试

● 面向对象分析与设计中的(37)是指一个模块在扩展性方面应该是开放的，而在更改性方面应该是封闭的；而(38)是指子类应当可以替换父类并出现在父类能够出现的任何地方。

- (37) A. 开闭原则 B. 替换原则 C. 依赖原则 D. 单一职责原则
(38) A. 开闭原则 B. 替换原则 C. 依赖原则 D. 单一职责原则

● 在选择某种面向对象语言进行软件开发时，不需要着重考虑的因素是，该语言(39)。

- (39) A. 将来是否能够占据市场主导地位
B. 类库是否丰富
C. 开发环境是否成熟
D. 是否支持全局变量和全局函数的定义

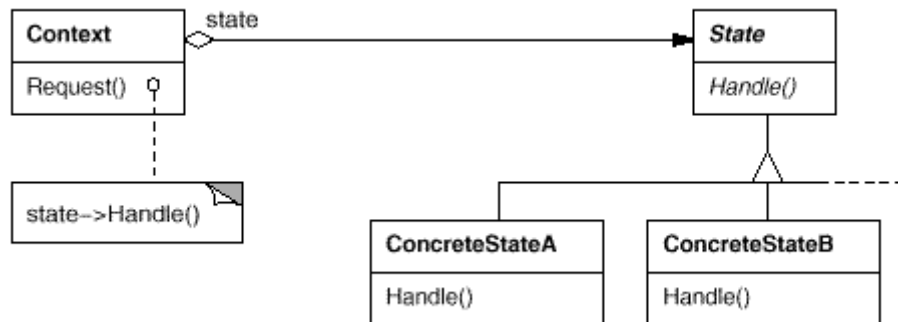
● (40) 限制了创建类的实例数量，而(41) 将一个类的接口转换成客户希望的另外一个接口，使得原本由于接口不兼容而不能一起工作的那些类可以一起工作。

- (40) A. 命令模式 (Command) B. 适配器模式 (Adapter)
C. 策略模式 (Strategy) D. 单例模式 (Singleton)
(41) A. 命令模式 (Command) B. 适配器模式 (Adapter)
C. 策略模式 (Strategy) D. 单例模式 (Singleton)

● (42) 是指在运行时把过程调用和响应调用所需要执行的代码加以结合。

(42) A. 绑定 B. 静态绑定 C. 动态绑定 D. 继承

● (43) 设计模式允许一个对象在其内部状态改变时改变它的行为。下图为这种设计模式的类图，已知类 State 为抽象类，则类 (44) 的实例代表了 Context 对象的状态。



(43) A. 单件 (Singleton) B. 桥接 (Bridge)

C. 组合 (Composite) D. 状态 (State)

(44) A. Context B. concreteStateA

C. Handle D. State

● 在 UML 的各种视图中，(45) 显示外部参与者观察到的系统功能；(46) 从系统的静态结构和动态行为角度显示系统内部如何实现系统的功能；(47) 显示的是源代码以及实际执行代码的组织结构。

(45) A. 用例视图 B. 进程视图 C. 实现视图 D. 逻辑视图

(46) A. 用例视图 B. 进程视图 C. 实现视图 D. 逻辑视图

(47) A. 用例视图 B. 进程视图 C. 实现视图 D. 逻辑视图

● 给定文法 $G[S]$ 及其非终结符 A ， $FIRST(A)$ 定义为：从 A 出发能推导出的终结符号的集合 (S 是文法的起始符号，为非终结符)。对于文法 $G[S]$ ：

$S \rightarrow [L] | a$

$L \rightarrow L, S | S$

其中， $G[S]$ 包含的四个终结符号分别为：

$a, [,],$

则 $FIRST(S)$ 的成员包括 (48)。

(48) A. a B. $a, [$ C. $a, [和]$ D. $a, [,]和,$

● 高级语言源程序的编译过程分若干个阶段，分配寄存器属于 (49) 阶段的工作。

(49) A. 词法分析 B. 语法分析 C. 语义分析 D. 代码生成

● 设某上下文无关文法如下： $S \rightarrow 11 \mid 1001 \mid S0 \mid SS$ ，则该文法所产生的所有二进制字符串都具有的特点是 (50)。

- (50) A. 能被 3 整除
B. 0、1 出现的次数相等
C. 0 和 1 的出现次数都为偶数
D. 能被 2 整除

● 某学校学生、教师和课程实体对应的关系模式如下：

学生 (学号, 姓名, 性别, 年龄, 家庭住址, 电话)

课程（课程号，课程名）

教师（职工号，姓名，年龄，家庭住址，电话）

如果一个学生可以选修多门课程，一门课程可以有多个学生选修；一个教师只能讲授一门课程，但一门课程可以有多个教师讲授。由于学生和课程之间是一个（51）的联系，所以（52）。又由于教师和课程之间是一个（53）的联系，所以（54）。

- (51) A. 1 对 1 B. 1 对多 C. 多对 1 D. 多对多
- (52) A. 不需要增加一个新的关系模式
B. 不需要增加一个新的关系模式，只需要将 1 端的码插入多端
C. 需要增加一个新的选课关系模式，该模式的主键应该为课程号
D. 需要增加一个新的选课关系模式，该模式的主键应该为课程号和学号
- (53) A. 1 对 1 B. 1 对多 C. 多对 1 D. 多对多
- (54) A. 不需要增加一个新的关系模式，只需要将职工号插入课程关系模式
B. 不需要增加一个新的关系模式，只需要将课程号插入教师关系模式
C. 需要增加一个新的选课关系模式，该模式的主键应该为课程号
D. 需要增加一个新的选课关系模式，该模式的主键应该为课程号和教师号

● 关系 R、S 如下图所示，关系代数表达式 $\pi_{1,5,6}(\sigma_{2=5}(R \times S)) = \underline{\hspace{2cm}}$ (55)，该表达式与 (56) 等价。

A	B	C
3	0	3
2	5	6
5	8	9
8	11	12

R

A	B	C
3	10	11
4	11	6
5	10	13
6	11	14

S

- (55) A.

A	B	C
3	0	3
5	8	9

B.

A	B	C
8	11	6
8	11	14

C.

A	B	C
5	10	11
5	10	13

D.

A	B	C
2	11	6
2	11	14

- (56) A. $\pi_{A,B,C}(\sigma_{B=B}(R \times S))$ B. $\pi_{R,A,R,B,R,C}(\sigma_{R=B=S,B}(R \times S))$
 C. $\pi_{R,A,S,B,S,C}(\sigma_{R=B=S,B}(R \times S))$ D. $\pi_{R,A,S,B,S,C}(\sigma_{R=B=S,C}(R \times S))$

● 一个具有 m 个结点的二叉树，其二叉链表结点（左、右孩子指针分别用 `left` 和 `right` 表示）中的空指针总数必定为 (57) 个。为形成中序（先序、后序）线索二叉树，现对该二叉链表所有结点进行如下操作：若结点 p 的左孩子指针为空，则将该左指针改为指向 p 在中序（先序、后序）遍历序列的前驱结点；若 p 的右孩子指针为空，则将该右指针改为指向 p 在中序（先序、后序）遍历序列的后继结点。假设指针 s 指向中序（先序、后序）线索二叉树中的某结点，则 (58)。

(57) A. $m+2$ B. $m+1$ C. m D. $m-1$

(58) A. $s \rightarrow \text{right}$ 指向的结点一定是 s 所指结点的直接后继结点

B. $s \rightarrow \text{left}$ 指向的结点一定是 s 所指结点的直接前驱结点

C. 从 s 所指结点出发的 `right` 链可能构成环

D. s 所指结点的 `left` 和 `right` 指针一定指向不同的结点

● (59) 的邻接矩阵是一个对称矩阵。

(59) A. 无向图 B. AOV 网 C. AOE 网 D. 有向图

● 将一个无序序列中的元素依次插入到一棵 (60)，并进行中序遍历，可得到一个有序序列。

(60) A. 完全二叉树 B. 最小生成树 C. 二叉排序树 D. 最优二叉树

● 广义表中的元素可以是原子，也可以是表，因此广义表的适用存储结构是 (61)。

(61) A. 链表 B. 静态数组 C. 动态数组 D. 散列表

● 某一维数组中依次存放了数据元素 12,23,30,38,41,52,54,76,85，在用折半（二分）查找方法（向上取整）查找元素 54 时，所经历“比较”运算的数据元素依次为 (62)。

(62) A. 41, 52, 54 B. 41, 76, 54 C. 41, 76, 52, 54 D. 41, 30, 76, 54

● 具有 n 个顶点、 e 条边的图采用邻接表存储结构，进行深度优先遍历和广度优先遍历运算的时间复杂度均为 (63)。

(63) A. $O(n^2)$ B. $O(e^2)$ C. $O(n \cdot e)$ D. $O(n+e)$

● 给定一组长度为 n 的无序序列，将其存储在一维数组 $a[0..n-1]$ 中。现采用如下方法找出其中的最大元素和最小元素：比较 $a[0]$ 和 $a[n-1]$ ，若 $a[0]$ 较大，则将二者的值进行交换；再比较 $a[1]$ 和 $a[n-2]$ ，若 $a[1]$ 较大，则交换二者的值；然后依次比较 $a[2]$ 和 $a[n-3]$ 、 $a[3]$ 和 $a[n-4]$ 、...，使得每一对元素中的较小者被交换到低下标端。重复上述方法，在数组的前 $n/2$ 个元素中查找最小元素，在后 $n/2$ 个元素查找最大元素，从而得到整个序列的最小元素和最大元素。上述方法采用的算法设计策略是 (64)。

(64) A. 动态规划法 B. 贪心法 C. 分治法 D. 回溯法

● 设某算法的计算时间表示为递推关系式 $T(n) = T(n-1) + n$ ($n > 0$) 及 $T(0) = 1$, 则该算法的时间复杂度为 (65)。

(65) A. $O(\lg n)$ B. $O(n \lg n)$ C. $O(n)$ D. $O(n^2)$

● ADSL 是一种宽带接入技术, 这种技术使用的传输介质是 (66)。

(66) A. 电话线 B. CATV 电缆
C. 基带同轴电缆 D. 无线通信网

● 下面关于网络系统设计原则的论述, 正确的是 (67)。

(67) A. 应尽量采用先进的网络设备, 获得最高的网络性能
B. 网络总体设计过程中, 只需要考虑近期目标即可, 不需要考虑扩展性
C. 网络系统应采用开放的标准和技术
D. 网络需求分析独立于应用系统的需求分析

● 下面的选项中, 属于本地回路地址的是 (68)。

(68) A. 120.168.10.1 B. 10.128.10.1
C. 127.0.0.1 D. 172.16.0.1

● 下面关于防火墙的说法, 正确的是 (69)。

(69) A. 防火墙一般由软件以及支持该软件运行的硬件系统构成
B. 防火墙只能防止未经授权的信息发送到内网
C. 防火墙能准确地检测出攻击来自哪一台计算机
D. 防火墙的主要支撑技术是加密技术

● Internet 上的 DNS 服务器中保存有 (70)。

(70) A. 主机名 B. 域名到 IP 地址的映射表
C. 所有主机的 MAC 地址 D. 路由表

● It should go without saying that the focus of UML is modeling. However, what that means, exactly, can be an open-ended question. (71) is a means to capture ideas, relationships, decisions, and requirements in a well-defined notation that can be applied to many different domains. Modeling not only means different things to different people, but also it can use different pieces of UML depending on what you are trying to convey. In general, a UML model is made up of one or more (72). A diagram graphically represents things, and the relationships between these things. These (73) can be representations of real-world objects, pure software constructs, or a description of the behavior of some other objects. It is common for an individual thing to show up on multiple diagrams; each diagram represents a particular interest, or view, of the thing being modeled. UML 2.0 divides diagrams into two categories:

structural diagrams and behavioral diagrams. (74) are used to capture the physical organization of the things in your system, i.e., how one object relates to another. (75) focus on the behavior of elements in a system. For example, you can use behavioral diagrams to capture requirements, operations, and internal state changes for elements.

- | | | | |
|---------------------------|------------------------|------------------------|-----------------------|
| (71) A. Programming | B. Analyzing | C. Designing | D. Modeling |
| (72) A. views | B. diagrams | C. user views | D. structure pictures |
| (73) A. things | B. pictures | C. languages | D. diagrams |
| (74) A. Activity diagrams | | B. Use-case diagrams | |
| | C. Structural diagrams | D. Behavioral diagrams | |
| (75) A. Activity diagrams | | B. Use-case diagrams | |
| | C. Structural diagrams | D. Behavioral diagrams | |

全国计算机技术与软件专业技术资格（水平）考试

2008 年下半年 软件设计师 下午试卷（B）

（考试时间 14:00~16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2008 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是(1)月(2)日。

因为正确的解答是“12 月 21 日”，故在答题纸的对应栏内写上“12”和“21”（参看下表）。

例题	解答栏
(1)	12
(2)	21

试题一（共 15 分）

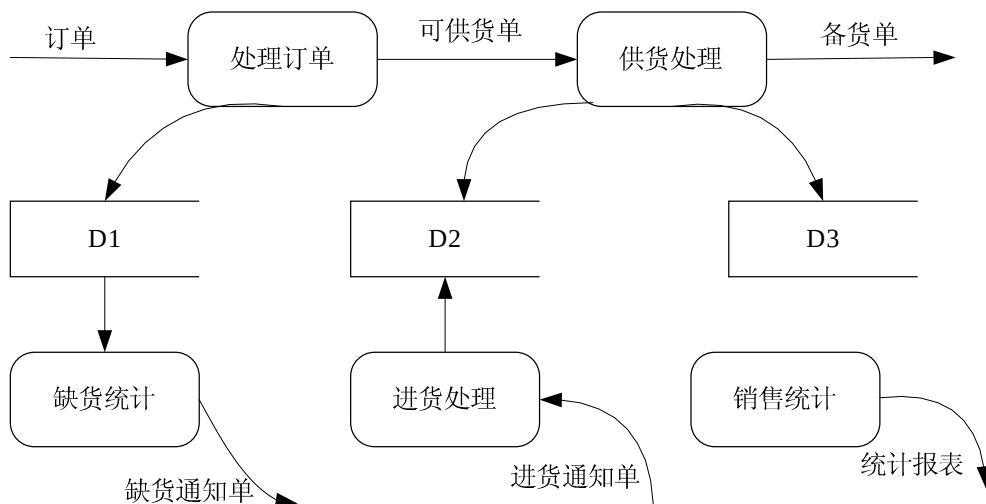
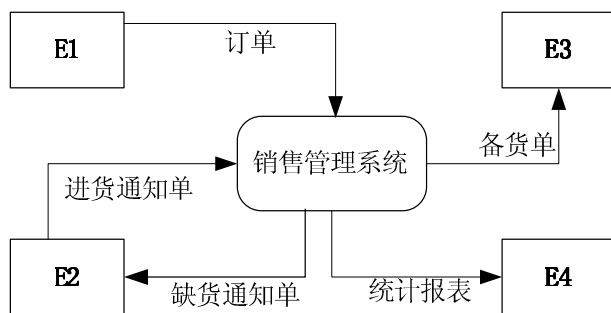
阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某营销企业拟开发一个销售管理系统，其主要功能描述如下：

1. 接受客户订单，检查库存货物是否满足订单要求。如果满足，进行供货处理：即修改库存记录文件，给库房开具备货单并且保留客户订单至订单记录文件；否则进行缺货处理：将缺货订单录入缺货记录文件。
2. 根据缺货记录文件进行缺货统计，将缺货通知单发给采购部门。
3. 根据采购部门提供的进货通知单进行进货处理：即修改库存记录文件，并从缺货记录文件中取出缺货订单进行供货处理。
4. 根据保留的客户订单进行销售统计，打印统计报表给经理。

现采用结构化方法对销售管理系统进行分析与设计，获得如图 1-1 所示的顶层数据流图和图 1-2 所示的 0 层数据流图。



【问题1】(4分)

使用说明中的词语，给出图 1-1 的外部实体 E1~E4 的名称。

【问题2】(3分)

使用说明中的词语，给出图 1-2 的数据存储 D1~D3 的名称。

【问题3】(8分)

数据流图 1-2 缺少了四条数据流，根据说明及数据流图 1-1 提供的信息，分别指出这四条数据流的起点和终点。

起 点	终 点

试题二（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某宾馆拟开发一个宾馆客房预订子系统，主要是针对客房的预订和入住等情况进行管理。

【需求分析结果】

1. 员工信息主要包括：员工号、姓名、出生年月、性别、部门、岗位、住址、联系电话和密码等信息。岗位有管理和服务两种。岗位为“管理”的员工可以更改（添加、删除和修改）员工表中的本部门员工的岗位和密码，要求将每一次更改前的信息保留；岗位为“服务”的员工只能修改员工表中本人的密码，且负责多个客房的清理等工作。

2. 部门信息主要包括：部门号、部门名称、部门负责人、电话等信息；一个员工只能属于一个部门，一个部门只有一位负责人。

3. 客房信息包括：客房号、类型、价格、状态等信息。其中类型是指单人间、三人间、普通标准间、豪华标准间等；状态是指空闲、入住和维修。

4. 客户信息包括：身份证号、姓名、性别、单位和联系电话。

5. 客房预定情况包括：客房号、预定日期、预定入住日期、预定入住天数、身份证号等信息。一条预定信息必须且仅对应一位客户，但一位客户可以有 multiple 预定信息。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示：

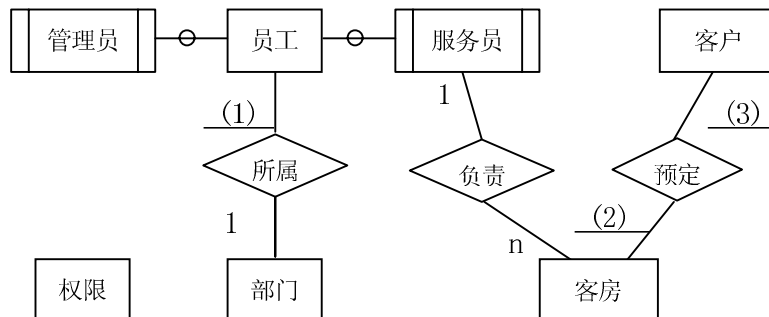


图 2-1 实体联系图

【逻辑结构设计】

逻辑结构设计阶段设计的部分关系模式（不完整）如下：

员工(_____(4)_____, 姓名, 出生年月, 性别, 岗位, 住址, 联系电话, 密码)
权限(岗位, 操作权限)

部门(部门号, 部门名称, 部门负责人, 电话)

客房(_____(5)_____, 类型, 价格, 状态, 入住日期, 入住时间, 员工号)

客户(_____(6)_____, 姓名, 性别, 单位, 联系电话)

更改权限(员工号, _____(7)_____, 密码, 更改日期, 更改时间, 管理员号)

预定情况(_____(8)_____, 预定日期, 预定入住日期, 预定入住天数)

【问题 1】(3 分)

根据问题描述,填写图 2-1 中(1)~(3)处联系的类型。联系类型分为一对一、一对多和多对多三种,分别使用 1:1, 1:n 或 1:*, m:n 或 *: *表示。

【问题 2】(2 分)

补充图 2-1 中的联系并指明其联系类型。

【问题 3】(7 分)

根据需求分析结果和图 2-1,将逻辑结构设计阶段生成的关系模式中的空(4)~(8)补充完整。(注:一个空可能需要填多个属性)

【问题 4】(3 分)

若去掉权限表,并将权限表中的操作权限属性放在员工表中(仍保持管理和服务岗位的操作权限规定),则与原有设计相比有什么优缺点(请从数据库设计的角度进行说明)?

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

在线会议审稿系统（ORS: Online Reviewing System）主要处理会议前期的投稿和审稿事务，其功能描述如下：

1. 用户在初始使用系统时，必须在系统中注册（register）成为作者或审稿人。
2. 作者登录（login）后提交稿件和浏览稿件审阅结果。提交稿件必须在规定提交时间范围内，其过程为先输入标题和摘要、选择稿件所属主题类型、选择稿件所在位置（存储位置）。上述几步若未完成，则重复；若完成，则上传稿件至数据库中，系统发送通知。
3. 审稿人登录后可设置兴趣领域、审阅稿件给出意见以及罗列录用和（或）拒绝的稿件。
4. 会议委员会主席是一个特殊审稿人，可以浏览提交的稿件、给审稿人分配稿件、罗列录用和（或）拒绝的稿件以及关闭审稿过程。其中关闭审稿过程须包括罗列录用和（或）拒绝的稿件。

系统采用面向对象方法开发，使用 UML 进行建模。在建模用例图时，常用的方式是先识别参与者，然后确定参与者如何使用系统来确定用例，每个用例可以构造一个活动图。参与者名称、用例和活动名称分别参见表 3-1、表 3-2 和表 3-3。系统的部分用例图和提交稿件的活动图分别如图 3-1 和图 3-2 所示。

表 3-1 参与者列表

名称	说明	名称	说明
User	用户	Author	作者
Reviewer	审稿人	PCChair	委员会主席

表 3-2 用例名称列表

名称	说明	名称	说明
login	登录系统	register	注册
submit paper	提交稿件	browse review results	浏览稿件审阅结果
close reviewing process	关闭审稿过程	assign paper to reviewer	分配稿件给审稿人
set preferences	设定兴趣领域	enter review	审阅稿件给出意见
list accepted/rejected papers	罗列录用或/和拒绝的稿件	browse submitted papers	浏览提交的稿件

表 3-3 活动名称列表

名称	说明	名称	说明
select paper location	选择稿件位置	upload paper	上传稿件
select subject group	选择主题类型	send notification	发送通知
enter title and abstract	输入标题和摘要		

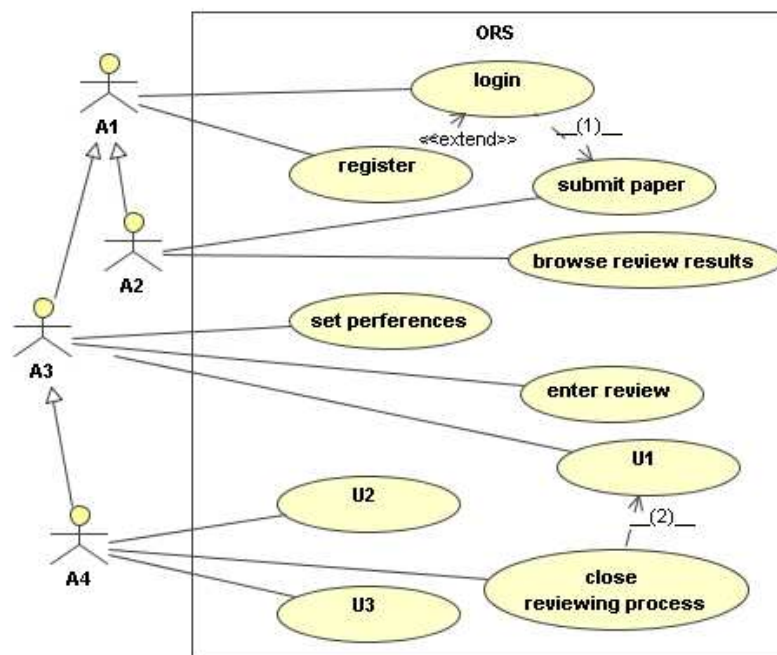


图 3-1 ORS 用例图

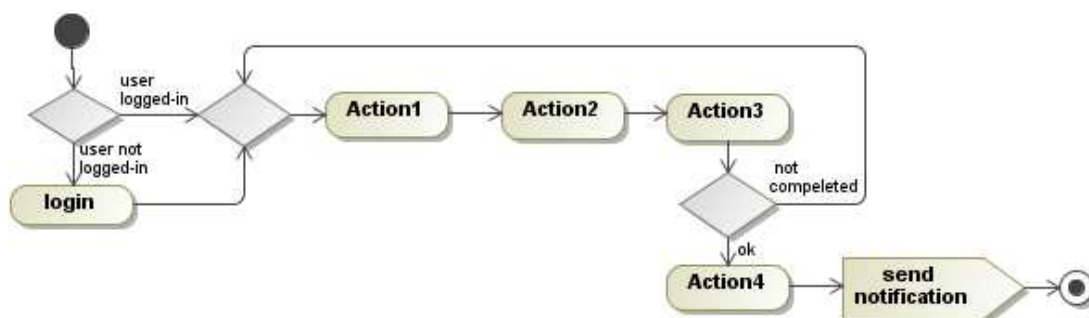


图 3-2 提交稿件过程的活动图

【问题 1】（4 分）

根据【说明】中的描述，使用表 3-1 中的英文名称，给出图 3-1 中 A1~A4 所对应的参与者。

【问题 2】（3 分）

根据【说明】中的描述，使用表 3-2 中的英文名称，给出图 3-1 中 U1~U3 所对应的用例。

【问题 3】（4 分）

根据【说明】中的描述，给出图 3-1 中 (1) 和 (2) 所对应的关系。

【问题 4】（4 分）

根据【说明】中的描述，使用表 3-2 和表 3-3 中的英文名称，给出图 3-2 中 Action1~Action4 对应的活动。

试题四（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某餐厅供应各种标准的营养套餐。假设菜单上共有 n 项食物 $m_1, m_2, \dots, m_n$ ，每项食物 m_i 的营养价值为 v_i ，价格为 p_i ，其中 $i=1,2,\dots,n$ ，套餐中每项食物至多出现一次。客人常需要一个算法来求解总价格不超过 M 的营养价值最大的套餐。

【问题 1】（9 分）

下面是用动态规划策略求解该问题的伪代码，请填充其中的空缺(1)、(2)和(3)处。

伪代码中的主要变量说明如下：

n ：总的食物项数；

v ：营养价值数组，下标从 1 到 n ，对应第 1 到第 n 项食物的营养价值；

p ：价格数组，下标从 1 到 n ，对应第 1 到第 n 项食物的价格；

M ：总价格标准，即套餐的价格不超过 M ；

x ：解向量（数组），下标从 1 到 n ，其元素值为 0 或 1，其中元素值为 0 表示对应的食物不出现在套餐中，元素值为 1 表示对应的食物出现在套餐中；

nv ： $n+1$ 行 $M+1$ 列的二维数组，其中行和列的下标均从 0 开始， $nv[i][j]$ 表示由前 i 项食物组合且价格不超过 j 的套餐的最大营养价值。问题最终要求的套餐的最大营养价值为 $nv[n][M]$ 。

伪代码如下：

MaxNutrientValue(n, v, p, M, x)

```
1  for i = 0 to n
2      nv[i][0] = 0
3  for j = 1 to M
4      nv[0][j] = 0
5  for i = 1 to n
6      for j = 1 to M
7          if j < p[i] //若食物  $m_i$  不能加入到套餐中
8              nv[i][j] = nv[i - 1][j]
9          else if (1)
10             nv[i][j] = nv[i - 1][j]
11          else
12             nv[i][j] = nv[i - 1][j - p[i]] + v[i]
13  j = M
14  for i = n downto 1
15      if (2)
16          x[i] = 0
17      else
18          x[i] = 1
19      (3)
20  return x and nv[n][M]
```

【问题2】(4分)

现有5项食物，每项食物的营养价值和价格如表4-1所示。

表4-1 食物营养价值及价格表

编 码	营养价值	价 格
m ₁	200	50
m ₂	180	30
m ₃	225	45
m ₄	200	25
m ₅	50	5

若要求总价格不超过100的营养价值最大的套餐，则套餐应包含的食物有(4)（用食物项的编码表示），对应的最大营养价值为(5)。

【问题3】(2分)

【问题1】中伪代码的时间复杂度为(6)（用O符号表示）。

从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五（共 15 分）

阅读下列说明和 C 函数，将应填入__ (n) __处的字句写在答题纸的对应栏内。

【说明】

已知集合 A 和 B 的元素分别用不含头结点的单链表存储，函数 Difference()用于求解集合 A 与 B 的差集，并将结果保存在集合 A 的单链表中。例如，若集合 A={5, 10, 20, 15, 25, 30}，集合 B={5, 15, 35, 25}，如图 5-1(a)所示，运算完成后的结果如图 5-1(b)所示。

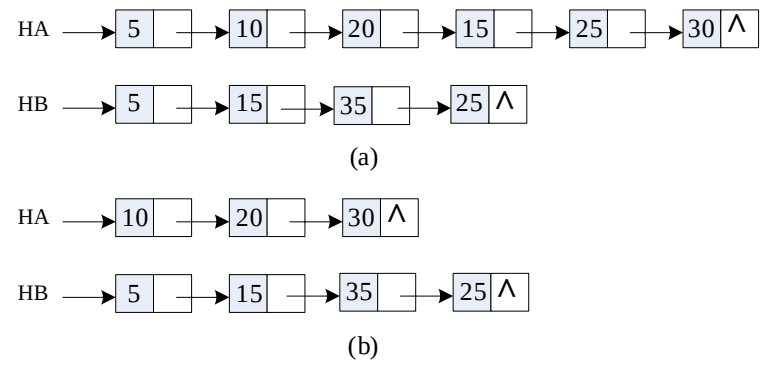


图 5-1 集合 A、B 运算前后示意图

链表结点的结构类型定义如下：

```
typedef struct Node{
    ElemType elem;
    struct Node *next;
}NodeType;
```

【C 函数】

```
void Difference(NodeType **LA, NodeType *LB)
{
    NodeType *pa, *pb, *pre, *q;

    pre = NULL;
    __ (1) __;
    while (pa) {
        pb = LB;
        while (__ (2) __)
            pb = pb->next;

        if (__ (3) __) {
```

```

    if (!pre)
        *LA = (4);
    else
        (5) = pa->next;
    q = pa;
    pa = pa->next;
    free(q);
}
else {
    (6);
    pa = pa->next;
}
}
}

```

试题六（共 15 分）

阅读下列说明和 C++代码，将应填入__(n)__处的字句写在答题纸的对应栏内。

【说明】

已知某类库开发商提供了一套类库，类库中定义了 **Application** 类和 **Document** 类，它们之间的关系如图 6-1 所示，其中，**Application** 类表示应用程序自身，而 **Document** 类则表示应用程序打开的文档。**Application** 类负责打开一个已有的以外部形式存储的文档，如一个文件，一旦从该文件中读出信息后，它就由一个 **Document** 对象表示。

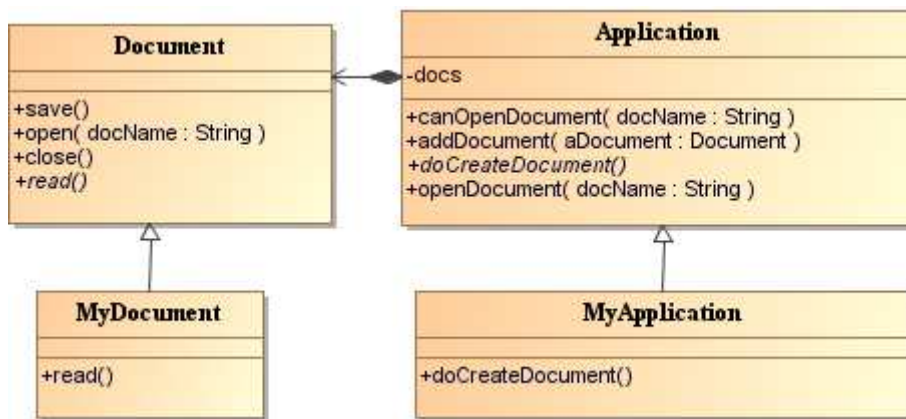


图 6-1 Application 与 Document 关系图

当开发一个具体的应用程序时，开发者需要分别创建自己的 **Application** 和 **Document** 子类，例如图 6-1 中的类 **MyApplication** 和类 **MyDocument**，并分别实现 **Application** 和 **Document** 类中的某些方法。

已知 **Application** 类中的 **openDocument** 方法采用了模板方法（Template Method）设计模式，该方法定义了打开文档的每一个主要步骤，如下所示：

1. 首先检查文档是否能够被打开，若不能打开，则给出出错信息并返回；
2. 创建文档对象；
3. 通过文档对象打开文档；
4. 通过文档对象读取文档信息；
5. 将文档对象加入到 **Application** 的文档对象集合中。

【C++代码】

```
#include <iostream>
#include <vector>
using namespace std;

class Document{
public:
    void save() { /*存储文档数据，此处代码省略*/ }
```

```

    void open(string docName){ /* 打开文档，此处代码省略 */ }
    void close(){ /* 关闭文档，此处代码省略*/ }
    virtual void read(string docName) = 0;
};

class Appplication{
private:
    vector <__ (1) __> docs; /*文档对象集合*/
public:
    bool canOpenDocument(string docName){
        /*判断是否可以打开指定文档，返回真值时表示可以打开，
        返回假值表示不可打开，此处代码省略*/
    }
    void addDocument(Document * aDocument){
        /*将文档对象添加到文档对象集合中*/
        docs.push_back(__ (2) __);
    }
    virtual Document * doCreateDocument() = 0; /*创建一个文档对象*/
    void openDocument(string docName){ /*打开文档*/
        if (__ (3) __){
            cout << "文档无法打开 !" << endl;
            return;
        }
        __ (4) __ adoc = __ (5) __;
        __ (6) __;
        __ (7) __;
        __ (8) __;
    }
};

```


试题七（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

已知某类库开发商提供了一套类库，类库中定义了 `Application` 类和 `Document` 类，它们之间的关系如图 7-1 所示，其中，`Application` 类表示应用程序自身，而 `Document` 类则表示应用程序打开的文档。`Application` 类负责打开一个已有的以外部形式存储的文档，如一个文件，一旦从该文件中读出信息后，它就由一个 `Document` 对象表示。

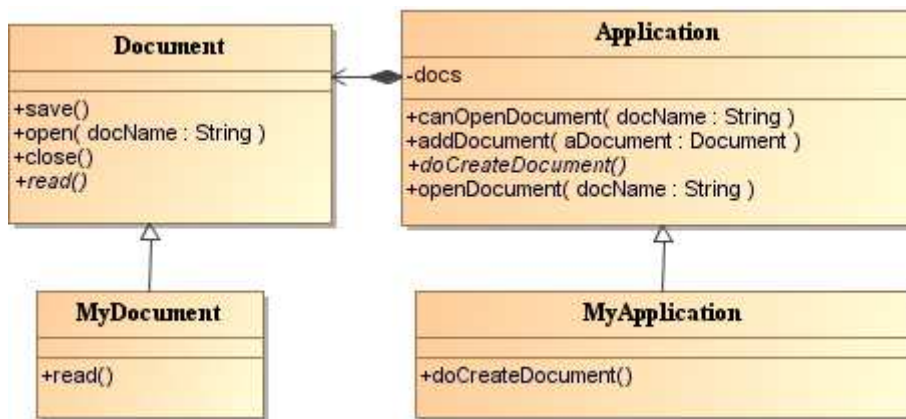


图 7-1 `Application` 与 `Document` 关系图

当开发一个具体的应用程序时，开发者需要分别创建自己的 `Application` 和 `Document` 子类，例如图 7-1 中的类 `MyApplication` 和类 `MyDocument`，并分别实现 `Application` 和 `Document` 类中的某些方法。

已知 `Application` 类中的 `openDocument` 方法采用了模板方法（Template Method）设计模式，该方法定义了打开文档的每一个主要步骤，如下所示：

1. 首先检查文档是否能够被打开，若不能打开，则给出出错信息并返回；
2. 创建文档对象；
3. 通过文档对象打开文档；
4. 通过文档对象读取文档信息；
5. 将文档对象加入到 `Application` 的文档对象集合中。

【Java 代码】

```
abstract class Document{
    public void save(){/*存储文档数据，此处代码省略*/ }
    public void open(String docName){ /* 打开文档，此处代码省略 */ }
    public void close(){/* 关闭文档，此处代码省略*/ }
    public abstract void read(String docName);
};
```

```

abstract class Appplication{
    private    Vector <__ (1) __>    docs; /*文档对象集合*/

    public    boolean canOpenDocument(String docName){
        /*判断是否可以打开指定文档，返回真值时表示可以打开，
        返回假值表示不可打开，此处代码省略*/
    }
    public void addDocument(Document aDocument){
        /*将文档对象添加到文档对象集合中*/
        docs.add(__ (2) __);
    }
    public abstract Document doCreateDocument(); /*创建一个文档对象*/
    public void openDocument(String docName){ /*打开文档*/
        if (__ (3) __){
            System.out.println( "文档无法打开 ！ ");
            return;
        }
        __ (4) __ adoc = __ (5) __;
        __ (6) __;
        __ (7) __;
        __ (8) __;
    }
};

```

全国计算机技术与软件专业技术资格（水平）考试

2009 年上半年 软件设计师 上午试卷

（考试时间 9:00～11:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2009 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是（88）月（89）日。

- | | | | |
|------------|-------|-------|-------|
| （88） A. 4 | B. 5 | C. 6 | D. 7 |
| （89） A. 20 | B. 21 | C. 22 | D. 23 |

因为考试日期是“5 月 23 日”，故（88）选 B，（89）选 D，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 D 填涂（参看答题卡）。

● 海明校验码是在 n 个数据位之外增设 k 个校验位，从而形成一个 $k+n$ 位的新的码字，使新的码字的码距比较均匀地拉大。 n 与 k 的关系是 (1)。

- (1) A. $2^k - 1 \geq n + k$ B. $2^n - 1 \leq n + k$ C. $n = k$ D. $n - 1 \leq k$

● 假设某硬盘由 5 个盘片构成（共有 8 个记录面），盘面有效记录区域的外直径为 30cm，内直径为 10cm，记录位密度为 250 位/mm，磁道密度为 16 道/mm，每磁道分 16 个扇区，每扇区 512 字节，则该硬盘的格式化容量约为 (2) MB。

- (2) A. $\frac{8 \cdot (30-10) \cdot 10 \cdot 250 \cdot 16}{8 \cdot 1024 \cdot 1024}$ B. $\frac{8 \cdot (30-10) \cdot 10 \cdot 16 \cdot 16 \cdot 512}{2 \cdot 1024 \cdot 1024}$
C. $\frac{8 \cdot (30-10) \cdot 10 \cdot 250 \cdot 16 \cdot 16}{8 \cdot 1024 \cdot 1024}$ D. $\frac{8 \cdot (30-10) \cdot 16 \cdot 16 \cdot 512}{2 \cdot 1024 \cdot 1024}$

● (3) 是指按内容访问的存储器。

- (3) A. 虚拟存储器 B. 相联存储器
C. 高速缓存 (Cache) D. 随机访问存储器

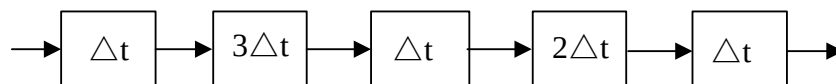
● 处理机主要由处理器、存储器和总线组成，总线包括 (4)。

- (4) A. 数据总线、地址总线、控制总线 B. 并行总线、串行总线、逻辑总线
C. 单工总线、双工总线、外部总线 D. 逻辑总线、物理总线、内部总线

● 计算机中常采用原码、反码、补码和移码表示数据，其中， ± 0 编码相同的是 (5)。

- (5) A. 原码和补码 B. 反码和补码
C. 补码和移码 D. 原码和移码

● 某指令流水线由 5 段组成，第 1、3、5 段所需时间为 Δt ，第 2、4 段所需时间分别为 $3\Delta t$ 、 $2\Delta t$ ，如下图所示，那么连续输入 n 条指令时的吞吐率（单位时间内执行的指令个数）TP 为 (6)。



- (6) A. $\frac{n}{5 \cdot (3+2)\Delta t}$ B. $\frac{n}{(3+3+2)\Delta t + 3(n-1)\Delta t}$
C. $\frac{n}{(3+2)\Delta t + (n-3)\Delta t}$ D. $\frac{n}{(3+2)\Delta t + 5 \cdot 3\Delta t}$

● 下面关于漏洞扫描系统的叙述，错误的是__ (7) __。

- (7) A. 漏洞扫描系统是一种自动检测目标主机安全弱点的程序
B. 黑客利用漏洞扫描系统可以发现目标主机的安全漏洞
C. 漏洞扫描系统可以用于发现网络入侵者
D. 漏洞扫描系统的实现依赖于系统漏洞库的完善

● 网络安全包含了网络信息的可用性、保密性、完整性和网络通信对象的真实性。其中，数字签名是对__ (8) __的保护。

- (8) A. 可用性 B. 保密性 C. 连通性 D. 真实性

● 计算机感染特洛伊木马后的典型现象是__ (9) __。

- (9) A. 程序异常退出 B. 有未知程序试图建立网络连接
C. 邮箱被垃圾邮件填满 D. Windows系统黑屏

● 关于软件著作权产生的时间，下面表述正确的是__ (10) __。

- (10) A. 自作品首次公开发表时
B. 自作者有创作意图时
C. 自作品得到国家著作权行政管理部门认可时
D. 自作品完成创作之日

● 程序员甲与同事乙在乙家探讨甲近期编写的程序，甲表示对该程序极不满意，说要弃之重写，并将程序手稿扔到乙家垃圾筒。后来乙将甲这一程序稍加修改，并署名发表。以下说法正确的是__ (11) __。

- (11) A. 乙的行为侵犯了甲的软件著作权
B. 乙的行为没有侵犯甲的软件著作权，因为甲已将程序手稿丢弃
C. 乙的行为没有侵犯甲的著作权，因为乙已将程序修改
D. 甲没有发表该程序并弃之，而乙将程序修改后发表，故乙应享有著作权

● PC机处理的音频信号主要是人耳能听得到的音频信号，它的频率范围是__ (12) __。

- (12) A. 300Hz~3400Hz B. 20Hz~20KHz
C. 10Hz~20KHz D. 20Hz~44KHz

● 多媒体计算机图像文件格式分为静态图像文件格式和动态图像文件格式，__ (13) __属于静态图像文件格式。

- (13) A. MPG B. AVS C. JPG D. AVI

● 计算机获取模拟视频信息的过程中首先要进行__(14)___。

- (14) A. A/D变换 B. 数据压缩 C. D/A变换 D. 数据存储

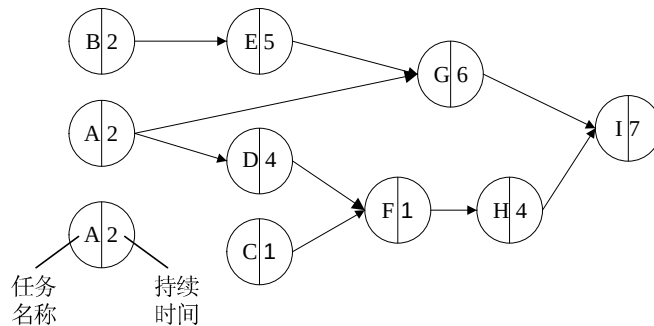
● 在采用面向对象技术构建软件系统时,很多敏捷方法都建议的一种重要的设计活动是__(15)___,它是一种重新组织的技术,可以简化构件的设计而无需改变其功能或行为。

- (15) A. 精化 B. 设计类 C. 重构 D. 抽象

● 一个软件开发过程描述了“谁做”、“做什么”、“怎么做”和“什么时候做”,RUP用__(16)___来表述“谁做”。

- (16) A. 角色 B. 活动 C. 制品 D. workflow

● 某项目主要由A~I任务构成,其计划图(如下图所示)展示了各任务之间的前后关系以及每个任务的工期(单位:天),该项目的关键路径是__(17)___。在不延误项目总工期的情况下,任务A最多可以推迟开始的时间是__(18)___天。



- (17) A. A→G→I B. A→D→F→H→I
C. B→E→G→I D. C→F→H→I

- (18) A. 0 B. 2 C. 5 D. 7

● 软件风险一般包含__(19)___两个特性。

- (19) A. 救火和危机管理 B. 已知风险和未知风险
C. 不确定性和损失 D. 员工和预算

● 函数调用时,基本的参数传递方式有传值与传地址两种,__(20)___。

- (20) A. 在传值方式下,形参将值传给实参
B. 在传值方式下,实参不能是数组元素
C. 在传地址方式下,形参和实参间可以实现数据的双向传递
D. 在传地址方式下,实参可以是任意的变量和表达式

● 已知某高级语言源程序 A 经编译后得到机器 C 上的目标程序 B，则(21)。

- (21) A. 对 B 进行反编译，不能还原出源程序 A
B. 对 B 进行反汇编，不能得到与源程序 A 等价的汇编程序代码
C. 对 B 进行反编译，得到的是源程序 A 的变量声明和算法流程
D. 对 A 和 B 进行交叉编译，可以产生在机器 C 上运行的动态链接库

● 下面关于程序语言的叙述，错误的是(22)。

- (22) A. 脚本语言属于动态语言，其程序结构可以在运行中改变
B. 脚本语言一般通过脚本引擎解释执行，不产生独立保存的目标程序
C. php、JavaScript 属于静态语言，其所有成分可在编译时确定
D. C 语言属于静态语言，其所有成分可在编译时确定

● 在 Windows XP 操作系统中，用户利用“磁盘管理”程序可以对磁盘进行初始化、创建卷，(23)。通常将“C:\Windows\myprogram.exe”文件设置成只读和隐藏属性，以便控制用户对该文件的访问，这一级安全管理称之为(24)安全管理。

- (23) A. 但只能使用 FAT 文件系统格式化卷
B. 但只能使用 FAT 32 文件系统格式化卷
C. 但只能使用 NTFS 文件系统格式化卷
D. 可以选择使用 FAT、FAT32 或 NTFS 文件系统格式化卷
(24) A. 文件级 B. 目录级 C. 用户级 D. 系统级

● 在移臂调度算法中，(25)算法可能会随时改变移动臂的运动方向。

- (25) A. 电梯调度和先来先服务
B. 先来先服务和最短寻找时间优先
C. 单向扫描和先来先服务
D. 电梯调度和最短寻找时间优先

● 设系统中有 R 类资源 m 个，现有 n 个进程互斥使用。若每个进程对 R 资源的最大需求为 w，那么当 m、n、w 取下表的值时，对于下表中的 a~e 五种情况，(26)两种情况可能会发生死锁。对于这两种情况，若将(27)，则不会发生死锁。

	a	b	c	d	e
m	2	2	2	4	4
n	1	2	2	3	3
w	2	1	2	2	3

- (26) A. a 和 b B. b 和 c C. c 和 d D. c 和 e
(27) A. n 加 1 或 w 加 1 B. m 加 1 或 w 减 1
C. m 减 1 或 w 加 1 D. m 减 1 或 w 减 1

● 某文件系统采用链式存储管理方案，磁盘块的大小为 1024 字节。文件 Myfile.doc 由 5 个逻辑记录组成，每个逻辑记录的大小与磁盘块的大小相等，并依次存放在 121、75、86、65 和 114 号磁盘块上。若需要存取文件的第 5120 逻辑字节处的信息，应该访问 (28) 号磁盘块。

- (28) A. 75 B. 85 C. 65 D. 114

● 软件能力成熟度模型 (CMM) 将软件能力成熟度自低到高依次划分为 5 级。目前，达到 CMM 第 3 级 (已定义级) 是许多组织努力的目标，该级的核心是 (29)。

- (29) A. 建立基本的项目管理和实践来跟踪项目费用、进度和功能特性
B. 使用标准开发过程 (或方法论) 构建 (或集成) 系统
C. 管理层寻求更主动地应对系统的开发问题
D. 连续地监督和改进标准化的系统开发过程

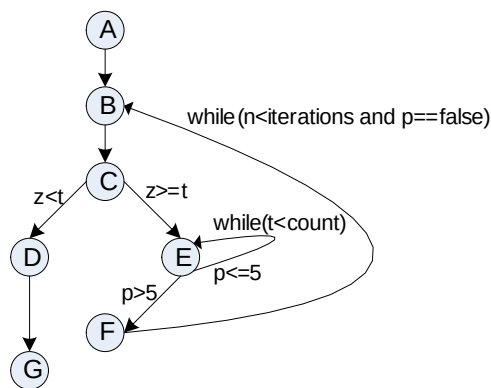
● RUP 在每个阶段都有主要目标，并在结束时产生一些制品。在 (30) 结束时产生 “在适当的平台上集成的软件产品”。

- (30) A. 初期阶段 B. 精化阶段 C. 构建阶段 D. 移交阶段

● 根据 ISO/IEC 9126 软件质量度量模型定义，一个软件的时间和资源质量特性属于 (31) 质量特性。

- (31) A. 功能性 B. 效率 C. 可靠性 D. 易使用性

● McCabe 度量法是通过定义环路复杂度，建立程序复杂性的度量，它基于一个程序模块的程序图中环路的个数。计算有向图 G 的环路复杂性的公式为： $V(G)=m-n+2$ ，其中 $V(G)$ 是有向图 G 中的环路个数， m 是 G 中的有向弧数， n 是 G 中的节点数。下图所示程序图的程序复杂度是 (32)。



- (32) A. 2 B. 3 C. 4 D. 5

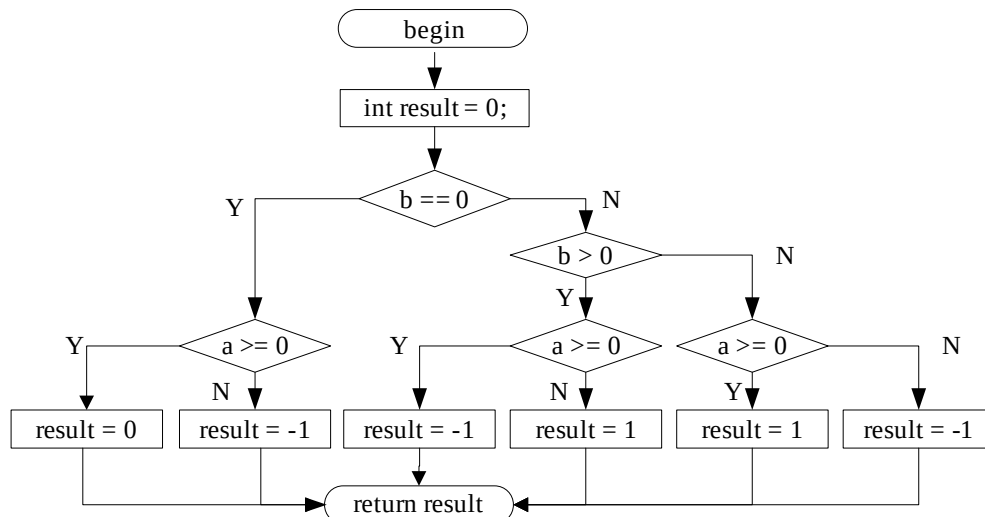
- 在开发信息系统时,用于系统开发人员与项目管理人员沟通的主要文档是_(33)。

(33) A. 系统开发合同 B. 系统设计说明书
C. 系统开发计划 D. 系统测试报告

- 软件工程每一个阶段结束前,应该着重对可维护性进行复审。在系统设计阶段的复审期间,应该从_(34)出发,评价软件的结构和过程。

(34) A. 指出可移植性问题以及可能影响软件维护的系统界面
B. 容易修改、模块化和功能独立的目的
C. 强调编码风格和内部说明文档
D. 可测试性

- 当用分支覆盖法对以下流程图进行测试时,至少需要设计_(35)个测试用例。



(35) A. 4 B. 5 C. 6 D. 8

- 某银行为了使其网上银行系统能够支持信用卡多币种付款功能而进行扩充升级,这需要对数据类型稍微进行一些改变,这一状况需要对网上银行系统进行_(36)维护。

(36) A. 正确性 B. 适应性 C. 完善性 D. 预防性

- 下面关于面向对象分析与面向对象设计的说法中,不正确的是_(37)。

(37) A. 面向对象分析侧重于理解问题
B. 面向对象设计侧重于理解解决方案
C. 面向对象分析描述软件要做什么
D. 面向对象设计一般不关注技术和实现层面的细节

● 在面向对象分析与设计中，__(38)__是应用领域中的核心类，一般用于保存系统中的信息以及提供针对这些信息的相关处理行为；__(39)__是系统内对象和系统外参与者的联系媒介；__(40)__主要是协调上述两种类对象之间的交互。

- (38) A. 控制类 B. 边界类 C. 实体类 D. 软件类
(39) A. 控制类 B. 边界类 C. 实体类 D. 软件类
(40) A. 控制类 B. 边界类 C. 实体类 D. 软件类

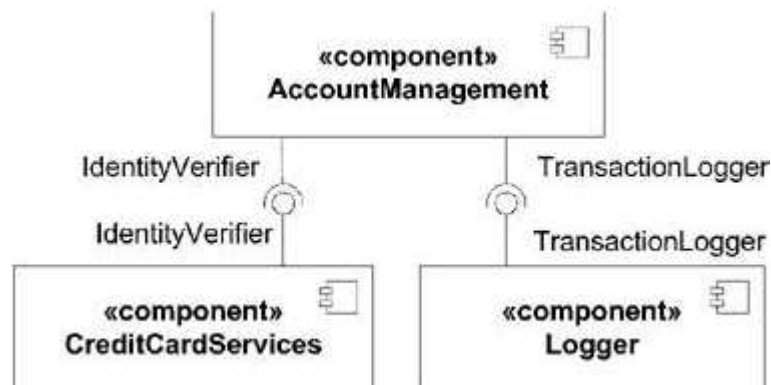
● 若类 A 仅在其方法 Method1 中定义并使用了类 B 的一个对象，类 A 其它部分的代码都不涉及类 B，那么类 A 与类 B 的关系应为__(41)__；若类 A 的某个属性是类 B 的一个对象，并且类 A 对象消失时，类 B 对象也随之消失，则类 A 与类 B 的关系应为__(42)__。

- (41) A. 关联 B. 依赖 C. 聚合 D. 组合
(42) A. 关联 B. 依赖 C. 聚合 D. 组合

● 当不适合采用生成子类的方法对已有的类进行扩充时，可以采用__(43)__设计模式动态地给一个对象添加一些额外的职责；当应用程序由于使用大量的对象，造成很大的存储开销时，可以采用__(44)__设计模式运用共享技术来有效地支持大量细粒度的对象；当想使用一个已经存在的类，但其接口不符合需求时，可以采用__(45)__设计模式将该类的接口转换成我们希望的接口。

- (43) A. 命令 (Command) B. 适配器 (Adapter)
 C. 装饰 (Decorate) D. 享元 (Flyweight)
(44) A. 命令 (Command) B. 适配器 (Adapter)
 C. 装饰 (Decorate) D. 享元 (Flyweight)
(45) A. 命令 (Command) B. 适配器 (Adapter)
 C. 装饰 (Decorate) D. 享元 (Flyweight)

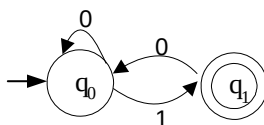
● 下图属于 UML 中的__(46)__，其中，AccountManagement 需要__(47)__。



- (46) A. 组件图 B. 部署图 C. 类图 D. 对象图

- (47) A. 实现 IdentityVerifier 接口并被 CreditCardServices 调用
B. 调用 CreditCardServices 实现的 IdentityVerifier 接口
C. 实现 IdentityVerifier 接口并被 Logger 调用
D. 调用 Logger 实现的 IdentityVerifier 接口

● 下图所示有限自动机的特点是 (48)。



- (48) A. 识别的 0、1 串是以 0 开头且以 1 结尾
B. 识别的 0、1 串中 1 的数目为偶数
C. 识别的 0、1 串中 0 后面必须是 1
D. 识别的 0、1 串中 1 不能连续出现

● 由 a、b 构造且仅包含偶数个 a 的串的集合用正规式表示为 (49)。

- (49) A. $(a^*a)^*b^*$ B. $(b^*(ab^*a)^*)^*$
C. $(a^*(ba^*)^*b)^*$ D. $(a|b)^*(aa)^*$

● 设某语言的语法规则用上下文无关文法 $G=(N,T,P,S)$ 表示，其中 N 是非终结符号的集合， T 是终结符号的集合， P 是产生式集合， S 是开始符号，令 $V=N \cup T$ ，那么符合该语言的句子是 (50)。

- (50) A. 从 S 出发推导的、仅包含 T 中符号的符号串
B. 从 N 中符号出发推导的、仅包含 T 中符号的符号串
C. 从 S 出发推导的、包含 V 中符号的符号串
D. 从 N 中符号出发推导的、包含 V 中符号的符号串

● 采用二维表格结构表达实体类型及实体间联系的数据模型是 (51)。

- (51) A. 层次模型 B. 网状模型
C. 关系模型 D. 面向对象模型

● 假设员工关系 EMP (员工号, 姓名, 部门, 部门电话, 部门负责人, 家庭住址, 家庭成员, 成员关系) 如下表所示。如果一个部门可以有多名员工, 一个员工可以有多个家庭成员, 那么关系 EMP 属于 (52), 且 (53) 问题; 为了解决这一问题, 应该将员工关系 EMP 分解为 (54)。

员工号	姓名	部门	部门电话	部门负责人	家庭住址	家庭成员	成员关系
0011	张晓明	开发部	808356	0012	北京海淀区 1 号	张大军	父亲
0011	张晓明	开发部	808356	0012	北京海淀区 1 号	胡敏铮	母亲
0011	张晓明	开发部	808356	0012	北京海淀区 1 号	张晓丽	妹妹
0012	吴俊	开发部	808356	0012	上海昆明路 15 号	吴胜利	父亲
0012	吴俊	开发部	808356	0012	上海昆明路 15 号	王若垚	母亲
0021	李立丽	市场部	808358	0021	西安雁塔路 8 号	李国庆	父亲
0021	李立丽	市场部	808358	0021	西安雁塔路 8 号	罗明	母亲
0022	王学强	市场部	808356	0021	西安太白路 2 号	王国钧	父亲
0031	吴俊	财务部	808360		西安科技路 18 号	吴鸿翔	父亲

(52) A. 1NF B. 2NF C. 3NF D. BCNF

- (53) A. 无冗余、无插入异常和删除异常
 B. 无冗余，但存在插入异常和删除异常
 C. 存在冗余，但不存在修改操作的不一致
 D. 存在冗余、修改操作的不一致，以及插入异常和删除异常

- (54) A. EMP1 (员工号, 姓名, 家庭住址)
 EMP2 (部门, 部门电话, 部门负责人)
 EMP3 (员工号, 家庭成员, 成员关系)
 B. EMP1 (员工号, 姓名, 部门, 家庭住址)
 EMP2 (部门, 部门电话, 部门负责人)
 EMP3 (员工号, 家庭成员, 成员关系)
 C. EMP1 (员工号, 姓名, 家庭住址)
 EMP2 (部门, 部门电话, 部门负责人, 家庭成员, 成员关系)
 D. EMP1 (员工号, 姓名, 部门, 部门电话, 部门负责人, 家庭住址)
 EMP2 (员工号, 家庭住址, 家庭成员, 成员关系)

● 关系 R、S 如下图所示，关系代数表达式 $\pi_{3,4,5}(\sigma_{1<6}(\mathbf{R} \times \mathbf{S})) = \underline{(55)}$ ，对关系 R、S 进行自然连接后的属性列数和元组个数分别为 (56)。

A	B	C
1	2	4
3	4	5
4	5	9
5	6	6

R

A	B	C
5	3	3
4	6	1
9	8	3
6	9	1

S

(55) A.

A	B	C
1	2	4
5	3	3

B.

A	B	C
5	3	4
9	8	4

C.	A	B	C
	5	3	3
	9	8	3

D.	A	B	C
	1	2	4
	3	4	5

(56) A. 3 和 0 B. 3 和 2 C. 6 和 0 D. 6 和 2

● 下面关于查找运算及查找表的叙述，错误的是__ (57) __。

- (57) A. 哈希表可以动态创建
 B. 二叉排序树属于动态查找表
 C. 二分查找要求查找表采用顺序存储结构或循环链表结构
 D. 顺序查找方法既适用于顺序存储结构，也适用于链表结构

● 下面关于图（网）的叙述，正确的是__ (58) __。

- (58) A. 连通无向网的最小生成树中，顶点数恰好比边数多 1
 B. 若有向图是强连通的，则其边数至少是顶点数的 2 倍
 C. 可以采用 AOV 网估算工程的工期
 D. 关键路径是 AOE 网中源点至汇点的最短路径

● 下面关于二叉排序树的叙述，错误的是__ (59) __。

- (59) A. 对二叉排序树进行中序遍历，必定得到结点关键字的有序序列
 B. 依据关键字无序的序列建立二叉排序树，也可能构造出单支树
 C. 若构造二叉排序树时进行平衡化处理，则根结点的左子树结点数与右子树结点数之差一定不超过 1
 D. 若构造二叉排序树时进行平衡化处理，则根结点的左子树高度与右子树高度的差值一定不超过 1

● 下面关于栈和队列的叙述，错误的是__ (60) __。

- (60) A. 栈和队列都是操作受限的线性表
 B. 队列采用单循环链表存储时，只需设置队尾指针就可使入队和出队操作的时间复杂度都为 $O(1)$
 C. 若队列的数据规模 n 可以确定，则采用顺序存储结构比链式存储结构效率更高
 D. 利用两个栈可以模拟一个队列的操作，反之亦可

● 下面关于二叉树的叙述，正确的是(61)。

- (61) A. 完全二叉树的高度 h 与其结点数 n 之间存在确定的关系
B. 在二叉树的顺序存储和链式存储结构中，完全二叉树更适合采用链式存储结构
C. 完全二叉树中一定不存在度为 1 的结点
D. 完全二叉树中必定有偶数个叶子结点

● 设 L 为广义表，将 $\text{head}(L)$ 定义为取非空广义表的第一个元素， $\text{tail}(L)$ 定义为取非空广义表除第一个元素外剩余元素构成的广义表。若广义表 $L=((x,y,z),a,(u,t,w))$ ，则从 L 中取出原子项 y 的运算是(62)。

- (62) A. $\text{head}(\text{tail}(\text{tail}(L)))$ B. $\text{tail}(\text{head}(\text{head}(L)))$
C. $\text{head}(\text{tail}(\text{head}(L)))$ D. $\text{tail}(\text{tail}(\text{head}(L)))$

● 现有 16 枚外形相同的硬币，其中有一枚比真币的重量轻的假币，若采用分治法找出这枚假币，至少比较(63) 次才能够找出该假币。

- (63) A. 3 B. 4 C. 5 D. 6

● 以下的算法设计方法中，(64) 以获取问题最优解为目标。

- (64) A. 回溯方法 B. 分治法 C. 动态规划 D. 递推

● 归并排序采用的算法设计方法属于(65)。

- (65) A. 归纳法 B. 分治法 C. 贪心法 D. 回溯方法

● 一个 B 类网络的子网掩码为 255.255.224.0，则这个网络被划分成了(66) 个子网。

- (66) A. 2 B. 4 C. 6 D. 8

● 在 Windows 系统中设置默认路由的作用是(67)。

- (67) A. 当主机接收到一个访问请求时首先选择的路由
B. 当没有其它路由可选时最后选择的路由
C. 访问本地主机的路由
D. 必须选择的路由

● HTML<body>元素中，(68) 属性用于定义超链接被鼠标点击后所显示的颜色。

- (68) A. `alink` B. `background` C. `bgcolor` D. `vlink`

● HTML 中<tr>标记用于定义表格的(69)。

- (69) A. 行 B. 列 C. 单元格 D. 标题

● 以下不符合 XML 文档语法规范的是__ (70) __。

- (70) A. 文档的第一行必须是 XML 文档声明
B. 文档必须包含根元素
C. 每个开始标记必须和结束标记配对使用
D. 标记之间可以交叉嵌套

● For nearly ten years, the Unified Modeling Language (UML) has been the industry standard for visualizing, specifying, constructing, and documenting the __ (71) __ of a software-intensive system. As the __ (72) __ standard modeling language, the UML facilitates communication and reduces confusion among project __ (73) __. The recent standardization of UML 2.0 has further extended the language's scope and viability. Its inherent expressiveness allows users to __ (74) __ everything from enterprise information systems and distributed Web-based applications to real-time embedded systems.

The UML is not limited to modeling software. In fact, it is expressive enough to model __ (75) __ systems, such as workflow in the legal system, the structure and behavior of a patient healthcare system, software engineering in aircraft combat systems, and the design of hardware.

To understand the UML, you need to form a conceptual model of the language, and this requires learning three major elements: the UML's basic building blocks, the rules that dictate how those building blocks may be put together, and some common mechanisms that apply throughout the UML.

- | | | | |
|----------------------|-----------------|--------------|-----------------|
| (71) A. classes | B. components | C. sequences | D. artifacts |
| (72) A. real | B. legal | C. de facto | D. illegal |
| (73) A. investors | B. developers | C. designers | D. stakeholders |
| (74) A. model | B. code | C. test | D. modify |
| (75) A. non-hardware | B. non-software | C. hardware | D. software |

全国计算机技术与软件专业技术资格（水平）考试

2009 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2009 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
_ (1) _ 月 _ (2) _ 日。

因为正确的解答是“5 月 23 日”，故在答题纸的对应栏内写上“5”和“23”
（参看下表）。

例题	解答栏
(1)	5
(2)	23

试题一至试题四是必答题

试题一（共 15 分）

阅读下列说明，回答问题 1 和问题 2，将解答填入答题纸的对应栏内。

【说明】

假设某大型商业企业由商品配送中心和连锁超市组成，其中商品配送中心包括采购、财务、配送等部门。为实现高效管理，设计了商品配送中心信息管理系统，其主要功能描述如下：

1. 系统接收由连锁超市提出的供货请求，并将其记录到供货请求记录文件。
2. 在接到供货请求后，从商品库存记录文件中进行商品库存信息查询。如果库存满足供货请求，则给配送处理发送配送通知；否则，向采购部门发出缺货通知。
3. 配送处理接到配送通知后，查询供货请求记录文件，更新商品库存记录文件，并向配送部门发送配送单，在配送货品的同时记录配送信息至商品配送记录文件。
4. 采购部门接到缺货通知后，与供货商洽谈，进行商品采购处理，合格商品入库，并记录采购清单至采购清单记录文件、向配送处理发出配送通知，同时通知财务部门给供货商支付货款。

该系统采用结构化方法进行开发，得到待修改的数据流图（如图 1-1 所示）。

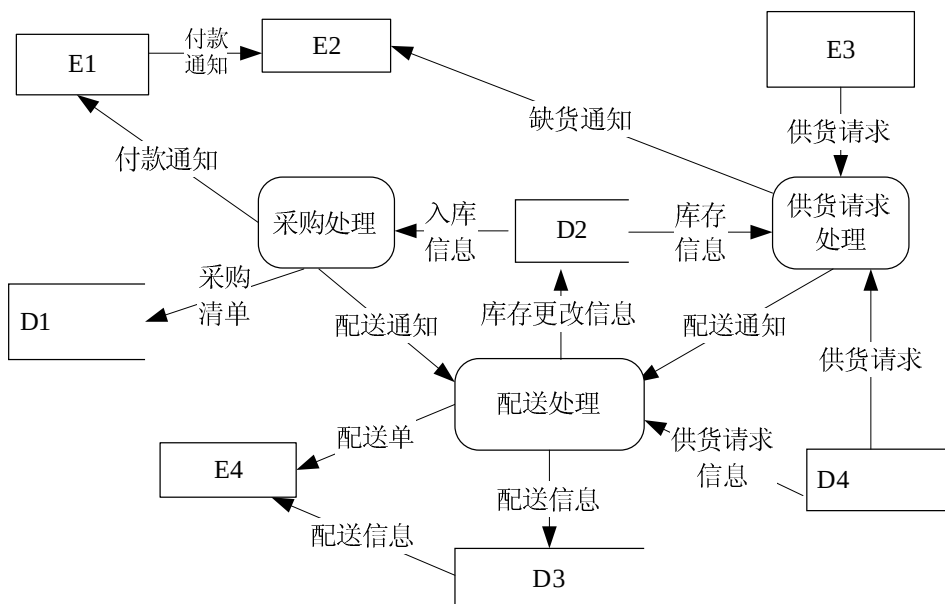


图 1-1 数据流图

【问题 1】（8 分）

使用【说明】中的词语，给出图 1-1 中外部实体 E1 至 E4 的名称和数据存储 D1 至 D4 的名称。

【问题2】（7分）

图 1-1 中存在四处错误数据流，请指出各自的起点和终点；若将上述四条错误数据流删除，为保证数据流图的正确性，应补充三条数据流，请给出所补充数据流的起点和终点。（起点和终点请采用数据流图 1-1 中的符号或名称）

错误数据流

起点	终点

补充的数据流

起点	终点

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某集团公司拥有多个大型连锁商场，公司需要构建一个数据库系统以方便管理其业务运作活动。

【需求分析结果】

1. 商场需要记录的信息包括商场编号（编号唯一），商场名称，地址和联系电话。某商场信息如表 2-1 所示。

表 2-1 商场信息表

商场编号	商场名称	地址	联系电话
PS2101	淮海商场	淮海中路 918 号	021-64158818
PS2902	西大街商场	西大街时代盛典大厦	029-87283220
PS2903	东大街商场	碑林区东大街 239 号	029-87450287
PS2901	长安商场	雁塔区长安中路 38 号	029-85264953

2. 每个商场包含有不同的部门，部门需要记录的信息包括部门编号（集团公司分配），部门名称，位置分布和联系电话。某商场的部门信息如表 2-2 所示。

表 2-2 部门信息表

部门编号	部门名称	位置分布	联系电话
DT002	财务部	商场大楼六层	82504342
DT007	后勤部	商场地下副一层	82504347
DT021	安保部	商场地下副一层	82504358
DT005	人事部	商场大楼六层	82504446
DT001	管理部	商场裙楼三层	82504668

3. 每个部门雇用多名员工处理日常事务，每名员工只能隶属于一个部门（新进员工在培训期不隶属于任何部门）。员工需要记录的信息包括员工编号（集团公司分配），姓名，岗位，电话号码和工资。员工信息如表 2-3 所示。

表 2-3 员工信息表

员工编号	姓名	岗位	电话号码	工资
XA3310	周 超	理货员	13609257638	1500.00
SH1075	刘 飞	防损员	13477293487	1500.00
XA0048	江雪花	广播员	15234567893	1428.00
BJ3123	张正华	部门主管	13345698432	1876.00

4. 每个部门的员工中有一名是经理，每个经理只能管理一个部门，系统需要记录每个经理的任职时间。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图和关系模式（不完整）如下：

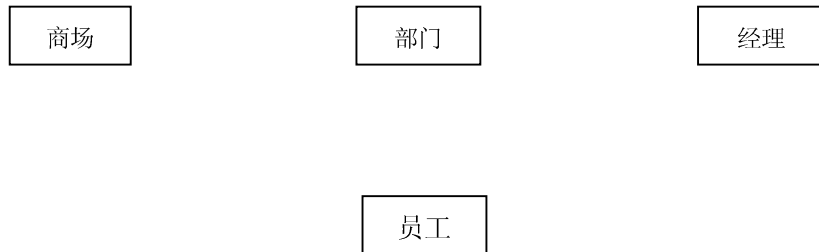


图 2-1 实体联系图

【关系模式设计】

商场（商场编号，商场名称，地址，联系电话）

部门（部门编号，部门名称，位置分布，联系电话，____（a）____）

员工（员工编号，员工姓名，岗位，电话号码，工资，____（b）____）

经理（____（c）____，任职时间）

【问题 1】（6 分）

根据问题描述，补充四个联系，完善图 2-1 的实体联系图。联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系的类型分为 1:1、1:n 和 m:n。

【问题 2】（6 分）

根据实体联系图，将关系模式中的空（a）~（c）补充完整，并分别给出部门、员工和经理关系模式的主键和外键。

【问题 3】（3 分）

为了使商场有紧急事务时能联系到轮休的员工，要求每位员工必须且只能登记一位紧急联系人的姓名和联系电话，不同的员工可以登记相同的紧急联系人。则在图 2-1 中还需添加的实体是____（1）____，该实体和图 2-1 中的员工存在____（2）____联系（填写联系类型）。给出该实体的关系模式。

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某银行计划开发一个自动存提款机模拟系统（ATM System）。系统通过读卡器（CardReader）读取 ATM 卡；系统与客户（Customer）的交互由客户控制台（CustomerConsole）实现；银行操作员（Operator）可控制系统的启动（System Startup）和停止（System Shutdown）；系统通过网络和银行系统（Bank）实现通信。

当读卡器判断用户已将 ATM 卡插入后，创建会话（Session）。会话开始后，读卡器进行读卡，并要求客户输入个人验证码（PIN）。系统将卡号和个人验证码信息送到银行系统进行验证。验证通过后，客户可从菜单选择如下事务（Transaction）：

- 1. 从 ATM 卡账户取款（Withdraw）；
- 2. 向 ATM 卡账户存款（Deposit）；
- 3. 进行转账（Transfer）；
- 4. 查询（Inquire）ATM 卡账户信息。

一次会话可以包含多个事务，每个事务处理也会将卡号和个人验证码信息送到银行系统进行验证。若个人验证码错误，则转个人验证码错误处理（Invalid PIN Process）。每个事务完成后，客户可选择继续上述事务或退卡。选择退卡时，系统弹出 ATM 卡，会话结束。

系统采用面向对象方法开发，使用 UML 进行建模。系统的顶层用例图如图 3-1 所示，一次会话的序列图（不考虑验证）如图 3-2 所示。消息名称参见表 3-1。

表 3-1 可能的消息名称列表

名称	说明	名称	说明
cardInserted()	ATM 卡已插入	performTransaction()	执行事务
performSession()	执行会话	readCard()	读卡
readPIN()	读取个人验证码	PIN	个人验证码信息
creat(atm, this, card, pin)	为当前会话创建事务	create(this)	为当前 ATM 创建会话
card	ATM 卡信息	doAgain	执行下一个事务
ejectCard()	弹出 ATM 卡		

【问题 1】（7 分）

根据【说明】中的描述，给出图 3-1 中 A1 和 A2 所对应的参与者，U1 至 U3 所对应的用例，以及该图中空（1）所对应的关系。（U1 至 U3 的可选用例包括：Session、Transaction、Insert Card、Invalid PIN Process 和 Transfer）

【问题 2】（6 分）

根据【说明】中的描述，使用表 3-1 中的英文名称，给出图 3-2 中 6~9 对应的消息。

【问题 3】（2 分）

解释图 3-1 中用例 U3 和用例 Withdraw、Deposit 等四个用例之间的关系及其内涵。

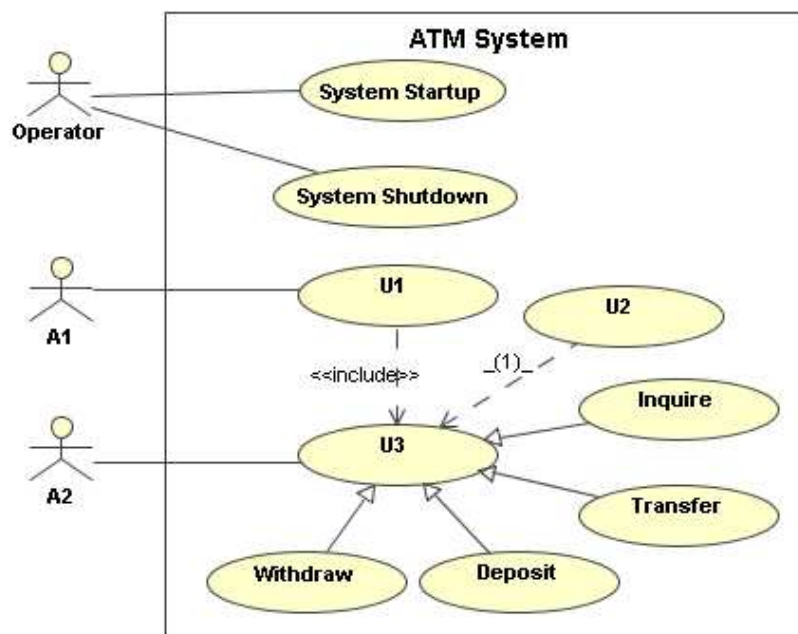


图 3-1 ATM 系统顶层用例图

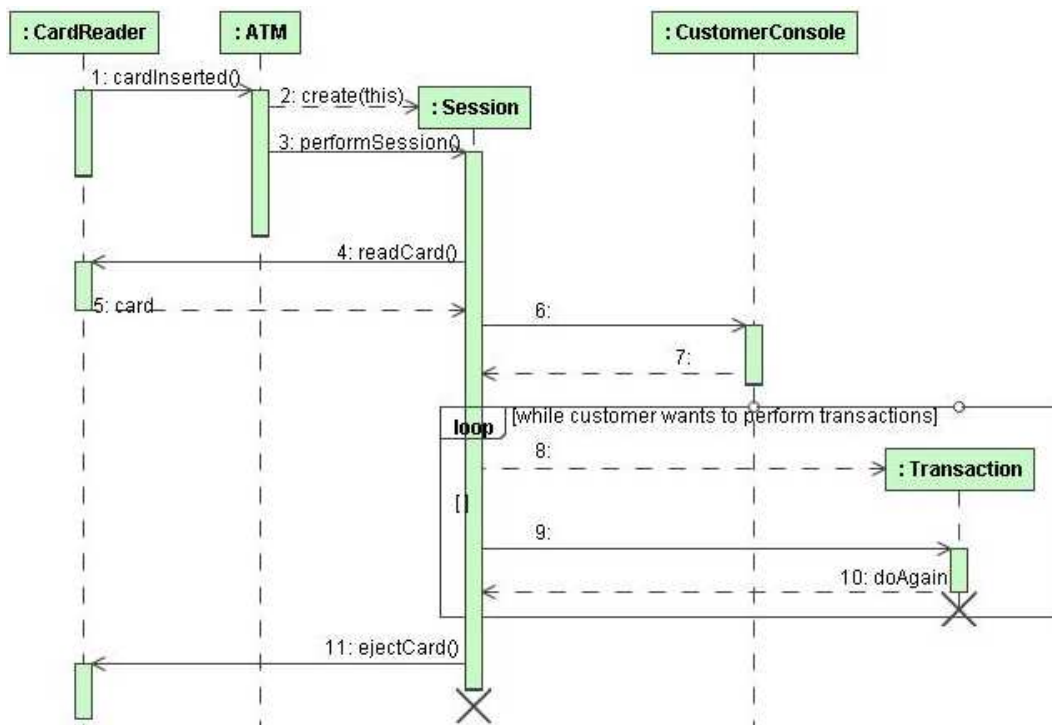


图 3-2 一次会话的序列图（无验证消息）

试题四（共 15 分）

阅读下列说明，回答问题 1 和问题 2，将解答填入答题纸的对应栏内。

【说明】

现需在某城市中选择一个社区建一个大型超市，使该城市的其它社区到该超市的距离总和最小。用图模型表示该城市的地图，其中顶点表示社区，边表示社区间的路线，边上的权重表示该路线的长度。

现设计一个算法来找到该大型超市的最佳位置：即在给定图中选择一个顶点，使该顶点到其它各顶点的最短路径之和最小。算法首先要求出每个顶点到其它任一顶点的最短路径，即需要计算任意两个顶点之间的最短路径；然后对每个顶点，计算其它各顶点到该顶点的最短路径之和；最后，选择最短路径之和最小的顶点作为建大型超市的最佳位置。

【问题 1】（12 分）

本题采用 **Floyd-Warshall** 算法求解任意两个顶点之间的最短路径。已知图 G 的顶点集合为 $V = \{1, 2, \dots, n\}$ ， $W = \{w_{ij}\}_{n \times n}$ 为权重矩阵。设 $d_{ij}^{(k)}$ 为从顶点 i 到顶点 j 的一条最短路径的权重。当 $k = 0$ 时，不存在中间顶点，因此 $d_{ij}^{(0)} = w_{ij}$ ；当 $k > 0$ 时，该最短路径上所有的中间顶点均属于集合 $\{1, 2, \dots, k\}$ 。若中间顶点包括顶点 k ，则 $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ ；若中间顶点不包括顶点 k ，则 $d_{ij}^{(k)} = d_{ij}^{(k-1)}$ 。于是得到如下递归式。

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & k = 0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & k > 0 \end{cases}$$

因为对于任意路径，所有的中间顶点都在集合 $\{1, 2, \dots, n\}$ 内，因此矩阵 $D^{(n)} = \{d_{ij}^{(n)}\}_{n \times n}$ 给出了任意两个顶点之间的最短路径，即对所有 $i, j \in V$ ， $d_{ij}^{(n)}$ 表示点 i 到点 j 的最短路径。

下面是求解该问题的伪代码，请填充其中空缺的 (1) 至 (6) 处。伪代码中的主要变量说明如下：

W: 权重矩阵

n: 图的顶点个数

SP: 最短路径权重之和数组，**SP[i]** 表示顶点 i 到其它各顶点的最短路径权重之和， i 从 1 到 n

min_SP: 最小的最短路径权重之和

min_v: 具有最小的最短路径权重之和的顶点

i: 循环控制变量

j: 循环控制变量
k: 循环控制变量

LOCATE-SHOPPINGMALL(W, n)

```
1  D(0) = W
2  for _____ (1) _____
3      for i = 1 to n
4          for j = 1 to n
5              if  $d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ 
6                  _____ (2) _____
7              else
8                  _____ (3) _____
9  for i = 1 to n
10     SP[i] = 0
11     for j = 1 to n
12         _____ (4) _____
13 min_SP = SP[1]
14 _____ (5) _____
15 for i = 2 to n
16     if min_SP > SP[i]
17         min_SP = SP[i]
18         min_v = i
19 return _____ (6) _____
```

【问题 2】(3 分)

【问题 1】中伪代码的时间复杂度为_____ (7) _____ (用 O 符号表示)。

从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五（共 15 分）

阅读下列说明和 C 函数代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

对二叉树进行遍历是二叉树的一个基本运算。遍历是指按某种策略访问二叉树的每个结点，且每个结点仅访问一次的过程。函数 `InOrder()` 借助栈实现二叉树的非递归中序遍历运算。

设二叉树采用二叉链表存储，结点类型定义如下：

```
typedef struct BtNode{
    ElemType data;           /*结点的数据域，ElemType 的具体定义省略*/
    struct BtNode *lchild,*rchild; /*结点的左、右孩子指针域*/
}BtNode, *BTree;
```

在函数 `InOrder()` 中，用栈暂存二叉树中各个结点的指针，并将栈表示为不含头结点的单向链表（简称链栈），其结点类型定义如下：

```
typedef struct StNode{           /*链栈的结点类型*/
    BTree elem;                 /*栈中的元素是指向二叉链表结点的指针*/
    struct StNode *link;
}StNode;
```

假设从栈顶到栈底的元素为 e_n 、 e_{n-1} 、...、 e_1 ，则不含头结点的链栈示意图如图 5-1 所示。

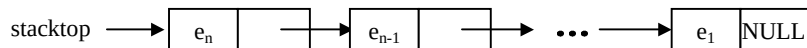


图 5-1 链栈示意图

【C 函数】

```
int InOrder(BTree root)          /* 实现二叉树的非递归中序遍历 */
{
    BTree ptr;                   /* ptr 用于指向二叉树中的结点 */
    StNode *q;                   /* q 暂存链栈中新创建或待删除的结点指针*/
    StNode *stacktop = NULL;     /* 初始化空栈的栈顶指针 stacktop */
    ptr = root;                  /* ptr 指向二叉树的根结点 */
    while ( ____ (1) ____ || stacktop != NULL) {
        while (ptr != NULL) {
            q = (StNode *)malloc(sizeof(StNode));
            if (q == NULL)
                return -1;
            q->elem = ptr;
            ____ (2) ____;
            stacktop = q;         /*stacktop 指向新的栈顶*/
            ptr = ____ (3) ____;  /*进入左子树*/
        }
        q = stacktop;
        ____ (4) ____;           /*栈顶元素出栈*/
        visit(q);                /*visit 是访问结点的函数，其具体定义省略*/
        ptr = ____ (5) ____;     /*进入右子树*/
        free(q);                 /*释放原栈顶元素的结点空间*/
    }
    return 0;
} /*InOrder*/
```

试题六（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

现欲实现一个图像浏览系统，要求该系统能够显示 BMP、JPEG 和 GIF 三种格式的文件，并且能够在 Windows 和 Linux 两种操作系统上运行。系统首先将 BMP、JPEG 和 GIF 三种格式的文件解析为像素矩阵，然后将像素矩阵显示在屏幕上。系统需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，采用桥接（Bridge）设计模式进行设计所得类图如图 6-1 所示。

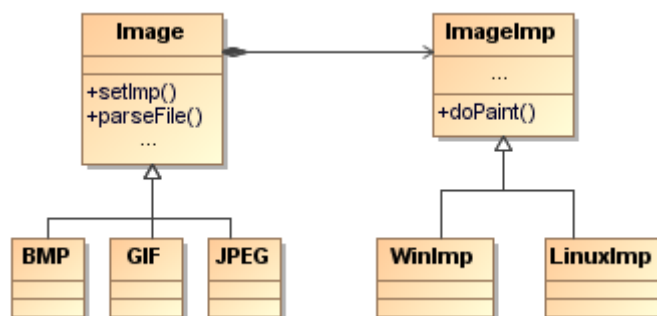


图 6-1 类图

采用该设计模式的原因在于：系统解析 BMP、GIF 与 JPEG 文件的代码仅与文件格式相关，而在屏幕上显示像素矩阵的代码则仅与操作系统相关。

【C++ 代码】

```
class Matrix{ //各种格式的文件最终都被转化为像素矩阵
    //此处代码省略
};

class ImageImp{
public:
    virtual void doPaint(Matrix m) = 0; //显示像素矩阵 m
};

class WinImp : public ImageImp{
public:
    void doPaint(Matrix m){ /*调用 windows 系统的绘制函数绘制像素矩阵*/ }
};

class LinuxImp : public ImageImp{
public:
    void doPaint(Matrix m){ /*调用 Linux 系统的绘制函数绘制像素矩阵*/ }
};
```

```

class Image {
public:
    void setImp(ImageImp *imp){__(1)__ = imp;}
    virtual void parseFile(string fileName) = 0;
protected:
    __(2)__ *imp;
};

class BMP : public Image{
public:
    void parseFile(string fileName){
        //此处解析 BMP 文件并获得一个像素矩阵对象 m
        __(3)__; // 显示像素矩阵 m
    }
};

class GIF : public Image{
    //此处代码省略
};

class JPEG : public Image{
    //此处代码省略
};

void main(){
    //在 windows 操作系统上查看 demo.bmp 图像文件
    Image *image1 = __(4__);
    ImageImp *imageImp1 = __(5__);
    __(6__);
    image1->parseFile("demo.bmp");
}

```

现假设该系统需要支持 10 种格式的图像文件和 5 种操作系统，不考虑类 **Matrix**，若采用桥接设计模式则至少需要设计__(7)__个类。

试题七（共 15 分）

阅读下列说明和 Java 代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

现欲实现一个图像浏览系统，要求该系统能够显示 BMP、JPEG 和 GIF 三种格式的文件，并且能够在 Windows 和 Linux 两种操作系统上运行。系统首先将 BMP、JPEG 和 GIF 三种格式的文件解析为像素矩阵，然后将像素矩阵显示在屏幕上。系统需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，采用桥接（Bridge）设计模式进行设计所得类图如图 7-1 所示。

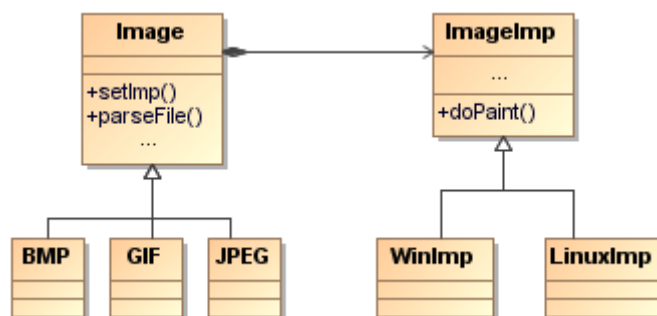


图 7-1 类图

采用该设计模式的原因在于：系统解析 BMP、GIF 与 JPEG 文件的代码仅与文件格式相关，而在屏幕上显示像素矩阵的代码则仅与操作系统相关。

【Java 代码】

```
class Matrix{ //各种格式的文件最终都被转化为像素矩阵
    //此处代码省略
};

abstract class ImageImp{
    public abstract void doPaint(Matrix m); //显示像素矩阵m
};

class WinImp extends ImageImp{
    public void doPaint(Matrix m){ /*调用windows系统的绘制函数绘制像素矩阵*/ }
};

class LinuxImp extends ImageImp{
    public void doPaint(Matrix m){ /*调用Linux系统的绘制函数绘制像素矩阵*/ }
};

abstract class Image {
    public void setImp(ImageImp imp){
```

```

        ____ (1) ____ = imp; }
    public abstract void parseFile(String fileName);
    protected ____ (2) ____ imp;
};

class BMP extends Image{
    public void parseFile(String fileName){
        //此处解析BMP文件并获得一个像素矩阵对象m
        ____ (3) ____; // 显示像素矩阵m
    }
};

class GIF extends Image{
    //此处代码省略
};

class JPEG extends Image{
    //此处代码省略
};

public class javaMain{
    public static void main(String[] args){
        //在windows操作系统上查看demo.bmp图像文件
        Image image1 = ____ (4) ____;
        ImageImp imageImp1 = ____ (5) ____;
        ____ (6) ____;
        image1.parseFile("demo.bmp");
    }
}

```

现假设该系统需要支持 10 种格式的图像文件和 5 种操作系统，不考虑类 **Matrix** 和类 **javaMain**，若采用桥接设计模式则至少需要设计 ____ (7) ____ 个类。

2009 年下半年软件设计师上午试题

●以下关于 CPU 的叙述中，错误的是（1）

- A. CPU 产生每条指令的操作信号并将操作信号送往相应的部件进行控制
- B. 程序计数器 PC 除了存放指令地址，也可以临时存储算术/逻辑运算结果
- C. CPU 中的控制器决定计算机运行过程的自动化
- D. 指令译码器是 CPU 控制器中的部件

●以下关于 CISC（Complex Instruction Set Computer, 复杂指令集计算机）和 RISC（Reduced Instruction Set Computer, 精简指令集计算机）的叙述中，错误的是（2）

- A. 在 CISC 中，其复杂指令都采用硬布线逻辑来执行
- B. 采用 CISC 技术的 CPU，其芯片设计复杂度更高
- C. 在 RISC 中，更适合采用硬布线逻辑执行指令
- C. 采用 RISC 技术，指令系统中的指令种类和寻址方式更少

●浮点数的一般表示形式为 $N=2^E \times F$ ，其中 E 为阶码，F 为尾数。以下关于浮点表示的叙述中，错误的是（3），两个浮点数进行相加运算，应首先（4）。

（3）

- A. 阶码的长度决定浮点表示的范围，尾数的长度决定浮点表示的精度
- B. 工业标准 IEEE754 浮点数格式中阶码采用移码、尾数采用原码表示
- C. 规格化指的是阶码采用移码、尾数采用补码
- C. 规格化表示要求交尾数的绝对值限定在区间[0.5, 1)

（4）

- A. 将较大的数进行规格化处理
- B. 将较小的数进行规格化处理
- C. 将这两个数的尾数相加
- D. 统一这两个数的阶码

●以下关于校验码的叙述中，正确的是(5)

- A. 海明码利用多组数位的奇偶性来检错和纠错
- B. 海明码的码距必须大于等于 1
- C. 循环冗余校验码具有很强的检错和纠错能力
- D. 循环冗余校验码的码距必定为 1

●以下关于 Cache 的叙述中，正确的是(6)

- A. 在容量确定的情况下，替换算法的时间复杂度是影响 Cache 命中率的关键因素
- B. Cache 的设计思想是在合理成本下提高命中率
- C. Cache 的设计目标是容量尽可能与主存容量相等
- D. CUP 中的 Cache 容量应大于 CUP 之外的 Cache 容量

●网络安全体系设计可从物理线路安全、网络安全、系统安全、应用安全等方面来进行。其中，数据库容灾属于 (7)

- A. 物理线路安全和网络安全
- B. 物理线路安全和应用安全
- C. 系统安全和网络安全
- D. 系统安全和应用安全

●包过滤防火墙对数据包的依据不包括 (8)

- A. 源 IP 地址
- B. 源端口号
- C. MAC 地址
- D. 目的 IP 地址

●某网站向 CA 申请了数字证书，用户通过 (9) 来验证网站的真伪

- A. CA 的签名
- B. 证书中的公钥
- C. 网站的私钥
- D. 用户的公钥

●下列智力成果中，能取得专利权的是 (10)

- A. 计算机程序代码
- B. 游戏的规则和方法
- C. 计算机算法

D. 用于控制测试过程的程序

●软件人与被许可方签订一份软件使用许可合同，若在该合同约定的时间和地域范围内，软件权利人不得再许可任何第三人以些相同的方法使用该项软件，但软件权利人可以自己使用，则该项许可使用是（11）

- A. 独家许可使用
- B. 独占许可使用
- C. 普通许可使用
- D. 部分许可使用

●多媒体中的“媒体”有两重含义，一是指存储信息的实体；二是指表达与传递信息的载体。（12）是存储信息的实体。

- A. 文字、图形、磁带、半导体存储器
- B. 磁盘、光盘、磁带、半导体存储器
- C. 文字、图形、图像、声音
- D. 声卡、磁带、半导体存储器

●RGB8：8：8 表示一帧彩色图像的颜色数为（13）种。

- A. 2^3
- B. 2^8
- C. 2^{24}
- D. 2^{512}

●位图与矢量图相比，位图（14）

- A. 占用空间较大，处理侧重于获取和复制，显示速度快
- B. 占用空间较小，处理侧重于绘制和创建，显示速度较慢
- C. 占用空间较大，处理侧重于获取和复制，显示速度较慢
- D. 占用空间较小，处理侧重于绘制和创建，显示速度快

●在采用结构化方法进行了系统分析时，根据分解与抽象的原则，按照系统中数据处理的流程，用（15）来建立系统的逻辑模型，从而完成分析工作。

A. ER 图 B. 数据流图 C. 程序流程图 D. 软件体系结构

●面积对象开发方法的基本思想是尽可能按照人类认识客观世界的方法来分析和解决问题,(16)方法不属于面积对象方法。

A. Booch B. Coad C. OMT D. Jackson

●确定构建软件系统所需要的人数时,无需考虑(17)

A. 系统的市场前景 B. 系统的规模 C. 系统的技术复杂性 D. 项目计划

●一个项目为了修正一个错误而进行了变更,但这个错误被修正后,却引起以前可以正确运行的代码出错.(18)最可能发现这一问题。

A. 单元测试 B. 接受测试 C. 回归测试 D. 安装测试

●风险预测从两个方面评估风险,即风险发生的可能性以及(19)

A. 风险产生的原因 B. 风险监控技术 C. 风险能否消除 D. 风险发生所产生的后果

●许多程序设计语言规定,程序中的数据都必须具有类型,其作用不包括(20)

A. 便于为数据合理分配存储单元
B. 便于对参与表达式计算的数据对象进行检查
C. 便于定义动态数据结构
D. 便于规定数据对象的取值范围及能够进行的运算

●以下关于 C/C++语言指针变量的叙述中,正确的是(21)

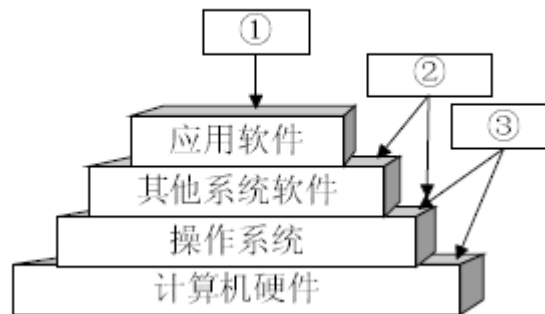
A. 指针变量可以是全局变量也可以是局部变量
B. 必须为指针变量与指针所指向的变量分配相同大小的存储空间
C. 对指针变量进行算术运算是没有意义的
D. 指针变量必须由动态产生的数据对象来赋值

●将高级语言源程序翻译为机器语言程序的过程中常引入中间代码。以下关于中间代码的叙述中,错误的是(22)

- A. 不同的高级程序语言可以产生同一种中间代码
- B. 使用中间代码有利于进行与机器无关的优化处理
- C. 使用中间代码有利于提高编译程序的可移植性
- D. 中间代码与机器语言代码在指令结构上必须一致

●操作系统是裸机上的第一层软件，其他系统软件（如（23）等）和应用软件都是建立在操作系统基础上的。

下图①②③分别表示（24）



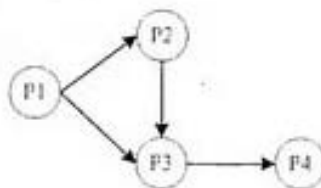
（23）

- A. 编译程序、财务软件和数据库管理系统软件
- B. 汇编程序、编译程序和 Java 解释器
- C. 编译程序、数据库管理系统软件和汽车防盗程序
- D. 语言处理程序、办公管理软件和气象预报软件

（24）

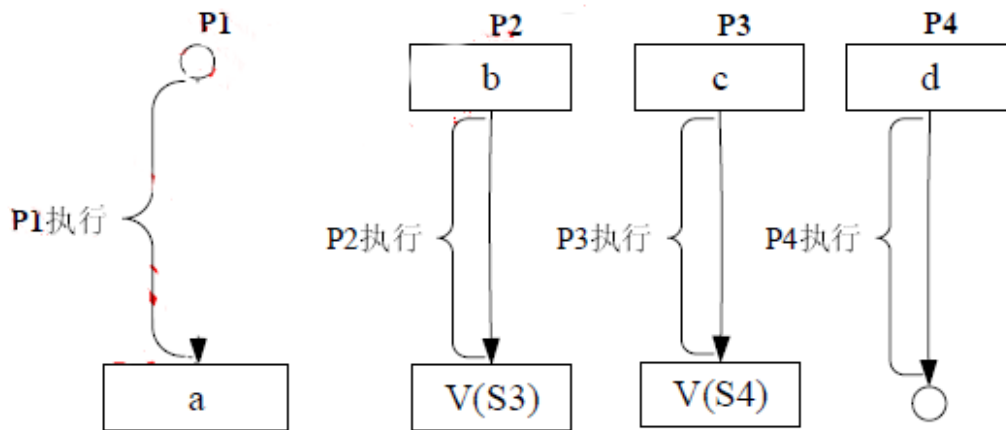
- A. 应用软件开发、最终用户和系统软件开发
- B. 应用软件开发、系统软件开发和最终用户
- C. 最终用户、系统软件开发和应用软件开发
- D. 最终用户、应用软件开发和系统软件开发

●进程 P1、P2、P3 和 P4 的前趋图如下：



若用 PV 操作控制这几个进程并发执行的过程，则需要设置 4 个信号量 S1、S2、S3 和 S4。且信号量初值都

等于零，图中 a 和 b 应分别填写 (25)，c 和 d 应分别填写 (26)。



(25)

- A. P(S1) P(S2)和 P(S3) B. P(S1) P(S2)和 V(S1) C. V(S1) V(S2)和 P(S1) D. V(S1) V(S2)和 V(S3)

(26)

- A. P(S1) P(S2)和 P(S4) B. P(S2) P(S3)和 P(S4) C. V(S1) V(S2)和 V(S4) D. V(S2) V(S3)和 V(S4)

●若系统正在将 (27) 文件修改的结果写回磁盘时系统发生崩溃，则对系统的影响相对较大。

- A. 空闲块 B. 目录 C. 用户数据 D. 用户程序

●UNIX 系统采用直接、一级、二级和三级间接索引技术访问文件，其索引结点有 13 个地址项 ($i_addr[0] \sim i_addr[12]$)。如果每个盘块的大小为 1KB，每个盘块号占 4B，则进程 A 访问文件 F 中第 11264 字节处的数据时，(28)

- A. 可直接寻址 B. 需要一次单位寻址 C. 需要二次间接寻址 D. 需要三次间接寻址

●软件能力成熟度模型 (CMM) 的第 4 级 (已管理级) 的核心是 (29)

- A. 建立基本的项目管理和实践来跟踪项目费用、进度和功能特性
B. 组织具有标准软件过程
C. 对软件过程和产品都有定量的理解和控制
D. 先进的新思想和新技术促进过程不断改进

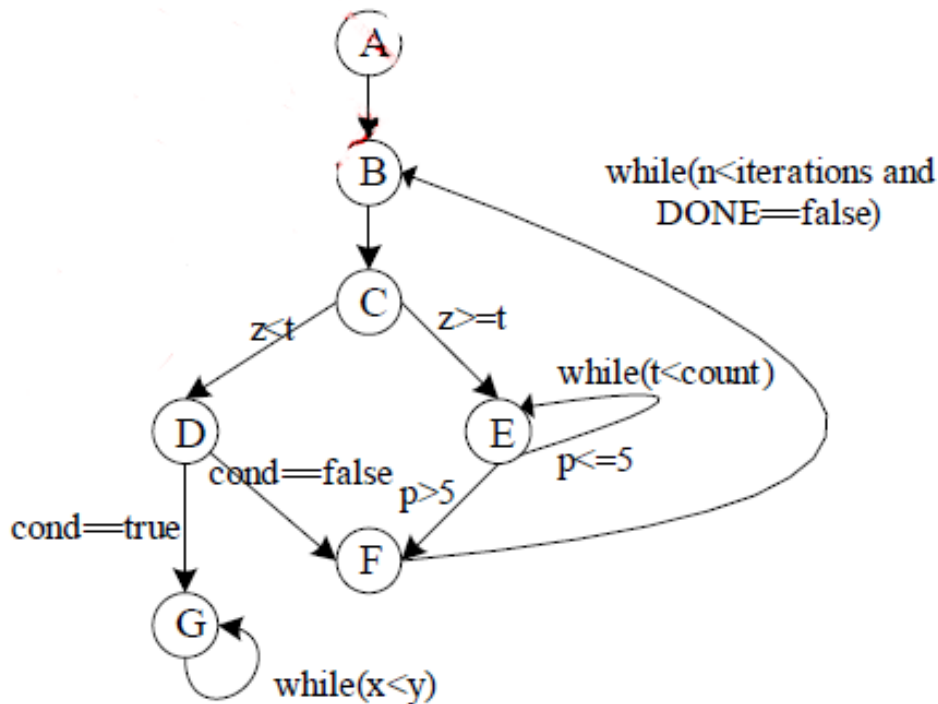
●软件系统设计的主要目的是为系统制定蓝图，(30) 并不是软件设计模型所关注的

- A. 系统总体结构 B. 数据结构 C. 界面模型 D. 项目范围

●ISO/IEC 9126 软件质量模型中,可靠性质特性包括多个子特性,一软件在故障发生后,要求在 90 秒内恢复其性能和受影响的数据,与达到此目的有半的软件属性为 (31)

- A. 容错性 B. 成熟性 C. 易恢复性 D. 易操作性

●某程序的程序图如下所示,运用 McCabe 度量法对其进行度量,其环路复杂度是 (32)



- A. 2 B. 3 C. 4 D. 5

●系统开发计划用于系统开发人员与项目管理人员在项目期内进行沟通,它包括 (33) 和预算分配表等。

- A. PERT 图 B. 总体规则 C. 测试规划 D. 开发合同

●改正在软件系统开发阶段已经发生而系统测试阶段清寒没有发现的错误,属于 (34) 维护。

- A. 正确性 B. 适应性 C. 完美性 D. 预防性

●某系统重用了第三方组件(但无法获得其源代码),则应采用 (35) 对组件进行测试。

- A. 基本路径 B. 分支覆盖 C. 环路覆盖 D. 黑盒测试

●极限编程(XP)由价值观、原则、实践和行为四个部分组成,其中价值观包括沟通、简单性、(36)。

- A. 好的计划 B. 不断的发布 C. 发愤和勇气 D. 持续集成

● 以下关于类和对象的叙述中，错误的是（37）

- A. 类是具有相同属性和服务的一组对象的集合
B. 类是一个对象模板，用它仅可以产生一个对象
C. 在客观世界中实际存在的是类的实例，好对象
D. 类为属于该类的全部对象提供了统一的抽象描述

● （38）是把对象的属性和服务结合成一个独立的系统单元，并尽可能隐藏对象的内部细节；（39）是指子类可以自动拥有父类的全部属性和服务；（40）是对象发出的服务请求，一般包含提供服务的对象标识、服务标识、输入信息和应答信息等。

- | | | | |
|-----------|-------|-------|-------|
| （38）A. 继承 | B. 多态 | C. 消息 | D. 封装 |
| （39）A. 继承 | B. 多态 | C. 消息 | D. 封装 |
| （40）A. 继承 | B. 多态 | C. 消息 | D. 封装 |

● 以下关于面向对象分析的叙述中，错误的是（41）

- A. 面向对象分析重分析问题域和系统责任
B. 面向对象分析需要考虑系统的测试问题
C. 面向对象分析忽略与系统实现有关的问题
D. 面积对象分析建立独立于实现的系统分析模型

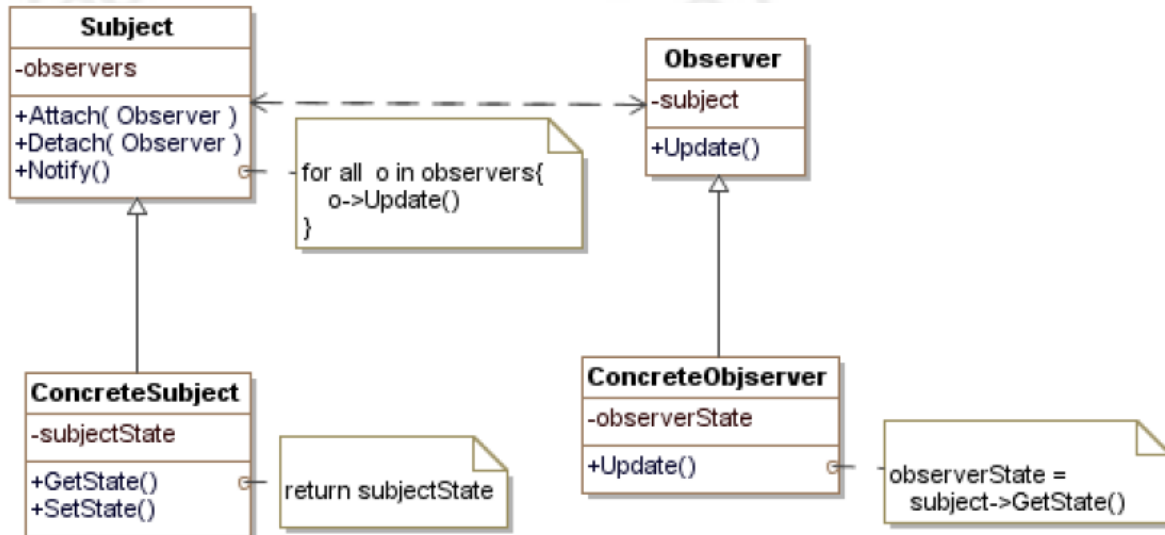
● 以下关于面向对象设计的叙述中，错误的是（42）

- A. 高层模块不应该依赖于底层模块
B. 抽象不应该依赖于细节
C. 细节可以依赖于抽象
D. 高层模块无法不依赖于底层模块

● 采用（43）设计模式可保证一个类仅有一个实例；采用（44）设计模式可将组合成树形结构以表示“部分-整体”的层次结构，使用户对单个对象和组合对象的使用具有一致性；采用（45）设计模式可动态地给一个对象添加一些额外的职责。

- (43)A. 命令 (Command) B. 单例 (Singleton) C. 装饰 (Decorate) D. 组合 (Composite)
- (44)A. 命令 (Command) B. 单例 (Singleton) C. 装饰 (Decorate) D. 组合 (Composite)
- (45)A. 命令 (Command) B. 单例 (Singleton) C. 装饰 (Decorate) D. 组合 (Composite)

● 下列 UML 类图表示的是 (46) 设计模式。该设计模式中, (47)



(46)

- A. 备忘录 (Memento) B. 策略 (Strategy) C. 状态 (State) D. 观察者 (Observer)

(47)

- A. 一个 Subject 对象对应多个 Observer 对象
- B. Subject 只能有一个 ConcreteSubject 子类
- C. Observer 只能有一个 ConcreteObserver 子类
- D. 一个 Subject 对象必须至少对应一个对象

● 以下关于编译系统对某高级语言进行翻译的叙述中, 错误的是 (48)

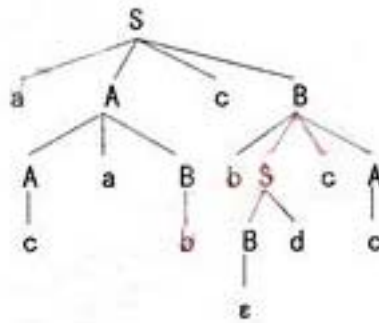
- A. 词法分析将把源程序看作一个线性字符序列进行分析
- B. 语法分析阶段可以发现程序中所有的语法错误
- C. 语义分析阶段可以发现程序所以的主义错误
- D. 目标代码生成阶段的工作与目标的体系结构相关

● 若一个程序语言可以提供链表的定义和运算, 则其运行时 (49)

- A. 数据空间适合采用静态存储分配策略

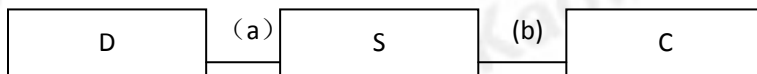
- B. 数据空间必须采用堆存储分配策略
- C. 指令空间需要采用栈结构
- D. 指令代码必须放入堆区

●由某上下文无关文法 $M[S]$ 推导出某句子的分析树如下图所示，则错误的叙述是 (50)



- A. 该文法推导出的句子必须以“a”开头
- B. $acabcbdcc$ 是该文法推导出的一个句子
- C. “ $S \rightarrow aAcB$ ”是该文法的终结符号集
- D. a, b, c, d 属于该文法的终结符号集

●假设有学生 S (学号, 姓名, 性别, 入学时间, 联系方式), 院系 D (院系号, 院系名称, 电话号码, 负责人) 和课程 C (课程号, 课程名) 三个实体, 若一名学生属于一个院系, 一个院系有多名学生; 一名学生可以选择多门课程, 一门课程可被多名学生选择, 刚图中 (a) 和 (b) 分别国 (51) 联系, 假设一对多联系不转换为一个独立的关系模式, 那么生成的关系模式 (52)



(51) A. 1 * 和 1 * B. 1 * 和 * 1 C. 1 * 和 * * D. * 1 和 * *

(52)

- A. S 中应加入关系模式 D 的主键
- B. S 中应加入关系模式 C 的主键
- C. D 中应加入关系模式 S 的主键
- D. C 中应加入关系模式 S 的主键

● 软硬件故障常造成数据库中的数据破坏。数据库恢复就是 (53)

- A. 重新安装数据库管理系统和应用程序
- B. 重新安装应用程序, 并将数据库镜像
- C. 重新安装数据库管理系统, 并将数据做镜像
- D. 在尽可能短的时间内, 把数据库恢复到故障发生前的状态

● 设有员工实体 Emp(员工号, 姓名, 性别, 年龄, 出生年月, 联系方式, 部门号), 其中“联系方式”要求记录该员工的手机号码和办公室电话, 部门号要求对照别一部门实体 Dept 的主码“部门号”。Emp 实体中存在派生属性和多值属性: (54); 对属性部门号应该进行 (55) 约束; 可以通过命令 (56) 修改表中的数据。

(54) A. 年龄和出生年月 B. 年龄和联系方式 C. 出生年月和联系方式 D. 出生年月和年龄

(55) A. 非空主键 B. 主键 C. 外键 D. 候选键

(56) A. INSERT B. DELETE C. UPDATE D. MODIFY

● 已知一个二叉树的先序遍历序列为①、②、③、④、⑤中序遍历序列为②、①、④、③、⑤, 则该二叉树的后序遍历序列为 (57), 对于任意一棵二叉树, 叙述错误的是 (58)

(57)

- A. ②、③、①、⑤、④
- B. ①、②、③、④、⑤
- C. ②、④、⑤、③、①
- D. ④、⑤、③、②、①

(58)

- A. 由其后序遍历序列和中序遍历序列可以构造该二叉树的先序遍历序列
- B. 由其先序遍历序列和后序遍历序列可以构造该二叉树的中序遍历序列
- C. 由其层序遍历序列和中序遍历序列可以构造该二叉树的先序遍历序列
- D. 由其层序遍历序列和中序遍历序列不能构造该二叉树的后序遍历序列

● 邻接矩阵和邻接表是图(网)的两种基本存储结构, 对于具有 N 个顶点、E 条边的图, (59)

- A. 进行深度优先遍历运算所消耗的时间与采用哪一种存储结构无关
- B. 进行广度优先遍历运算所消耗的时间与采用哪一种存储结构无关
- C. 采用邻接表表示图时, 查找所有顶点的邻接顶点的时间复杂度为 $O(n \cdot c)$

D. 采用邻接矩阵表示图时, 查找所有顶点的邻接顶点的时间复杂度为 $O(n^2)$

● 单向链表中往往含有一个头结点, 该结点不存储数据元素, 一般令链表的头指针指向该结点, 而该结点指针域的值域为第一个元素结点的指针, 以下关于单链表头结点的叙述中, 错误的是 (60)

- A. 若在头结点中存入链表长度值, 则求链表长度运算的时间复杂度为 $O(1)$
- B. 在链表的任何一个元素前后进行插入和删除操作可用一致的方式进行处理
- C. 加入头结点后, 代表链表的头指针不会因为链表的空而改变
- D. 加入头结点后, 在链表中进行查找运算的时间复杂度为 $O(1)$

● 对于长度为 $m(m>1)$ 的指定序列, 通过初始为空的一个栈, 一个队列后, 错误的叙述是 (61)

- A. 若入栈和入队的序列相同, 则出栈序列和出队序列可能相同
- B. 若入栈和入队的序列相同, 则出栈序列和出队序列可以互为逆序
- C. 入队序列与出队序列关系为 $1:1$, 而入栈序列与出栈序列关系是 $1:n(n\geq 1)$
- D. 入栈序列与出队序列关系为 $1:1$, 而入队序列与出栈序列关系是 $1:n(n\geq 1)$

● 字符串采用链表存储方式时, 每个结点存储多个字符有助于提高存储密度。若采用结点大小相同的链表存储串, 则串比较、求子串、串连接、串替换等串的基本运算中, (62)

- A. 进行串的比较运算最不方便
- B. 进行求子串运算最不方便
- C. 进行串连接最不方便
- D. 进行串替换最不方便

● 某算法的时间复杂度表达式为 $T(n)=an^2+bn\lg n+cn+d$, 其中, n 为问题的规模, a 、 b 、 c 和 d 为常数, 用 O 表示其渐近时间复杂度为 (63)

- A. $O(n^2)$
- B. $O(n)$
- C. $O(n\lg n)$
- D. $O(1)$

● 以下关于快速排序算法的描述中, 错误的是 (64)。在快速排序过程中, 需要设立基准元素并划分序列来进行排序, 若序列由元素 {12, 25, 30, 45, 52, 67, 85} 构成, 则初始排列为 (65), 排序效率最高 (令序列的第一个元素为基准元素)。

(65)

- A. 快速排序算法是不稳定的排序算法
- B. 快速排序算法是最坏情况下的时间复杂度为 $O(n \lg n)$
- C. 快速排序算法是一种分治算法
- D. 当输入数据基本有序时，快速排序算法具有最坏情况下的时间复杂度

(66)

- A. 45, 12, 30, 25, 67, 52, 85
- B. 85, 67, 52, 45, 30, 25, 12
- C. 12, 25, 30, 45, 52, 67, 85
- D. 45, 12, 25, 30, 85, 67, 52

●下列网络互连设备中，属于物理层的是 (66)，属于网络层的是 (67)

- (66) A. 中继器 B. 交换机 C. 路由器 D. 网桥
- (67) A. 中继器 B. 交换机 C. 路由器 D. 网桥

●下图是 HTML 文件 test.html 在 IE 中的显示效果，实现图中①处效果的 HTML 语句是 (68)，实现图中②处效果的 HTML 语句是 (69)，实现图中③处效果的 HTML 语句是 (70)



(68)

- A. <TITLE>我的主页</TITLE>
- B. <HEAD>的主页</HEAD>
- C. <BODY>我的主页</BODY>
- D. <H1>我的主页</H1>

- (69) A. <HR>
- B. <TR></TR>
- C. <CELL></CELL>
- D. <TR></TR>

(70)

- A. Welcome B. Welcome
C. <I>Welcome</I> D. <H>Welcome</H>

● Why is (71) fun? What delights may its practitioner expect his reward? First is the sheer joy of making things. As the child delights in his mud pie, so the adult enjoys building things, especially things of his own design. Second is the pleasure of making things that are useful to other people. Third is the fascination of fashioning complex puzzle-like objects of interlocking moving parts and watching them work in subtle eyes, playing out the consequences of principles built in from the beginning. Fourth is the joy of always learning, which springs from the (72) nature of the task. In one way or another the problem is ever new, and its solver learns something: sometimes (73), sometimes theoretical, and sometimes both. Finally, there is the delight of working in such a tractable medium. The (74), like the poet, works only slightly removed from pure thought-stuff. Few media of creation are so flexible, so easy to polish and rework, so readily capable of realizing grand conceptual structures. Yet the program (75), unlike the poet's words, is real in the sense that it moves and works, producing visible outputs separate from the construct itself. It prints results, draws pictures, produces sounds, moves arms. Programming then is fun because it gratifies creative longings built deep within us and delights sensibilities we have in common with all men.

- | | | | |
|----------------|--------------|------------------|----------------|
| A. programming | B. composing | C. working | D. writing |
| A. repeating | B. basic | C. non-repeating | D. advance |
| A. semantic | B. practical | C. lexical | D. syntactical |
| A. poet | B. architect | C. doctor | D. programmer |
| A. construct | B. code | C. size | D. scale |

全国计算机技术与软件专业技术资格（水平）考试

2009 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道，每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2009 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月（2）日。

因为正确的解答是“11 月 14 日”，故在答题纸的对应栏内写上“11”和“14”（参看下表）。

例题	解答栏
(1)	11
(2)	14

试题一(共 15 分)

阅读以下说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

现准备为某银行开发一个信用卡管理系统 CCMS，该系统的基本功能为：

1. 信用卡申请。非信用卡客户填写信用卡申请表，说明所要申请的信用卡类型及申请者的基本信息，提交 CCMS。如果信用卡申请被银行接受，CCMS 将记录该客户的基本信息，并发送确认函给该客户，告知客户信用卡的有效期及信贷限额；否则该客户将会收到一封拒绝函。非信用卡客户收到确认函后成为信用卡客户。

2. 信用卡激活。信用卡客户向 CCMS 提交激活请求，用信用卡号和密码激活该信用卡。激活操作结束后，CCMS 将激活通知发送给客户，告知客户其信用卡是否被成功激活。

3. 信用卡客户信息管理。信用卡客户的个人信息可以在 CCMS 中进行在线管理。每位信用卡客户可以在线查询和修改个人信息。

4. 交易信息查询。信用卡客户使用信用卡进行的每一笔交易都会记录在 CCMS 中。信用卡客户可以通过 CCMS 查询并核实其交易信息（包括信用卡交易记录及交易额）。

【问题 1】(3 分)

根据【说明】，将图 1-1 中的 E1-E3 填充完整。

【问题 2】(3 分)

图 1-1 中缺少三条数据流，根据【说明】，分别指出这三条数据流的起点和终点。（注：数据流的起点和终点均采用图中的符号和描述）

【问题 3】(5 分)

图 1-2 中有两条数据流是错误的，请指出这两条数据流的名称，并改正。（注：数据流的起点和终点均采用图中的符号和描述）

【问题 4】(4 分)

根据【说明】，将图 1-2 中 P1-P4 的处理名称填充完整。

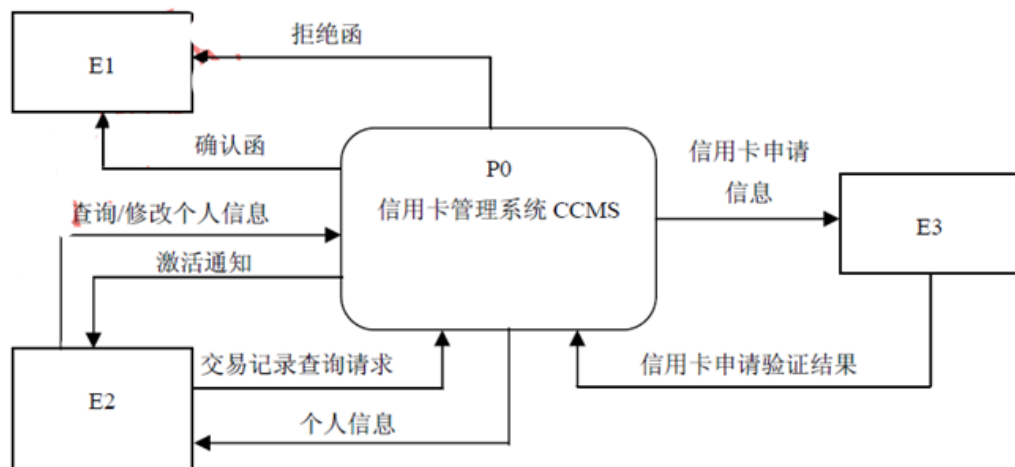


图 1-1 顶层数据流图

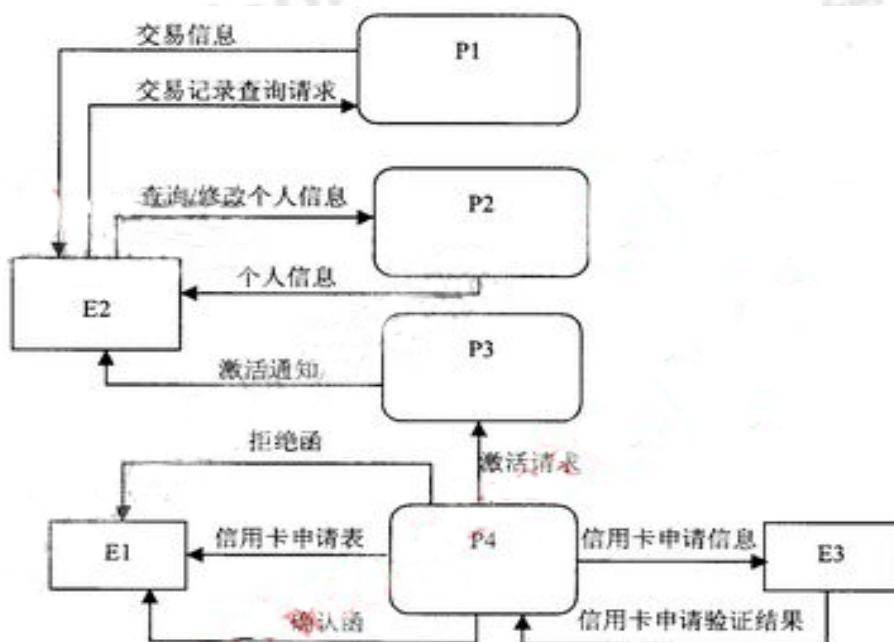


图 1-2 0层数据流图

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某公司拟开发一多用户电子邮件客户端系统，部分功能的初步需求分析结果如下：

(1) 邮件客户端系统支持多个用户，用户信息主要包括用户名和用户密码，且系统中的用户名不可重复。

(2) 邮件帐号信息包括邮件地址及其相应的密码，一个用户可以拥有多个邮件地址（如 user1@123.com）。

(3) 一个用户可拥有一个地址簿，地址簿信息包括联系人编号、姓名、电话、单位地址、邮件地址 1、邮件地址 2、邮件地址 3 等信息。地址簿中一个联系人只能属于一个用户，且联系人编号唯一标识一个联系人。

(4) 一个邮件帐号可以含有多封邮件，一封邮件可以含有多个附件。邮件主要包括邮件号、发件人地址、收件人地址、邮件状态、邮件主题、邮件内容、发送时间、接收时间。其中，邮件号在整个系统内唯一标识一封邮件，邮件状态有已接收、待发送、已发送和已删除 4 种，分别表示邮件是属于收件箱、发件箱、已发送箱和废件箱。一封邮件可以发送给多个用户。附件信息主要包括附件号、附件文件名、附件大小。一个附件只属于一封邮件，附件号仅在一封邮件内唯一。

【问题 1】(5 分)

根据以上说明设计的 E-R 图如图 2-1 所示，请指出地址簿与用户、电子邮件帐号与邮件、邮件与附件之间的联系类型。

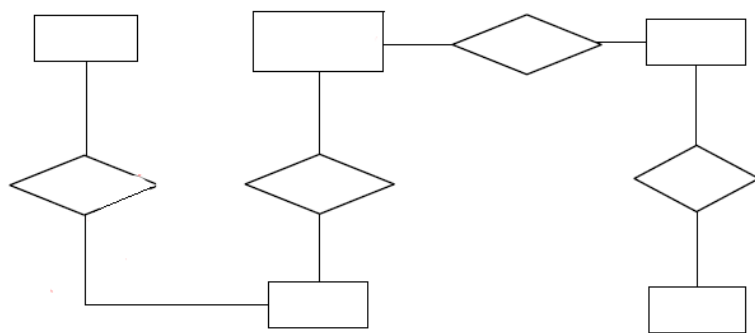


图 2-1 电子邮件客户端系统 E-R 图

【问题 2】(4 分)

该邮件客户端系统的主要关系模式如下，请填补 (a) ~ (c) 的空缺部分。

用户 (用户名, 用户密码)

地址簿 (_____(a)_____, 联系人编号, 姓名, 电话, 单位地址, 邮件地址 1, 邮件地址 2, 邮件地址 3)

邮件帐号 (邮件地址, 邮件密码, 用户名)

邮件 (_____(b)_____, 收件人地址, 邮件状态, 邮件主题, 邮件内容, 发送时间, 接收时间)

附件 (_____(c)_____, 附件号, 附件文件名, 附件大小)

【问题 3】(6 分)

(1) 请指出【问题 2】中给出的地址簿、邮件和附件关系模式的主键，如果关系模式存在

外键请指出。

(2) 附件属于弱实体吗? 请用 50 字以内的文字说明原因。

试题三 (共 15 分)

阅读下列说明和 UML 图, 回答问题 1 至问题 4, 将解答填入答题纸的对应栏内。

【说明】

某企业为了方便员工用餐, 为餐厅开发了一个订餐系统(COS: Cafeteria Ordering System), 企业员工可通过企业内联网使用该系统。

企业的任何员工都可以查看菜单和今日特价。

系统的顾客是注册到系统的员工, 可以订餐(如果未登录, 需先登录)、注册工资支付、预约规律的订餐, 在特别情况下可以覆盖预订。

餐厅员工是特殊顾客, 可以进行备餐、生成付费请求和请求送餐, 其中对于注册工资支付的顾客生成付费请求并发送给工资系统。

菜单管理员是餐厅特定员工, 可以管理菜单。

送餐员可以打印送餐说明, 记录送餐信息(如送餐时间)以及记录收费(对于没有注册工资支付的顾客, 由送餐员收取现金后记录)。

顾客订餐过程如下:

1. 顾客请求查看菜单;
2. 系统显示菜单和今日特价;
3. 顾客选菜;
4. 系统显示订单和价格;
5. 顾客确认订单;
6. 系统显示可送餐时间;
7. 顾客指定送餐时间、地点和支付方式;
8. 系统确认接受订单, 然后发送 Email 给顾客以确认订餐, 同时发送相关订餐信息通知给餐厅员工。

系统采用面向对象方法开发, 使用 UML 进行建模。系统的顶层用例图和一次订餐的活动图初稿分别如图 3-1 和图 3-2 所示。

【问题 1】(2 分)

根据【说明】中的描述, 给出图 3-1 中 A1 和 A2 所对应的参与者。

【问题 2】(8 分)

根据【说明】中的描述，给出图 3-1 中缺少的四个用例及其所对应的参与者。

【问题 3】(4 分)

根据【说明】中的描述，给出图 3-2 中 (1) ~ (4) 处对应的活动名称或图形符号。

【问题 4】(1 分)

指出图 3-1 中员工和顾客之间是什么关系，并解释该关系的内涵。

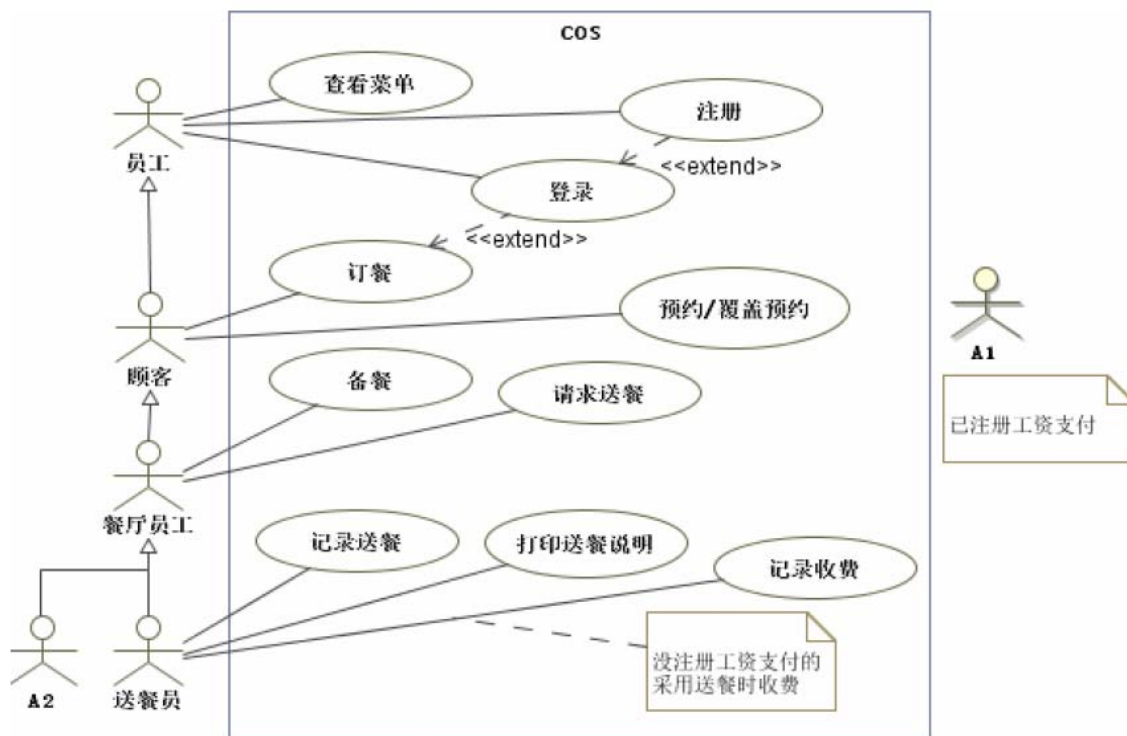


图 3-1 COS 系统顶层用例图

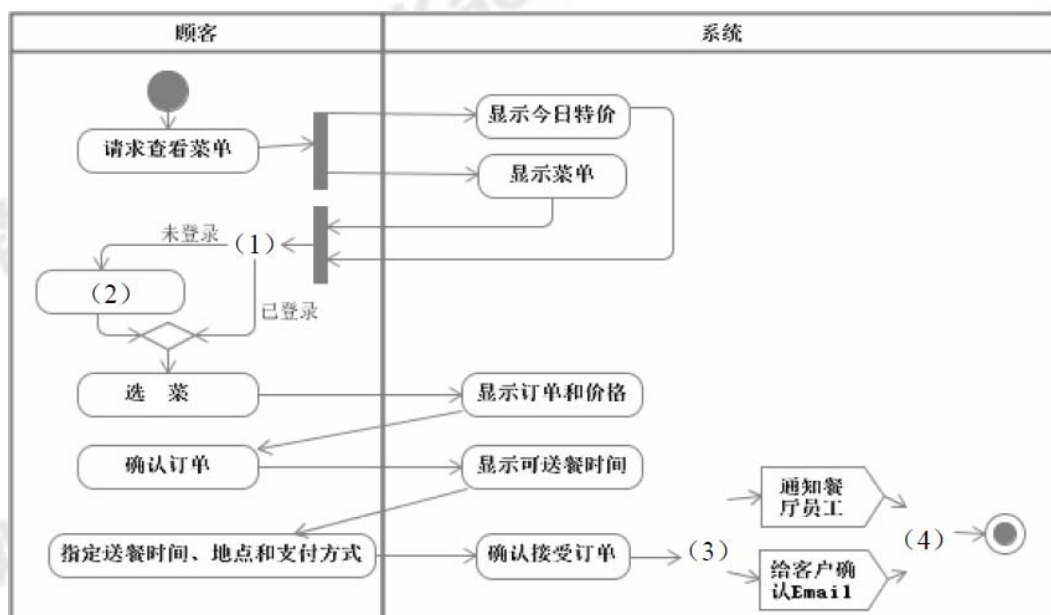


图 3-2 一次订餐的活动图

试题四（共 15 分）

阅读下列说明，回答问题 1 至问题 2，将解答填入答题纸的对应栏内。

【说明】

0-1 背包问题可以描述为：有 n 个物品，对 $i=1, 2, \dots, n$ ，第 i 个物品价值为 v_i ，重量为 w_i (v_i 和 w_i 为非负数)，背包容量为 W (W 为非负数)，选择其中一些物品装入背包，使装入背包物品的总价值最大，即 $\max \sum_{i=1}^n v_i x_i$ ，且总重量不超过背包容量，即 $\sum_{i=1}^n w_i x_i \leq W$ ，其中， $x_i \in \{0, 1\}$ ， $x_i=0$ 表示第 i 个物品放入背包。

【问题 1】（8 分）

用回溯法求解此 0-1 背包问题，请填充下面伪代码中 (1)~(4) 处空缺。

回溯法是一种系统的搜索方法，在确定解空间后，回溯法从根结点开始，按照深度优先策略遍历空间树，搜索满足约束条件的解。对每一个当前结点，若扩展该结点已经不满足约束条件，则不再继续扩展。为了进一步提高算法的搜索效率，往往假设已经设计了界函数，判断并剪枝那些即扩展了也不能得到最优解的结点。现在假设已经设计了 $BOUND(v, w, k, W)$ 函数，其中 v, w, k 和 W 分别表示当前已经获得的价值、当前背包的重量、已经确定是否选择的物品数和背包的总容量。对应于搜索树中的某个结点，该函数值表示确定了部分物品是否选择之后，对剩下的物品在满足约束条件的前提下进行选择可能获得最大的价值，若该价值小于等于当前已经得到的最优解，则该结点无需再扩展。

下面给出 0-1 背包问题的回溯法伪代码。

函数参数说明如下：

W ：背包容量； n ：物品个数； w ：重量数组； v ：价值数组； fw ：获得最大价值时背包的重量； fp ：背包获得的最大价值； X ：问题的最优解。

变量说明如下：

cw ：当前的背包重量； cp ：当前获得的价值； k ：当前考虑的物品编号； Y ：当前已获得的部分解。

BKNAP (W, n, w, v, fw, fp, X)

1 $cw \leftarrow cp \leftarrow 0$

2 (1)

3 $fp \leftarrow -1$

```

4 while true
5     while  $k \leq n$  and  $cw + w[k] \leq W$  do
6         (1)
7          $cp \leftarrow cp + v[k]$ 
8          $Y[k] \leftarrow 1$ 
9          $k \leftarrow k + 1$ 
10    if  $k > n$  then
11        if  $fp < cp$  then
12             $fp \leftarrow cp$ 
13             $fw \leftarrow cw$ 
14             $k \leftarrow n$ 
15             $X \leftarrow Y$ 
16        else  $Y(k) \leftarrow 0$ 
17    while  $BOUND(cp, cw, k, W) \leq fp$  do
18        while  $k \neq 0$  and  $Y(k) \neq 1$  do
19            (3)
20        if  $k = 0$  then return
21         $Y[k] \leftarrow 0$ 
22         $cw \leftarrow cw - w[k]$ 
23         $cp \leftarrow cp - v[k]$ 
24        (4)

```

【问题 2】(7 分)

考虑表 4-1 的实例，假设有 3 个物品，背包容量为 22。图 4-1 中是根据上述算法构造的搜索树，其中结点的编号表示了搜索树生成的顺序，边上的数字 1/0 分别表示选择/不选择对应物品，除了根结点之外，每个左孩子结点旁边的上下两个数字分别表示当前背包的重量和已获得的价值，右孩子结点旁边的数字表示扩展了该结点后最多可能获得的值。为获得最优解，应该选择特(5)，获得的值为(6)

表 4-1 0-1 背包问题实例

	物品 1	物品 2	物品 3
重量	15	10	10

价值	30	18	17
单位价值	2	1.8	1.7

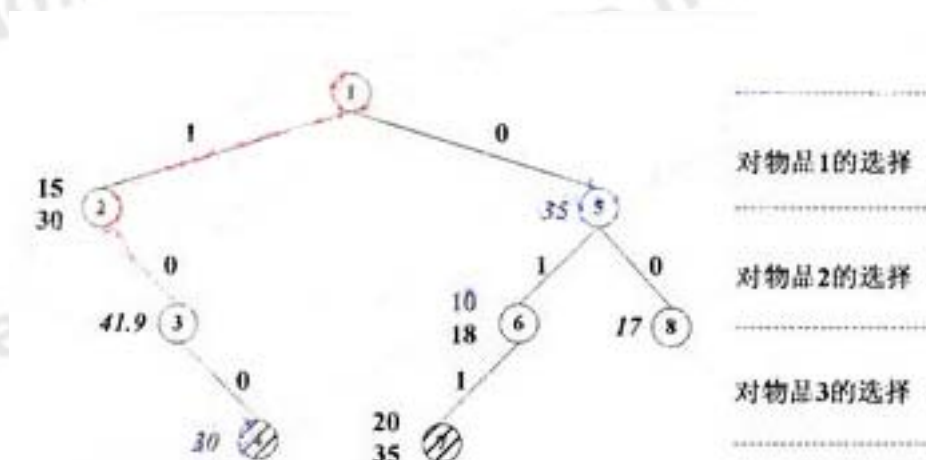


图 4-1 表 4-1 实例的搜索树

于表 4-1 的实例，若采用穷举法搜索整个解空间，则搜索树的结点数为 (7)，而用了上述的回溯法，搜索树的结点的为 (8)。

从下列的 3 道试题（试题五至试题七）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

现欲构造一文/目录树，采用组合 (Composite) 设计模式来设计，得到的类图如 5-1 所示：

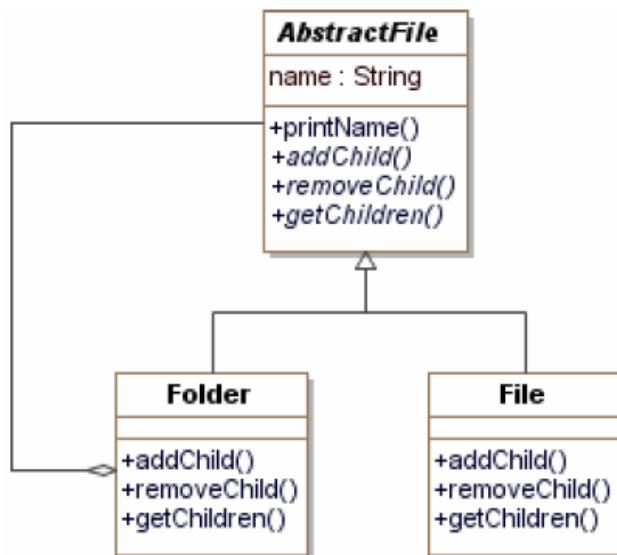


图 5-1 类图

【C++代码】

```
#include <list>
```

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class AbstractFile {
```

```
protected :
```

```
    string name; // 文件或目录名称
```

```
public:
```

```
    void printName() {cout << name;} // 打印文件或目录名称
```

```
    virtual void addChild(AbstractFile *file)=0; // 给一个目录增加子目录或文件
```

```
    virtual void removeChild(AbstractFile *file)=0; // 删除一个目录的子目录或文件
```

```
    virtual list<AbstractFile*> *getChildren()=0; // 获得一个目录的子目录或文件
```

```
};
```

```
class File : public AbstractFile {
```

```
public :
```

```
    File(string name) { (1) = name; }
```

```
    void addChild(AbstractFile *file) { return ; }
```

```
    void removeChild(AbstractFile *file) { return ; }
```

```
(2) getChildren() { return (3) ; }  
};  
  
class Folder :public AbstractFile {  
private :  
    list <AbstractFile*> childList; // 存储子目录或文件  
public :  
    Folder(string name) { (4) = name; }  
    void addChild(AbstractFile *file) { childList.push_back(file); }  
    void removeChild(AbstractFile *file) { childList.remove(file); }  
    list<AbstractFile*> *getChildren() { return (5) ; }  
};  
  
void main()  
    // 构造一个树形的文件/目录结构  
    AbstractFile *rootFolder = new Folder("c:\\");  
    AbstractFile *compositeFolder = new Folder("composite");  
    AbstractFile *windowsFolder = new Folder("windows");  
    AbstractFile *file = new File("TestComposite.java");  
    rootFolder->addChild(compositeFolder);  
    rootFolder->addChild(windowsFolder);  
    compositeFolder->addChild(file);  
}
```

试题六（共 15 分）

阅读下列说明和 JAVA 代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

现欲构造一文/目录树，采用组合(Composite)设计模式来设计，得到的类图如 6-1 所示：

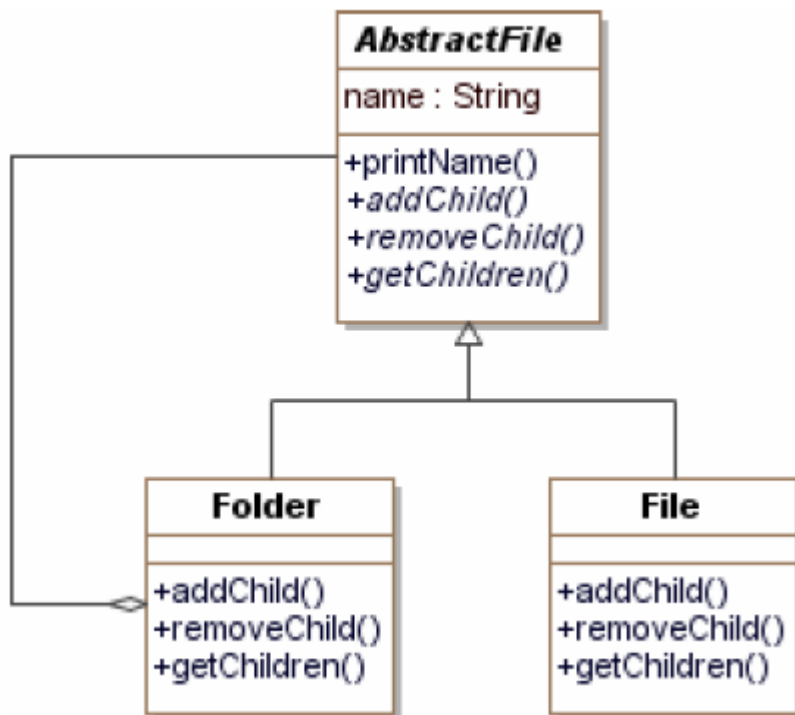


图 6-1 类图

【Java 代码】

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
(1) class AbstractFile {
```

```
    protected String name;
```

```
    public void printName() {System.out.println(name);} 
```

```
    public abstract boolean addChild(AbstractFile file);
```

```
    public abstract boolean removeChild(AbstractFile file);
```

```
    public abstract List<AbstractFile> getChildren();
```

```
}
```

```
class File extends AbstractFile {
```

```
    public File(String name) { this.name = name; }
```

```
    public boolean addChild(AbstractFile file) { return false; }
```

```
    public boolean removeChild(AbstractFile file) { return false; }
```

```
    public List<AbstractFile> getChildren() { return (2) ; }
```

```
}
```



```
class Folder extends AbstractFile {
    private List <AbstractFile> childList;
    public Folder(String name) {
        this.name = name;
        this.childList = new ArrayList<AbstractFile>();
    }

    public boolean addChild(AbstractFile file) { return childList.add(file); }
    public boolean removeChild(AbstractFile file) { return childList.remove(file); }
    public (3) <AbstractFile> getChildren() { return (4) ; }
}

public class Client {
    public static void main(String[] args) {
        // 构造一个树形的文件/目录结构
        AbstractFile rootFolder = new Folder("c:\\");
        AbstractFile compositeFolder = new Folder("composite");
        AbstractFile windowsFolder = new Folder("windows");
        AbstractFile file = new File("TestComposite.java");
        rootFolder.addChild(compositeFolder);
        rootFolder.addChild(windowsFolder);
        compositeFolder.addChild(file);

        // 打印目录文件树
        printTree(rootFolder);
    }

    private static void printTree(AbstractFile ifile) {
        ifile.printName();
        List <AbstractFile> children = ifile.getChildren();
        if(children == null) return;
        for (AbstractFile file:children) {
            (5) ;
        }
    }
}
```

```
}  
  
}  
  
}
```

该程序运行后输出结果为：

c:\

composite

TestComposite.java

Windows

试题七（共 15 分）

阅读以下说明和 C 程序，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

现有 n ($n < 1000$) 节火车车厢，顺序编号为 $1, 2, 3, \dots, n$ ，按编号连续依次从 A 方向的铁轨驶入，从 B 方向铁轨驶出，一旦车厢进入车站（Station）就不能再回到 A 方向的铁轨上；一旦车厢驶入 B 方向铁轨就不能再回到车站，如图 7-1 所示，其中 Station 为栈结构，初始为空且最多能停放 1000 节车厢。

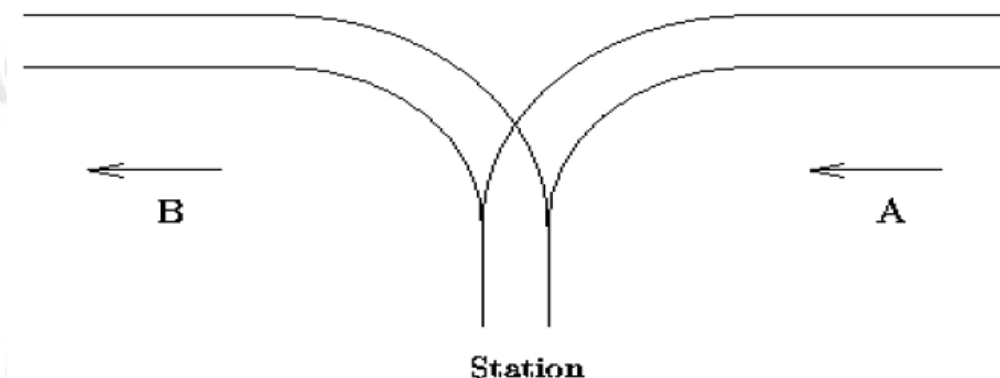


图 7-1 车站示意图

下面的 C 程序判断能否从 B 方向驶出预先指定的车厢序列，程序中使用了栈类型 STACK，关于栈基本操作的函数原型说明如下：

void InitStack(STACK *s)：初始化栈。

void Push(STACK *s, int e)：将一个整数压栈，栈中元素数目增 1。

void Pop(STACK *s)：栈顶元素出栈，栈中元素数目减 1。

int Top(STACK s)：返回非空栈的栈顶元素值，栈中元素数目不变。

int IsEmpty(STACK s): 若是空栈则返回 1, 否则返回 0。

【C 程序】

```
#include<stdio.h>
```

```
/*此处为栈类型及其基本操作的定义, 省略*/
```

```
int main( ){
```

```
    STACK station;
```

```
    int state[1000];
```

```
    int n;          /*车厢数*/
```

```
    int begin, i, j, maxNo; /*maxNo 为 A 端正待入栈的车厢编号*/
```

```
    printf("请输入车厢数: ");
```

```
    scanf("%d",&n);
```

```
    printf("请输入需要判断的车厢编号序列 (以空格分隔): ");
```

```
    if (n < 1) return -1;
```

```
    for (i = 0; i<n; i++) /* 读入需要驶出的车厢编号序列, 存入数组 state[] */
```

```
        scanf("%d",&state[i]);
```

```
    (1) ; /*初始化栈*/
```

```
    maxNo = 1;
```

```
    for(i = 0; i < n; ){/*检查输出序列中的每个车厢号 state[i]是否能从栈中获取*/
```

```
        if ( (2) ){/*当栈不为空时*/
```

```
            if (state[i] == Top(station)){ /*栈顶车厢号等于被检查车厢号*/
```

```
                printf("%d ",Top(station));
```

```
                Pop(&station); i++;
```

```
        }
```

```
        else
```

```
            if ( (3) ){
```

```
                printf("error\n");
```

```
                return 1;
```

```
            }
```

```
        else {
```

```
            begin = (4) ;
```

```
for(j = begin+1; j<=state[i]; j++) {  
    Push(&station, j);  
}  
}  
  
else { /*当栈为空时*/  
    begin = maxNo;  
    for(j = begin; j<=state[i]; j++){  
        Push(&station, j);  
    }  
    maxNo = (5) ;  
}  
}  
printf("OK");  
return 0;  
}
```

(本试题的参考答案请在软考网下载。网址是 <http://www.RuanKao.net>)

全国计算机技术与软件专业技术资格（水平）考试

2010 年上半年 软件设计师 上午试卷

（考试时间 9:00~11:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确写入你的姓名和准考证号，并用正规 2B 铅笔在你写入的准考证号下填涂准考证号。
2. 本试卷的试题中共有 75 个空格，需要全部解答，每个空格 1 分，满分 75 分。
3. 每个空格对应一个序号，有 A、B、C、D 四个选项，请选择一个最恰当的选项作为解答，在答题卡相应序号下填涂该选项。
4. 解答前务必阅读例题和答题卡上的例题填涂样式及填涂注意事项。解答时用正规 2B 铅笔正确填涂选项，如需修改，请用橡皮擦干净，否则会导致不能正确评分。

例题

● 2010 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
____（88）____ 月 ____（89）____ 日。

- | | | | | |
|------|-------|-------|-------|-------|
| （88） | A. 4 | B. 5 | C. 6 | D. 7 |
| （89） | A. 19 | B. 20 | C. 21 | D. 22 |

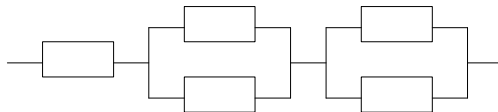
因为考试日期是“5 月 22 日”，故（88）选 B，（89）选 D，应在答题卡序号 88 下对 B 填涂，在序号 89 下对 D 填涂（参看答题卡）。

● 为实现程序指令的顺序执行，CPU (1) 中的值将自动加 1。

- (1) A. 指令寄存器 (IR) B. 程序计数器 (PC)
C. 地址寄存器 (AR) D. 指令译码器 (ID)

● 某计算机系统由下图所示的部件构成，假定每个部件的千小时可靠度都为 R ，则该系统的千小时可靠度为 (2)。

- (2) A. $R + 2R/4$ B. $R + R^2/4$ C. $R(1 - (1 - R)^2)$ D. $R(1 - (1 - R)^2)^2$



● 以下关于计算机系统中断概念的叙述中，正确的是 (3)。

- (3) A. 由 I/O 设备提出的中断请求和电源掉电都是可屏蔽中断
B. 由 I/O 设备提出的中断请求和电源掉电都是不可屏蔽中断
C. 由 I/O 设备提出的中断请求是可屏蔽中断，电源掉电是不可屏蔽中断
D. 由 I/O 设备提出的中断请求是不可屏蔽中断，电源掉电是可屏蔽中断

● 与 $\overline{A} \oplus B$ 等价的逻辑表达式是 (4)。 ($\oplus$ 表示逻辑异或，+ 表示逻辑加)

- (4) A. $A + \overline{B}$ B. $A \oplus \overline{B}$ C. $A \oplus B$ D. $AB + \overline{A}\overline{B}$

● 计算机指令一般包括操作码和地址码两部分，为分析执行一条指令，其 (5)。

- (5) A. 操作码应存入指令寄存器 (IR)，地址码应存入程序计数器 (PC)
B. 操作码应存入程序计数器 (PC)，地址码应存入指令寄存器 (IR)
C. 操作码和地址码都应存入指令寄存器 (IR)
D. 操作码和地址码都应存入程序计数器 (PC)

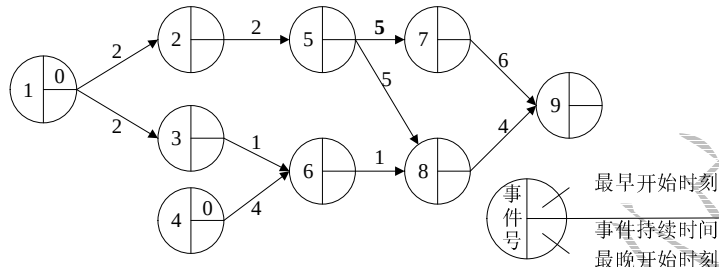
● 关于 64 位和 32 位微处理器，不能以 2 倍关系描述的是 (6)。

- (6) A. 通用寄存器的位数 B. 数据总线的宽度
C. 运算速度 D. 能同时进行运算的位数

● Outlook Express 作为邮件代理软件有诸多优点，以下说法中，错误的是 (7)。

- (7) A. 可以脱机处理邮件
B. 可以管理多个邮件账号
C. 可以使用通讯簿存储和检索电子邮件地址
D. 不能发送和接收安全邮件

● 进度安排的常用图形描述方法有 Gantt 图和 PERT 图。Gantt 图不能清晰地描述 (18)；PERT 图可以给出哪些任务完成后才能开始另一些任务。下图所示的 PERT 图中，事件 6 的最晚开始时刻是 (19)。



- (18) A. 每个任务从何时开始
B. 每个任务到何时结束
C. 每个任务的进展情况
D. 各任务之间的依赖关系
- (19) A. 0
B. 3
C. 10
D. 11

- 若某整数的 16 位补码为 FFFF_{H} (H 表示十六进制)，则该数的十进制值为 (20)。
- (20) A. 0
B. -1
C. $2^{16} - 1$
D. $-2^{16} + 1$

● 逻辑表达式 “ $a \wedge b \vee c \wedge (b \vee x > 0)$ ” 的后缀式为 (21)。(其中 $\wedge$ 、 $\vee$ 分别表示逻辑与、逻辑或， $>$ 表示关系运算大于，对逻辑表达式进行短路求值)

- (21) A. $abcbx0 > \vee \wedge \wedge \vee$
B. $ab \wedge c \vee b \wedge x0 > \vee$
C. $ab \wedge cb \wedge x0 > \vee \vee$
D. $ab \wedge cbx0 > \vee \wedge \vee$

- 编译程序对 C 语言源程序进行语法分析时，可以确定 (22)。

- (22) A. 变量是否定义 (或声明)
B. 变量的值是否正确
C. 循环语句的执行次数
D. 循环条件是否正确

● 如果系统采用信箱通信方式，当进程调用 Send 原语被设置成 “等信箱” 状态时，其原因是 (23)。

- (23) A. 指定的信箱不存在
B. 调用时没有设置参数
C. 指定的信箱中无信件
D. 指定的信箱中存满了信件

● 若在系统中有若干个互斥资源 R，6 个并发进程，每个进程都需要 2 个资源 R，那么使系统不发生死锁的资源 R 的最少数目为 (24)。

- (24) A. 6
B. 7
C. 9
D. 12

● 某进程有 5 个页面，页号为 0~4，页面变换表如下所示。表中状态位等于 0 和 1 分别表示页面不在内存或在内存。若系统给该进程分配了 3 个存储块，当访问的页面 3 不在内存时，应该淘汰表中页号为 (25) 的页面。假定页面大小为 4K，逻辑地址为十六进制 2C25H，该地址经过变换后，其物理地址应为十六进制 (26)。

页号	页帧号	状态位	访问位	修改位
0	3	1	1	0
1	—	0	0	0
2	4	1	1	1
3	—	0	0	0
4	1	1	1	1

- (25) A. 0 B. 1 C. 2 D. 4
 (26) A. 2C25H B. 4096H C. 4C25H D. 8C25H

● 假设某磁盘的每个磁道划分成 9 个物理块，每块存放 1 个逻辑记录。逻辑记录 R0, R1, ..., R8 存放在同一个磁道上，记录的安排顺序如下表所示：

物理块	1	2	3	4	5	6	7	8	9
逻辑记录	R0	R1	R2	R3	R4	R5	R6	R7	R8

如果磁盘的旋转速度为 27ms/周，磁头当前处在 R0 的开始处。若系统顺序处理这些记录，使用单缓冲区，每个记录处理时间为 3ms，则处理这 9 个记录的最长时间为 (27)；若对信息存储进行优化分布后，处理 9 个记录的最少时间为 (28)。

- (27) A. 54ms B. 108ms C. 222ms D. 243ms
 (28) A. 27ms B. 54ms C. 108ms D. 216ms

● 对于一个大型软件来说，不加控制的变更很快就会引起混乱。为有效地实现变更控制，需借助于配置数据库和基线的概念。 (29) 不属于配置数据库。

- (29) A. 开发库 B. 受控库 C. 信息库 D. 产品库

● 软件设计时需要遵循抽象、模块化、信息隐蔽和模块独立原则。在划分软件系统模块时，应尽量做到 (30)。

- (30) A. 高内聚高耦合 B. 高内聚低耦合
 C. 低内聚高耦合 D. 低内聚低耦合

● 能力成熟度集成模型 CMMI 是 CMM 模型的最新版本，它有连续式和阶段式两种表示方式。基于连续式表示的 CMMI 共有 6 个 (0~5) 能力等级，每个能力等级对应到一个一般目标以及一组一般执行方法和特定方法，其中能力等级 (31) 主要关注过程的组织标准化和部署。

- (31) A. 1 B. 2 C. 3 D. 4

● 统一过程（UP）定义了初启阶段、精化阶段、构建阶段、移交阶段和产生阶段，每个阶段以达到某个里程碑时结束，其中（32）的里程碑是生命周期架构。

(32) A. 初启阶段 B. 精化阶段 C. 构建阶段 D. 移交阶段

● 程序的三种基本控制结构是（33）。

(33) A. 过程、子程序和分程序 B. 顺序、选择和重复
C. 递归、堆栈和队列 D. 调用、返回和跳转

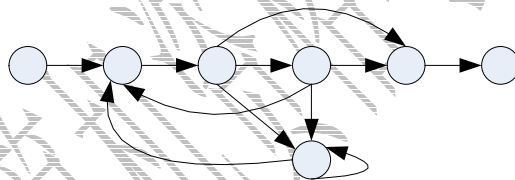
● （34）不属于软件配置管理的活动。

(34) A. 变更标识 B. 变更控制 C. 质量控制 D. 版本控制

● 一个功能模块 M1 中的函数 F1 有一个参数需要接收指向整型的指针，但是在功能模块 M2 中调用 F1 时传递了一个整型值，在软件测试中，（35）最可能测出这一问题。

(35) A. M1 的单元测试 B. M2 的单元测试
C. M1 和 M2 的集成测试 D. 确认测试

● 某程序的程序图如下图所示，运用 McCabe 度量法对其进行度量，其环路复杂度是（36）。



(36) A. 4 B. 5 C. 6 D. 8

● 以下关于面向对象方法中继承的叙述中，错误的是（37）。

(37) A. 继承是父类和子类之间共享数据和方法的机制
B. 继承定义了一种类与类之间的关系
C. 继承关系中的子类将拥有父类的全部属性和方法
D. 继承仅仅允许单重继承，即不允许一个子类有多个父类

● 不同的对象收到同一消息可以产生完全不同的结果，这一现象叫做（38）。绑定是一个把过程调用和响应调用所需要执行的代码加以结合的过程。在一般的程序设计语言中，绑定在编译时进行，叫做（39）；而（40）则在运行时进行，即一个给定的过程调用和执行代码的结合直到调用发生时才进行。

(38) A. 继承 B. 多态 C. 动态绑定 D. 静态绑定
(39) A. 继承 B. 多态 C. 动态绑定 D. 静态绑定
(40) A. 继承 B. 多态 C. 动态绑定 D. 静态绑定

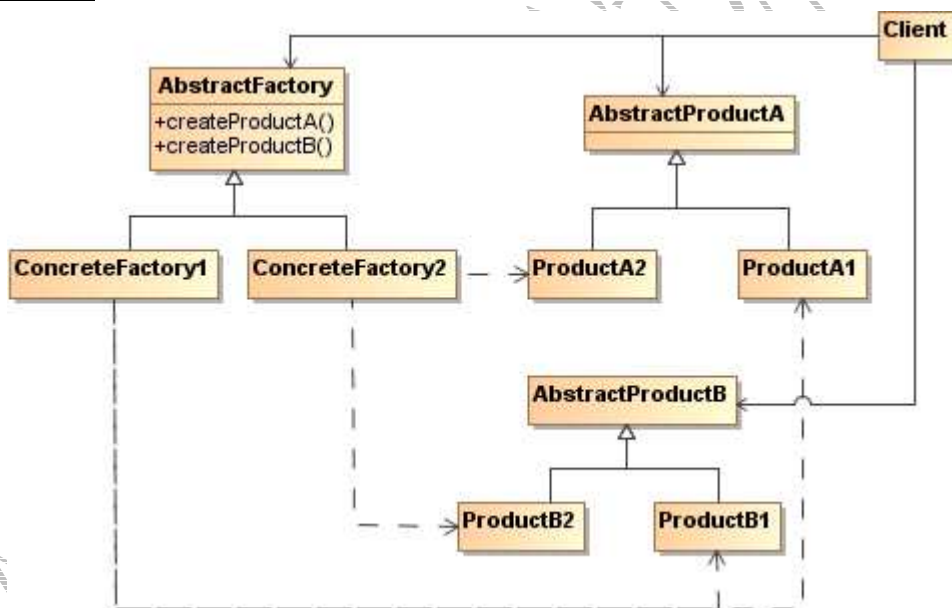
● (41) 不是面向对象分析阶段需要完成的。

- (41) A. 认定对象 B. 组织对象
C. 实现对象及其相互关系 D. 描述对象间的相互作用

● 以下关于面向对象设计的叙述中，错误的是 (42)。

- (42) A. 面向对象设计应在面向对象分析之前，因为只有产生了设计结果才可对其进行分析
B. 面向对象设计与面向对象分析是面向对象软件过程中两个重要的阶段
C. 面向对象设计应该依赖于面向对象分析的结果
D. 面向对象设计产生的结果在形式上可以与面向对象分析产生的结果类似，例如都可以使用 UML 表达

● 如下 UML 类图表示的是 (43) 设计模式。以下关于该设计模式的叙述中，错误的是 (44)。



- (43) A. 工厂方法 B. 策略 C. 抽象工厂 D. 观察者
(44) A. 提供创建一系列相关或相互依赖的对象的接口，而无需指定这些对象所属的具体类
B. 可应用于一个系统要由多个产品系列中的一个来配置的时候
C. 可应用于强调一系列相关产品对象的设计以便进行联合使用的时候
D. 可应用于希望使用已经存在的类，但其接口不符合需求的时候

● UML 类图中类与类之间的关系有五种：依赖、关联、聚合、组合与继承。若类 A 需要使用标准数学函数类库中提供的功能，那么类 A 与标准类库提供的类之间存在 (45) 关系；若类 A 中包含了其它类的实例，且当类 A 的实例消失时，其包含的其它类的实例也消失，则类 A 和它所包含的类之间存在 (46) 关系；若类 A 的实例消失时，其它类的实例仍然存在并继续工作，那么类 A 和它所包含的类之间存在 (47) 关系。

- (45) A. 依赖 B. 关联 C. 聚合 D. 组合
 (46) A. 依赖 B. 关联 C. 聚合 D. 组合
 (47) A. 依赖 B. 关联 C. 聚合 D. 组合

● 以下关于高级语言程序的编译和解释的叙述中，正确的是 (48)。

- (48) A. 编译方式下，可以省略对源程序的词法分析、语法分析
 B. 解释方式下，可以省略对源程序的词法分析、语法分析
 C. 编译方式下，在机器上运行的目标程序完全独立于源程序
 D. 解释方式下，在机器上运行的目标程序完全独立于源程序

● 标记语言用一系列约定好的标记来对电子文档进行标记，以实现对电子文档的语义、结构及格式的定义。(49) 不是标记语言。

- (49) A. HTML B. XML C. WML D. PHP

● 对于正规式 $0^*(10^*1)^*0^*$ ，其正规集中字符串的特点是 (50)。

- (50) A. 开头和结尾必须是 0 B. 1 必须出现偶数次
 C. 0 不能连续出现 D. 1 不能连续出现

● 确定系统边界和关系规范化分别在数据库设计的 (51) 阶段进行。

- (51) A. 需求分析和逻辑设计 B. 需求分析和概念设计
 C. 需求分析和物理设计 D. 逻辑设计和概念设计

● 若关系 R 、 S 如下图所示，则关系代数表达式 $\pi_{1,3,7}(\sigma_{3<6}(R \times S))$ 与 (52) 等价。

A	B	C	D
1	2	4	6
2	3	3	1
3	4	1	3

R

C	D	E
3	4	2
8	9	3

S

- (52) A. $\pi_{A,C,E}(\sigma_{C<D}(R \times S))$ B. $\pi_{A,R,C,E}(\sigma_{R.C<S.D}(R \times S))$
 C. $\pi_{A,S,C,S,E}(\sigma_{R.C<S.D}(R \times S))$ D. $\pi_{R,A,R,C,R,E}(\sigma_{R.C<S.D}(R \times S))$

● 某销售公司数据库的零件 P (零件号, 零件名称, 供应商, 供应商所在地, 库存量) 关系如下表所示, 其中同一种零件可由不同的供应商供应, 一个供应商可以供应多种零件。零件关系的主键为 (53)。

零件号	零件名称	供应商	供应商所在地	单价(元)	库存量
010023	P2	S1	北京市海淀区 58 号	22.80	380
010024	P3	S1	北京市海淀区 58 号	280.00	1350
010022	P1	S2	陕西省西安市雁塔区 2 号	65.60	160
010023	P2	S2	陕西省西安市雁塔区 2 号	28.00	1280
010024	P3	S2	陕西省西安市雁塔区 2 号	260.00	3900
010022	P1	S3	北京市新城区 65 号	66.80	2860
...	...	...	...	...	...

查询各种零件的平均单价、最高单价与最低单价之间差距的 SQL 语句为:

SELECT 零件号, _____ (54)
FROM P
_____ (55);

- (53) A. 零件号, 零件名称 B. 零件号, 供应商
C. 零件号, 供应商所在地 D. 供应商, 供应商所在地
- (54) A. 零件名称, AVG (单价), MAX (单价) - MIN (单价)
B. 供应商, AVG (单价), MAX (单价) - MIN (单价)
C. 零件名称, AVG 单价, MAX 单价 - MIN 单价
D. 供应商, AVG 单价, MAX 单价 - MIN 单价
- (55) A. ORDER BY 供应商 B. ORDER BY 零件号
C. GROUP BY 供应商 D. GROUP BY 零件号

该关系存在冗余以及插入异常和删除异常等问题。为了解决这一问题需要将零件关系分解为 (56)。

- (56) A. P1 (零件号, 零件名称, 单价)、P2 (供应商, 供应商所在地, 库存量)
B. P1 (零件号, 零件名称)、P2 (供应商, 供应商所在地, 单价, 库存量)
C. P1 (零件号, 零件名称)、P2 (零件号, 供应商, 单价, 库存量)、
P3 (供应商, 供应商所在地)
D. P1 (零件号, 零件名称)、P2 (零件号, 单价, 库存量)、
P3 (供应商, 供应商所在地)、P4 (供应商所在地, 库存量)

● 对 n 个元素的有序表 A[1..n] 进行二分 (折半) 查找 (除 2 取商时向下取整), 查找元素 A[i] ($1 \leq i \leq n$) 时, 最多与 A 中的 (57) 个元素进行比较。

- (57) A. n B. $\lfloor \log_2 n \rfloor - 1$ C. n/2 D. $\lfloor \log_2 n \rfloor + 1$

● 设有如下所示的下三角矩阵 $A[0..8,0..8]$ ，将该三角矩阵的非零元素（即行下标不小于列下标的所有元素）按行优先压缩存储在数组 $M[1..m]$ 中，则元素 $A[i,j]$ ($0 \leq i \leq 8, j \leq i$) 存储在数组 M 的 (58) 中。

$$\begin{bmatrix} A_{0,0} & & & & & & & & \\ A_{1,0} & A_{1,1} & & & & & & & \\ . & & . & & & & 0 & & \\ . & & & . & & & & & \\ . & & & & . & & & & \\ A_{7,0} & A_{7,1} & A_{7,2} & \dots & & & A_{7,7} & & \\ A_{8,0} & A_{8,1} & A_{8,2} & A_{8,3} & \dots & & & & A_{8,8} \end{bmatrix}$$

- (58) A. $M[\frac{i(i+1)}{2} + j + 1]$ B. $M[\frac{i(i+1)}{2} + j]$
 C. $M[\frac{i(i-1)}{2} + j]$ D. $M[\frac{i(i-1)}{2} + j + 1]$

● 若用 n 个权值构造一棵最优二叉树（哈夫曼树），则该二叉树的结点总数为 (59)。

- (59) A. $2n$ B. $2n-1$ C. $2n+1$ D. $2n+2$

● 栈是一种按“后进先出”原则进行插入和删除操作的数据结构，因此，(60) 必须用栈。

- (60) A. 实现函数或过程的递归调用及返回处理时
 B. 将一个元素序列进行逆置
 C. 链表结点的申请和释放
 D. 可执行程序装入和卸载

● 对以下四个序列用直接插入排序方法由小到大进行排序时，元素比较次数最少的是 (61)。

- (61) A. 89, 27, 35, 78, 41, 15 B. 27, 35, 41, 16, 89, 70
 C. 15, 27, 46, 40, 64, 85 D. 90, 80, 45, 38, 30, 25

● 对于哈希表，如果将装填因子 α 定义为表中装入的记录数与表的长度之比，那么向表中加入新记录时，(62)。

- (62) A. α 的值随冲突次数的增加而递减 B. α 越大发生冲突的可能性就越大
 C. α 等于 1 时不会再发生冲突 D. α 低于 0.5 时不会发生冲突

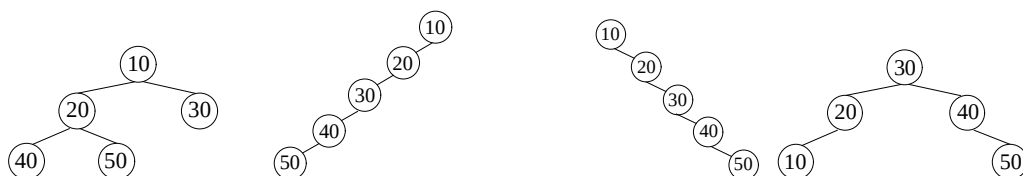
- 用关键字序列 10、20、30、40、50 构造的二叉排序树（二叉查找树）为 (63)。

(63) A.

B.

C.

D.



- 若某算法在问题规模为 n 时，其基本操作的重复次数可由下式表示，则该算法的时间复杂度为 (64)。

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n-1) + n & n > 1 \end{cases}$$

(64) A. $O(n)$

B. $O(n^2)$

C. $O(\log n)$

D. $O(n \log n)$

- 若对一个链表最常用的操作是在末尾插入结点和删除尾结点，则采用仅设尾指针的单向循环链表（不含头结点）时，(65)。

(65) A. 插入和删除操作的时间复杂度都为 $O(1)$

B. 插入和删除操作的时间复杂度都为 $O(n)$

C. 插入操作的时间复杂度为 $O(1)$ ，删除操作的时间复杂度为 $O(n)$

D. 插入操作的时间复杂度为 $O(n)$ ，删除操作的时间复杂度为 $O(1)$

- IP 地址块 222.125.80.128/26 包含了 (66) 个可用主机地址，其中最小地址是 (67)，最大地址是 (68)。

(66) A. 14

B. 30

C. 62

D. 126

(67) A. 222.125.80.128

B. 222.125.80.129

C. 222.125.80.159

D. 222.125.80.160

(68) A. 222.125.80.128

B. 222.125.80.190

C. 222.125.80.192

D. 222.125.80.254

- 以下 HTML 代码中，创建指向邮箱地址的链接正确的是 (69)。

(69) A. `<a href="email:test@test.com">test@test.com</a>`

B. `<a href="mailto:test@test.com">test@test.com</a>`

C. `<a href="mail:test@test.com">test@test.com</a>`

D. `<a href="mailto:test@test.com">test@test.com</a>`

- POP3 服务默认的 TCP 端口号是 (70)。

(70) A. 20

B. 25

C. 80

D. 110

● Observe that for the programmer, as for the chef, the urgency of the patron (顾客) may govern the scheduled completion of the task, but it cannot govern the actual completion. An omelette (煎鸡蛋), promised in two minutes, may appear to be progressing nicely. But when it has not set in two minutes, the customer has two choices—waits or eats it raw. Software customers have had (71) choices.

Now I do not think software (72) have less inherent courage and firmness than chefs, nor than other engineering managers. But false (73) to match the patron's desired date is much more common in our discipline than elsewhere in engineering. It is very (74) to make a vigorous, plausible, and job risking defense of an estimate that is derived by no quantitative method, supported by little data, and certified chiefly by the hunches of the managers.

Clearly two solutions are needed. We need to develop and publicize productivity figures, bug-incidence figures, estimating rules, and so on. The whole profession can only profit from (75) such data. Until estimating is on a sounder basis, individual managers will need to stiffen their backbones and defend their estimates with the assurance that their poor hunches are better than wish derived estimates.

- | | |
|-----------------|-----------------|
| (71) A. no | B. the same |
| C. other | D. lots of |
| (72) A. testers | B. constructors |
| C. managers | D. architects |
| (73) A. tasks | B. jobs |
| C. works | D. scheduling |
| (74) A. easy | B. difficult |
| C. simple | D. painless |
| (75) A. sharing | B. excluding |
| C. omitting | D. ignoring |

全国计算机技术与软件专业技术资格（水平）考试

2010 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题纸

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五和试题六选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面例题，将解答写在答题纸的对应栏内。

例题

2010 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是(1)月(2)日。

因为正确的解答是“5 月 22 日”，故在答题纸的对应栏内写上“5”和“22”（参看下表）。

例题	解答栏
(1)	5
(2)	22

试题一（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某大型企业的数据中心为了集中管理、控制用户对数据的访问并支持大量的连接需求，欲构建数据管理中间件，其主要功能如下：

（1）数据管理员可通过中间件进行用户管理、操作管理和权限管理。用户管理维护用户信息，用户信息（用户名、密码）存储在用户表中；操作管理维护数据实体的标准操作及其所属的后端数据库信息，标准操作和后端数据库信息存放在操作表中；权限管理维护权限表，该表存储用户可执行的操作信息。

（2）中间件验证前端应用提供的用户信息。若验证不通过，返回非法用户信息；若验证通过，中间件将等待前端应用提交操作请求。

（3）前端应用提交操作请求后，中间件先对请求进行格式检查。如果格式不正确，返回格式错误信息；如果格式正确，则进行权限验证（验证用户是否有权执行请求的操作），若用户无权执行该操作，则返回权限不足信息，否则进行连接管理。

（4）连接管理连接相应的后台数据库并提交操作。连接管理先检查是否存在空闲的数据库连接，如果不存在，新建连接；如果存在，则重用连接。

（5）后端数据库执行操作并将结果传给中间件，中间件对收到的操作结果进行处理后，将其返回给前端应用。

现采用结构化方法对系统进行分析与设计，获得如图 1-1 所示的顶层数据流图和图 1-2 所示的 0 层数据流图。

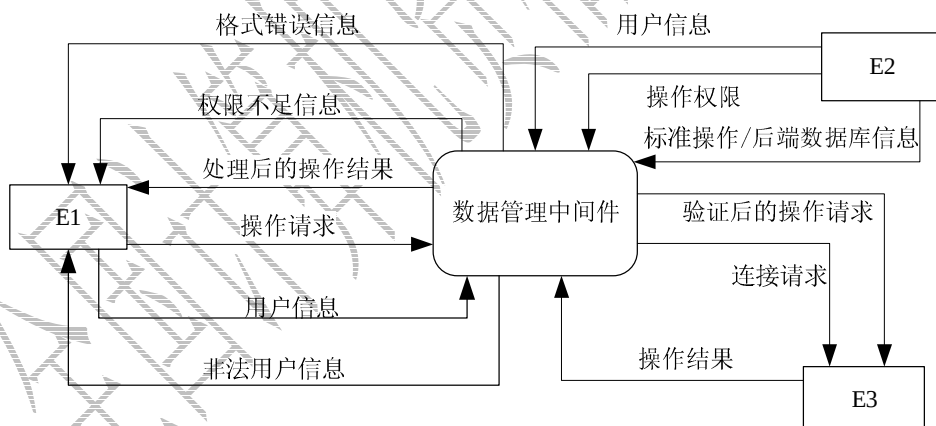


图 1-1 顶层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（3 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D3 的名称。

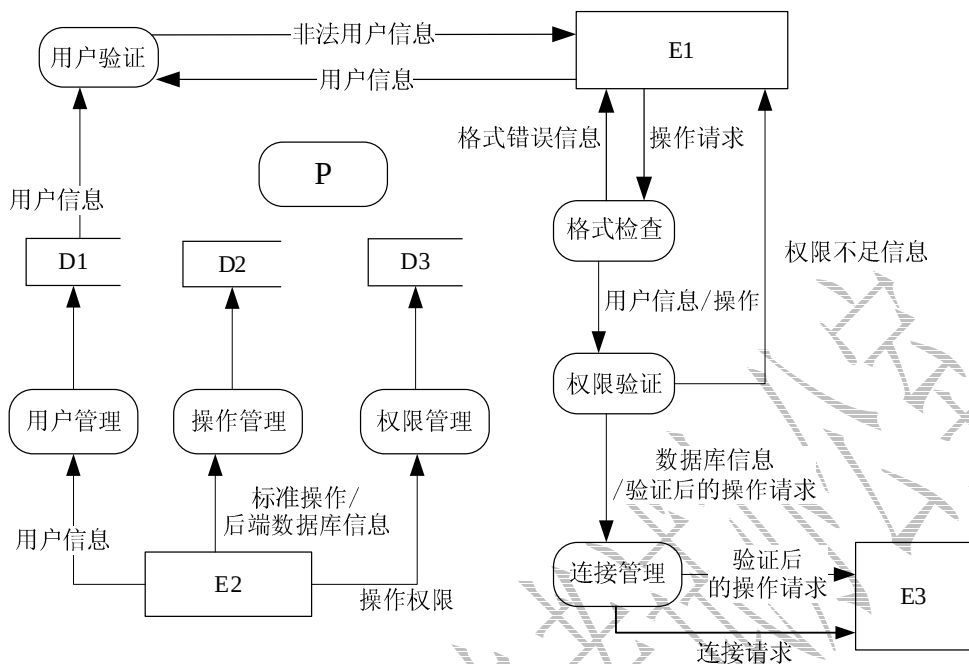


图 1-2 0 层数据流图

【问题 3】（6 分）

给出图 1-2 中加工 P 的名称及其输入、输出流。

	名 称	起 点	终 点
输入流			P
输出流		P	

除加工 P 的输入与输出流外，图 1-2 还缺失了两条数据流，请给出这两条数据流的起点和终点。

起 点	终 点

注：名称使用说明中的词汇，起点和终点均使用图 1-2 中的符号或词汇。

【问题 4】（3 分）

在绘制数据流图时，需要注意加工的绘制。请给出三种在绘制加工的输入、输出时可能出现的错误。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某学校拟开发一套实验管理系统，对各课程的实验安排情况进行管理。

【需求分析】

一个实验室可进行多种类型不同的实验。由于实验室和实验员资源有限，需根据学生人数分批次安排实验室和实验员。一门课程可以为多个班级开设，每个班级每学期可以开设多门课程。一门课程的一种实验可以根据人数、实验室的可容纳人数和实验类型，分批次开设在多个实验室的不同时间段。一个实验室的一次实验可以分配多个实验员负责辅导实验，实验员给出学生的每次实验成绩。

（1）课程信息包括：课程编号、课程名称、实验学时、授课学期和开课的班级等信息；实验信息记录该课程的实验进度信息，包括：实验名、实验类型、学时、安排周次等信息，如表 2-1 所示。

表 2-1 课程及实验信息

课程编号	15054037	课程名称	数字电视原理		实验学时	12
班级	电 0501,信 0501,计 0501	授课院系	机械与电气工程		授课学期	第三学期
序号	实验名		实验类	难度	学时	安排周次
1505403701	音视频 AD-DA 实验		验证性	1	2	3
1505403702	音频编码实验		验证性	2	2	5
1505403703	视频编码实验		演示性	0.5	1	9

（2）以课程为单位制定实验安排计划信息，包括：实验地点，实验时间、实验员等信息，实验计划如表 2-2 所示。

表 2-2 实验安排计划

课程编号	15054037	课程名称	数字电视原理	安排学期	2009 年秋	总人数	220
实验编号	实验名	实验员	实验时间	地点	批次号	人数	
1505403701	音视频 AD-DA 实验	盛×，陈×	第 3 周周四晚上	实验三楼 310	1	60	
1505403701	音视频 AD-DA 实验	盛×，陈×	第 3 周周四晚上	实验三楼 310	2	60	
1505403701	音视频 AD-DA 实验	吴×，刘×	第 3 周周五晚上	实验三楼 311	3	60	
1505403701	音视频 AD-DA 实验	吴×	第 3 周周五晚上	实验三楼 311	4	40	
1505403702	音频编码实验	盛×，刘×	第 5 周一下午	实验四楼 410	1	70	

（3）由实验员给出每个学生每次实验的成绩，包括：实验名、学号、姓名、班级、实验成绩等信息，实验成绩如表 2-3 所示。

表 2-3 实验成绩

实验员： 盛×

实验名	音视频 AD-DA 实验	课程名	数字电视原理
学号	姓名	班级	实验成绩
030501001	陈民	信 0501	87
030501002	刘志	信 0501	78
040501001	张勤	计 0501	86

(4) 学生的实验课程总成绩根据每次实验的成绩以及每次实验的难度来计算。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

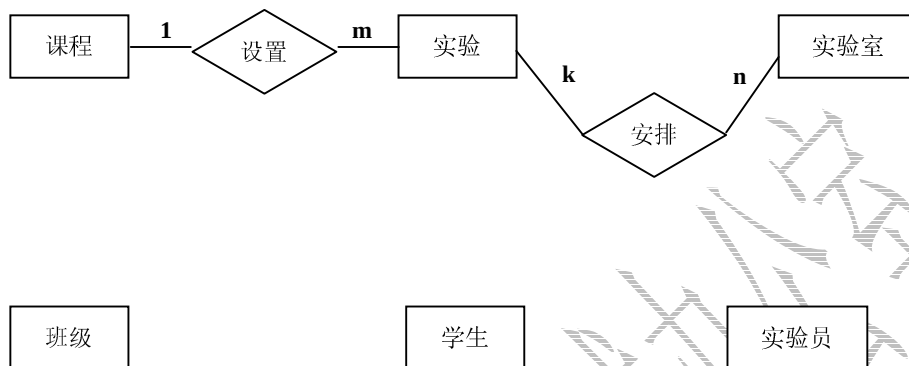


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

课程（课程编号，课程名称，授课院系，实验学时）

班级（班级号，专业，所属系）

开课情况（_____（1）_____，授课学期）

实验（_____（2）_____，实验类型，难度，学时，安排周次）

实验计划（_____（3）_____，实验时间，人数）

实验员（_____（4）_____，级别）

实验室（实验室编号，地点，开放时间，可容纳人数，实验类型）

学生（_____（5）_____，姓名，年龄，性别）

实验成绩（_____（6）_____，实验成绩，评分实验员）

【问题 1】（6 分）

补充图 2-1 中的联系和联系的类型。

【问题 2】（6 分）

根据图 2-1，将逻辑结构设计阶段生成的关系模式中的空（1）~（6）补充完整并用下划线指出这六个关系模式的主键。

【问题 3】（3 分）

如果需要记录课程的授课教师，新增加“授课教师”实体。请对图 2-1 进行修改，画出修改后的实体间联系和联系的类型。

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某运输公司决定为新的售票机开发车票销售的控制软件。图 3-1 给出了售票机的面板示意图以及相关的控制部件。

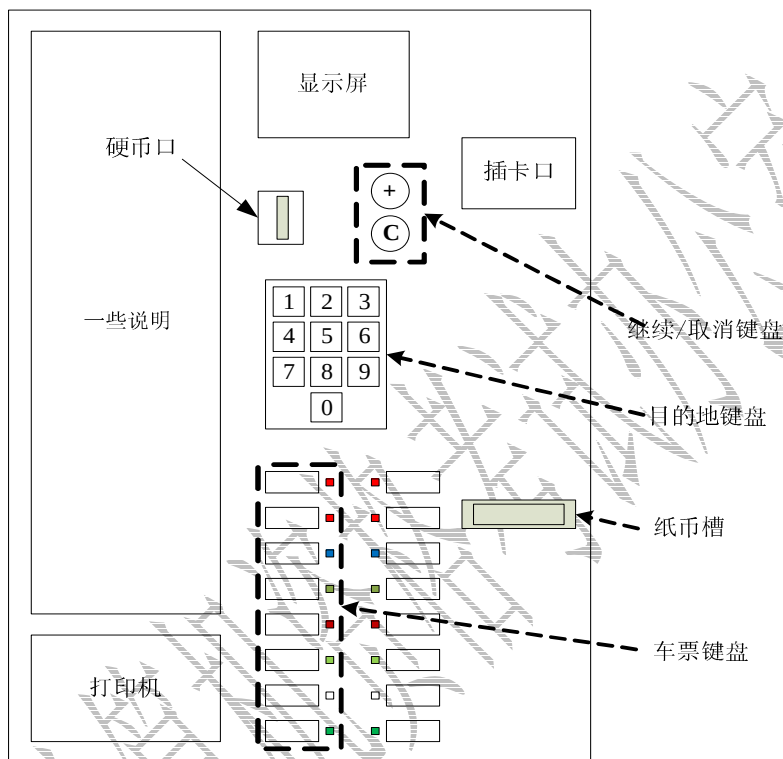


图 3-1 售票机面板示意图

售票机相关部件的作用如下所述：

- (1) 目的地键盘用来输入行程目的地的代码（例如，200 表示总站）。
- (2) 乘客可以通过车票键盘选择车票种类（单程票、多次往返票和座席种类）。
- (3) 继续/取消键盘上的取消按钮用于取消购票过程，继续按钮允许乘客连续购买多张票。
- (4) 显示屏显示所有的系统输出和用户提示信息。
- (5) 插卡口接受 MCard（现金卡），硬币口和纸币槽接受现金。
- (6) 打印机用于输出车票。

假设乘客总是支付恰好需要的金额而无需找零，售票机的维护工作（取回现金、放入空白车票等）由服务技术人员完成。

系统采用面向对象方法开发，使用 UML 进行建模。系统的顶层用例图和类图分别如图 3-2 和图 3-3 所示。

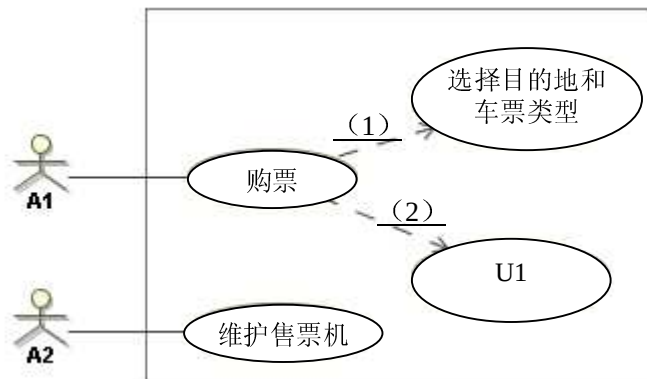


图 3-2 顶层用例图

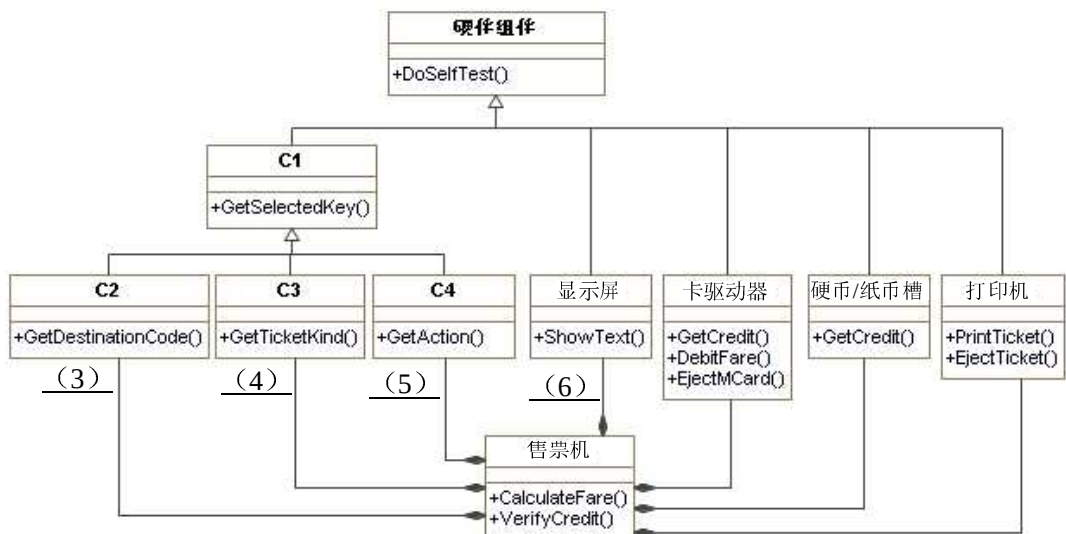


图 3-3 类图

【问题 1】(5 分)

根据说明中的描述，给出图 3-2 中 A1 和 A2 所对应的参与者，U1 所对应的用例，以及 (1)、(2) 处所对应的关系。

【问题 2】(7 分)

根据说明中的描述，给出图 3-3 中缺少的 C1~C4 所对应的类名以及 (3) ~ (6) 处所对应的多重度。

【问题 3】(3 分)

图 3-3 中的类图设计采用了中介者 (Mediator) 设计模式，请说明该模式的内涵。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答写在答题纸的对应栏内。

【说明】

对有向图进行拓扑排序的方法是：

- (1) 初始时拓扑序列为空；
- (2) 任意选择一个入度为 0 的顶点，将其放入拓扑序列中，同时从图中删除该顶点以及从该顶点出发的弧；
- (3) 重复 (2)，直到不存在入度为 0 的顶点为止（若所有顶点都进入拓扑序列则完成拓扑排序，否则由于有向图中存在回路无法完成拓扑排序）。

函数 `int* TopSort(LinkedDigraph G)` 的功能是对有向图 `G` 中的顶点进行拓扑排序，返回拓扑序列中的顶点编号序列，若不能完成拓扑排序，则返回空指针。其中，图 `G` 中的顶点从 1 开始依次编号，顶点序列为 `v1, v2, ..., vn`，图 `G` 采用邻接表表示，其数据类型定义如下：

```
#define MAXVNUM 50          /* 最大顶点数 */
typedef struct ArcNode{      /* 表结点类型 */
    int adjvex;              /* 邻接顶点编号 */
    struct ArcNode *nextarc; /* 指示下一个邻接顶点 */
}ArcNode;
typedef struct AdjList{      /* 头结点类型 */
    char vdata;              /* 顶点的数据信息 */
    ArcNode *firstarc;       /* 指向邻接表的第一个表结点 */
}AdjList;
typedef struct LinkedDigraph{ /* 图的类型 */
    int n;                   /* 图中顶点个数 */
    AdjList Vhead[MAXVNUM]; /* 所有顶点的头结点数组 */
}LinkedDigraph;
```

例如，某有向图 `G` 如图 4-1 所示，其邻接表如图 4-2 所示。

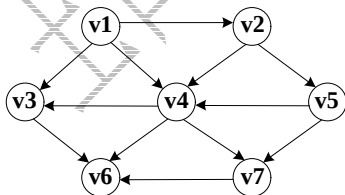


图 4-1 有向图 G

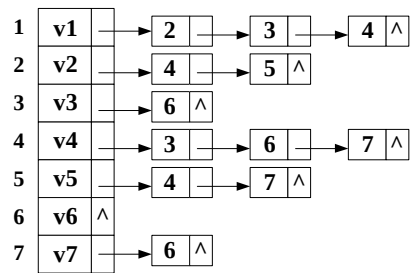


图 4-2 有向图 G 的邻接表示意图

函数 TopSort 中用到了队列结构 (Queue 的定义省略)，实现队列基本操作的函数原型如下表所示：

函数原型	说 明
void InitQueue(Queue *Q)	初始化队列（构造一个空队列）
bool IsEmpty(Queue Q)	判断队列是否为空，若是则返回 true, 否则返回 false
void EnQueue(Queue *Q, int e)	元素入队列
void DeQueue(Queue *Q, int *p)	元素出队列

【C 代码】

```
int *TopSort(LinkedDigraph G) {
    ArcNode *p;          /* 临时指针，指示表结点 */
    Queue Q;             /* 临时队列，保存入度为 0 的顶点编号 */
    int k = 0;           /* 临时变量，用作数组元素的下标 */
    int j = 0, w = 0;     /* 临时变量，用作顶点编号 */
    int *topOrder, *inDegree;
    topOrder = (int *)malloc((G.n+1) * sizeof(int)); /* 存储拓扑序列中的顶点编号 */
    inDegree = (int *)malloc((G.n+1) * sizeof(int)); /* 存储图 G 中各顶点的入度 */
    if (!inDegree || !topOrder) return NULL;

    ____ (1) ____;      /* 构造一个空队列 */

    for (j = 1; j <= G.n; j++) { /* 初始化 */
        topOrder[j] = 0;    inDegree[j] = 0;
    }

    for (j = 1; j <= G.n; j++) /* 求图 G 中各顶点的入度 */
        for (p = G.Vhead[j].firstarc; p; p = p->nextarc)
            inDegree[p->adjvex] += 1;

    for (j = 1; j <= G.n; j++) /* 将图 G 中入度为 0 的顶点保存在队列中 */
        if (0 == inDegree[j]) EnQueue(&Q, j);
}
```

```

while (!IsEmpty(Q)) {
    _____ (2) _____;    /* 队头顶点出队列并用 w 保存该顶点的编号 */
    topOrder[k++] = w;
/* 将顶点 w 的所有邻接顶点的入度减 1 (模拟删除顶点 w 及从该顶点出发的弧的操作) */
    for(p = G.Vhead[w].firstarc; p; p = p->nextarc) {
        _____ (3) _____ -= 1;
        if (0 == _____ (4) _____)    EnQueue(&Q, p->adjvex);
    }/* for */
}/* while */

free(inDegree);

if (_____ (5) _____)
    return NULL;
return topOrder;
} /*TopSort*/

```

【问题 1】(9 分)

根据以上说明和 C 代码，填充 C 代码中的空 (1) ~ (5)。

【问题 2】(2 分)

对于图 4-1 所示的有向图 G，写出函数 TopSort 执行后得到的拓扑序列。若将函数 TopSort 中的队列改为栈，写出函数 TopSort 执行后得到的拓扑序列。

【问题 3】(4 分)

设某有向无环图的顶点个数为 n、弧数为 e，那么用邻接表存储该图时，实现上述拓扑排序算法的函数 TopSort 的时间复杂度是_____ (6) _____。

若有向图采用邻接矩阵表示（例如，图 4-1 所示有向图的邻接矩阵如图 4-3 所示），且将函数 TopSort 中有关邻接表的操作修改为针对邻接矩阵的操作，那么对于有 n 个顶点、e 条弧的有向无环图，实现上述拓扑排序算法的时间复杂度是_____ (7) _____。

	v1	v2	v3	v4	v5	v6	v7
v1	0	1	1	1	0	0	0
v2	0	0	0	1	1	0	0
v3	0	0	0	0	0	1	0
v4	0	0	1	0	0	1	1
v5	0	0	0	1	0	0	1
v6	0	0	0	0	0	0	0
v7	0	0	0	0	0	1	0

图 4-3 有向图 G 的邻接矩阵

从下列的 2 道试题（试题五和试题六）中任选 1 道解答。
如果解答的试题数超过 1 道，则题号小的 1 道解答有效。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某软件公司现欲开发一款飞机飞行模拟系统，该系统主要模拟不同种类飞机的飞行特征与起飞特征。需要模拟的飞机种类及其特征如表 5-1 所示。

表 5-1

飞机种类	起飞特征	飞行特征
直升机 (Helicopter)	垂直起飞 (VerticalTakeOff)	亚音速飞行 (SubSonicFly)
客机 (AirPlane)	长距离起飞 (LongDistanceTakeOff)	亚音速飞行 (SubSonicFly)
歼击机 (Fighter)	长距离起飞 (LongDistanceTakeOff)	超音速飞行 (SuperSonicFly)
鹞式战斗机 (Harrier)	垂直起飞 (VerticalTakeOff)	超音速飞行 (SuperSonicFly)

为支持将来模拟更多种类的飞机，采用策略设计模式（Strategy）设计的类图如图 5-1 所示。

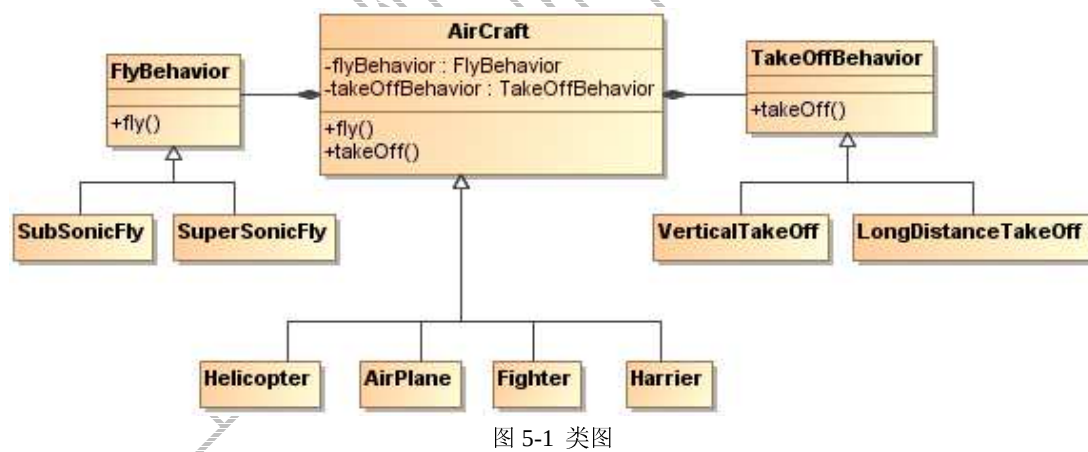


图 5-1 中，AirCraft 为抽象类，描述了抽象的飞机，而类 Helicopter、AirPlane、Fighter 和 Harrier 分别描述具体的飞机种类，方法 fly() 和 takeOff() 分别表示不同飞机都具有飞行特征和起飞特征；类 FlyBehavior 与 TakeOffBehavior 为抽象类，分别用于表示抽象的飞行行为与起飞行为；类 SubSonicFly 与 SuperSonicFly 分别描述亚音速飞行和超音速飞行的行为；类 VerticalTakeOff 与 LongDistanceTakeOff 分别描述垂直起飞与长距离起飞的行为。

【C++代码】

```
#include<iostream>
using namespace std;
class FlyBehavior {
    public: virtual void fly() = 0;
};
class SubSonicFly:public FlyBehavior{
    public: void fly(){ cout << "亚音速飞行 !" << endl; }
};
class SuperSonicFly:public FlyBehavior{
    public: void fly(){ cout << "超音速飞行 !" << endl; }
};

class TakeOffBehavior {
    public: virtual void takeOff() = 0;
};
class VerticalTakeOff:public TakeOffBehavior{
    public: void takeOff(){ cout << "垂直起飞 !" << endl; }
};
class LongDistanceTakeOff:public TakeOffBehavior{
    public: void takeOff (){ cout << "长距离起飞 !" << endl; }
};

class AirCraft{
protected:
    (1) ;
    (2) ;
public:
    void fly(){ (3) ; }
    void takeOff() { (4) ; };
};
class Helicopter: public AirCraft {
public:
    Helicopter (){
        flyBehavior = new (5) ;
        takeOffBehavior = new (6) ;
    }
    (7) {
        if(!flyBehavior) delete flyBehavior;
        if(!takeOffBehavior) delete takeOffBehavior;
    }
};
//其它代码省略
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某软件公司现欲开发一款飞机飞行模拟系统，该系统主要模拟不同种类飞机的飞行特征与起飞特征。需要模拟的飞机种类及其特征如表 6-1 所示。

表 6-1

飞机种类	起飞特征	飞行特征
直升机 (Helicopter)	垂直起飞 (VerticalTakeOff)	亚音速飞行 (SubSonicFly)
客机 (AirPlane)	长距离起飞 (LongDistanceTakeOff)	亚音速飞行 (SubSonicFly)
歼击机 (Fighter)	长距离起飞 (LongDistanceTakeOff)	超音速飞行 (SuperSonicFly)
鹞式战斗机 (Harrier)	垂直起飞 (VerticalTakeOff)	超音速飞行 (SuperSonicFly)

为支持将来模拟更多种类的飞机，采用策略设计模式 (Strategy) 设计的类图如图 6-1 所示。

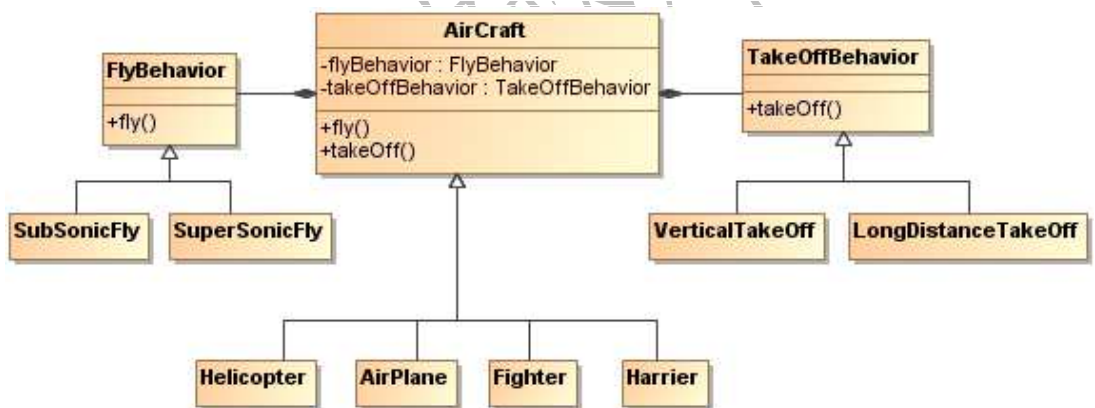


图 6-1 类图

图 6-1 中，**Aircraft** 为抽象类，描述了抽象的飞机，而类 **Helicopter**、**AirPlane**、**Fighter** 和 **Harrier** 分别描述具体的飞机种类，方法 `fly()` 和 `takeOff()` 分别表示不同飞机都具有飞行特征和起飞特征；类 **FlyBehavior** 与 **TakeOffBehavior** 为抽象类，分别用于表示抽象的飞行行为与起飞行为；类 **SubSonicFly** 与 **SuperSonicFly** 分别描述亚音速飞行和超音速飞行的行为；类 **VerticalTakeOff** 与 **LongDistanceTakeOff** 分别描述垂直起飞与长距离起飞的行为。

【Java 代码】

```
interface FlyBehavior {
    public void fly();
};

class SubSonicFly implements FlyBehavior{
    public void fly(){ System.out.println("亚音速飞行 !"); }
};

class SuperSonicFly implements FlyBehavior{
    public void fly(){ System.out.println("超音速飞行 !"); }
};

interface TakeOffBehavior {
    public void takeOff();
};

class VerticalTakeOff implements TakeOffBehavior {
    public void takeOff (){ System.out.println("垂直起飞 !"); }
};

class LongDistanceTakeOff implements TakeOffBehavior {
    public void takeOff(){ System.out.println("长距离起飞 !"); }
};

abstract class AirCraft {
    protected ____ (1) ____;
    protected ____ (2) ____;
    public void fly(){ ____ (3) ____; }
    public void takeOff() { ____ (4) ____; };
};

class Helicopter ____ (5) ____ AirCraft{
    public Helicopter (){
        flyBehavior = new ____ (6) ____;
        takeOffBehavior = new ____ (7) ____;
    }
};

//其它代码省略
```

2004 年上半年软件设计师上午试题答案

(1)	B	(16)	D	(31)	B	(46)	B	(61)	B
(2)	B	(17)	D	(32)	D	(47)	C	(62)	D
(3)	D	(18)	D	(33)	C	(48)	C	(63)	D
(4)	D	(19)	B	(34)	B	(49)	D	(64)	C
(5)	C	(20)	A	(35)	A	(50)	B	(65)	C
(6)	D	(21)	C	(36)	A	(51)	A	(66)	B
(7)	B	(22)	B	(37)	D	(52)	B	(67)	C
(8)	D	(23)	C	(38)	A	(53)	B	(68)	D
(9)	C	(24)	B	(39)	B	(54)	D	(69)	C
(10)	C	(25)	D	(40)	B	(55)	D	(70)	C
(11)	B	(26)	A	(41)	A	(56)	C	(71)	C
(12)	A	(27)	C	(42)	B	(57)	D	(72)	A
(13)	B	(28)	C	(43)	B	(58)	A	(73)	D
(14)	C	(29)	A	(44)	B	(59)	B	(74)	A
(15)	D	(30)	C	(45)	B	(60)	D	(75)	B

2004 年上半年软件设计师下午试题答案

试题一

[问题 1]

A: 传感器: B: 报警器

[问题 2]

监控传感器、显示信息和状态

[问题 3]

1、“读传感器”添加输入数据流“传感器状态”

2、“显示格式”添加输出数据流“TV 信号”

3、“拨号”添加输出数据流“电话拨号”

[问题 4]

逻辑数据流图说明应该具有哪些加工,而不关心这些加工是如何实现的:物理数据流图则要说明加工是如何实现的。

试题二

[问题 1]

(1)1 (2)col (3)row (4)2 (5)col (6)row (7)k

[问题 2]

判断条件 1: b

判断条件 2: e

判断条件 3: f

试题三

[问题 1]

A: 浏览客户信息 B: 修改个人信息 C: 登录 D: 删除客户信息

[问题 2]

(1)1 (2)0..* (3)0..1 (4)0..1

[问题 3]

(1)4 种关系的含义:

依赖表示类之间的使用关系。

概括表示一般类和特殊类之间的关系。

关联和聚集都表示实例之间的结构关系。

(2)关联和聚集的区别:

关联指明一个类的对象与另一类的对象间的联系:两个类之间的关联表示了两个同等地位类之间的结构关系,这两个类在概念上是同级别的。

聚集是一种特殊的关联,它表示整体/部分关系。

试题四

(1)JMP LAST

(2)SUB GR0, WDT, GR1

(3)ST GR2, BTASC, GR1

(4)ADD GR0, C48

(5)ST GR0, BTASC, GR1

试题五

- (1) `p&& p->data !=e` 或 `p&&(*p).data !=e`
- (2) `p->Lchild` 或 `(*p).Lchild`
- (3) `s->Rchild` 或 `(*s).Rchild`
- (4) `p->Lchild` 或 `(*p).Lchild`
- (5) `p==pp->Lchild` 或 `p (*pp).Lchild`

试题六

- (1) `classArray<T>`
- (2) `new T[iRsz*iCs2]`
- (3) `throw 'e'`
- (4) `tBody, iCurrentRow=i`
- (5) `tBody(iRsz, iCs2)`

2004 年下半年软件设计师上午试题答案

(1)	C	(16)	C	(31)	B	(46)	C	(61)	B
(2)	B	(17)	B	(32)	C	(47)	A	(62)	D
(3)	C	(18)	B	(33)	A	(48)	D	(63)	C
(4)	D	(19)	B	(34)	B	(49)	C	(64)	A
(5)	B	(20)	D	(35)	C	(50)	B	(65)	D
(6)	C	(21)	D	(36)	B	(51)	A	(66)	A
(7)	D	(22)	A	(37)	B	(52)	B	(67)	A
(8)	B	(23)	B	(38)	D	(53)	D	(68)	D
(9)	B	(24)	A	(39)	B	(54)	A	(69)	B
(10)	D	(25)	C	(40)	B	(55)	C	(70)	B
(11)	C	(26)	B	(41)	D	(56)	B	(71)	C
(12)	A	(27)	C	(42)	A	(57)	B	(72)	A
(13)	A	(28)	B	(43)	D	(58)	C	(73)	B
(14)	C	(29)	A	(44)	C	(59)	A	(74)	C
(15)	A	(30)	D	(45)	A	(60)	A	(75)	A

2004 年下半年软件设计师下午试题答案

试题一

- [问题 1] 起点：读者文件 终点，登记读者信息或 3
 起点：处理查询请求或 2
 终点：读者文件
- [问题 2] 起点：图书目录文件 终点：图书信息查询或 2.2
 起点：借书文件 终点：读者信息查询或 2.1
 起点：借书文件 终点：图书信息查询或 2.2
- [问题 3]
(1) [入库单 | 借书单 | 还书单 | 注销单]
(2) 分类目录号+书名+作者+价格+数量+购书日期

试题二

- [问题 1]
Customers(cid, cname, address, cardnum), 主码: cid
orders(Ordernum, Orderdate, cid) 主码: Ordernum ;外码: cid
Books<bid, title, author, qty_in_stock, year_publicshed, price>,
主码: bid
Orderlist(bid, ordernum, qty, ship_date) 其中 bid 和 ordernum 是主码
也是外码
注：以上四个关系模式和每个模式中的属性可按任意次序书写。

- [问题 2]
(1) PRIMARY KEY(cid)
(2) UNIQUE(cardnum)
注：(1) 和 (2) 的次序可以颠倒。

- [问题 3] (4 分)
(3) not in
(4) c
(5) C. ordernum

试题三(共 15 分)

- [问题 1]
锁的编号、安全级别、锁的当前状态
- [问题 2]
(1) 中断事件 (2) 读取用户指纹 (3) 读取用户开锁权限
(4) 读取锁的安全级别 (5) 判断用户是否有权限开锁或用户是否可以开锁

- [问题 3]
组装和聚集都表示实例之间的整体/部分关系。组装是聚集的一种形式。
聚集是概念性的，只是区分整体与部分。
组装具有很强的归属关系，而且整体与部分的对象生存周期是一致的。
或者回答：如果没有成分对象，组装对象也不存在；在任何时候，每个给定的成分对象只能是组装对象的组成部分。

试题四

[问题 1]

(1) $P(S_1)$ (2) $V(S_2)$ (3) $P(S_2)$ (4) $V(S_1)$

[问题 2]

(5) 1, 表示允许同时对缓冲区进行写操作的进程数量 (0.5 分) (6) 1, 表示允许同时对缓冲区进行读操作的进程数量 (0.5 分)

解法 1: $P(S_A)$ 插入位置 (b), $V(S_A)$ 插入位置 (f), $P(S_B)$ 插入位置 (h) (若将 $P(S_B)$ 插入位置 (g), 则扣 0.5 分), $V(S_B)$ 插入位置 (k);

解法 2: $P(S_B)$ 插入位置 (b), $V(S_B)$ 插入位置 (f), $P(S_A)$ 插入位置 (h) (若将 $P(S_A)$ 插入位置 (g), 则扣 0.5 分), $V(S_A)$ 插入位置 (k);

	进程 A	进程 B
t1	阻塞(等待 I/O 结束)	运行
t2	阻塞(等待 I/O 结束)	阻塞(等待 R 资源)
t3	运行	阻塞(等待 I/O 结束)
t4	就绪	运行

试题五

- (1) $\text{indegree}[p \rightarrow \text{adjvex}]++$, 及其等价形式
- (2) $\text{Stack}[\text{top}--]$, 及其等价形式
- (3) $\text{indegree}[p \rightarrow \text{adjvex}]--$, 及其等价形式
- (4) $\text{ve}[w] + p \rightarrow \text{weight} > \text{ve}[p \rightarrow \text{adjvex}]$, 及其等价形式
- (5) $\text{ve}[w]$, 及其等价形式

试题六(共 15 分)

- (1) `private`
- (2) `Configure *Configure: : _inStance`
- (3) `new Configure`
- (4) `_inStance`
- (5) `Configure::Instance()`

试题七(共 15 分)

- (1) `try`
- (2) `Exception e` 或者: `EmptyQueueException e`, `e` 是对象名, 可用任意合法标识符替换 (3 分)
- (3) `throws EmptyQueueException`
- (4) `throw (new EmptyQueueException())`
- (5) `Exception`

2005 年上半年软件设计师上午试题答案

(1)	C	(16)	B	(31)	A	(46)	B	(61)	C
(2)	D	(17)	A	(32)	C	(47)	D	(62)	C
(3)	C	(18)	D	(33)	B	(48)	B	(63)	A
(4)	A	(19)	D	(34)	C	(49)	D	(64)	B
(5)	C	(20)	C	(35)	D	(50)	C	(65)	B
(6)	D	(21)	D	(36)	A	(51)	D	(66)	B
(7)	B	(22)	A	(37)	D	(52)	C	(67)	C
(8)	C	(23)	A	(38)	D	(53)	C	(68)	B
(9)	B	(24)	C	(39)	B	(54)	D	(69)	D
(10)	C	(25)	D	(40)	D	(55)	D	(70)	A
(11)	B	(26)	D	(41)	C	(56)	D	(71)	A
(12)	B	(27)	A	(42)	B	(57)	C	(72)	C
(13)	C	(28)	B	(43)	B	(58)	D	(73)	B
(14)	D	(29)	A	(44)	C	(59)	C	(74)	D
(15)	C	(30)	C	(45)	C	(60)	D	(75)	A

2005 年上半年软件设计师下午试题答案

试题一

[问题 1]

(1) 起点：学生住宿服务系统 终点：房主

数据流名：费用信息 或 交纳的费用 或 费用

(2) 起点：房屋文件 终点：查询房屋 或 4

[问题 2]

房主文件 和 学生文件

[问题 3]

姓名+现住址+电话号码+出生日期+性别

试题二

[问题 1]

(1) 主键：职员号 外键：部门号

(2) 主键：部门号，或部门名 外键：负责人代码

[问题 2]

(a) PRIMARY KEY

(b) FOREIGN KEY (部门号)，REFERENCES 部门

(c) Count (*), Sum (月工资), Avg (月工资)

(d) GROUP by 部门号 HAVING count (*)

[问题 3]

(1) 该行不能插入“职员”关系，它违反了用户定义完整性中月工资的取值范围必须大于等于 1000 元，小于等于 8000 元。

(2) 该行不能插入“职员”关系，因为职员号“60802”在表 2-1 中已存在，违反了实体完整性中主键必须唯一区分关系中的每一个属性。

(3) 该行可以插入“职员”关系，尽管部门号、电话和办公室为空，但是它表示该职员没有分配到某个部门。

[问题 4]

“职员”关系模式主要的问题是：

数据冗余问题。因为某部门的职员人数有多少，其办公室和电话将要重复存入多少。

数据修改不一致问题。因为某部门的办公室变了可能会导致某些职员的修改了，某些职员的未修改。

将关系模式修改为：职员（职员号，职员姓名，月工资，部门号）

部门（部门号，部门名，负责人代码，任职时间，办公室，电话）

试题三

(1) [i] 或 i

(2) [c] 或 c

(3) [d] 或 d

- (4) [a] 或 a
- (5) [h] 或 h

试题四

- (1) $k \geq N$ 或 $k = N$
- (2) $\text{cose} + c[k][i] < \text{mincost}$
- (3) i
- (4) k+1
- (5) $\text{worker}[i] = 0$

试题五

- (1) Shape
- (2) Xcircle
- (3) DisplayIt()
- (4) Shape *
- (5) getShapeInstance(type)
- (6) delete s

试题六

- (1) implements Shape
- (2) XCircle
- (3) XCircle()
- (4) displayIt()
- (5) Shape
- (6) getShapeInstance(type)

试题七

- (1) Implements IShape
- (2) XCircle
- (3) New XCircle
- (4) Get IShape_Color
- (5) Let Ishape_Color
- (6) bridged.displayIt

2005 年下半年软件设计师上午试题答案

(1)	B	(16)	B	(31)	C	(46)	D	(61)	D
(2)	D	(17)	B	(32)	D	(47)	A	(62)	A
(3)	C	(18)	C	(33)	B	(48)	B	(63)	B
(4)	C	(19)	B	(34)	C	(49)	D	(64)	B
(5)	C	(20)	A	(35)	B	(50)	A	(65)	B
(6)	A	(21)	D	(36)	A	(51)	B	(66)	C
(7)	A	(22)	A	(37)	D	(52)	C	(67)	B
(8)	C	(23)	D	(38)	B	(53)	C	(68)	A
(9)	C	(24)	C	(39)	A	(54)	B	(69)	D
(10)	C	(25)	A	(40)	D	(55)	D	(70)	A
(11)	C	(26)	C	(41)	C	(56)	B	(71)	C
(12)	B	(27)	D	(42)	C	(57)	B	(72)	B
(13)	C	(28)	C	(43)	D	(58)	C	(73)	A
(14)	D	(29)	D	(44)	B	(59)	D	(74)	D
(15)	A	(30)	C	(45)	A	(60)	A	(75)	A

2005 年下半年软件设计师下午试题答案

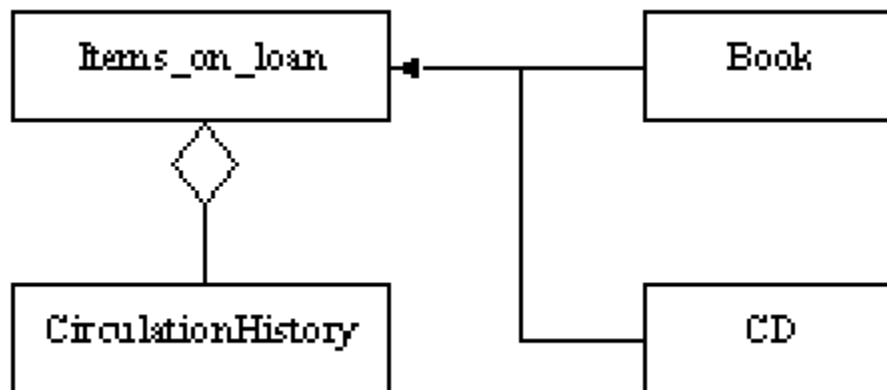
试题一

[问题 1]

属性: title

方法: Reference Title

[问题 2]



[问题 3]

(1) 1

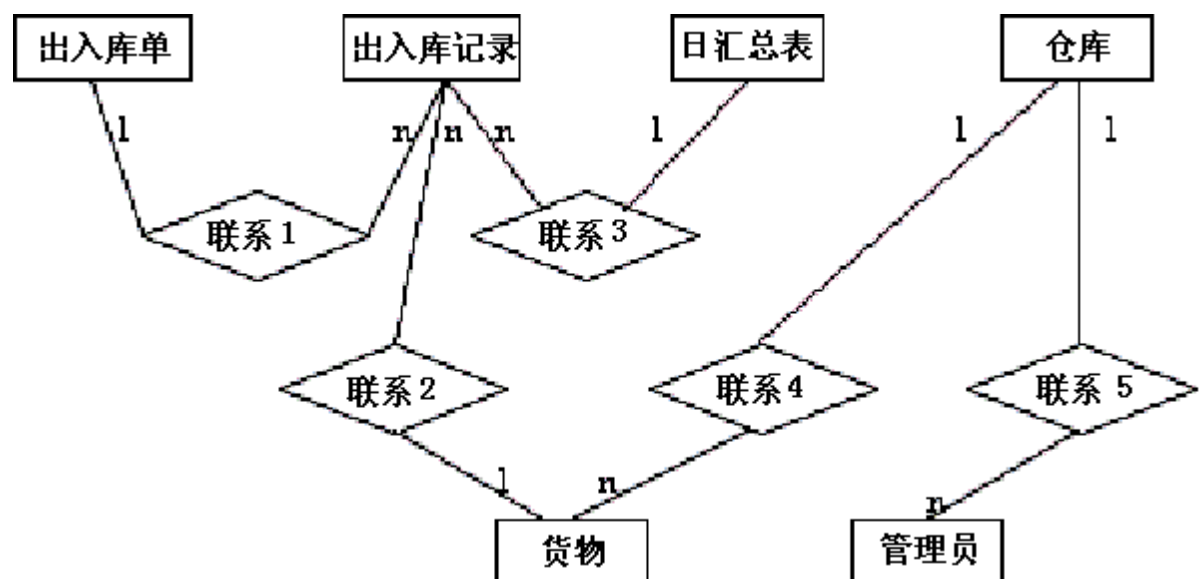
(2) 图书和唱碟种类数

试题二

[问题 1]

货物编码, 货物名称, 单价, 仓库号, 库存量

[问题 2]



[问题 3]

出入库单的主键：流水号

出入库记录的主键：流水号，货物编码

日汇总表的主键：日期，货物编码

仓库的主键：仓库号

管理员的主键：管理员号

货物的主键：货物编码

试题三

问题 1	方框 a	A 的私钥
	方框 b	A 的公钥
问题 2	方框 c	摘要算法
	方框 d	会话密钥
	方框 e	会话密钥
	方框 f	摘要算法
问题 3	对称密钥体系的加解密效率非常高，适合于加解密大量的数据，因此可用于加密消息。公钥密码体系的加解密效率较低，更适合用于加密少量数据的场合，因此可用于加密消息摘要和会话密钥。	
问题 4	[b]，或根据输入数据获取输出数据的时间非常短	
	[e]，或根据输出数据无法还原出输入数据	

试题四

- (1) $\text{Index} = \text{NewElemKey} \% P$ ，或 $\text{Index} = \text{Hash}(\text{NewElemKey})$
- (2) $i < \text{ITEMS}$
- (3) $\text{front} = \&\text{Bucket}[\text{Index}]$ ，或 $\text{front} = \text{Bucket} + \text{Index}$
- (4) $k = \text{ITEMS}$ ，或 $k \geq \text{ITEMS}$
- (5) $t = \text{NULL}$
- (6) $\text{front} \rightarrow \text{Link} = s$

试题五

- (1) `class OfficeDoc`
- (2) `OfficeDoc`
- (3) `virtual`
- (4) `update(this)`
- (5) `attach(this)`

试题六

- (1) `Subject subject`
- (2) `Observers ()`
- (3) `update (this)`
- (4) `Observer`
- (5) `Subject subject`

试题七

- (1) * func
- (2) struct DocExplorer *
- (3) doc->index-1
- (4) doc, doc->myObs[loop]
- (5) notifyObs (&doc)

2006 年上半年软件设计师上午试题答案

(1)	D	(16)	D	(31)	C	(46)	D	(61)	A
(2)	A	(17)	B	(32)	A	(47)	C	(62)	C
(3)	D	(18)	C	(33)	A	(48)	B	(63)	A
(4)	D	(19)	D	(34)	B	(49)	C	(64)	D
(5)	A	(20)	A	(35)	C	(50)	B	(65)	B
(6)	C	(21)	B	(36)	B	(51)	A	(66)	B
(7)	A	(22)	C	(37)	B	(52)	B	(67)	C
(8)	D	(23)	D	(38)	B	(53)	A	(68)	A
(9)	C	(24)	A	(39)	D	(54)	C	(69)	D
(10)	A	(25)	B	(40)	A	(55)	C	(70)	B
(11)	B	(26)	D	(41)	B	(56)	D	(71)	A
(12)	B	(27)	D	(42)	B	(57)	C	(72)	C
(13)	A	(28)	C	(43)	A	(58)	D	(73)	B
(14)	C	(29)	C	(44)	B	(59)	B	(74)	A
(15)	C	(30)	D	(45)	B	(60)	A	(75)	D

2006 年上半年软件设计师下午试题答案

试题一

[问题 1]

外部实体：(选课) 学生、(任课) 老师

数据存储：作业成绩统计文件

[问题 2]

- (1) (选课) 学生
- (2) (选课) 学生
- (3) (选课) 学生
- (4) (选课) 学生
- (5) 作业成绩
- (6) DB
- (7) 作业成绩统计文件
- (8) 作业成绩
- (9) (任课) 老师
- (10) DB
- (11) 作业
- (12) (选课) 学生
- (13) (任课) 老师

[问题 3]

- 错误 1：外部实体 A 和 B 之间不能存在数据流
- 错误 2：外部实体 A 和数据存储 H 之间不能存在数据流
- 错误 3：加工 2 的输入/输出数据流名字相同
- 错误 4：加工 4 只有输入没有输出
- 错误 5：加工 5 只有输出，没有输入。

试题二

[问题 1]

Customer 的属性：客户标识

Person 的属性：姓名、住宅电话、email

[问题 2]

- (1) 1 (2) 0..* (3) 1 (4) 1 (5) 1 (6) 1..*

[问题 3]

CustomerInformationSystem 的方法：

addCustomer, getCustomer, removeCustomer。

InstitutionalCustomer 的方法：

addContact, getContact, removeContact。

试题三

[问题 1]

- (1) n
- (2) m

- (3) 1
 (4) n 或 m
 (5) m
 (6) n

[问题 2]

- (a) 读者 ID, 图书 ID
 (b) 读者 ID, ISBN 号

[问题 3]

关系模式	主键	外键
读者	读者 ID	/
书目	ISBN 号	/
图书	图书 ID	ISBN 号
借还记录	读者 ID, 图书 ID, 借书时间	读者 ID, 图书 ID
预约登记	读者 ID, ISBN 号, 预约时间	读者 ID, ISBN 号, 图书 ID

试题四

[问题 1]

1. 开发人员无需重新设计业务模式, 只需在原系统的界面和中间层添加 webservice 层, 就能继承原系统的所有业务。
2. 有可能以较少的开发时间和工作量, 将系统升级为 B/S 应用模式。
3. 可以不修改原有的 web 服务和中间层, 直接扩展新服务。

[问题 2]

WSDL 是 Web Service 的描述语言, 用于描述 Web Service 的服务、接口绑定等, 为用户提供详细的接口说明书。

SOAP 是通信协议, 用于以服务的方式在互联网上发布有用的程序模块。

UDDI 提供了一种统一的发布、查找和定位 Web 服务的方法。

[问题 3]

在 Web Service 模型的解决方案中, 服务提供者定义并实现 Web Service, 使用服务描述语言 (WSDL) 描述 Web Service, 然后将服务描述发布到服务请求者或服务注册中心; 服务请求者使用查找操作从本地或服务注册中心检索服务描述, 然后使用服务描述与服务提供者进行绑定并调用 Web Service。服务注册中心是整个模型中的可选角色, 它是连接服务提供者和服务请求者的纽带。

试题五

- (1) $p \rightarrow \text{numkeys}$
 (2) $p \rightarrow K[\text{mid}] > \text{akey}$
 (3) $p \rightarrow A[\text{hi}]$
 (4) root, akey, &f
 (5) $t \ \&\& \ t \rightarrow \text{numkeys} == M - 1$

试题六

- (1) `items[k].getProductptr() ->getProductId()`

- (2) items[k].getQuantity()
- (3) catalog.getProductByIndex(k)
- (4) it != sales.End()
- (5) it->getOrderedAmount

试题七

- (1) implements Iterable<Order>
- (2) this.orders.iterator()
- (3) this.catalog
- (4) this.sales
- (5) OrderItem item

2006 年下半年软件设计师上午试题答案

上午答案

(1)	C	(16)	D	(31)	B	(46)	B	(61)	A
(2)	A	(17)	B	(32)	D	(47)	C	(62)	A
(3)	C	(18)	C	(33)	C	(48)	B	(63)	B
(4)	B	(19)	D	(34)	D	(49)	D	(64)	A
(5)	C	(20)	A	(35)	A	(50)	C	(65)	B
(6)	B	(21)	B	(36)	B	(51)	B	(66)	C
(7)	D	(22)	D	(37)	A	(52)	D	(67)	B
(8)	A	(23)	A	(38)	C	(53)	A	(68)	C
(9)	B	(24)	C	(39)	D	(54)	A	(69)	A
(10)	C	(25)	D	(40)	B	(55)	B	(70)	C
(11)	B	(26)	B	(41)	A	(56)	C	(71)	B
(12)	C	(27)	D	(42)	A	(57)	B	(72)	C
(13)	B	(28)	C	(43)	B	(58)	B	(73)	B
(14)	A	(29)	C	(44)	C	(59)	D	(74)	D
(15)	A	(30)	A	(45)	B	(60)	C	(75)	A

2006 年下半年软件设计师下午试题答案

下午答案

试题一

[问题 1] 初录数据、复录数据

[问题 2] 0 层图中，数据清除处理（加工 6）没有输入数据流

[问题 3] ①

[问题 4] ①、②、④

[问题 5] 手工分户帐=初录分户帐+复录分户帐

试题二

[问题 1] (1) 房间号，身份证号

[问题 2]

住宿主键：房间号，身份证号，入住日期

住宿外键：房间号，身份证号，

[问题 3] (2) 住宿. 身份证号 (3) HAVING (4) ORDER BY 2 DESC

[问题 4]

表：住宿 属性：入住日期 类型：聚簇索引

原因：表中记录的物理顺序与索引项的顺序一致，根据索引访问数据时，一次读取操作
可以获取多条记录数据，因而可减少查询时间。

试题三

[问题 1] (1) 0..n (2) 1 (3) 0..n (4) 1..n (5) 1 (6) 0..n

[问题 2]

(1) getCategories

(2) getCommodities

(3) createPromotion

(4) addCommodities

[问题 3]

关系：聚集（聚合）是关联的特例。不同点：聚集表示部分与整体关系的关联。

试题四

- (1) $f[0][0] = e[0] + a[0][0]$
 $f[1][0] = e[1] + a[1][0]$
- (2) $f[0][j-1] + a[0][j]$
- (3) $f[1][j-1] + a[1][j] < f[0][j-1] + t[0][j-1] + a[1][j]$
- (4) $f_i = f[0][n-1] + x[0]$
 $li = 0$
- (5) $f_i = f[1][n-1] + x[1]$
 $li = 1$

试题五

- (1) EnQueue(&tempQ, root)
- (2) brotherptr = brotherptr -> nextbrother
- (3) !IsEmpty(tempQ)
- (4) DeQueue(&tempQ, &ptr)
- (5) !ptr->firstchild
- (6) EnQueue(&tempQ, ptr->firstchild)
- (7) brotherptr = brotherptr -> nextbrother

试题六

- (1) state == CLOSED || state == CLOSING
- (2) state == OPENING || state == STAYOPEN
- (3) state == OPEN
- (4) state ->click()
- (5) state ->timeout()
- (6) state ->complete()
- (7) door->setState(door->OPENING)

试题七

- (1) `state == CLOSED || state == CLOSING`
- (2) `state == OPENING || state == STAYOPEN`
- (3) `state == OPEN`
- (4) `state.click()`
- (5) `state.timeout()`
- (6) `state.complete()`
- (7) `door.setState(door.OPENING)`

2007 年上半年软件设计师上午试题答案

(1)	C	(16)	D	(31)	C	(46)	B	(61)	C
(2)	B	(17)	C	(32)	C	(47)	A	(62)	B
(3)	C	(18)	C	(33)	D	(48)	A	(63)	C
(4)	C	(19)	D	(34)	B	(49)	B	(64)	A
(5)	B	(20)	B	(35)	B	(50)	D	(65)	B
(6)	B	(21)	A	(36)	C	(51)	B	(66)	D
(7)	C	(22)	B	(37)	B	(52)	A	(67)	B
(8)	B	(23)	C	(38)	D	(53)	B	(68)	D
(9)	D	(24)	B	(39)	A	(54)	A	(69)	B
(10)	B	(25)	B	(40)	B	(55)	A	(70)	B
(11)	A	(26)	B	(41)	A	(56)	D	(71)	C
(12)	D	(27)	B	(42)	C	(57)	D	(72)	D
(13)	A	(28)	C	(43)	D	(58)	C	(73)	A
(14)	C	(29)	A	(44)	B	(59)	D	(74)	B
(15)	A	(30)	C	(45)	C	(60)	B	(75)	D

2007 年上半年软件设计师下午试题答案

试题一

【问题 1】 (1) 费用单

(2) 待租赁房屋列表

(3) 看房请求

(4) 变更房屋状态请求

【问题 2】 (5) 房主信息文件

(6) 租赁者信息文件

(7) 房屋信息文件

(8) 看房记录文件

【问题 3】 (1) 起点: 房主 终点: 变更房屋状态 数据流名称: 变更房屋状态请求

(2) 起点: 租赁者 终点: 登记租赁者信息 数据流名称: 租赁者信息

(3) 起点: 租赁者 终点: 安排租赁者看房 数据流名称: 看房请求

试题二

【问题 1】 (1) 1

(2) *, 或 n , 或 m

(3) *, 或 n , 或 m

(4) *, 或 n , 或 m

【问题 2】 缺少的联系数: 3

挂号单: 收银员

挂号单: 医师

挂号单: 门诊处方

【问题 3】 (5) 收银员

(6) 就诊号

(7) 药品编码, 数量, 单价

(8) 类型, 库存, 货架编号, 单位, 价格, 单价

挂号单主键: 就诊号

门诊处方主键: 就诊号

处方明细主键: 就诊号、药品编码

试题三

【问题 1】 (a) 资源目录 (b) 图书 (c) 唱片

【问题 2】

CatalogItem 的属性: 索引号、名称、出版时间、资源状态

图书的属性: 作者、页数

唱片的属性: 演唱者、介质类型

【问题 3】 (1) 1 (2) 0..* (3) 1 (4) 0..* (5) 1 (6) 1 或者 0..1

试题四

- (1) $k=0$
- (2) $j \leq N$
- (3) $k=k+1$
- (4) $d[i] + 6$
- (5) $O(N)$

试题五

- (1) root
- (2) root->childptr[0]
- (3) root->childptr[i]
- (4) placeBoosters(p)
- (5) degradation

试题六

- (1) std
- (2) virtual void
- (3) virtual void
- (4) flyBehavior
- (5) quackBehavior
- (6) flyBehavior->fly()
- (7) quackBehavior->quack()
- (8) FlyNoWay()
- (9) Squeak()

试题七

- (1) interface
- (2) interface
- (3) flyBehavior
- (4) quackBehavior
- (5) flyBehavior.fly
- (6) quackBehavior.quack()
- (7) abstract
- (8) FlyNoWay()
- (9) Squeak()

2007年下半年软件设计师上午试题答案

(1)	B	(16)	D	(31)	A	(46)	D	(61)	B
(2)	A	(17)	A	(32)	B	(47)	B	(62)	D
(3)	B	(18)	A	(33)	C	(48)	D	(63)	A
(4)	C	(19)	B	(34)	B	(49)	C	(64)	A
(5)	B	(20)	C	(35)	A	(50)	B	(65)	A
(6)	C	(21)	D	(36)	B	(51)	A	(66)	B
(7)	A	(22)	C	(37)	A	(52)	D	(67)	B
(8)	D	(23)	A	(38)	C	(53)	B	(68)	D
(9)	D	(24)	B	(39)	C	(54)	A	(69)	D
(10)	B	(25)	D	(40)	A	(55)	D	(70)	A
(11)	B	(26)	C	(41)	B	(56)	D	(71)	B
(12)	B	(27)	B	(42)	D	(57)	A	(72)	D
(13)	D	(28)	D	(43)	D	(58)	C	(73)	A
(14)	C	(29)	D	(44)	A	(59)	B	(74)	C
(15)	C	(30)	A	(45)	C	(60)	C	(75)	D

2007 年下半年软件设计师下午试题答案

试题一

[问题 1] E1：考试委员会； E2：主讲教师； E3：学生或选课学生； E4：教务处

[问题 2]

D1：学生信息文件； D2：课程单元信息文件； D3：课程信息文件；

D4：课程成绩文件； D5：无效成绩文件

[问题 3]

起点	终点
D4 或 课程成绩文件	4 或 生成成绩列表
D1 或 学生信息文件	5 或 生成最终成绩单
4 或 生成成绩列表	5 或 生成最终成绩单

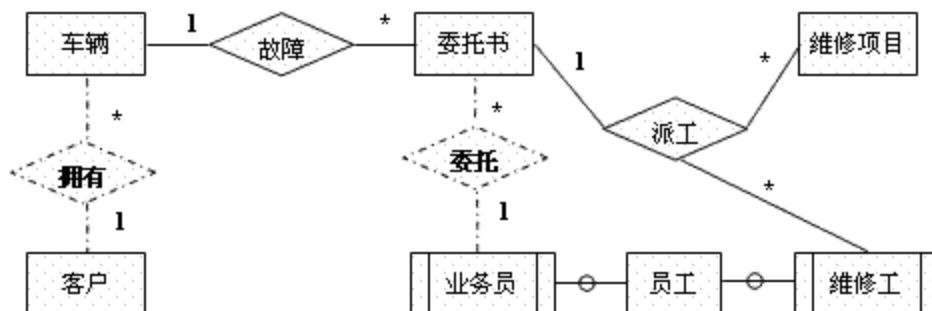
[问题 4] 程序流程图通常在进行详细设计时使用，用来描述程序的逻辑结构。

试题二

[问题 1]

- (1) * (或 n 或 m)
- (2) 1
- (3) * (或 n 或 m)
- (4) * (或 n 或 m)

[问题 2]



[问题 3]

- (5) 客户编号，客户名称，客户性质
- (6) 委托书编号，客户编号，车牌号，业务员编号
或：委托书编号，车牌号，业务员编号
- (7) 委托书编号，维修工编号，维修项目编号
- (8) 员工编号，员工姓名

[问题 4]

客户：客户编号 委托书：委托书编号
派工单：委托书编号，维修项目编号，维修工编号

试题三

[问题 1]

A: Artist B: Song C: Band D: Musician E: Track F: Album

[问题 2] (1) 0..* (2) 2..* (3) 0..1 (4) 1..* (5) 1..* (6) 1

[问题 3]

类	多重度
Track 或 E	0..1
Track 或 E	0..1

【问题 4】按任意键，选择歌曲

试题四

(1) $i \leq n$ (2) $d[J[r]] > d[i]$ (3) $J[r + 1] = i$, 或 $J[q + 1] = i$
(4) 1, 2, 4, 5 或 job1、job2、job4、job5 (5) 220 (6) 能 (7) 不能

试题五

(1) `this->center` (2) `this->center->common`
(3) `p_circle->center` (4) `va_start` (5) `C(P(20, 40), 10)`

试题六

(1) `ProcessRequest(aRequest)` (2) `Approver *` (3) `Approver::`
(4) `&Tammy` (5) `&Meeting` (6) `&Sam` (7) `Larry`

试题七

(1) `ProcessRequest(aRequest)` (2) `Approver` (3) `super`
(4) `Tammy` (5) `Meeting` (6) `Sam` (7) `Larry`

2008年上半年软件设计师上午试题答案

(1)	C	(16)	C	(31)	C	(46)	D	(61)	C
(2)	D	(17)	D	(32)	C	(47)	B	(62)	B
(3)	D	(18)	A	(33)	D	(48)	B	(63)	C
(4)	B	(19)	B	(34)	C	(49)	C	(64)	B
(5)	D	(20)	C	(35)	C	(50)	C	(65)	C
(6)	C	(21)	B	(36)	A	(51)	C	(66)	B
(7)	C	(22)	C	(37)	A	(52)	D	(67)	A
(8)	A	(23)	B	(38)	C	(53)	D	(68)	D
(9)	D	(24)	A	(39)	B	(54)	C	(69)	C
(10)	D	(25)	D	(40)	C	(55)	A	(70)	A
(11)	B	(26)	C	(41)	A	(56)	C	(71)	A
(12)	B	(27)	C	(42)	D	(57)	C	(72)	C
(13)	A	(28)	B	(43)	B	(58)	A	(73)	B
(14)	D	(29)	C	(44)	D	(59)	B	(74)	C
(15)	A	(30)	C	(45)	A	(60)	B	(75)	D

2008 年上半年软件设计师下午试题答案

试题一

参考答案

问题一

E1: 客户

问题二

D1: 客户信息文件

D2: 音像制品信息文件

D3: 租借记录文件

D4: 预约记录文件

问题三

起点	终点
E1 或客户	4 或创建新客户
5 或创建预约记录	E1 或新客户
6 或归还音像制品	7 或履行预约服务

注意：3 条数据流无前后顺序区分

问题四

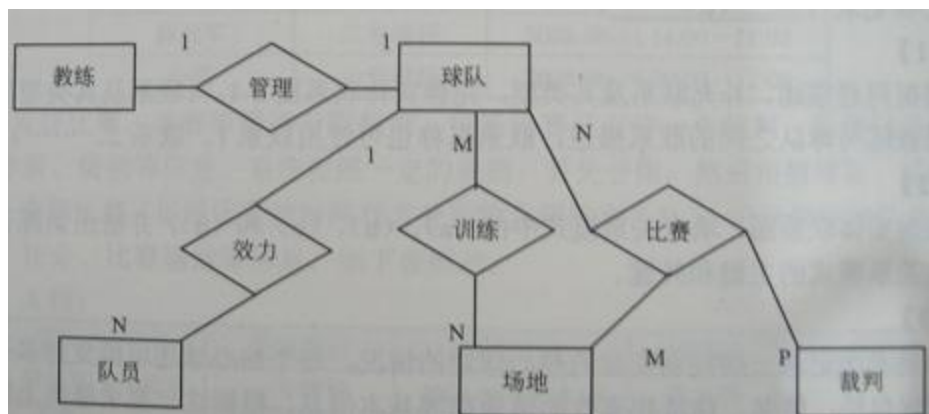
面向数据结构的设计方法以数据结构作为设计的基础，它根据输入/输出数据结构导出程序的结构。

面向数据结构的设计方法用于规模不大的数据处理系统。

试题二

问题一

（对联系名称不做要求，但不能出现重名，图中的 M、N、P 也可以表示为*）



问题二

(1) 球队名称

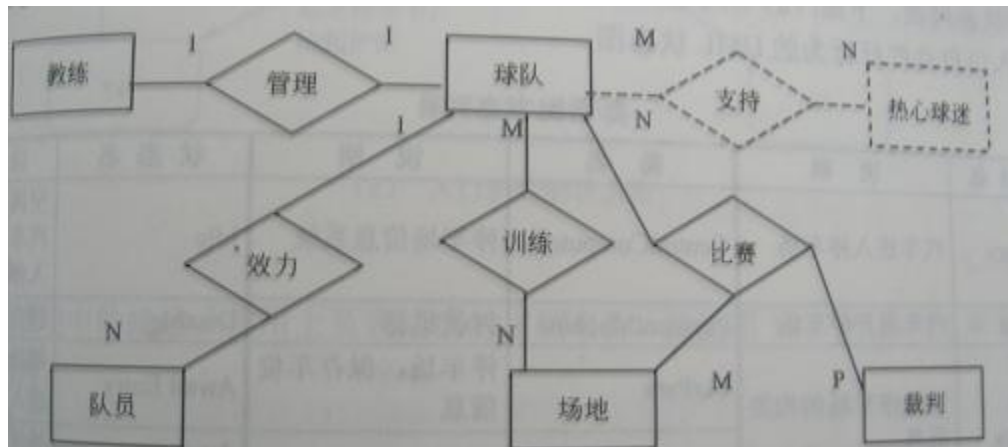
(2) 教练编号

(3) 球队名称, 场地名称, 开始时间, 结束时间

(4) 甲队, 乙队, 比赛时间, 场地名称, 比分, 裁判, 分组

训练记录	主键	(球队, 开始时间) 或 (场地名称, 开始时间) 或 (球队, 结束时间) 或 (场地名称, 结束时间)
	外键	球队名称, 场地名称
比赛记录	主键	(甲队, 比赛时间) 或 (场地名称, 比赛时间) 或 (裁判, 比赛时间) 或 (乙队, 比赛时间)
	外键	甲队, 乙队, 场地名称, 裁判

问题三



关系模式:

热心球迷 (球迷编号, 姓名, 住址, 俱乐部)

支持球队 (球迷编号, 球队)

试题三

问题一

U1: Car entry

U2: Car exit

U3: Car entry when full

问题二

A: Carpark B: Barrier C: EntryBarrier

D: ExitBarrier

其中, C、D 的答案可以互换

问题三

S1: Idle S2: Await Ticket Take S3: Await Enable S4: Await Entry

问题四

用例之间的延伸关系用于对用户看作是可选系统行为的用例的一部分建模。通过这种方式, 可以把可选行为从必需的行为中分离出来。

试题四

问题一

- (1) $A[i+1]$
- (2) $A[r]$
- (3) $i+1$

注：空（1）和空（2）答案可以互换

问题二

- (4) $O(n \lg n)$ 或 $O(n \log_2 n)$
- (5) $O(n \lg n)$ 或 $O(n \log_2 n)$
- (6) $O(n^2)$
- (7) 最坏

问题三

- (8) $A[i]$
- (9) $A[r]$
- (10) 否

注：空（8）和空（9）答案可以互换

试题五

- (1) $S=NULL // S \rightarrow pTop == Null$
- (2) $S \rightarrow pTop \rightarrow data$
- (3) $newNode$
- (4) $S \rightarrow pTop \rightarrow next$, 或 $lastTop \rightarrow next$
- (5) 24 4

试题六

- (1) $commands[button]$
- (2) $trunLight(0)$
- (3) $setChannel(1)$
- (4) $\&lightCommand$
- (5) $RemoteContrller$
- (6) $Light$
- (7) TV

试题七

- (1) $commands[button]$
- (2) $trunLight(0)$
- (3) $setChannel(1)$
- (4) $lightCommand$
- (5) $RemoteContrller$
- (6) $Light$
- (7) TV

2008年下半年软件设计师上午试题答案

(1)	B	(16)	A	(31)	C	(46)	D	(61)	A
(2)	D	(17)	D	(32)	A	(47)	C	(62)	B
(3)	B	(18)	A	(33)	D	(48)	B	(63)	D
(4)	C	(19)	C	(34)	C	(49)	D	(64)	C
(5)	B	(20)	A	(35)	B	(50)	A	(65)	C
(6)	A	(21)	B	(36)	C	(51)	D	(66)	A
(7)	D	(22)	D	(37)	A	(52)	D	(67)	C
(8)	A	(23)	B	(38)	B	(53)	C	(68)	C
(9)	C	(24)	D	(39)	D	(54)	B	(69)	A
(10)	A	(25)	D	(40)	D	(55)	B	(70)	B
(11)	C	(26)	C	(41)	B	(56)	C	(71)	D
(12)	D	(27)	B	(42)	C	(57)	B	(72)	B
(13)	D	(28)	A	(43)	D	(58)	C	(73)	A
(14)	D	(29)	C	(44)	B	(59)	A	(74)	C
(15)	B	(30)	D	(45)	A	(60)	C	(75)	D

2008 年下半年软件设计师下午试题答案

试题 1

问题 1

E1:客户 E2: 采购部门 E3: 库房 E4: 经理

问题 2

D1: 缺货记录文件 D2:库存记录文件 D3:订单记录文件

问题 3

起点	终点
库存记录文件或 D2	处理订单
进货处理	供货处理
订单记录文件或 D3	销售统计
缺货记录文件或 D1	进货处理

试题 2

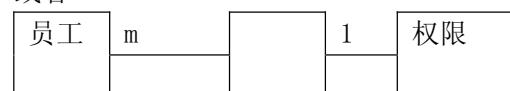
问题 1

- (1) n, 或 m, 或*
- (2) n, 或 m, 或*
- (3) n, 或 m, 或*

问题 2

需要增加员工和权限之间 m: 1 的联系

或者



问题 3

- (4) 员工号, 部门号
- (5) 客房号
- (6) 身份证号
- (7) 岗位
- (8) 客房号 身份证号

问题 4

若将权限表中的操作权限属性放在员工表中, 则相同岗位的操作权限在员工表中重复存储, 存在数据冗余

试题 3

问题 1

A1:User A2:Author A3:reviewer A4:PCChair

问题 2

U1:list accepted/rejected papers U2:browse submitted papers

U3:assign paper to reviewer

问题 3

(1) <<extend>>

(2) <<include>>

问题 4

Action1:enter title and abstract

Action2:select subject group

Action3:select paper location

Action4:upload paper

试题 4

问题 1

(1) $nv[i-1][j] \geq nv[i-1][j-p[i]] + v[i]$

(2) $nv[i][j] = nv[i-1][j]$

(3) $j = j - p[i]$

问题 2

(4) m2 m3 m4

(5) 605

问题 3

(6) $O(nM)$, 或 $O(n \times M)$, 或 $O(n * M)$

试题 5

(1) $pa = *LA$

(2) $pb \&\&pb \rightarrow elem \neq pa \rightarrow elem$, 或其等价表示

(3) pb 或 $pb \neq NULL$

(4) $pa \rightarrow next$, 或 $(*pa).next$, 或其等价表示

(5) $pre \rightarrow next$, 或 $(*pre).next$

(6) $pre = pa$

试题 6

(1) Document*

(2) aDocument

(3) !canOpenDocumengt(docName)

(4) Document*

(5) doCreateDocument()

(6) adoc->open(docName)

- (7) adoc->read(docName)
- (8) addDocument(adoc)

试题 7

- (1) Document
- (2) aDocument
- (3) !canOpenDocumengt(docName
- (4))Document
- (5) doCreateDocument()
- (6) adoc.open(docName)
- (7) adoc.read(docName)
- (8) addDocument(adoc)

2009年上半年软件设计师上午试题答案

(1)	A	(16)	A	(31)	B	(46)	B	(61)	A
(2)	B	(17)	C	(32)	B	(47)	B	(62)	C
(3)	B	(18)	C	(33)	B	(48)	D	(63)	B
(4)	A	(19)	C	(34)	B	(49)	B	(64)	A
(5)	C	(20)	C	(35)	C	(50)	D	(65)	B
(6)	B	(21)	D	(36)	B	(51)	C	(66)	C
(7)	B	(22)	C	(37)	D	(52)	A	(67)	B
(8)	D	(23)	D	(38)	C	(53)	D	(68)	D
(9)	B	(24)	C	(39)	B	(54)	B	(69)	A
(10)	B	(25)	B	(40)	A	(55)	C	(70)	D
(11)	A	(26)	D	(41)	A	(56)	A	(71)	A
(12)	B	(27)	B	(42)	B	(57)	C	(72)	C
(13)	C	(28)	D	(43)	C	(58)	A	(73)	C
(14)	A	(29)	B	(44)	D	(59)	C	(74)	A
(15)	D	(30)	C	(45)	B	(60)	D	(75)	B

2009 年上半年软件设计师下午试题答案

注意：本答案仅做参考

试题一

[问题 1]

E1: 采购部; E2:财务部; E3: 连锁超市; E4: 配送部门

D1: 采购清单记录文件; D2: 商品库存记录文件; D3: 配送记录文件; D4: 供货请求记录文件

[问题 2]

错误数据流

起点	终点
D4	供货请求处理
供货请求处理	E2
D3	E4
D2	采购处理

补充数据流

起点	终点
供货请求处理	D4
供货请求处理	采购处理
采购处理	D2

试题二

[问题 1]

商场 (1) ——联系 1——部门 (n)

部门 (1) ——联系 2——经理 (1)

部门 (1) ——联系 3——员工 (m)

经理 (1) ——联系 4——员工 (p)

[问题 2]

(a) 商场编号; (b) 部门编号; (c) 员工编号, 部门编码号

部门: PrimaryKey 部门编号 ForeignKey 商场编号

员工: PriamryKey 员工编号 ForeignKey 部门编号

经理: PriamryKey 员工编号任职时间 Foreign 部门编号

[问题 3]

紧急联系人; N: 1; 紧急联系人 (N) ——联系人——员工 (1)

试题三

[问题 1]

A1: CardReader; A2: CustomerConsole

U1: Session; U2: InvalidPinProcess; U3: Transaction

(1): extend

[问题 2]

```
6: ReadPIN ();  
7: PIN ;  
8: creat(atm,this,card,pin);  
9: performTransaction()
```

[问题 3]

U3 和 Withdraw、Deposit 等四个用例之间的关系是 Extended(扩展关系), 内涵为 Withdraw、Deposit 等四个用例都是充分条件, 只有满足了才执行 U3。

试题四

- (1) $k=1$ ton
- (2) $Dij(k)=dij(k-1)$
- (3) $dij(k)=diK(k-1)+dkj(k-1)$
- (4) $sp[i]=sp[i]+dij(j)$
- (5) $min_v=1$
- (6) min_SP
- (7) $O(n^3)$

试题五、

- (1) $ptr!=NULL$
- (2) $q->link=stacktop$
- (3) $ptr->lchild$
- (4) $stacktop=stacktop->link$
- (5) $q->elem->rchild$

2009 年下半年软件设计师上午试题答案

本答案为互联网收集，仅供参考。更新答案信息请关注 RuanKao.net

一、选择题

1-5 : B、A、C、A、A、

6-10:A、C、A、B、C、

11-15:A、B、C、D、C、

16-20:D、A、A、D、B、

21-25:B、D、B、D、C、

26-30:A、A、B、C、C、

31-35:A、C、B、D、D、

36-40:C、C、D、A、C、

41-45:A、A、B、D、C、

46-50:D、D、D、C、A、

51-55:C、A、D、B、C、

56-60:D、D、C、C、B、

61-65:B、B、A、D、A、

66-70:A、C、A、B、C、

71-75:A、C、B、D、A、

2009 年下半年软件设计师下午试题答案

本答案为互联网收集，仅供参考。更新答案信息请关注 RuanKao.net

试题一：

1、E1 非信用卡客户 E2 信用卡客户 E3 银行

2、P0-E2 E1-P0 P0-E1

3、(1) 信用卡申请表 更改：E1-P4

(2) 个人信息 更改：P4-P2

4、交易信息查询，信用卡客户信息管理，信用卡激活，信息卡申请

试题二：

1、一对一，一对多，一对多

2、用户名，邮件号，邮件号

3、地址簿主键：用户名和联系人编号，邮件主键：邮件号，附件中的主键：附件号 外键是邮件号

试题三：

1、顾客，菜单管理员

2、生成付费请求，参加者：餐厅员工 今日特价，参加者：员工 注册工资支付，参加者：顾客

生成付费请求并发送给工资系统，参与者：餐厅员工

3、请求登录，注册到系统，发送订单信息，顾客接受订单

4、包含关系 顾客包含在员工内 只要是顾客，必定对应员工里相关信息

试题四：

1、 $k < -0$, $cw < -cw + w[k]$, $Y[k] < -X[k]$,

2、2, 18, 8, 4

试题六：

```
abstract , null , List , childList , System.out.println(ifile.printName());
```

2010 上半年标准答案（希赛提供）

1.B 2.D 3.C 4.B.D 5.C
 6.C 7.D 8.D 9.B 10.B
 11.A 12.A 13.B 14.D 15.C
 16.D 17.A 18.D 19.C 20.B
 21.D 22.A 23.D 24.B 25.A
 26.C 27.C 28.B 29.C 30.B
 31.C 32.B 33.B 34.C 35.C
 36.C 37.D 38.B 39.D 40.C
 41.C 42.A 43.C 44.D 45.A
 46.D 47.C 48.C 49.D 50.B
 51.A 52.B 53.B 54.A 55.D
 56.C 57.D 58.A 59.B 60.A
 61.C 62.B 63.C 64.B 65.C
 66.C 67.B 68.B 69.D 70.D
 71.B 72.C 73.D 74.B 75.A

试题一(15 分)

问题 1（3 分，各 1 分）

E1: 前端应用 E2:数据管理员 E3:后端数据库

问题 2（3 分，各 1 分）

D1: 用户表 D2: 操作表 D3:权限表

问题 3（6 分）

P 的名称：操作结果处理（1 分）

	名称	起点	终点
输入流	操作结果（1 分）	E3（0.5 分）	P
输出流	处理后的操作结果（1 分）	P	E1(0.5 分)

缺少的数据流（每行 1 分，共 2 分）

起点	终点
D2	权限验证
D3	权限验证

问题 4（3 分）

在绘制数据流图的加工时，可能出现的输入，输出错误：

只有输入而无输出 或者 黑洞（1 分）

只有输出而无输入 或者 奇迹（1 分）

输入的数据流无法通过加工产生输出流 或者 灰洞（1 分）

输入的数据流与输出的数据流名称相同（1 分）

注：总分 3 分，答对上述 1 个即给 1 分，多答不多给分。

试题二

问题 1 (6 分)

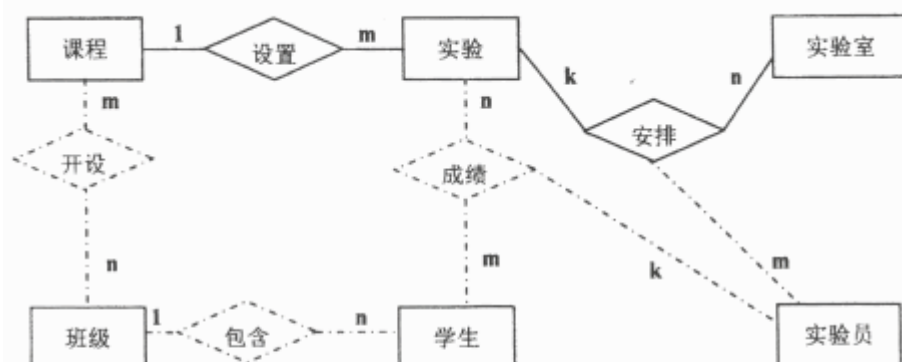


图 2-1 实体联系图

课程与班级之间的联系 1 分，联系的类型 1 分

班级与学生之间的联系 1 分，联系的类型 0.5 分

实验、实验员与学生之间的联系 1 分，联系的类型 1 分

实验、实验员与实验室之间的联系及类型 0.5 分

问题 2 (6 分)

- (1) 课程编号, 班级号
- (2) 实验编号, 课程编号
- (3) 实验编号, 批次号, 安排学期, 实验室编号, 实验员编号
- (4) 实验员编号, 实验员姓名
- (5) 学号, 班级号
- (6) 实验编号, 学号

注：每个空 0.5 分，每个主键 0.5 分。

问题 3 (3 分)

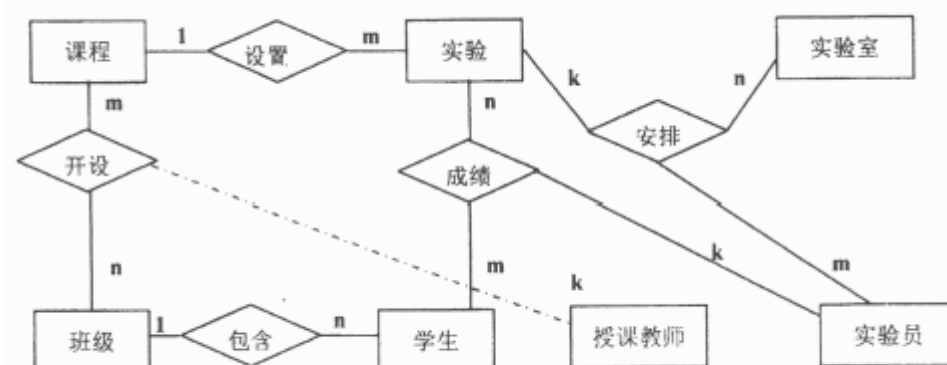


图 2-2 修改后的实体联系图

授课教师、课程与班级之间的联系 1.5 分，联系的类型 1.5 分。

试题三

问题 1 (5 分, 各1分)

A1: 乘客 A2:服务技术人员

U1: 支付 (1)<<include >> (2)<<include >>

问题 2 (7 分)

C1:键盘 (2分)

C2:目的地键盘 (1分)

C3:车票键盘 (1分)

C4:继续/取消键盘 (1分)

(3)~(6):1 (各 0.5分)

问题 3 (3分)

使用 Mediator 模式,可以使各个对象间的耦合松散(1分),只需关心和 Mediator 的关系,使多对多的关系变成一对多的关系(1分),可以降低系统的复杂性,提高可修改扩展性。(1分)。

试题四

问题 1 (9分)

(1) InitQueue(&Q) (1分) 注:函数名与参数必须完全正确才可得分

(2) DeQueue(&Q,&W) (2分) 注:函数名与参数必须完全正确才可得分

(3) inDegree[p->adjvex] 及其等价形式 (2分)

(4)inDegree[p->adjvex] 及其等价形式 (2分)

(5) $k \leq G..n$ 或 $kl = G..n$ (2分)

问题 2 (2分)

队列方式: v1 v2 v5 v4 v3 v7 v6(或 1 2 5 4 3 7 6) (1分)

栈方式: v1 v2 v5 v4 v7 v3 v6(或 1 2 5 4 7 3 6) (1分)

问题 3 (4分)

(6) $O(n+e)$ (2分)

(7) $O(n^2)$ (2分)

试题五

(1) FlyBehavior*flyBehavior (2分)

(2) TakeOffBehavior*takeOffBehavior (2分)

(3) flyBehavior->fly() (2分)

(4) takeOffBehavior->takeOff() (2分)

(5) SubSonicFly() (2分)

(6) VerticalTakeOff() (2分)

(7) ~Helicopter() (3分)

注 空1与空2的答案可以互换

试题六

(1) FlyBehavior.flyBehavior (2分)

(2) TakeOffBehavior.takeOffBehavior (2分)

(3) flyBehavior.fly() (2分)

(4) takeOffBehavior.takeOff() (2分)

(5) extends (3分)

(6) SubSonicFly() (2分)

(7) VerticalTakeOff() (2分)

注 空1与空2的答案可以互换

网友讨论

【答案】1.B

程序计数器 PC 自动加 1，指向下一条指令。

【答案】2.D

并联的可靠度为 $1-(1-R)(1-R)$ 串联的可靠度等于各个部分的乘积。

故系统可靠度为 $R \times (1-(1-R))(1-(1-R))$ 。

【答案】3.C

输入输出设备的中断是可屏蔽的，而掉电什么都白扯，机器断电即玩完，掉电不可屏蔽。

【答案】4.B.D

数学逻辑推导，或维恩图验证，或者代入数值计算，原始 A、B 相同时为真，不同时为假。

【答案】5.C

在分析指令的时候需要将指令(包括操作码和地址码)首先存放到指令寄存器 IR 中，然后再按操作码进行译码，利用地址码计算相应的物理地址。

【答案】6.C

64 位和 32 位通常代表设计时候的数据总线宽度、CPU 内寄存器位数、ALU 运算位数。与运算数度没有必然的联系。

【答案】7.D

一直没有用 Outlook，但是知道可以脱机处理，管理多个邮件帐号。至于 C 项，微软那么强大，做这么个小功能还是小菜的。D 不能发送和接受安全邮件这点应该是能够做到的。猜选 D 项。

【答案】8.D 9.B

Macro 是宏的意思，C 语言中可是有这个东西的，那么这个病毒自然联想到宏病毒。宏病毒是利用 VBS 脚本编写的，寄生在 Word 和 Excel，一般设置安全低的环境，打开文件即执行了宏病毒。一直想看看 VBS，想做个签名的 word 插件，可惜手头没有书，之前在 HIT 的图书馆见过。

PS:这是一个浪漫的爱情故事。一个男孩遇到了一个女孩。女孩靠跳舞赚钱，男孩回家为那个女孩编写计算机病毒。这个计算机病毒后来流传了出去，造成了数以百万计美元的损失。

美国新泽西州警方宣布，Melissa 电子邮件病毒的制造者已经被抓获。这位 30 岁的新泽西州男子名叫大卫·史密斯，人称“电脑小子”，是一家电脑公司的网络程序员。他被指控包括妨碍公共通信在内的多项罪名。新泽西州检察长的发言人称，史密斯是星期四晚上在邻近的伊顿城他哥哥家中被捕的。

据新泽西州检察当局指称，史密斯之所以给他发明的病毒取名 Melissa，是为了纪念他在佛罗里达居住时认识的一名脱衣舞娘。警方发言人维尔尼罗说，在美国在线的帮助下，办案人员获取了史密斯连入互联网的历史记录，并最终查获到他病毒送上网时使用的电话线路。维尔尼罗称，史密斯这么做，不过是为了出名。史密斯最大的梦想就是能有一种成就感，现在他终于“成功”了。

【答案】10.B

专利和注册商标一样，谁先申请谁有权，甚至有理说不清。

看到讨论，大家多虑了，这个出题人没有那么阴人的爱好。找到证据如下：

法条：两个以上的申请人分别就同样的发明创造申请专利的，专利权授予最先申请的人。

释义：本条是关于两个以上的申请人就同样的发明创造提出专利申请时，应当如何处理的规定。

一、按照本条的规定，我国专利法在处理两个以上的主体分别就同样的发明创造申请专利时，实行申请在先的原则。申请在先原则是指当两个以上的主体就同样的发明创造申请专利时，不论谁发明在先，专利权授予最先提出专利申请的人。实行这一原则，可以克服先发明原则的弊端，有利于鼓励发明人及时提出专利申请，充分发挥专利制度促进技术公开和交流的作用；同时，由于只需要看谁的申请在先，而不需花费大量的时间和精力去确定谁是先发明人，可以大大提高对专利申请审查的效率。

二、两个以上的主体就同样的发明创造提出专利申请时，如何确定申请时间的先后，按照我国专利法实施细则的规定，我国实行按日计算的办法。而根据“专利不重复”的原则，对同样的发明创造，只能向一个主体授予一项专利权。按照专利法实施细则的规定，两个以上的申请人在同一日分别就同样的发明创造申请专利的，应当在收到专利机关的通知后自行协商确定申请人。在这种情况下，申请各方可以协商确定一个申请人，也可以协商确定各方作为共同申请人。如果各方协商不成的，按照国务院专利行政部门发布的《专利审查指南》的规定，则只能驳回各方的专利申请。

三、对于如何判断两个以上主体申请专利的发明创造是否属于“同样的发明创造”，在《专利审查指南》中分别对是否属于同样的发明、实用新型和外观设计的判断标准作了具体规定，应按此规定执行。

【答案】11.A

在工作范围内的作品属于职务作品，著作权归公司。至于 C 项的复制权是否有，就不确定了，那么按照“不求甚解，只要定解”的心态选择了 A，那么程序员就不享有复制权了。呵呵.....

【答案】12. A 13. B 14. D

12、13 题谢谢网友 zhenhao2008 和 xyxz341 的提供，我回来翻了一下教材第六章都漏掉了，呵呵~

我自己选的是 DCD，复习的时候多媒体部分直接藐视，现在任意发挥型。因为 VCD 比较早，但估计有不是最早的，所以在 BD 中徘徊，给了 D 一个面子，因为长得很特殊嘛。至于视频 DVD，只知道 mp4 是 C 项，但是具体用没用在 DVD 中就不知了。最后，估计 MPEG-7 是个高科技、新标准，估计可能以后就不用进行压缩了，所以选 MPEG-7 了。

PS：MPEG-7 标准被称为“多媒体内容描述接口”，为各类多媒体信息提供一种标准化的描述，这种描述将与内容本身有关，允许快速和有效的查询用户感兴趣的资料。它将扩展现有内容识别专用解决方案的有限的能力，特别是它还包括了更多的数据类型。换言之，MPEG-7 规定一个用于描述各种不同类型多媒体信息的描述符的标准集合。

【答案】15.C

是复用的一种技术，采用构件库进行开发，那么首先要正确合格，然后应该适当修改成你所需要的，没有 100%合适的复用的。复用复用，本来就不是为你专门开发的，当然要修改啦。

【答案】16.D

需求分析的目的，理解用户需求，知道要做什么。]

【答案】17.A

白盒测试要拿到源代码，明白内部逻辑，然后根据覆盖标准才能测试。

【答案】18.D 19.C

Gantt 图只能看到开始时间，结束时间和持续时间，无法看到各个子任务的关联。

关键路径 12579 长度为 15 天，用 $15-4-1=10$ 可得 6 开始的最晚时间。

【答案】20.B

FFFFH=1111 1111 1111 1111(补码)=1000 0000 0000 0001(原码)=-1。

【答案】21.B

(dannie 更正：D。其实想一想以前数据结构里面怎么把一个算式改成后缀式的代码实现就很理解了。后缀式运算是从左至右扫描，见到字符就存入栈内，如果遇到运算符，则从栈顶弹出相应的数量的字符（一元运算符弹出一个，二元弹出两个，一次类推），进行计算，然后将计算所得结果存入栈内。通过这个思想对答案进行验证即可。此外需要注意的是，与运算比或运算运算级高)

希赛：D

抛硬币可得，因为加上一个短路求值，这样就不会算了。有会的大侠告诉我一下。

我仍然坚持 B，抛硬币真理啊！撞了南墙不回头啊。

大家注意两点：1.与、或优先级别应该相同；2.这道题目里明显加入了短路求值的考虑。

先判断 a 与 b 为真，跳过 c 和或运算符，接着判断 b，如果 b 为真，直接进行最后的与操作。

【答案】22.A

编译原理也基本没有看，徘徊在 AB 间，石头剪子布得出答案。大侠帮我。

【答案】23.D

简单理解成 mail 了，没有发出去的话应该是对方满了，自以为是得到的答案。

【答案】24.B

简单的鸽巢原理，还是喜欢当年离散数学的鸽洞原理称谓，总想起王老师。

【答案】25. A 26. C

这题目出的也够烂的，连修改位的 0 和 1 代表什么都不说一声，自己意会吧。因为淘汰页面，必然替换在内存中已有的页，内存中有 0、2、4 号页，观其修改为，0 页为 0，2、4 页为 1，推断修改位为 1 代表正在被使用，故选择 A 选项 0 号页。

+-----+	块 1
4K	
+-----+	块 2
4K	
+-----+	块 3
4K	

+-----+

26 题现在才发现我一直看错了，晕啊，我错选成 A 项了。

2C25H=10 1100 0010 1001 其中块号为 10=2，而页号为 2 的页帧号为 4，故选择 C。

【答案】27.D 28.B

感觉是历年的真题，一点没有改动，这次往年未改动的真题出了几道，或者是我记错了。但是确定是见过这道题目，当时觉得这道题目的计算有点勉强。

最长时间：每次要读的下个数据块恰好刚转过去。

第一块 R0 读入 3ms+处理 3ms+(第二条等待 24ms+读入 3ms+处理 3ms)×8 块-3ms=243ms。最后减去的 3ms 是因为有单缓冲区的缘故，即处理 R0 和等待 R1 时重叠 3ms。呵呵…… 很复杂吧！能看懂你就是高手了。。。

28 空，记得当时给的是 $(3+3) \times 9 = 54\text{ms}$ ，但我觉得单缓存的二级流水应是更少时间，唉……

希赛：27C

27 题确定选择 D，大家可以 google 题目的第一行，就会发现类似的题目，看看解题过程。另外，2007 年下半年的数据库系统工程师上午试卷考过这道题目，只是选项变了，如下：看两题的共同选项！！

● 假设磁盘上每个磁道划分成 9 个物理块，每块存放 1 个逻辑记录。逻辑记录 R1, R2, ..., R9 存放在同一个磁道上，记录的安排顺序如下表所示：

物理块	1	2	3	4	5	6	7	8	9
逻辑记录	R1	R2	R3	R4	R5	R6	R7	R8	R9

假定磁盘的旋转速度为 27ms/周，磁头当前处在 R1 的开始处，系统使用单缓冲区，当缓冲区内容处理完时才能再存放新的记录，且每个记录的处理时间为 3ms。若系统顺序处理这些记录，则处理这 9 个记录的最长时间为 (20) ；若对信息存储进行优化分布后，处理 9 个记录的最少时间为 (21) 。

(20) A. 243ms B. 246ms C. 254ms D. 280ms

(21) A. 30ms B. 36ms C. 54ms D. 60ms

我觉得没必要再讨论 27 题了，出题方已经搬石头砸自己的脚了，必然

27 题选什么都会给分的，请看官方出的 07 年答案分析，题目如上，只

有记录块名称从 R1 开始编号而已，其它的都一样。

【答案】29.C

自己用骰子测算而知 A。等待大仙显灵……

回帖网友 cuijj102 说见过原题，应该选 C，谢谢提供。如果有异议，请自行查找。

【答案】30.B

模块的独立性要求。

【答案】31.C

经过本人占卜得到的结果是 B，结果回来查一下，应该是 C 的定义级。如下：

0 不完整级 1 执行级 2 管理级 3 定义级 4 量化管理级 5
最佳化级

考了无数年的 CMM 五级，突然弄出个 CMMI 六级，而且还不是对应的，要命啊!!

【答案】32.B

某年的真题，我见过。初期阶段里程碑是生命周期目标建立，架构是在净化阶段。

【答案】33.B

此题目有鄙视人的嫌疑，无论学什么程序设计都讲的基础。

【答案】34.C

经过大仙托梦给我的。

【答案】35. C

是 M1 和 M2 的集成测试最可能，当然 M1 当中也可能，只选最最最……

好像写了很长了，看着不舒服，决定再起一篇文章续上。

PS:原答案是 CC 答得,后面括号内红色部分是 dannie 根据网友反馈重新更正的。

【答案】36.C

历年考过两次以上了，公式：弧数目-节点数目+2=11-7+2=6.

【答案】37.D

类的继承是允许多继承的。

【答案】38.B 39.D 40.C

(dannie 更正：38:B (多态的显示定义)。某个人，这种蒙都能蒙对的题目你也能错!!!)

纯粹定义理解。呵呵～ 38 题考试中是选择的多态，昨晚整理答案时弄错了。

【答案】 41.C

分析阶段如果连对象都实现了，那么设计和实现阶段干什么活呢？做人厚道，不抢他人饭碗，大家发财嘛。

【答案】 42.A

一定是先 OOA，然后再 OOD。

【答案】 43. C 44. C

希赛：44D

周公告诉我选的是 AC，回来发现更像 CC。

PS：工厂方法模式对简单工厂模式进行了抽象。有一个抽象的 Factory 类（可以是抽象类和接口），这个类将不再负责具体的产品生产，而是只制定一些规范，具体的生产工作由其子类去完成。在这个模式中，工厂类和产品类往往可以依次对应。即一个抽象工厂对应一个抽象产品，一个具体工厂对应一个具体产品，这个具体的工厂就负责生产对应的产品。

每一个模式都是针对一定问题的解决方案，工厂方法模式针对的是一个产品等级结构；而抽象工厂模式针对的是多个产品等级结果。产品族：是指位于不同产品等级结构中，功能相关联的产品组成的家族。当有多个不同的等级结构的产品时，如果使用工厂方法模式就势必要使用多个独立的工厂等级结构来对付产品的等级结构。如果这些产品等级结构是平行的，会导致多个平行的工厂等级结构。抽象工厂模式使用同一个工厂等级结构负责不同产品等级结构产品对象的创建。

【答案】 45.A 46.D 47.C

概念理解。

【答案】 48.C

编译将源代码生成目标程序，目标程序可以独立于源程序运行。

【答案】 49.D

PHP 脚本语言，WML(Wireless Markup Language - 无线标记语言)。

【答案】 50.B

历年有类似的题，自动机与形式语言的知识，较简单，可举反例排除 ACD。

【答案】 51.A

转铅笔得到错误的真谛 D。

正解：系统边界，即系统包含的功能与系统不包含的功能之间的界限。一般在系统分析阶段定义，只有明确了系统边界，才能继续进行下面的分析、设计等工作。关系模式是在数据库设计的逻辑结构设计中的。

【答案】 52.B

数据库运算，笛卡尔积、选择、投影。

【答案】 53.B 54.B 55.D 56.C

53、54 没说的，55 要按照零件号组的进行求值，使同一类的数量可以统计。

希赛：54A

【答案】 57.D

折半查找不成功时候需要比较最多，二叉树的高度那么多次，为 D 项。

【答案】 58.A

注意 M 数组从 1 开始, $A[i,j]$ 前有 i 行, 元素个数分别为 1, 2, 3, ..., i 共 $i*(i+1)/2$ 个, 加上 $A[i,0], A[i,1], \dots, A[i,j]$ 共 $j+1$ 个, 总计 $i*(i+1)/2+j+1$.
对应关系: $M[1] \rightarrow A[0,0], M[2] \rightarrow A[1,0], \dots, M[i*(i+1)/2+j+1] \rightarrow A[i,j]$.

【答案】 59.B

树的数学知识, n 个叶子, $n-1$ 个非叶节点, 共 $2n-1$ 个。

【答案】 60.A

栈在实际中的典型应用。

【答案】 61. C

N 个数, 插入排序当按照要求已经排好序, 只需要比较 N 次, 比较次数是最少的。

【答案】 62. B

装填因子越大, 代表装入记录数占表长的比例变大, 当所有的表都有至少一条记录时, 必然会存在冲突。这题目也不是很确定。

【答案】 63. C

二叉排序树, 右子树节点均大于根节点。

【答案】 64. B

历年考过的,

$T(n)=T(n-1)+n=T(n-2)+n-1=\dots=T(1)+n+(n-1)+(n-2)+\dots+2=n\times(n+1)/2$ ，故时间复杂度为 $O(n^2)$ 。

【答案】65. C

单循环链表特性，以前做过的题目，结果还是选 A 了，唉……。

插入直接插入尾指针 rear 的下一个，而删除需要获得尾指针 rear 的上一个元素，上一个需要循环一遍链表才能得到。

【答案】66. C 67. B 68. B

(dannie 更正：66:C(还余下 $32-26=6$ 位，故可以主机地址有 $2^6-2=62$ 位);68:B(计算 $10111110=190$)。真是不懂，某个人，从第 27 题都能看出来啊，你网络全还给老师去了啊，还给总给人布网线，真不知道说啥了。)

自己选的 DBD，计算机网络的知识没有看，至于这题目祈祷了一会，呵呵……自己弄吧。从 67 到 68 进行组合，看是否满足 66 的结果，最后敲定 129 到 254 共 126 个 IP 地址，故填上凑数。

正解：222. 125. 80. 128/26 中的 26 是指前 26 位二进制数不变，那么还有 $32-26=6$ 位可用于子网内主机，即 $2^6-2=64-2=62$ (去掉六位全 0 和全 1)。

【答案】69. D

代码 mailto 之前写过一次。

【答案】 70. D

唉，蒙错了，自己选择了 B，回来查了一下是 110 端口。

【答案】 71. B 72. C 73. D 74. B 75. A

71 相同的选择，根据打的煎鸡蛋比方。

错以为：72 不像 managers，那么应该是编程或者构建者。

73 schedule 与第一句话中对应。

74 语义

75 根据 publicize 这个词，公开化，那么数据应该是分享的。

《The Mythical-Man Month》中的原文如下：

“Observe that for the programmer, as for the chef, the urgency of the patron may govern the scheduled completion of the task, but it cannot govern the actual completion. An omelette, promised in two minutes, may appear to be progressing nicely. But when it has not set in two minutes, the customer has two choices—wait or eat it raw. Software customers have had [the same](#) choices.

The cook has another choice; he can turn up the heat. The result is often an omelet nothing can save—burned in one part, raw in another. (此段被出题者略去)

Now I do not think software [managers](#) have less inherent courage and firmness than chefs, nor than other engineering

managers. But false [scheduling](#) to match the patron's desired date is much more common in our discipline than elsewhere in engineering. It is very [difficult](#) to make a vigorous, plausible, and job-risking defense of an estimate that is derived by no quantitative method, supported by little data, and certified chiefly by the hunches of the managers.

Clearly two solutions are needed. We need to develop and publicize productivity figures, bug-incidence figures, estimating rules, and so on. The whole profession can only profit from [sharing](#) such data. Until estimating is on a sounder basis, individual managers will need to stiffen their backbones and defend their estimates with the assurance that their poor hunches are better than wish-derived estimates."

The Mythical-Man Month,

Frederick P. Brooks, page 21